



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

TrasmUM

Luís Ferreira
a98286

José Fernandes
a106937

Gabriel Dantas
a107291

Simão Oliveira
a107322

Maio, 2026

LI4

Data da Receção	
Responsável	
Avaliação	
Observações	

TrasmUM

Luís Ferreira
a98286

José Fernandes
a106937

Gabriel Dantas
a107291

Simão Oliveira
a107322

Maio, 2026

Resumo

O presente relatório documenta o desenvolvimento do sistema TrasmUM, uma solução de gestão integrada para uma rede de três lojas de conveniência no distrito de Bragança, complementada por uma sede central. O principal desafio consistiu em substituir um modelo de operação integralmente analógico — assente em registos em papel e recolha telefónica diária de dados — por uma solução tecnológica federada e resiliente, capaz de dotar cada loja de autonomia operacional plena enquanto centraliza a informação relevante para a tomada de decisão estratégica.

O trabalho desenvolveu-se ao longo de quatro etapas — engenharia de requisitos, arquitetura e design, implementação e, por fim, verificação e validação — em todas as quais modelos de linguagem de grande dimensão (LLMs) foram utilizados como agentes de apoio, com registo sistemático dos *prompts* e reflexão crítica sobre cada interação. A análise de requisitos produziu a especificação formal de 23 requisitos funcionais e 18 requisitos não funcionais, traduzidos posteriormente em casos de uso, diagramas de máquinas de estado e numa arquitetura documentada através de registos de decisão arquitetural. A solução materializou-se em dois softwares independentes: o Software de Loja, um terminal de ponto de venda com persistência local e sincronização assíncrona de integridade verificável; e o Software de Servidor Central, uma aplicação web de supervisão da rede.

O sistema encontra-se operacional, com a totalidade dos requisitos funcionais verificados, uma suíte de 54 testes unitários sem falhas e cobertura de instrução superior a 80% nas classes da camada de serviço. Ao longo do relatório, o processo de desenvolvimento é apresentado em articulação com as interações com LLMs e as decisões técnicas que as acompanharam.

Área de Aplicação: Engenharia de Software Assistida por LLM; Sistemas de Informação para Gestão de Retalho.

Palavras-Chave: Gestão de Lojas de Conveniência, JavaFX, Javalin, Vue.js, MySQL, Docker, *Large Language Models*, Engenharia de Requisitos, Arquitetura de Software, Testes Unitários, JUnit 5, JaCoCo

Índice

1. Introdução	1
1.1. Âmbito do Projeto	1
1.2. Metodologia de Utilização de LLMs	1
1.3. Estrutura do Relatório	2
2. Definição do Domínio do Sistema	4
2.1. Utilização de LLMs na Definição do Domínio do Sistema	4
2.2. Contextualização	4
2.3. Motivação e Objetivos	5
2.4. Recursos Necessários	6
2.5. Análise de Contexto e Restrições	6
2.6. Análise de Viabilidade	7
2.7. Medidas de Sucesso	7
2.8. Plano de Execução do Projeto	8
2.9. Revisão e Aprovação do Projeto	9
3. Levantamento e Análise de Requisitos	11
3.1. Utilização de LLMs no Levantamento e Análise de Requisitos	11
3.2. Métodos de Levantamento de Requisitos Utilizados	11
3.3. Levantamento de Requisitos	17
3.4. Refinamento de Requisitos Ambíguos	17
3.5. Requisitos Funcionais	20
3.6. Requisitos Não Funcionais	23
3.7. Casos de Uso	25
3.8. Diagramas de Máquinas de Estado	31
3.9. Revisão e Aprovação do Projeto	36
4. Arquitetura e Design do Software utilizando LLM	38
4.1. Utilização de LLMs na Fase de Arquitetura	38
4.2. Estrutura dos Softwares	38
4.3. Registos de Decisão Arquitetural	39
4.4. Diagrama de Componentes - Software de Loja	40
4.5. Diagramas de Classe - Software de Loja	41
4.6. Diagrama de Componentes - Software do Servidor Central	49
4.7. Diagramas de Classe - Software do Servidor Central	49
4.8. Diagramas de Sequência	54
4.9. Esboço Das Interfaces do Sistema	57
4.9.1. Estrutura Geral das Interfaces	57
4.9.2. Caracterização das Interfaces - Software de Loja	58
4.9.3. Caracterização das Interfaces - Software do Servidor Central	64
4.10. Conceção do Sistema de Dados - Software de Loja	66
4.10.1. Modelo Lógico	66
4.10.2. Dicionário de Dados e Decisões de Design	66
4.10.3. Normalização	70
4.11. Conceção do Sistema de Dados - Software de Servidor Central	70
4.11.1. Modelo Lógico	70
4.11.2. Dicionário de Dados e Decisões de Design	71

4.11.3. Normalização	73
4.12. Especificação da API e Utilização do Swagger	74
4.13. Revisão e Aprovação do Projeto	75
5. Implementação e Desenvolvimento Assistido por LLM	77
5.1. Utilização de LLMs na Fase de Implementação	77
5.2. Processo de Implementação e Divisão de Trabalho	77
5.3. Configuração do Ambiente de Trabalho	78
5.4. Desenvolvimento da Camada de Domínio - Software de Loja	79
5.5. Desenvolvimento da Camada de Repositório - Software de Loja	80
5.6. Desenvolvimento da Camada de Serviço - Software de Loja	81
5.7. Desenvolvimento da Camada de Apresentação - Software de Loja	83
5.8. Gateway de Sincronização e raiz de composição - Software de Loja	86
5.9. Desenvolvimento do Backend - Servidor Central	87
5.10. Desenvolvimento do Frontend - Servidor Central	88
5.11. Revisão de Código, Detecção de <i>Smells</i> e Refactoring	90
5.12. Desvios ao Design Original	91
5.13. Containerização do Software do Servidor Central	93
5.14. Apresentação do Protótipo Funcional	94
6. Verificação, Validação e Avaliação da Qualidade do Software Produzido	95
6.1. Utilização de LLMs na Fase de Verificação e Validação	95
6.2. Estratégia de Testes	95
6.3. Testes Unitários	96
6.4. Testes de Integração — Endpoints HTTP	96
6.5. Verificação dos Requisitos (SRS)	97
6.6. Métricas de Qualidade	99
7. Guia de Instalação, Operação e Manutenção	101
7.1. Pré-requisitos	101
7.2. Instalação e Arranque via Docker Compose	101
7.3. Instalação e Arranque Local (sem Docker)	102
7.3.1. Preparação da Base de Dados	102
7.3.2. Arranque do Servidor Central	102
7.3.3. Arranque do Software de Loja	102
7.4. Configuração	103
7.4.1. Software de Loja	103
7.4.2. Software do Servidor Central	103
7.5. Credenciais e Primeiro Acesso	104
7.6. Operação Corrente	104
7.6.1. Ciclo Diário	104
7.6.2. Monitorização	104
7.7. Manutenção	105
7.7.1. Cópia de Segurança da Base de Dados	105
7.7.2. Restauro da Base de Dados	105
7.7.3. Atualização do Software	105
7.8. Resolução de Problemas Comuns	105
8. Conclusões e Trabalho Futuro	107
8.1. Resultados Alcançados	107
8.2. Reflexão sobre a Utilização de LLMs	107
8.3. Trabalho Futuro	109
Referências	110
Lista de Siglas e Acrónimos	111
Anexos	112
Prompts Utilizados	157

Anexo 1	Logo da Universidade do Minho	113
Anexo 2	Entrevista realizada a Hélder Cerqueira	114
Anexo 3	Entrevista realizada a Cátia Das Unhas Pintadas	117
Anexo 4	Lista completa de requisitos levantados	119
Anexo 5	Especificação Completa dos Requisitos Funcionais	120
Anexo 6	Especificação Completa dos Requisitos Não Funcionais	127
Anexo 7	Especificação Completa dos Casos de Uso	130
Anexo 8	Registos de Decisão Arquitetural	135
Anexo 9	Diagrama de componentes — Software de Loja	139
Anexo 10	Diagrama de componentes — Servidor Central	140
Anexo 11	Diagrama de Sequência — Autênticação Loja	141
Anexo 12	Diagrama de Sequência — Registo de Venda	142
Anexo 13	Diagrama de Sequência — Fecho de dia e Sincronização	143
Anexo 14	Diagrama de Sequência — Receção de Pacotes no Servidor Central	144
Anexo 15	Diagrama de Sequência — Processamento da Devolução	145
Anexo 16	Diagrama de Sequência — Registo de Remessa.	146
Anexo 17	Diagrama de Sequência — Autênticação Servidor Central	147
Anexo 18	Exemplo de payloads e respostas da API	148
Anexo 19	Lista completa de desvios — Software de Loja	149
Anexo 20	Lista completa de desvios — Software de Servidor Central	152
Anexo 21	Listagem completa dos testes unitários — Servidor Central	153
Anexo 22	Listagem completa dos testes unitários — Servidor Central	154

Lista de Figuras

Figura 1	Ata da reunião de planeamento de execução do projeto.	8
Figura 2	Diagrama de Gantt elaborado. ¹	9
Figura 3	Ata da reunião de revisão e aprovação do projeto.	10
Figura 4	Ata da reunião para a definição do conjunto das técnicas a utilizar.	12
Figura 5	Excerto da entrevista realizada a Hélder Cerqueira.	13
Figura 6	Questionário realizado aos operadores de loja.	14
Figura 7	Exemplar de uma fatura física emitida na rede de lojas TrasmUM.	15
Figura 8	Exemplar de uma guia de remessa física rececionada na rede TrasmUM.	16
Figura 9	Excerto dos requisitos levantados.	17
Figura 10	Diagrama de Casos de Uso: Terminal de Loja — Operação Diária.	28
Figura 11	Diagrama de Casos de Uso: Gestão de Inventário e Fornecedores.	29
Figura 12	Diagrama de Casos de Uso: Gestão Administrativa.	29
Figura 13	Diagrama de Casos de Uso: Servidor Central — Supervisão Estratégica.	30
Figura 14	Diagrama de Casos de Uso: Visão Global do Sistema TrasmUM.	31
Figura 15	Diagrama de Máquina de Estado: Ciclo de Vida da Sessão do Utilizador no Terminal de uma Loja.	32
Figura 16	Diagrama de Máquina de Estado: Ciclo de Vida de uma Venda.	33
Figura 17	Diagrama de Máquina de Estado: Ciclo de Vida de um Lote de Produto.	34
Figura 18	Diagrama de Máquina de Estado: Ciclo de Vida de um Pedido de Remessa.	34
Figura 19	Diagrama de Máquina de Estado: Ciclo de Vida de um Pagamento a Fornecedor.	35
Figura 20	Diagrama de Máquina de Estado: Ciclo de Vida do Fecho de Dia.	36
Figura 21	Ata da reunião de revisão e aprovação do levantamento de requisitos.	37
Figura 22	Arquitetura Global do Sistema TrasmUM	38
Figura 23	Diagrama de classes — Domínio: Core, Segurança e Auditoria.	41
Figura 24	Diagrama de classes — Domínio: Catálogo, Inventário e Fornecedores.	42
Figura 25	Diagrama de classes — Domínio: Vendas e Faturas.	43
Figura 26	Diagrama de classes — Domínio: Tesouraria e Caixa.	44
Figura 27	Diagrama de classes — Repositório: Core e Segurança.	45
Figura 28	Diagrama de classes — Repositório: Catálogo, Inventário e Fornecedores.	45
Figura 29	Diagrama de classes — Repositório: Vendas e Devoluções.	45
Figura 30	Diagrama de classes — Repositório: Tesouraria e Caixa.	46
Figura 31	Diagrama de classes — Serviços: Core e Segurança.	46
Figura 32	Diagrama de classes — Serviços: Catálogo e Inventário.	47
Figura 33	Diagrama de classes — Serviços: Vendas e Devoluções.	47
Figura 34	Diagrama de classes — Serviços: Tesouraria e Caixa.	47
Figura 35	Diagrama de classes — Serviços: Gestão e BackOffice.	48
Figura 36	Diagrama de classes — Serviços: Fecho e Sincronização.	48
Figura 37	Diagrama de classes — Apresentação: Controllers Core.	48
Figura 38	Diagrama de classes — Apresentação: Controllers de Gestão.	49
Figura 39	Diagrama de classes — Domínio: Entidades Core.	50
Figura 40	Diagrama de classes — Domínio: Entidades Transacionais.	51
Figura 41	Diagrama de classes — Domínio: Value Objects de Análise.	51
Figura 42	Diagrama de classes — Domínio: DTOs de Transferência.	52
Figura 43	Diagrama de classes — Repositório: Core e Autenticação.	52

¹Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 8](#).

Figura 44	Diagrama de classes — Repositório: Vendas, Devoluções e Sessões de Caixa.	52
Figura 45	Diagrama de classes — Repositório: Remessas, Pagamentos e Auditoria.	52
Figura 46	Diagrama de classes — Serviços: Autenticação e Receção.	53
Figura 47	Diagrama de classes — Serviços: Análise.	53
Figura 48	Diagrama de classes — Apresentação: Handlers HTTP.	53
Figura 49	Esboço da “Página de Início de Sessão” (ECR-01)“	58
Figura 50	Esboço da “Página de Bloqueio de Inicio de Sessão” (ECR-01B)	59
Figura 51	Esboço da “Página de Venda da Caixa” (ECR-06)	59
Figura 52	Esboço da “Página de Devoluções da Caixa”	60
Figura 53	Esboço da “Página de Consulta da Caixa”	60
Figura 54	Esboço da “Página principal do Gerente”	61
Figura 55	Esboço da “Página de gestão de Inventário do Gerente”	61
Figura 56	Esboço da “Página de gestão dos Fornecedores do Gerente”	62
Figura 57	Esboço da “Página de configurações do Gerente”	63
Figura 58	Esboço da “Página de Fecho do dia do Gerente”	63
Figura 59	Esboço da “Página de Sincronização de dados da Cátia”	64
Figura 60	Esboço da “Página de Estatísticas da Cátia”	65
Figura 61	Esboço da “Página de Gestão de Remessas da Cátia”	65
Figura 62	Modelo lógico da base de dados do Software de Loja.	66
Figura 63	Modelo lógico da base de dados do Software de Servidor Central.	71
Figura 64	Ata da reunião de revisão e aprovação da arquitetura e design do software.	76
Figura 65	Divisão de tarefas de implementação por par da equipa.	78
Figura 66	Botões de controlo de escala da interface.	83
Figura 67	Menu lateral adaptativo: à esquerda, sessão de Funcionário com acesso restrito às operações de frente de loja; à direita, sessão de CEO com a totalidade dos acessos de gestão.	84
Figura 68	Ecrã de autenticação após cinco tentativas falhadas, com cronómetro regressivo e bloqueio dos campos de credenciais.	84
Figura 69	Vista de Venda com TabPane Venda/Devolução: integração dos dois fluxos funcionalmente relacionados num único contexto de operação.	85
Figura 70	Vista de Catálogo com painel de alertas ativos: ruturas de stock e lotes em alerta de validade, ordenados por criticidade.	85
Figura 71	Vista de Aprovisionamento com filtragem dinâmica de produtos por fornecedor, refletindo a relação FornecedorProduto introduzida na fase de implementação.	86
Figura 72	Dashboard global do Servidor Central: métricas agregadas da rede, decomposição por loja, totais de despesas e lucro líquido — resultado final da integração entre o <i>frontend</i> Vue e o <i>backend</i> Javalin.	88
Figura 73	Monitor de Receção de Dados: estado consolidado da rede no momento corrente, com indicação visual por loja sobre o pacote diário de fecho — Recebido, Em Falta ou Erro.	89
Figura 74	Dashboard por loja: decomposição das métricas analíticas para um estabelecimento individual.	90
Figura 75	Ata da reunião da apresentação do protótipo Funcional.	94

Lista de Tabelas

Tabela 1	User Stories — Funcionário da Caixa	16
Tabela 2	User Stories — Manager	16
Tabela 3	Refinamento do Requisito RF03 — Registo de Vendas e Identificação de Artigos	18
Tabela 4	Refinamento do Requisito RF05 — Processamento de Devoluções	18
Tabela 5	Refinamento do Requisito RF06 — Registo de Remessa, Gestão de Validades e Pagamentos	18
Tabela 6	Refinamento do Requisito RF22 — Aplicação de Descontos por Aproximação do Prazo de Validade	19
Tabela 7	Requisitos removidos e respetivas justificações	19
Tabela 8	Especificação do Requisito Funcional 1	20
Tabela 9	Especificação do Requisito Funcional 2	21
Tabela 10	Especificação do Requisito Funcional 3	21
Tabela 11	Especificação do Requisito Funcional 4	22
Tabela 12	Especificação do Requisito Funcional 5	22
Tabela 13	Especificação do Requisito Funcional 6	23
Tabela 14	Especificação do Requisito Não Funcional 1	24
Tabela 15	Especificação do Requisito Não Funcional 2	24
Tabela 16	Especificação do Requisito Não Funcional 3	24
Tabela 17	Especificação do Requisito Não Funcional 4	25
Tabela 18	Caso de uso — Registar Venda	25
Tabela 19	Caso de uso — Registar Devolução	26
Tabela 20	Caso de uso — Consultar Dashboards e Relatórios Analíticos	27
Tabela 21	Dicionário de dados — Utilizador	67
Tabela 22	Dicionário de dados — Lote	67
Tabela 23	Dicionário de dados — Venda	68
Tabela 24	Dicionário de dados — Pagamento	68
Tabela 25	Dicionário de dados — DetalheNumerario	69
Tabela 26	Dicionário de dados — LinhaRemessa	69
Tabela 27	Dicionário de dados — CEO	71
Tabela 28	Dicionário de dados — Loja	72
Tabela 29	Dicionário de dados — FechoDiaCentral	72
Tabela 30	Dicionário de dados — PagamentoCentral	72
Tabela 31	Dicionário de dados — SessaoCaixaCentral	73
Tabela 32	Endpoints — Autenticação	74
Tabela 33	Endpoints — Gestão de Contas CEO	74
Tabela 34	Endpoints — Sincronização	74
Tabela 35	Endpoints — Dashboard e Métricas	75
Tabela 36	Endpoints — Relatórios Analíticos	75
Tabela 37	Endpoints — Remessas da Rede	75
Tabela 38	Endpoints — Estado de Consolidação	75
Tabela 39	Endpoints HTTP expostos pelo Servidor Central	87
Tabela 40	Desvios ao design original — Software de Loja (selecção)	91
Tabela 41	Desvios ao design original — Software de Servidor Central (selecção)	92
Tabela 42	Verificação dos Requisitos Funcionais	97
Tabela 43	Verificação dos Requisitos Não Funcionais	98

Tabela 44	Cobertura de código por classe testada — Software de Loja	99
Tabela 45	Cobertura de código por classe testada — Servidor Central	99
Tabela 46	Pré-requisitos por modo de implantação	101
Tabela 47	Parâmetros de configuração do Software de Loja	103
Tabela 48	Parâmetros de configuração do Servidor Central	103
Tabela 49	Problemas frequentes e respectivas soluções	105
Tabela 50	Especificação do Requisito Funcional 1	120
Tabela 51	Especificação do Requisito Funcional 2	120
Tabela 52	Especificação do Requisito Funcional 3	121
Tabela 53	Especificação do Requisito Funcional 4	121
Tabela 54	Especificação do Requisito Funcional 5	121
Tabela 55	Especificação do Requisito Funcional 6	122
Tabela 56	Especificação do Requisito Funcional 7	122
Tabela 57	Especificação do Requisito Funcional 8	122
Tabela 58	Especificação do Requisito Funcional 9	123
Tabela 59	Especificação do Requisito Funcional 10	123
Tabela 60	Especificação do Requisito Funcional 11	123
Tabela 61	Especificação do Requisito Funcional 12	123
Tabela 62	Especificação do Requisito Funcional 13	124
Tabela 63	Especificação do Requisito Funcional 14	124
Tabela 64	Especificação do Requisito Funcional 15	124
Tabela 65	Especificação do Requisito Funcional 16	125
Tabela 66	Especificação do Requisito Funcional 17	125
Tabela 67	Especificação do Requisito Funcional 18	125
Tabela 68	Especificação do Requisito Funcional 19	125
Tabela 69	Especificação do Requisito Funcional 20	126
Tabela 70	Especificação do Requisito Funcional 21	126
Tabela 71	Especificação do Requisito Funcional 22	126
Tabela 72	Especificação do Requisito Funcional 23	127
Tabela 73	Especificação do Requisito Não Funcional 1	127
Tabela 74	Especificação do Requisito Não Funcional 2	127
Tabela 75	Especificação do Requisito Não Funcional 3	127
Tabela 76	Especificação do Requisito Não Funcional 4	127
Tabela 77	Especificação do Requisito Não Funcional 5	127
Tabela 78	Especificação do Requisito Não Funcional 6	128
Tabela 79	Especificação do Requisito Não Funcional 7	128
Tabela 80	Especificação do Requisito Não Funcional 8	128
Tabela 81	Especificação do Requisito Não Funcional 9	128
Tabela 82	Especificação do Requisito Não Funcional 10	128
Tabela 83	Especificação do Requisito Não Funcional 11	128
Tabela 84	Especificação do Requisito Não Funcional 12	129
Tabela 85	Especificação do Requisito Não Funcional 13	129
Tabela 86	Especificação do Requisito Não Funcional 14	129
Tabela 87	Especificação do Requisito Não Funcional 15	129
Tabela 88	Especificação do Requisito Não Funcional 16	129
Tabela 89	Especificação do Requisito Não Funcional 17	129
Tabela 90	Especificação do Requisito Não Funcional 18	129
Tabela 91	Caso de uso — Iniciar Sessão no Terminal da Loja	130
Tabela 92	Caso de uso — Consultar e Pesquisar Produtos	130
Tabela 93	Caso de uso — Monitorizar Caixa e Registrar Sangria	130
Tabela 94	Caso de uso — Criar Pedidos de Remessa	131
Tabela 95	Caso de uso — Registrar Receção de Remessa	131
Tabela 96	Caso de uso — Gerir Documentos de Fornecedores	132
Tabela 97	Caso de uso — Gerir Pagamentos a Fornecedores	132
Tabela 98	Caso de uso — Aplicar Desconto por Aproximação do Prazo de Validade	132

Tabela 99	Caso de uso — Gerir Catálogo de Produtos	133
Tabela 100	Caso de uso — Realizar Fecho de Dia	133
Tabela 101	Caso de uso — Gerir Utilizadores do Sistema	133
Tabela 102	Caso de uso — Consultar Remessas da Rede	134
Tabela 103	Caso de uso — Consultar Estado de Consolidação da Rede	134
Tabela 104	Caso de uso — Iniciar Sessão no Servidor Central	134
Tabela 105	ADR-01 — Arquitetura de Distribuição Global	135
Tabela 106	ADR-02 — Dois Softwares Distintos e Independentes	135
Tabela 107	ADR-03 — Base de Dados Partilhada por Loja	136
Tabela 108	ADR-04 — Delta Sync com Três Estados de Sincronização	136
Tabela 109	ADR-05 — Idempotência por Chave Composta nas Tabelas Centrais	136
Tabela 110	ADR-06 — Bootstrapping por Conta de Instalação Temporária	137
Tabela 111	ADR-07 — Logs de Auditoria Incluídos no Pacote de Fecho	137
Tabela 112	ADR-08 — Comunicação Segura com HTTPS	137
Tabela 113	ADR-09 — Layered Architecture com MVC como Padrão de Organização Interna	138
Tabela 114	ADR-10 — Stack Tecnológica: Java, JavaFX, Javalin e MySQL	138
Tabela 115	Exemplos de pedido e resposta — Autenticação	148
Tabela 116	Exemplos de pedido e resposta — Sincronização	148
Tabela 117	Exemplos de resposta — Dashboard e Métricas	148
Tabela 118	Exemplos de pedido e resposta — Relatórios Analíticos	148
Tabela 119	Exemplos de pedido e resposta — Remessas da Rede	149
Tabela 120	Exemplos de resposta — Estado de Consolidação	149
Tabela 121	Desvios Design - Software de Loja	149
Tabela 122	Desvios Design - Software de Servidor Central	152
Tabela 123	Testes unitários — Software de Loja	153
Tabela 124	Testes unitários — Software de Servidor Central	154
Tabela 125	Especificação do Requisito Funcional	166
Tabela 126	Especificação do Requisito Não Funcional	167
Tabela 127	Caso de uso Registrar Venda	167

1. Introdução

1.1. Âmbito do Projeto

A digitalização de processos de negócio constitui, para muitas pequenas empresas de retalho, uma condição de sobrevivência face ao crescimento da sua operação. A rede TrasmUM — um conjunto de três lojas de conveniência no distrito de Bragança, geridas a partir de uma sede central — ilustra esse desafio: herdada de uma gestão integralmente analógica, assente em registos em papel, controlo manual de inventário e recolha telefónica diária de dados de vendas, a rede confrontava-se com discrepâncias sistemáticas de stock, morosidade no apuramento de caixa e extravio frequente de documentação de fornecedores. Estas ineficiências, detalhadas no Capítulo 2, comprometiam tanto a rentabilidade como a capacidade de expansão do negócio.

O trabalho documentado neste relatório responde a esse problema com o desenvolvimento do sistema TrasmUM, uma solução de gestão integrada desenvolvida no âmbito da unidade curricular de Laboratórios de Informática IV da Licenciatura em Engenharia Informática da Universidade do Minho. A solução adota uma arquitetura federada e resiliente, que dota cada loja de autonomia operacional plena durante o horário de funcionamento e centraliza no Servidor Central apenas a informação estritamente necessária à tomada de decisão estratégica.

1.2. Metodologia de Utilização de LLMs

Antes de proceder à descrição do trabalho realizado, importa clarificar a metodologia adotada para a utilização de modelos de linguagem de grande dimensão ao longo deste projeto, por forma a garantir a transparência que o uso destas ferramentas exige num contexto académico.

Os modelos utilizados foram o Claude (Anthropic, 2025) (Anthropic) e o Gemini (Google, 2025) (Google). A escolha entre os dois modelos foi determinada pelo tipo de tarefa em causa e pela disponibilidade de cada ferramenta em cada momento, sendo que ambos foram utilizados de forma complementar ao longo das diferentes fases do trabalho.

A interação com os modelos seguiu sempre o *template* de registo de *prompts* apresentado no final desta secção, que documenta o *prompt* original, o *output* gerado, as fontes fornecidas como contexto e uma reflexão final sobre o processo. Este registo sistemático serve um duplo propósito: por um lado, garante a rastreabilidade de todas as interações com os modelos; por outro, torna explícito o grau de intervenção humana em cada *output* incluído no relatório.

Para facilitar a leitura e a auditoria deste documento, todos os *prompts* reais utilizados encontram-se compilados na secção de Anexos. Sempre que um conteúdo gerado com apoio de IA é apresentado ao longo dos próximos capítulos, é feita uma referência direta para a respetiva entrada no Anexo. Adicionalmente, cada entrada nesse anexo possui um link interativo que permite ao leitor voltar imediatamente à secção do relatório que estava a ler.

Na prática, os *outputs* dos modelos enquadraram-se em quatro categorias distintas:

- Em alguns casos, o *output* foi aceite sem alterações significativas, quando a resposta era factualmente correta, coerente com o contexto do projeto e adequada ao registo académico pretendido.

- Com maior frequência, o *output* foi usado como base e refinado pela equipa, sendo adaptado em termos de terminologia, estrutura ou nível de detalhe.
- Noutros casos, o *output* foi rejeitado e o *prompt* reformulado, tipicamente quando a resposta revelava falta de contexto suficiente ou quando divergia das decisões tomadas pela equipa.
- Por fim, em algumas situações, o *output* foi apenas parcialmente aproveitado, sendo o restante redigido manualmente pela equipa.

Em todos os casos, a inclusão de qualquer conteúdo gerado por LLM no relatório foi precedida da revisão por pelo menos um membro da equipa, que assumiu a responsabilidade pela correção e adequação do texto ao contexto do projeto. Esta prática assegura que os modelos funcionaram como ferramentas de apoio à produção de conteúdo e não como substitutos do julgamento técnico e académico da equipa.

Os critérios que guiaram a aceitação ou rejeição de *outputs* foram os seguintes: coerência com os requisitos e decisões já documentadas no projeto; correção técnica verificável pela equipa; adequação ao registo académico formal do relatório; e ausência de contradições com o domínio específico do sistema. *Outputs* que não cumprissem estes critérios foram sempre rejeitados ou substancialmente reescritos, independentemente da sua qualidade aparente.

O *template* padrão definido para a formulação e registo de *prompts* assume a seguinte estrutura:

Descrição	Breve descrição do pedido efetuado ao LLM, explicitando o intuito do <i>prompt</i> escrito.
LLM Utilizado	Qual foi o LLM utilizado.
Prompt	Prompt originalmente utilizado.
Fontes Fornecidas	Fontes previamente fornecidas para a geração de contexto (bibliografia, anexos, enunciado do trabalho prático, etc.).
Reflexão Final	Reflexão do processo.
↑ Voltar à secção	

Prompt 1: Template para a formulação e registo de prompts

1.3. Estrutura do Relatório

Para além do presente capítulo introdutório, o relatório encontra-se organizado em mais cinco capítulos, que acompanham a sequência natural das etapas de desenvolvimento do sistema TrasmUM.

O Capítulo 2, *Definição do Domínio do Sistema*, apresenta o caso de estudo que enquadra o projeto, expondo a contextualização do negócio, a motivação e os objetivos da solução, a identificação dos recursos necessários, a análise de contexto e restrições, o estudo de viabilidade, as medidas de sucesso definidas e o plano de execução do projeto.

O Capítulo 3, *Levantamento e Análise de Requisitos*, detalha o processo de elicitação de requisitos e os respetivos métodos aplicados, culminando na especificação formal dos requisitos funcionais e não funcionais, dos casos de uso e dos diagramas de máquinas de estado que modelam o comportamento do sistema.

O Capítulo 4, *Arquitetura e Design do Software*, descreve as decisões estruturantes da solução, formalizadas através de registos de decisão arquitetural (ADR), e apresenta os diagramas de componentes, de classes e de sequência de ambos os softwares, o esboço das interfaces, a conceção do sistema de dados e a especificação da API do Servidor Central.

O Capítulo 5, *Implementação e Desenvolvimento Assistido por LLM*, documenta o processo de codificação do sistema, abordando a configuração do ambiente de trabalho, o desenvolvimento das diferentes camadas de cada software, a revisão de código e os desvios introduzidos relativamente ao design original.

O Capítulo 6, *Verificação, Validação e Avaliação da Qualidade do Software Produzido*, apresenta a estratégia de testes adotada, os testes unitários produzidos, a verificação do cumprimento dos requisitos e a avaliação da qualidade do software à luz da norma ISO/IEC 25010 (International Organization for Standardization, 2011).

O relatório encerra com um guia de instalação, operação e manutenção do sistema, seguido das conclusões e da identificação de possíveis linhas de trabalho futuro. Em anexo, encontram-se compilados os artefactos de suporte ao desenvolvimento, incluindo a especificação completa dos requisitos e casos de uso, os registos de decisão arquitetural, os diagramas produzidos e a totalidade dos *prompts* utilizados nas interações com os modelos de linguagem.

2. Definição do Domínio do Sistema

2.1. Utilização de LLMs na Definição do Domínio do Sistema

Nesta secção, a utilização de LLMs centrou-se na geração assistida de conteúdo narrativo para a definição do domínio do sistema. Dado que o domínio do negócio — uma rede de lojas de conveniência — foi definido e contextualizado pela própria equipa através de uma narrativa fictícia criada para o efeito, o modelo foi utilizado para transformar essa narrativa em linguagem técnica estruturada, cobrindo a contextualização, a motivação e objetivos, a identificação de *stakeholders*, a análise de contexto e restrições, a análise de viabilidade, as medidas de sucesso e o plano de execução do projeto.

A abordagem adotada consistiu em fornecer ao modelo o enunciado do trabalho e a narrativa do caso de estudo como contexto, e solicitar a geração de cada subsecção de forma progressiva e iterativa. Em cada iteração, o output foi revisto pela equipa e, quando necessário, o *prompt* foi reformulado para incorporar correções ou ajustes ao conteúdo gerado. O principal critério de validação aplicado nesta secção foi a coerência com a narrativa original do caso de estudo — qualquer detalhe introduzido pelo modelo que não estivesse explícita ou implicitamente presente na narrativa foi removido ou assinalado como suposição da equipa.

2.2. Contextualização

No dia 13 de fevereiro de 2026, Hélder Cerqueira, o próspero e conhecido proprietário da empresa *Prestige Limousines*, decidiu elevar a fasquia das surpresas românticas ao adquirir uma cadeia de pequenas lojas de conveniência, denominada como TrasmUM. A rede, composta por exatamente três estabelecimentos comerciais espalhados pelo distrito de Bragança, foi o presente inusitado escolhido para oferecer à sua esposa, carinhosamente conhecida por todos como Cátia das unhas pintadas, durante um jantar romântico no restaurante O Santril.

O negócio concretizou-se de forma amigável com as anteriores proprietárias, a Dona Zulmira e a Dona Acácia, que decidiram passar o testemunho devido à sua idade já avançada e à natural dificuldade em acompanhar o ritmo acelerado de crescimento comercial que a cadeia vinha a registar nos últimos tempos.

Assim que Hélder oficializou a grande surpresa à sua esposa e ambos começaram a explorar a sua nova aquisição, depararam-se com uma realidade peculiar: cada loja era gerida de forma completamente independente, sem qualquer mecanismo de comunicação ou partilha de informação entre estabelecimentos. Perante este cenário, Hélder decidiu de imediato contactar um grupo de quatro jovens estudantes de engenharia informática para assumirem o desenvolvimento do novo sistema informático da rede. Esta escolha de equipa não foi feita ao acaso, sendo antes motivada pela excelente experiência prévia do empresário com estes mesmos jovens, que no passado lhe tinham entregue um trabalho irrepreensível quando encomendou o exigente software de gestão para a sua frota de limousines.²

²Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 1](#).

2.3. Motivação e Objetivos

Quando Hélder e Cátia das unhas pintadas iniciaram a transição do negócio e analisaram detalhadamente os processos operacionais deixados pela Dona Zulmira e pela Dona Acácia, constataram um grau de ineficiência severo. O controlo de inventário das três lojas era suportado exclusivamente por registos físicos em cadernos e folhas soltas. Esta dependência de uma metodologia inteiramente manual resultava frequentemente em graves discrepâncias nos níveis de stock. Observava-se uma ocorrência sistemática de ruturas de produtos essenciais nas prateleiras e, em simultâneo, a acumulação excessiva de artigos perecíveis, cuja validade expirava antes da sua comercialização. Este desequilíbrio contínuo na gestão de existências traduzia-se em perdas financeiras consideráveis e na diminuição da rentabilidade global da operação.

Uma segunda ineficiência crítica verificava-se no procedimento diário de fecho de caixa e no apuramento de resultados. Dada a ausência de qualquer infraestrutura de comunicação centralizada entre os estabelecimentos espalhados pelo distrito de Bragança, as antigas proprietárias despendiam várias horas ao telefone com cada uma das três lojas para coligir os dados de vendas diárias. Este processo de consolidação financeira era inerentemente moroso e altamente suscetível a erros de cálculo humano. A transcrição e soma manual dos valores ditados à distância originavam inconsistências frequentes no apuramento dos lucros da jornada, exigindo sucessivas recontagens e prolongando o fecho da operação muito para lá do horário de expediente.

Por fim, a gestão do aprovisionamento e a relação com os fornecedores apresentavam fragilidades estruturais profundas, agravadas por métodos de arquivo manifestamente inadequados. Os documentos essenciais de suporte, tais como faturas e guias de remessa, eram rotineiramente arquivados em simples caixas de sapatos, desprovidos de qualquer sistema de categorização lógica ou cronológica. Esta desorganização documental propiciava o extravio frequente de faturamentos, resultando na impossibilidade de liquidar pagamentos a tempo e horas. O incumprimento reiterado dos prazos de pagamento gerava atritos substanciais com os distribuidores parceiros, dificultando a capacidade de cada loja gerir de forma autónoma o seu próprio aprovisionamento.

A dispersão geográfica dos três estabelecimentos pelo distrito de Bragança, aliada à necessidade de manter cada loja operacional independentemente de falhas de comunicação externas, tornou evidente que a solução mais robusta passaria por dotar cada estabelecimento de autonomia operacional plena, centralizando apenas o que é estritamente necessário para a supervisão estratégica da rede.³

Para concretizar esta ambiciosa transição, a confiança depositada na equipa de engenharia informática assume um papel decisivo. Hélder Cerqueira reconhece nestes jovens a competência técnica e o brio profissional demonstrados anteriormente na criação do bem-sucedido software para a sua frota de limousines. A motivação atual passa por replicar esse grau de excelência na rede de lojas TrasmUM. A expectativa da administração é que esta parceria tecnológica dote a cadeia de lojas de uma infraestrutura robusta, capaz de agregar a informação relevante e transformar os dados diários numa verdadeira ferramenta estratégica para o futuro.

De forma a mitigar as falhas identificadas e operacionalizar a visão estratégica da administração, o sistema deverá atingir um conjunto de objetivos estruturantes. Em primeiro lugar, deverá permitir o registo estruturado das vendas efetuadas em cada loja, incluindo devoluções e anulações, garantindo a atualização automática dos níveis de stock e a correta integração da informação nos apuramentos financeiros. Complementarmente, deverá estabelecer um mecanismo de receção de dados no Servidor Central que consolide, ao final de cada dia, os pacotes de fecho enviados autonomamente por cada loja, para efeitos de análise e auditoria. No plano do inventário, o sistema deverá monitorizar automaticamente os níveis de stock, emitindo alertas que previnam ruturas de produtos essenciais e minimizem o desperdício de artigos perecíveis. A digitalização do fluxo documental — nomeadamente o arquivo e categorização de faturas e guias de remessa — eliminará a dependência de suportes físicos e garantirá a rastreabilidade dos pagamentos a fornecedores. O processo de fecho de caixa diário será igualmente automatizado, substituindo a recolha telefónica de dados por processos eletrónicos que assegurem a integridade dos valores apurados. Para suporte à decisão estratégica, serão disponibilizados painéis de controlo com

³ Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 2](#).

métricas de desempenho atualizadas, permitindo à Diretora-Geral avaliar a rentabilidade de cada loja com base nos dados consolidados. Por fim, o sistema deverá garantir que cada estabelecimento opera de forma completamente independente — com gestão local de produtos, preços, funcionários e stock — atribuindo a cada loja um gestor responsável pela administração do seu próprio catálogo e equipa.⁴

2.4. Recursos Necessários

Para a realização do projeto TrasmUM, serão necessários diversos recursos humanos, que podem ser divididos nas categorias de pessoal externo e pessoal interno. O pessoal externo é constituído pela equipa de desenvolvimento de software, composta pelos membros Luís Ferreira, José Fernandes, Gabriel Dantas e Simão Oliveira, estudantes de Engenharia Informática na Universidade do Minho. O pessoal interno é constituído pelos representantes da organização TrasmUM envolvidos no projeto: Hélder Cerqueira, proprietário e principal promotor financeiro da modernização tecnológica, e Cátia das Unhas Pintadas, Diretora-Geral da rede e principal *stakeholder* estratégica, responsável pela validação dos requisitos e aprovação das fases do projeto. Os managers e funcionários das três lojas constituem igualmente *stakeholders* internos, enquanto utilizadores finais do Software de Loja, cujas necessidades operacionais orientam diretamente o levantamento de requisitos. Os clientes da rede, embora não interajam diretamente com o sistema, beneficiam indiretamente da modernização através de um atendimento mais rápido e fiável.

Também serão necessários recursos materiais, divididos em hardware e software. Ao nível do hardware, serão necessários quatro computadores pessoais, um por cada elemento da equipa de desenvolvimento, para que estes possam realizar o seu trabalho. Ao nível do software, serão necessárias ferramentas de suporte ao desenvolvimento, documentação, modelação e gestão do projeto. Para controlo de versões e colaboração entre os membros da equipa, será utilizado o Git com alojamento no GitHub. Para comunicação e coordenação interna, a equipa recorrerá ao Discord e ao WhatsApp. Para modelação UML, serão utilizados o PlantUML o Visual Paradigm e Mermaid. Para a redação do presente relatório, será utilizado o Typst (Madje e Haug, 2024).

No que respeita à pilha tecnológica de implementação, foi decidido que o sistema será desenvolvido em Java, com MySQL como sistema de gestão de base de dados em todas as instâncias do sistema — tanto nas lojas como no servidor central. As decisões relativas a *frameworks* de interface gráfica, bibliotecas de comunicação e demais componentes tecnológicos serão tomadas e documentadas na fase de Arquitetura e Design do Software. Para escrita do código, cada elemento da equipa utilizará o Visual Studio Code como ambiente de desenvolvimento.

Para apoio ao desenvolvimento, a equipa recorrerá a modelos de linguagem de grande dimensão, cuja metodologia de utilização é documentada nas secções correspondentes de cada fase do projeto.⁵

2.5. Análise de Contexto e Restrições

O Sistema de Gestão Integrada para a rede TrasmUM irá operar num modelo federado e descentralizado, abrangendo dois ambientes operacionais distintos. O Terminal de Loja é instalado nos estabelecimentos da rede e é responsável pela totalidade da operação diária de cada estabelecimento, funcionando de forma completamente independente durante o horário de funcionamento. Cada estabelecimento dispõe de uma única instância de base de dados local, partilhada entre todos os seus terminais via rede local, constituindo a fonte de verdade da loja durante o horário de funcionamento. O Servidor Central opera exclusivamente na máquina central da rede, sendo responsável pela receção passiva dos pacotes de fecho de dia enviados por cada loja, pela persistência na base de dados central e pela disponibilização de *dashboards* e relatórios analíticos para consulta pela CEO. Este sistema foca-se primordialmente na otimização dos fluxos vitais de informação, abrangendo o registo de vendas, o controlo de stocks e a gestão de fornecedores. O propósito central desta consolidação informática é dotar a administração de dados estruturados e atualizados,

⁴Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 3](#).

⁵Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 4](#).

apoando ativamente os processos de tomada de decisão estratégica e eliminando a dependência do caos administrativo anterior.

O propósito central desta arquitetura federada é dotar cada loja de autonomia operacional plena, e simultaneamente fornecer à Cátia os dados agregados necessários para a tomada de decisão estratégica. Por conseguinte, foram estabelecidas fronteiras rigorosas para o projeto, delimitando o seu âmbito de forma a garantir a sua exequibilidade técnica. O sistema limitar-se-á a registar logicamente o método de pagamento escolhido para cada transação, não existindo qualquer comunicação com *gateways* bancárias, redes de pagamento externas ou terminais físicos Multibanco. No que respeita à integração com hardware, o sistema não comunicará via APIs nativas com periféricos POS, embora a interface de caixa esteja preparada para receber input de leitores de código de barras standard, uma vez que estes operam como simples emuladores de teclado. O sistema não enviará igualmente notificações automáticas a fornecedores, limitando-se ao registo interno dos documentos, nem contemplará a importação dos registos analógicos anteriores, arrancando sem qualquer mecanismo de digitalização retroativa dos dados das anteriores proprietárias. Por fim, os dados operacionais da rede fluem exclusivamente das lojas para o Servidor Central, nunca no sentido inverso — a única comunicação no sentido contrário consiste em sinais técnicos de confirmação de receção que não transportam dados operacionais.⁶

2.6. Análise de Viabilidade

A viabilidade do desenvolvimento e implementação do novo Sistema de Gestão Integrada apresenta-se manifestamente positiva quando avaliada sob a perspetiva dos diversos intervenientes da rede de lojas espalhadas pelo distrito de Bragança. Para a administração central, encabeçada por Hélder Cerqueira e pela Diretora-Geral, Cátia das unhas pintadas, a transição de um modelo integralmente dependente de registos manuais em papel para uma infraestrutura tecnológica unificada justifica em absoluto o investimento. Esta modernização digital afigura-se como o único caminho viável para assegurar o crescimento comercial e mitigar os graves problemas operacionais deixados pela Dona Zulmira e pela Dona Acácia.

Do ponto de vista operacional, os funcionários das três lojas de conveniência beneficiarão de um fluxo de trabalho otimizado. Em paralelo, os fornecedores e distribuidores, peças vitais da operação, encontram nesta reestruturação a garantia de estabilidade da cadeia de abastecimento. Com a digitalização do fluxo documental, concluiu-se que o sistema informático seria capaz de produzir melhorias substanciais em múltiplas frentes operacionais. Ao nível do registo de vendas, prevê-se uma redução superior a 90% nos erros de registo de vendas e devoluções, assegurando a atualização automática do stock e a correta integração da informação nos relatórios financeiros. No que respeita ao fecho de caixa, estima-se uma redução de 85% no tempo despendido no apuramento financeiro diário, eliminando por completo a morosa recolha telefónica de dados. A gestão de existências beneficiará igualmente de uma diminuição estimada de 75% na ocorrência de ruturas de produtos essenciais e no desperdício de artigos perecíveis, fruto da monitorização automática dos níveis de stock. A pontualidade na liquidação de faturas a fornecedores deverá melhorar 90%, impulsionada pelo novo arquivo digital e categorização de guias de remessa. Por fim, antecipa-se uma redução de 95% das atuais discrepâncias de stock físico, substituindo as folhas soltas por uma base de dados local autónoma em cada estabelecimento.⁷

2.7. Medidas de Sucesso

Para assegurar que o desenvolvimento de software traduz os ganhos de viabilidade em resultados tangíveis, Hélder Cerqueira e Cátia das unhas pintadas definiram critérios de avaliação rigorosos para a equipa dos quatro jovens estudantes de engenharia informática. Estas métricas operam como a ponte entre os objetivos estratégicos da empresa e os alvos técnicos do sistema, fornecendo um guião objetivo e mensurável para orientar a implementação e validar o sucesso final da arquitetura.

⁶Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 5](#).

⁷Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 6](#).

Os indicadores definidos para atestar o sucesso da digitalização desta cadeia retalhista estabelecem as seguintes metas de desempenho. O sistema deverá garantir que 90% das vendas e devoluções são registadas sem erros ou omissões, assegurando a atualização automática do stock e a correta integração nos relatórios financeiros. No que respeita à receção do fecho de dia, o Servidor Central deve ser capaz de receber e persistir o pacote de dados enviado por cada loja num tempo máximo de cinco minutos por loja, garantindo que os dados agregados estão disponíveis para análise pela Diretora-Geral na manhã seguinte. O módulo de controlo de inventário deverá garantir uma taxa de correspondência de 99,5% entre o registo digital e a contagem física nas prateleiras e no armazém. Ao nível da adoção, o sistema deverá registar uma taxa de utilização ativa de 100% por parte dos operadores no prazo máximo de três semanas após o *deployment*. Por fim, cada loja deve ser capaz de completar a totalidade das operações diárias — vendas, devoluções e gestão de remessas — de forma completamente autónoma, sem dependência de comunicação com o Servidor Central durante o horário de funcionamento.⁸

2.8. Plano de Execução do Projeto

O planeamento operacional do sistema de gestão integrada para a rede TrasmUM foi consolidado através de uma reunião de alinhamento estratégico entre a equipa de desenvolvimento e a administração, representada por Helder Cerqueira e Cátia. Durante esta sessão, foram detalhadas as fases do ciclo de vida do software a implementar num horizonte temporal estimado de cinco meses, garantindo a concordância entre as capacidades técnicas dos programadores e as necessidades do negócio. A calendarização detalhada destas etapas, incluindo os marcos críticos e a interdependência de tarefas, encontra-se formalizada no Diagrama de Gantt ilustrado na figura seguinte



Figura 1: Ata da reunião de planeamento de execução do projeto.

⁸Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 6](#).

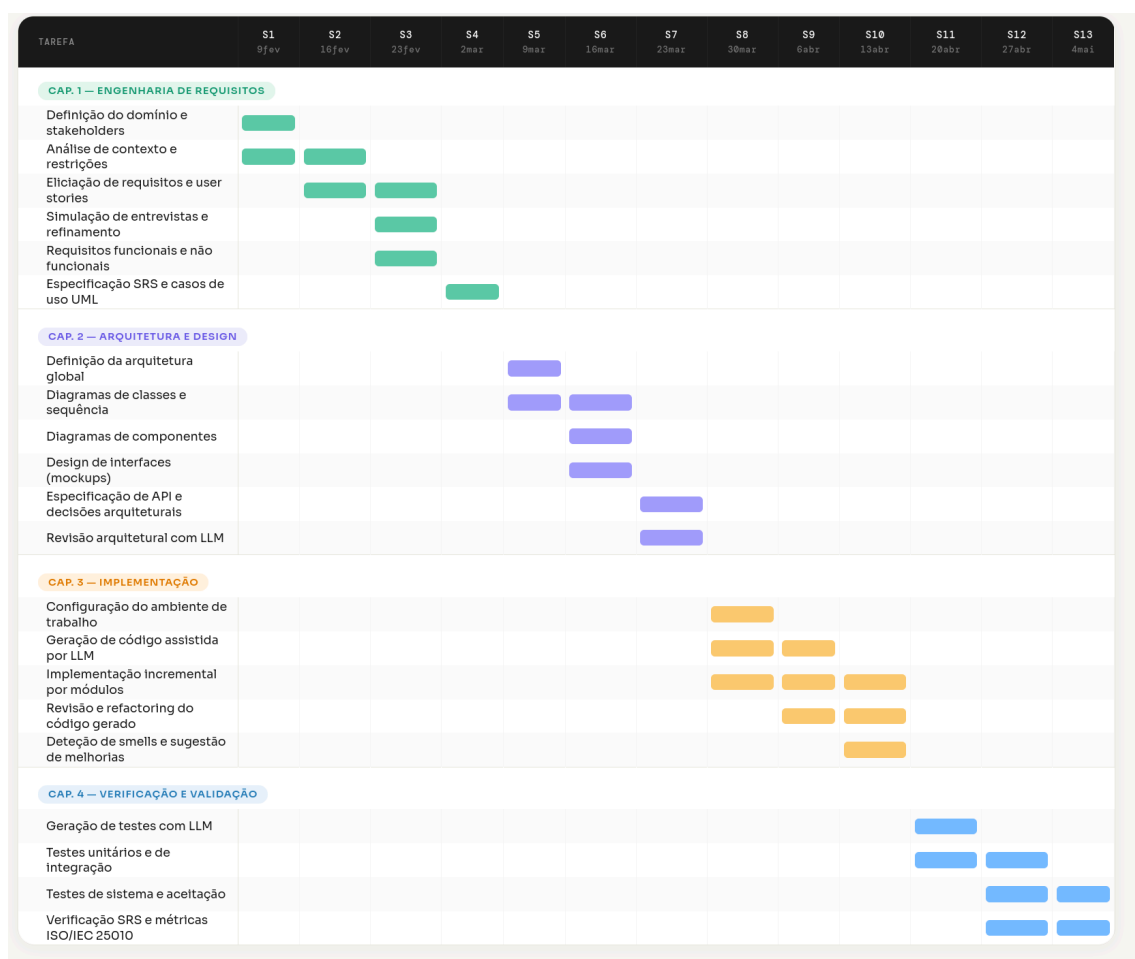


Figura 2: Diagrama de Gantt elaborado.⁹

2.9. Revisão e Aprovação do Projeto

A fase de oficialização do projeto culminou numa reunião final de validação com os principais *stakeholders*, Hélder Cerqueira e Cátia, destinada à revisão exaustiva dos objetivos propostos, métricas de desempenho e do cronograma de cinco meses. A sessão resultou na aprovação formal da estratégia de digitalização pela administração, que validou a viabilidade da solução para a cadeia de lojas de conveniência. Esta anuência conferiu a autorização necessária para que a equipa avance de imediato para a etapa subsequente de Levantamento e Análise de Requisitos, assegurando que o desenvolvimento técnico permanece estritamente alinhado com as expectativas da direção.¹⁰

⁹Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 8](#).

¹⁰Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 7](#).



TrasmUM – Reunião 2

Data: 2026/02/20

Duração prevista: 30 minutos

Horário de início: 08h30

Horário de término: 08h55

Participantes: Hélder Teixeira (helder@trasmum.pt)
Cátia (unhasPintadas@trasmum.pt)
Gabriel Dantas (a107291@uminho.pt)
José Fernandes (106937@uminho.pt)

Pauta da reunião:

- Revisão dos objetivos propostos
- Aprovação formal do projeto.

Figura 3: Ata da reunião de revisão e aprovação do projeto.

3. Levantamento e Análise de Requisitos

3.1. Utilização de LLMs no Levantamento e Análise de Requisitos

Nesta secção, a utilização de LLMs assumiu um papel diversificado, cobrindo múltiplos tipos de tarefas: a simulação de entrevistas com *stakeholders*, a geração de *user stories*, o levantamento e refinamento de requisitos, e a estruturação formal de requisitos funcionais, não funcionais, casos de uso e diagramas de máquinas de estado. Em todas as fases, os outputs foram sujeitos a revisão pela equipa, sendo rejeitados ou reformulados os conteúdos inconsistentes com o perfil dos *stakeholders* ou sem fonte de elicitação identificável. O critério de validação transversal a toda esta secção foi a rastreabilidade — cada requisito, caso de uso e diagrama deve ser justificável a partir de pelo menos uma fonte identificada. No término desta fase, foi ainda conduzida uma revisão crítica do documento completo com recurso a LLM, com o objetivo de detetar ambiguidades, inconsistências internas e contradições entre secções, sendo os problemas identificados avaliados pela equipa antes da submissão para aprovação.

3.2. Métodos de Levantamento de Requisitos Utilizados

Os requisitos do Sistema de Gestão Integrada para a rede de lojas TrasmUM foram levantados e especificados baseando-se no método de Sommerville (Sommerville, 2015, pp. 102–134), adicionalmente, recorreu-se à criação e aplicação de questionários, como técnica complementar de recolha de informação.

Para assegurar uma extração abrangente, exata e alinhada com as necessidades operacionais e estratégicas da organização, delineou-se um plano de elicitação de requisitos assente em cinco técnicas complementares: uma reunião formal com a administração, entrevistas semiestruturadas, aplicação de questionários, análise documental e redação de *user stories*.¹¹

A reunião formal com a administração, representada por Hélder Cerqueira e Cátia, constituiu o marco inicial do processo de descoberta e compreensão de requisitos. O objetivo primordial deste encontro centrou-se no alinhamento estratégico e na definição do conjunto de técnicas que a equipa de desenvolvimento estaria autorizada a utilizar. Durante a sessão, a direção aprovou formalmente a consulta ao acervo documental da empresa — nomeadamente o acesso às guias de remessa e faturas físicas atualmente arquivadas. Embora o foco principal da reunião tenha sido a delimitação metodológica, a dinâmica exploratória da conversa acabou por fomentar a identificação preliminar de alguns requisitos, que emergiram de forma orgânica à medida que a conversa fluía com os *stakeholders*.¹²

¹¹Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 9](#).

¹²Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 10](#).



TrasmUM – Reunião 3

Data: 2026/02/24

Duração prevista: 90 minutos

Horário de início: 14h30

Horário de término: 16h15

Participantes: Hélder Teixeira (helder@trasmum.pt)
Cátia (unhasPintadas@trasmum.pt)
Gabriel Dantas (a107291@uminho.pt)
José Fernandes (a106937@uminho.pt)
Luís Ferreira (a98286@uminho.pt)
Simão Oliveira (a107322@uminho.pt)

Pauta da reunião:

- Definição do conjunto de fontes de informação e técnicas que a equipa de desenvolvimento estaria autorizada a utilizar.
- Levantamento de requisitos.

Figura 4: Ata da reunião para a definição do conjunto das técnicas a utilizar.

A realização de entrevistas semiestruturadas constituiu uma técnica central no processo de elicitação de requisitos de alto nível, seguindo as orientações de Sommerville (Sommerville, 2015). De acordo com o autor, as entrevistas permitem obter uma compreensão global do que os *stakeholders* fazem e das dificuldades que enfrentam com os sistemas atuais, sendo particularmente eficazes quando combinam questões predefinidas com uma exploração mais aberta dos temas que vão emergindo naturalmente. Foram realizadas duas sessões distintas, cada uma dirigida a um *stakeholder* com perspetivas complementares sobre o sistema. A primeira entrevista foi conduzida com Hélder Cerqueira nas instalações da sede TrasmUM a 2 de março de 2026, com foco na visão macro do negócio e nas expectativas de alto nível do investidor relativamente à escalabilidade, segurança e tempos de resposta do sistema.

Registo de Entrevista - Elicitação de Requisitos

Entrevistado(a):	Hélder Cerqueira (Administração)
Equipa/Entrevistador:	Equipa de Análise de Sistemas
Data e Hora:	02 de Março de 2026, 16:00
Local/Plataforma:	Instalações da Sede - TrasmUM
Objetivo Principal:	Compreender as ineficiências do modelo atual de gestão de fornecedores e apuramento de requisitos.

Transcrição da Entrevista

P: Boa tarde, Hélder. Para começar, gostaríamos de saber quais são as suas principais expectativas para o novo sistema que iremos desenvolver.

R: Boa tarde! Olhem, acima de tudo quero que o sistema nos traga controlo e confiança nos números, porque hoje em dia isso simplesmente não existe. Quero conseguir ver as vendas e o stock das lojas sem depender de telefonemas nem de papelada perdida. Também é importante que ajude a equipa nas lojas a trabalhar mais rápido e com menos erros — os operadores perdem demasiado tempo com coisas que deviam ser automáticas. No fundo, procuro uma solução simples de usar, mas robusta o suficiente para suportar o crescimento da rede.

P: No que diz respeito ao funcionário, como é que este deve, num dia normal de funcionamento, interagir com o sistema?

R: Bom, o funcionário chega, autentica-se no sistema — e aqui é importante que seja apenas um por loja, por turno, para sabermos sempre quem é responsável pelo quê — e durante o dia basicamente regista as vendas e trata de eventuais devoluções. No final do dia, o manager faz o fecho de dia e o sistema encarrega-se de enviar tudo automaticamente para a sede. Sem telefonemas, sem papelada — é isso que queremos.

Figura 5: Excerto da entrevista realizada a Hélder Cerqueira.

A segunda entrevista foi realizada com a Cátia a 3 de março de 2026, explorando as suas necessidades enquanto manager no Terminal de Loja e enquanto CEO na interface do Servidor Central, nomeadamente a supervisão estratégica via *dashboards* e relatórios analíticos, a gestão hierárquica de utilizadores no Terminal de Loja e o controlo de pagamentos a fornecedores.

Ambas as sessões foram conduzidas com recurso a um guião estruturado que garantiu a cobertura dos tópicos essenciais, permitindo simultaneamente que a conversa fluísse de forma orgânica. Esta abordagem mista reflete precisamente o equilíbrio recomendado por Sommerville (Sommerville, 2015) entre entrevistas fechadas e abertas, evitando discussões completamente livres, que raramente produzem resultados úteis, mas sem suprimir a emergência espontânea de requisitos. O autor alerta ainda que os *stakeholders* raramente sugerem requisitos específicos e detalhados sem um contexto concreto; por essa razão, as perguntas foram elaboradas com base nos problemas operacionais previamente identificados, funcionando como âncoras para a discussão.¹³ As transcrições integrais de ambas as entrevistas encontram-se disponíveis no Anexo 2 e Anexo 3.¹⁴

A aplicação de questionários permitiu uma recolha sistemática de informação junto dos colaboradores da linha da frente, procurando identificar as dificuldades mais críticas do atual registo manual de vendas e quantificar a frequência de erros no apuramento de caixa. O principal objetivo foi comprovar a necessidade de automatizar o sistema, com especial foco na atualização do stock em tempo real e numa maior eficiência operacional durante o fecho de expediente. Desta forma, assegurou-se uma perspetiva fidedigna sobre o verdadeiro fluxo de trabalho diário nas três lojas da rede TrasmUM.¹⁵

¹³ Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 13](#).

¹⁴ Simulação de Entrevista com Hélder com o apoio de IA — ver [Anexo de IA, Entrada 12](#).

¹⁵ Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 11](#).

Questionário de Levantamento de Requisitos — Operadores de Caixa

Olá, Simão. Quando submeter este formulário, o proprietário verá o seu nome e endereço de e-mail.

1. Que dificuldades encontra atualmente no registo manual de vendas? 

Introduza a sua resposta

2. Com que frequência ocorrem erros no apuramento de valores de caixa? 

☐ Nunca

☐ Raramente

☐ Ocasionalmente

☐ Frequentemente

☐ Muito frequentemente

3. Considera útil que o sistema atualize automaticamente o stock após cada venda? 

☐ Nada útil

☐ Pouco útil

☐ Útil

☐ Muito útil

☐ Essencial

4. Tem alguma sugestão adicional para melhorar o processo de vendas e fecho de caixa? 

Introduza a sua resposta

Figura 6: Questionário realizado aos operadores de loja.

A análise documental iniciou-se com o exame das faturas físicas atualmente emitidas na rede de lojas TrasmUM. A avaliação deste documento visou identificar a estrutura de dados inerente ao processo de venda ao cliente, mapeando os campos de informação essenciais, os detalhes dos artigos transacionados e as respetivas obrigações fiscais. Este levantamento garante que o futuro sistema informático acomoda todos os requisitos práticos e legais de faturação, permitindo uma transição segura do registo em papel para o formato digital. A figura seguinte ilustra um exemplar de uma fatura recolhida durante esta análise.



Figura 7: Exemplar de uma fatura física emitida na rede de lojas TrasmUM.

De seguida, procedeu-se à análise das guias de remessa físicas, utilizadas no quotidiano das lojas para registar a entrega de mercadorias por parte dos fornecedores. O estudo destes documentos permitiu mapear o fluxo de entrada de produtos e identificar as informações cruciais para a gestão de inventário, como quantidades, lotes e identificação de fornecedores. A compreensão destas variáveis operacionais é essencial para assegurar que a futura solução automatizada consiga processar a atualização do stock de forma fiável e atempada. A figura abaixo apresenta uma das guias de remessa avaliadas no contexto deste levantamento de requisitos.¹⁶

¹⁶Conteúdo gerado com apoio de IA — ver [Anexo de IA, Entrada 14](#).

amazUM Delivery, LDA.
Rua Amália da Costa Lima, n.º 706
4990-350, Ponte de Lima
Telefone: 210 000 000
Email: amazUMdelivery@gmail.com

GUIA DE REMESSA
N.º Documento: GR 2025/001
Data: 25 de novembro de 2025

Local de Carga:
Armazém Central
Rua Amália da Costa Lima, Ponte de Lima

Dados do Cliente / Local de Descarga:
Dona Acácia, TrasmUM
Rua Acácio Mariano, 456
5300-200 Bragança
NIF: 268 999 343

Ref.	Descrição do Artigo	Qtd.	Unid.	Validade
ART-001	Alheira de Aves Minho fumeiro	17	un	30/12/2025
ART-002	Carne bovino	10	kg	30/11/2025
ART-003	Ice Tea Limão 500ml	15	un	30/08/2026
ART-004	Detergente Manual Loixa Ultra Limão Fairy	13	un	N/A

Observações: Sujeito a conferência física no ato da entrega.

Total a Pagar: 250,00 €

Informações de Pagamento:
Banco: Caixa Geral de Levantamentos
IBAN: PT50 0000 0000 1234 5678 9012 3
Condições: Pagamento a 30 dias após a data de entrega.

A Entregar (Motorista/Expedidor)
Data: __ / __ / ____

O Recetor (Cliente)
Data: __ / __ / ____

1

Figura 8: Exemplar de uma guia de remessa física rececionada na rede TrasmUM.

O levantamento de requisitos foi complementado pela recolha de testemunhos diretos de dois perfis de utilizador distintos: um funcionário de caixa com experiência operacional no dia a dia¹⁷, e um manager responsável pela gestão autónoma do seu estabelecimento.¹⁸ A análise qualitativa destes relatos permitiu identificar as principais necessidades funcionais e não funcionais do futuro sistema, descritas de seguida sob a forma de histórias de utilizador.

Utilizador	Funcionário da Caixa
US01	Como operador, quero que o sistema calcule automaticamente o troco a devolver ao cliente quando o pagamento é feito em numerário, para que não precise de fazer cálculos mentais ao fim de um dia de trabalho e reduza o risco de erros.
US02	Como operador, quero pesquisar rapidamente qualquer artigo no sistema através de uma barra de pesquisa digital, consultando stock, preços e validades, para que não perca tempo a folhear listas em papel ou a procurar fisicamente produtos no armazém.

Tabela 1: User Stories — Funcionário da Caixa

Utilizador	Manager
US-M01	Como manager, quero registar a receção de remessas de fornecedores, para que o stock da minha loja seja atualizado automaticamente sem depender de registos manuais.

¹⁷User Story gerada com o apoio de IA — ver [Anexo de IA, Entrada 15](#).

¹⁸User Story gerada com o apoio de IA — ver [Anexo de IA, Entrada 16](#).

US-M02	Como manager, quero criar e atualizar os produtos e preços do catálogo da minha loja, para que os funcionários operem sempre com informação correta e atualizada.
US-M03	Como manager, quero criar e gerir as contas dos funcionários da minha loja, para que cada operador aceda ao sistema com credenciais próprias e todas as ações fiquem rastreáveis.
US-M04	Como manager, quero realizar o fecho de dia e enviar automaticamente os dados para o Servidor Central, para que a Cátia tenha acesso às métricas do meu estabelecimento sem necessidade de contacto telefónico.

Tabela 2: User Stories — Manager

3.3. Levantamento de Requisitos

Com base nas orientações metodológicas e no plano de elicitação detalhado na secção anterior, procedeu-se ao levantamento exaustivo das necessidades do Sistema de Gestão Integrada para a rede de lojas TrasmUM. A aplicação conjugada de diferentes técnicas — nomeadamente reuniões formais, entrevistas com a administração, questionários aplicados aos operadores de caixa, *user stories* e análise documental de faturas e guias de remessa físicas — permitiu compilar um conjunto inicial de requisitos em bruto.

De seguida, apresenta-se um excerto ilustrativo de alguns dos requisitos levantados na sua forma original. Importa salientar que a lista integral e exaustiva de todos os requisitos recolhidos — incluindo o respetivo analista responsável e a fonte metodológica associada — encontra-se disponível para consulta no Anexo 4, num estado prévio às fases subsequentes de refinamento e especificação formal.

TrasmUM — Levantamento de Requisitos (ordem de elicitação) Versão PRÉ-refinamento					
● Fundo vermelho = requisito removido (redundante, inviável ou reformulado) ● Fundo amarelo = texto corrigido na fase de refinamento Texto original mantido nesta folha					
ID	Tipo	Requisito (texto original)	Analista	Fonte	Estado
R1	Funcional	O sistema deve permitir a operação e autenticação de apenas um funcionário por loja, associando-lhe todas as operações do turno.	José Fernandes	Entrevista — Cátia das Unhas Pintadas	Mantido
R2	Funcional	O sistema deve monitorizar o valor em numerário na caixa, assumindo um fundo inicial de 100€, e sugerir a retirada de dinheiro por questões de segurança.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Questionário — Operadores de Caixa	Mantido
R3	Funcional	O operador deve poder realizar o fecho de caixa diário, confirmando o valor apurado e despoletando o envio dos dados do dia para o servidor central.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira	Mantido
R4	Funcional	A administração deve poder criar pedidos de remessas e direcioná-los especificamente para reabastecimento de lojas selecionadas.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira	Corrigido
R5	Funcional	No final do dia, a loja deve remeter todas as operações de vendas, devoluções e remessas para a sede de forma automática.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas	Corrigido
R6	Funcional	A administração deve poder redigir e enviar mensagens e anúncios informativos para os terminais dos funcionários nas lojas.	Gabriel Dantas	User Story — Cátia das Unhas Pintadas	Removido — Funcionalidade fora do âmbito definido; ausência de requisito técnico correspondente e sem caso de uso associado
R7	Não Funcional	O sistema da loja deve garantir disponibilidade mínima de 99% mensal durante o horário de funcionamento definido.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Hélder Cerqueira	Mantido
R8	Não Funcional	O servidor central deve realizar backups automáticos diários com retenção mínima de 30 dias.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Hélder Cerqueira	Corrigido
R9	Funcional	O funcionário deve poder anular uma venda que ainda se encontre em curso, antes da sua finalização, sem impacto no stock nem no apuramento de caixa.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Mantido
R10	Funcional	O funcionário deve poder registar as vendas dos produtos na loja, atualizando o sistema automaticamente.	Gabriel Dantas	Questionário — Operadores de Caixa	Corrigido
R11	Funcional	O funcionário deve poder processar a devolução de produtos, corrigindo o valor em caixa e repondo o stock disponível, estritamente caso o artigo se encontre dentro do prazo de validade.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Corrigido

Figura 9: Excerto dos requisitos levantados.

3.4. Refinamento de Requisitos Ambíguos

Concluída a fase de levantamento, procedeu-se à revisão crítica do conjunto de requisitos recolhidos. Dada a natureza da linguagem natural e a complexidade inerente ao domínio do negócio retalhista, verificou-se que alguns requisitos apresentavam formulações suscetíveis de interpretações múltiplas ou insuficientemente precisas para suportar uma especificação técnica inequívoca. Nos casos identificados, a ambiguidade poderia comprometer a estabilidade arquitetural do sistema — nomeadamente na gestão rastreável de stock, fluxos financeiros e logística — ou gerar implementações divergentes das expectativas dos *stakeholders*, pelo que se procedeu ao seu refinamento estrutural antes da formalização definitiva.

Requisito Original	O funcionário deve poder registar as vendas dos produtos na loja, atualizando o sistema automaticamente.
Ambiguidade Identificada	O requisito menciona o registo de “produtos”, omitindo a forma como o sistema distingue fisicamente unidades do mesmo produto com validades diferentes (lotes). Se o sistema registar apenas o código genérico do produto na caixa, não conseguirá abater o stock com precisão nem aplicar os descontos específicos de fim de validade a unidades individuais, inviabilizando a rastreabilidade exigida pela operação retalhista.
Correção Elaborada	O funcionário deve poder registar as vendas através da leitura do código de barras genérico do produto. O sistema deve deduzir automaticamente uma unidade ao lote desse produto com a data de validade mais próxima de expirar (lógica FEFO) e aplicar o preço correspondente. Caso se trate de um artigo com desconto de fim de validade, o funcionário aciona o botão ‘Desconto Validade’ na interface de caixa. O sistema apresenta a lista de lotes sinalizados com desconto ativo, o funcionário seleciona o lote pretendido, e o sistema aplica o preço com desconto e deduz a unidade a esse lote específico, sobrepondo a lógica FEFO.

Tabela 3: Refinamento do Requisito RF03 — Registo de Vendas e Identificação de Artigos

Requisito Original	O funcionário deve poder processar a devolução de produtos, corrigindo o valor em caixa e repondo o stock disponível, estritamente caso o artigo se encontre dentro do prazo de validade.
Ambiguidade Identificada	A formulação não esclarece a autonomia do sistema na aceitação comercial, sugerindo que o sistema poderia tomar essa decisão de forma autónoma. Adicionalmente, omite a forma como o sistema identifica a validade exata da unidade devolvida e não estabelece se a devolução depende de uma venda registada previamente.
Correção Elaborada	A devolução exige a identificação prévia da transação original para evitar fraudes. O operador deve inserir o ID/Número da fatura de venda. O sistema valida a fatura e resgata os artigos lá vendidos. Após a decisão física de aceitar o produto, o operador introduz a data de validade impressa na embalagem devolvida, permitindo ao sistema mapear o artigo ao lote correto. Se o lote ainda estiver dentro do prazo, o stock é incrementado; caso contrário, não é repostado em stock. O sistema local deve validar se a fatura inserida foi emitida na base de dados dessa mesma loja. Apenas são aceites devoluções de transações originadas localmente, rejeitando faturas de outras filiais.

Tabela 4: Refinamento do Requisito RF05 — Processamento de Devoluções

Requisito Original	O manager deve poder registar a entrada de uma remessa de fornecedores, adicionando imediatamente as novas quantidades ao stock da loja e associando-lhes a data de validade correspondente ao lote recebido.
Ambiguidade Identificada	O requisito isola a entrada de mercadoria na loja do fluxo logístico e financeiro da sede. Não explica como esta entrada afeta as encomendas pendentes nem como gera as obrigações de tesouraria. Adicionalmente, pode ser interpretado como uma substituição global da validade anterior pela do novo lote, resultando na perda de rastreabilidade.
Correção Elaborada	O registo de uma remessa deve ser efetuado pelo Manager mediante a introdução da respetiva Guia de Remessa. Esta ação deve alterar automaticamente o estado do Pedido de Remessa correspondente para concluído e utilizar o valor financeiro registado na guia para gerar um Pagamento Pendente associado a esse fornecedor. O Manager é responsável pela gestão destes pagamentos localmente. O registo de pagamentos pendentes fica disponível para consulta no Servidor Central após o envio do fecho de dia. O sistema deve adicionar as novas unidades ao stock, atribuindo-lhes um código único de lote e a data de validade da entrega. As unidades previamente existentes em stock mantêm os seus próprios códigos de lote e validades independentes.

Tabela 5: Refinamento do Requisito RF06 — Registo de Remessa, Gestão de Validades e Pagamentos

Requisito Original	As lojas devem analisar diariamente as validades dos produtos e o sistema sugerir/alertar os produtos em risco, cabendo à administração clicar num botão para “Aplicar Desconto” de forma manual.
Ambiguidade Identificada	A utilização da conjunção “sugerir/alertar” introduz ambiguidade quanto ao comportamento efetivo do sistema. Mais criticamente, o texto refere a aplicação do desconto aos “produtos”, o que implicaria a redução de preço de todas as unidades da loja, afetando erradamente os lotes mais recentes que não estão em risco de expiração.
Correção Elaborada	O terminal local de cada loja deve executar diariamente uma rotina de verificação que identifique todos os lotes cujos produtos apresentem uma data de validade inferior ou igual a um limiar configurável pelo Manager. O sistema apresenta ao Manager um alerta com os lotes identificados e um botão de “Aplicar Desconto” para cada um. A aplicação do desconto incide exclusivamente sobre o lote específico sinalizado, sem afetar o preço dos restantes lotes do mesmo produto. Na caixa, o funcionário aplica o desconto através de um botão específico no ecrã no momento da transação, abatendo a unidade a esse lote sinalizado. O desconto fica registado localmente — o fecho de dia reporta apenas o valor real pelo qual o produto foi vendido, sem distinção explícita de desconto de validade.

Tabela 6: Refinamento do Requisito RF22 — Aplicação de Descontos por Aproximação do Prazo de Validade

Importa salientar que, para além dos requisitos detalhados nas tabelas seguintes — cujo grau de ambiguidade representava um risco arquitetural, financeiro ou logístico significativo para o projeto —, o processo de revisão abrangeu a totalidade dos requisitos levantados. Durante esta fase, foram também realizadas diversas correções menores noutros requisitos, focadas essencialmente em ajustes de sintaxe, uniformização de vocabulário técnico e clarificação de termos. Dado que estas alterações mais simples não modificaram a lógica central do sistema, optou-se por omiti-las desta secção, privilegiando assim a clareza, a concisão e o foco nos aspetos mais críticos do documento.

Adicionalmente, no decurso da revisão crítica do conjunto de requisitos levantados, foram identificados seis requisitos que não reuniam condições para integrar a especificação formal do sistema. A sua remoção foi decidida por unanimidade pela equipa, com base nos seguintes critérios: ausência de fonte de elicitação válida, redundância face a requisitos já especificados, inviabilidade técnica no âmbito e prazo do projeto, ou contradição com decisões arquiteturais já formalizadas. Os requisitos removidos e as respetivas justificações encontram-se documentados na tabela de requisitos levantados, apresentada.¹⁹

R6	A administração deve poder redigir e enviar mensagens e anúncios informativos para os terminais dos funcionários nas lojas.	Funcionalidade fora do âmbito definido; ausência de requisito técnico correspondente nos dois terminais do sistema e de caso de uso associado.
R47	O sistema deve permitir ao manager exportar o catálogo de produtos da loja para um ficheiro CSV.	Funcionalidade não confirmada por nenhum <i>stakeholder</i> ; ausência de fonte de elicitação válida e de caso de uso correspondente.
R48	O funcionário deve poder consultar o histórico completo de vendas da loja nos últimos 12 meses a partir do terminal de caixa.	Redundante face ao requisito RF20; a visibilidade histórica agregada é exclusiva do Servidor Central e do perfil CEO.
R49	O sistema deve enviar notificações automáticas por email ao manager quando o stock de um produto atingir o limiar mínimo.	Restrição de âmbito: o sistema não realiza comunicações externas automáticas, conforme definido na secção de Análise de Contexto e Restrições. A funcionalidade de alerta está coberta pelo RF10 (alerta visual interno).
R50	O sistema deve permitir à CEO definir centralmente os preços dos produtos de todas as lojas a partir do Servidor Central.	Contradiz a decisão arquitetural de gestão local autónoma do catálogo por cada manager

¹⁹Texto e tabelas de refinamento de requisitos gerados com apoio de IA — ver [Anexo de IA, Entrada 18](#).

		(RF17) e viola o princípio de autonomia operacional por loja.
R51	O sistema deve registrar automaticamente a localização GPS de cada terminal no momento do fecho de dia.	Inviável no âmbito do projeto; ausência de hardware adequado e de requisito do <i>stakeholder</i> ; nenhuma fonte de elicitação identificada.
R52	O manager deve poder aprovar individualmente cada venda antes de ser finalizada pelo funcionário.	Contradiz os requisitos de fluxo de caixa e usabilidade (RNF14, RNF15); introduziria um bloqueio operacional inaceitável na linha da frente.

Tabela 7: Requisitos removidos e respetivas justificações

3.5. Requisitos Funcionais

Os requisitos funcionais do sistema TrasmUM definem os serviços e as funções específicas que a plataforma deve executar em resposta a estímulos dos utilizadores ou de outros sistemas. De acordo com a metodologia de Ian Sommerville (Sommerville, 2015), estes requisitos descrevem detalhadamente o comportamento do software perante entradas particulares, especificando o que o sistema deve fazer para satisfazer as necessidades operacionais da rede de lojas. Dado que a operação da rede TrasmUM se divide em dois contextos físicos e funcionais distintos, os requisitos funcionais encontram-se distribuídos entre o Ambiente de Loja e o Ambiente de Servidor Central, estando cada requisito explicitamente atribuído à área do sistema a que pertence. A especificação completa e exaustiva de todos requisitos funcionais encontra-se disponível no Anexo 5. As subsecções seguintes apresentam três requisitos selecionados a título ilustrativo, escolhidos por cobrirem cenários de complexidade e natureza distintos: o RF1, que estabelece a restrição de sessão única por utilizador e exemplifica um requisito de controlo com impacto arquitetural direto; o RF3, que modela a lógica FEFO de seleção de lotes e exemplifica um requisito funcional com lógica de negócio não trivial; e o RF6, que integra o registo de remessas com a geração de pagamentos pendentes e exemplifica um requisito que agrega múltiplos fluxos operacionais num único ponto de entrada.²⁰

A restrição da operação a um único utilizador em sessão ativa visa eliminar a diluição de responsabilidades, garantindo que as operações diárias e as eventuais discrepâncias de caixa sejam inequivocamente rastreáveis e atribuíveis. Cada loja dispõe de vários terminais físicos.

Requisito de Utilizador	O sistema deve impedir que o mesmo utilizador esteja autenticado em mais do que um terminal simultaneamente, independentemente do número de terminais disponíveis na loja, independentemente do seu perfil de autenticação, associando-lhe todas as operações realizadas durante o turno. Numa loja com dois terminais, podem coexistir duas sessões simultâneas — uma por terminal —, sem que tal constitua violação desta restrição.
Analista	José Fernandes
Fonte	Entrevista — Cátia
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve verificar, no momento da autenticação, se o utilizador já possui uma sessão ativa noutro terminal, bloqueando o <i>login</i> caso isso se verifique. 2. O sistema deve registar internamente o identificador único do utilizador autenticado e associá-lo a todas as operações de escrita na base de dados local durante a sessão.

²⁰Texto e tabelas de requisitos gerados com apoio de IA — ver [Anexo de IA, Entrada 21](#).

	3. A restrição aplica-se ao utilizador, não ao terminal: dois utilizadores distintos podem ter sessões ativas em terminais diferentes da mesma loja simultaneamente.
Relevância para a Aplicação	Resolve a ineficiência de responsabilização (o “jogo do empurra” entre turnos), assegurando a integridade da autoria das transações financeiras independentemente do nível hierárquico do utilizador em funções. A existência de dois terminais por loja garante ainda a continuidade operacional da caixa, eliminando a necessidade de interromper o atendimento ao cliente para a realização de tarefas de gestão.

Tabela 8: Especificação do Requisito Funcional 1

Para assegurar a confidencialidade e a integridade do sistema no ponto de venda, a autenticação individual substitui o acesso livre e partilhado, prevenindo a manipulação indevida dos dados de faturação e acessos não autorizados. O comportamento do sistema após autenticação é determinado pelo perfil do utilizador. O ficheiro de configuração local, definido no momento da instalação, identifica a loja a que o terminal pertence, não determinando o acesso a funcionalidades.

Requisito de Utilizador	O sistema deve exigir credenciais únicas (utilizador e palavra-passe) para acesso ao terminal, desbloqueando funcionalidades distintas consoante o perfil autenticado — Funcionário, Manager ou CEO.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve apresentar um único ecrã de início de sessão que exija a inserção de utilizador e palavra-passe, comum a todos os perfis. 2. O sistema deve validar as credenciais contra a base de dados local de utilizadores ativos antes de permitir o acesso. 3. O sistema deve ler, no arranque, um ficheiro de configuração local que contenha o identificador único da loja a que o terminal pertence. Este ficheiro é definido no momento da instalação e não é alterável através da interface da aplicação. 4. Após autenticação bem-sucedida, o sistema determina as funcionalidades disponíveis com base no perfil do utilizador, de acordo com as seguintes regras: • O perfil Funcionário acede ao registo de vendas, devoluções e consulta de produtos. • O perfil Manager acede adicionalmente à gestão de remessas, catálogo, preços, funcionários e fecho de dia. • O perfil CEO acede a todas as funcionalidades do Manager, acrescentando a gestão de contas de manager do mesmo estabelecimento.
Relevância para a Aplicação	Garante a segregação de responsabilidades e o sigilo da informação financeira e operacional, impedindo que utilizadores acedam a funcionalidades ou dados fora do âmbito do seu cargo. A autenticação individual substitui o acesso livre e partilhado, prevenindo a manipulação indevida dos dados de faturação e acessos não autorizados.

Tabela 9: Especificação do Requisito Funcional 2

A automação do registo de vendas por lote responde diretamente ao problema das frequentes ruturas e perdas financeiras causadas pelo controlo analógico de stock, permitindo rastrear exatamente a validade e a quantidade do que é vendido.

Requisito de Utilizador	O funcionário deve poder registar as vendas dos produtos identificando o código de barras do produto, para atualização automática de stock e obtenção do seu preço. Esta funcionalidade está disponível a todos os perfis de utilizador.
Analista	Gabriel Dantas
Fonte	Questionário — Operadores de Caixa

Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. A interface de caixa deve manter o campo de inserção automaticamente em foco, processando de imediato a entrada do código de barras genérico do produto. 2. O sistema deve deduzir automaticamente uma unidade ao lote desse produto com a data de validade mais próxima a expirar (FEFO) na base de dados local, exceto quando o funcionário aciona o botão “Desconto Validade”, caso em que o sistema sobrepõe a lógica FEFO e abate a unidade ao lote sinalizado com desconto selecionado pelo funcionário. 3. O sistema deve apresentar o preço de venda atualizado, aplicando automaticamente eventuais descontos de validade já configurados para esse lote.
Relevância para a Aplicação	A lógica FEFO garante que os lotes mais antigos são sempre os primeiros a ser vendidos. O botão ‘Desconto Validade’ permite ao funcionário desviar-se desta lógica de forma controlada, assegurando que os lotes sinalizados pelo manager são escoados sem necessidade de hardware adicional de impressão de etiquetas.

Tabela 10: Especificação do Requisito Funcional 3

A capacidade de anulação ágil durante o atendimento previne erros operacionais e retrabalho, evitando correções morosas, stresse no momento do pagamento e registos indevidos no final do dia.

Requisito de Utilizador	O funcionário deve poder anular uma venda em curso antes de a finalizar, sem afetar o apuramento final ou os stocks.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. A interface deve disponibilizar uma opção para cancelar a transação corrente antes da emissão do comprovativo. 2. O sistema deve limpar os produtos selecionados da memória temporária sem registar movimentos financeiros ou alterações no inventário local.
Relevância para a Aplicação	Aumenta a eficiência da linha de frente e previne a contaminação dos dados contabilísticos com vendas não efetivadas.

Tabela 11: Especificação do Requisito Funcional 4

A correta gestão de devoluções assegura que o inventário reflete apenas produtos aptos para revenda, evitando que produtos fora da validade regressem ao stock e penalizem a rentabilidade.

Requisito de Utilizador	O funcionário deve poder registar devoluções associadas a uma fatura de venda prévia, deduzindo o valor exato em caixa e repondo o stock de um artigo apenas se este estiver dentro da validade. O sistema local deve validar se a fatura inserida foi emitida na base de dados dessa mesma loja. Apenas são aceites devoluções de transações originadas localmente, rejeitando faturas de outras filiais.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve exigir a inserção do identificador da fatura original, do artigo e da data de validade física impressa na embalagem, para mapeamento automático ao código de lote correspondente. 2. O sistema deve validar a existência do artigo na fatura e bloquear a operação se este já tiver sido alvo de devolução prévia. A validação opera ao nível da unidade: o

	sistema deve verificar quantas unidades os do artigo constam na fatura e quantas já foram devolvidas, bloqueando quando a quantidade devolvida igualar a quantidade faturada. 3. O sistema deve resgatar o valor exato pago na altura da compra e subtrair-lo ao apuramento atual de caixa. 4. O sistema deve comparar a data de validade do lote com a data atual e, se válida, incrementar em um a quantidade de stock local para esse lote; caso contrário, não deve alterar a quantidade em stock.
Relevância para a Aplicação	Assegura a exatidão financeira do apuramento diário e previne a comercialização indevida de artigos perecíveis expirados.

Tabela 12: Especificação do Requisito Funcional 5

O registo automatizado de remessas na loja extingue o caótico arquivo em caixas de sapatos, assegurando a comunicação imediata de entregas à sede e mantendo o rastreamento individual das validades por lote.

Requisito de Utilizador	O manager deve registar a receção de remessas introduzindo a respetiva guia, de forma a atualizar o stock local com lotes independentes e gerar um registo de pagamento pendente associado ao fornecedor.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve permitir ao manager a introdução dos dados da guia de remessa (fornecedor, produtos, quantidades, data, valor) e a sua submissão local. 2. O sistema deve criar registos independentes de stock (novos lotes) para as unidades recebidas, atribuindo-lhes um código único de lote e a data de validade da entrega, sem sobrepor as validades dos lotes pré-existentes. 3. O sistema deve gerar localmente um registo de pagamento pendente associado ao fornecedor, com base no valor financeiro registado na guia, para inclusão no pacote de fecho de dia. 4. O sistema deve permitir ao manager atualizar o estado de um registo de pagamento pendente para “pago”, ficando essa alteração incluída no próximo pacote de fecho de dia.
Relevância para a Aplicação	Elimina o arquivo físico caótico em caixas de sapatos, assegura a rastreabilidade individual dos lotes e garante que as obrigações de tesouraria ficam registadas localmente para posterior consolidação no Servidor Central.

Tabela 13: Especificação do Requisito Funcional 6

Os restantes requisitos funcionais encontram-se especificados na íntegra no Anexo 5, mantendo a mesma estrutura, numeração e nível de detalhe utilizados nos exemplos acima.

3.6. Requisitos Não Funcionais

A especificação que se segue detalha os requisitos não funcionais do sistema TrasmUM, os quais definem as propriedades emergentes da plataforma — como fiabilidade, desempenho e segurança — e as restrições operacionais que moldam a sua implementação técnica. Fundamentada na taxonomia de Ian Sommerville (Sommerville, 2015), esta análise categoriza cada requisito como Requisito de Produto, Requisito Organizacional ou Requisito Externo. A especificação completa dos não funcionais encontra-se disponível no Anexo 6. As subsecções seguintes apresentam quatro requisitos a título ilustrativo, selecionados por representarem categorias e domínios distintos: o RNF1 e o RNF2, que estabelecem restrições

de desempenho no ponto de venda; o RNF3, que define o tempo máximo de transmissão do pacote de fecho de dia; e o RNF4, que impõe requisitos de responsividade à interface de análise da CEO.²¹

O primeiro requisito não funcional trata-se de um Requisito de Produto, especificamente no domínio do Desempenho. O propósito técnico desta restrição temporal é garantir que o processamento das transações no ponto de venda seja célere, evitando o bloqueio da interface do operador durante o ato de pagamento e o acesso à base de dados. Ao estipular um tempo máximo de dois segundos para a persistência da transação, o sistema assegura a fluidez do atendimento nas horas de ponta, protegendo a viabilidade comercial da loja ao mitigar a formação de longas filas de espera e o consequente descontentamento dos clientes da rede TrasmUM.

Requisito de Utilizador	O registo de uma venda deve ser concluído em ≤ 2 segundos, medido desde a confirmação da operação até à persistência da transação.
Analista	José Fernandes
Fonte	Questionário — Operadores de Caixa
Relevância para a Aplicação	Garante um fluxo de atendimento contínuo e ininterrupto, reduzindo a pressão sobre os funcionários na linha da frente e assegurando que as limitações do software não prejudicam a capacidade de resposta da loja.

Tabela 14: Especificação do Requisito Não Funcional 1

Classificado como um Requisito de Produto focado no Desempenho, este critério estabelece um teto máximo de latência para a produção do documento digital comprovativo da transação. A restrição técnica impõe que a arquitetura do software optimize a compilação de dados da venda de modo a não interromper o fluxo final de atendimento. Esta métrica protege a operação diária das lojas TrasmUM ao garantir que, logo após o pagamento, a libertação do cliente decorre de forma quase instantânea, mantendo o ritmo acelerado e eficiente exigido no retalho de conveniência.

Requisito de Utilizador	A geração de fatura deve ocorrer em ≤ 3 segundos após confirmação da venda.
Analista	Gabriel Dantas
Fonte	Análise Documental — Fatura física TrasmUM
Relevância para a Aplicação	Evita estrangulamentos na conclusão do processo de compra, assegurando que o cliente não aguarda desnecessariamente pelo talão e que o operador fica rapidamente livre para o próximo atendimento.

Tabela 15: Especificação do Requisito Não Funcional 2

Trata-se de um Requisito de Produto inserido na vertente de Eficiência e Desempenho. O estabelecimento de um tempo máximo de transmissão por loja garante que o envio do pacote de fecho de dia é eficiente e previsível, sem constituir uma janela temporal de receção no Servidor Central — que recebe pacotes de forma passiva e contínua, independentemente do horário de envio. Este requisito protege a infraestrutura de rede e os recursos computacionais do Servidor Central, impedindo cenários de sobrecarga e assegurando que o apuramento total da rede TrasmUM, com as suas três lojas, esteja concluído a tempo de fornecer relatórios exatos à gestão na manhã seguinte.

Requisito de Utilizador	O envio e processamento do pacote de fecho de dia de cada loja deve ser concluído em ≤ 5 minutos, considerando o volume médio diário de transações.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira
Relevância para a Aplicação	Garante a consolidação ágil de todos os dados financeiros e logísticos diários no Servidor Central, assegurando que os dados de todas as lojas estão disponíveis para análise pela Cátia na manhã seguinte.

²¹Texto e tabelas de requisitos gerados com apoio de IA — ver [Anexo de IA, Entrada 20](#).

Tabela 16: Especificação do Requisito Não Funcional 3

Este parâmetro é enquadrado como um Requisito de Produto relativo ao Desempenho e à Usabilidade. A exigência de um carregamento rápido das interfaces analíticas impõe que o cálculo de métricas complexas (como níveis de stock e volumes de vendas globais) seja pré-processado na base de dados, em vez de calculado em tempo real durante o pedido. Desta forma, protege-se o tempo útil da Diretora-Geral, fomentando a adoção do sistema e assegurando que a consulta de dados críticos para a tomada de decisão estratégica decorre de forma fluida, sem bloqueios na interface administrativa.

Requisito de Utilizador	As <i>dashboards</i> devem carregar em ≤ 3 segundos para consultas padrão, sobre os dados consolidados recebidos via fecho de dia.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Relevância para a Aplicação	Proporciona à Cátia uma experiência analítica reativa e eficiente, essencial para o acompanhamento contínuo da rentabilidade da rede e planeamento tático imediato.

Tabela 17: Especificação do Requisito Não Funcional 4

Os restantes requisitos não funcionais encontram-se especificados na íntegra no Anexo 6, mantendo a estrutura e a numeração de referência utilizadas ao longo deste documento.

3.7. Casos de Uso

Os casos de uso apresentados nesta secção transpõem os requisitos funcionais especificados para um conjunto de interações concretas entre os atores do sistema e a plataforma TrasmUM. De acordo com a metodologia de Sommerville (Sommerville, 2015), esta representação permite evidenciar, de forma clara e estruturada, o que o sistema deve fazer em resposta a estímulos externos, servindo de ponte entre a especificação técnica e a validação com os *stakeholders*. Os dois únicos atores identificados são o Operador de Loja, que interage com o terminal local de cada estabelecimento, e a Administração, responsável pela supervisão e análise da rede a partir da interface do Servidor Central. Sempre que requisitos funcionais apresentassem uma coesão lógica e operacional suficiente, foram agrupados num único caso de uso, garantindo a exaustividade sem fragmentação desnecessária do modelo. A especificação completa dos catorze casos de uso identificados encontra-se disponível no Anexo 7. As subsecções seguintes apresentam três casos de uso a título ilustrativo, selecionados por cobrirem cenários de complexidade distinta e por envolverem os dois atores e as duas fronteiras de sistema: o caso de uso Registrar Venda, que modela o fluxo transaccional mais frequente do sistema com múltiplos fluxos alternativos; o caso de uso Registrar Devolução, que exemplifica um fluxo com cadeia de validações sequenciais interdependentes; e o caso de uso Consultar Dashboards e Relatórios Analíticos, que representa a perspetiva exclusiva do ator Administração no Terminal de Servidor Central.²²

O caso de uso que se segue tem origem nos requisitos RF03, RF04, RF09, RF11 e RF12. Cobre o fluxo completo de uma transação de venda, desde a adição de artigos por código de lote até à emissão da fatura digital e cálculo do troco, incluindo a possibilidade de anulação antes da finalização.

Caso de uso	Registrar Venda
Requisitos de Origem	RF03, RF04, RF09, RF11, RF12
Ator	Funcionário / Manager / CEO
Área do Sistema	Terminal de Loja

²²Texto e casos de uso gerados com apoio de IA — ver [Anexo de IA, Entrada 21](#).

Descrição	Registo de uma venda com identificação de artigos por lote, atualização automática de stock, cálculo de troco em numerário e emissão de fatura digital, com possibilidade de anulação antes da finalização.
Pré-condição	O utilizador tem sessão iniciada e o catálogo local está disponível na base de dados local da loja.
Pós Condição	O stock do lote correspondente é decrementado, a transação fica registada na base de dados local e é emitida uma fatura digital. Se a venda for anulada, nenhuma alteração é persistida.
Fluxo normal	1. O utilizador lê o código de barras genérico do artigo (ou digita manualmente). 2. O sistema identifica o artigo, apresenta o preço base e deduz uma unidade ao lote com a validade mais próxima a expirar (FEFO) na base de dados local . 3. O utilizador repete os passos 1–2 para cada artigo e confirma a transação. 4. O utilizador seleciona o método de pagamento em numerário. 5. O utilizador insere o valor entregue pelo cliente; o sistema calcula e apresenta o troco. 6. O sistema gera a fatura digital com os dados da transação e disponibiliza-a na interface para visualização. 7. O sistema regista a venda concluída na base de dados local .
Fluxo alternativo 1	[Pagamento por cartão] (passo 4) 4.1. O utilizador seleciona o método de pagamento por cartão. 4.2. O sistema regista o método de pagamento sem cálculo de troco e avança para o passo 6.
Fluxo alternativo 2	[Código de lote não encontrado ou stock insuficiente] (passo 2) 2.1. O sistema apresenta mensagem de erro descritiva e cancela a operação, voltando para o passo 1.
Fluxo alternativo 3	[Artigo com Desconto de Validade] (passo 1) 1.1. O funcionário aciona o botão “Desconto Validade” na interface antes de confirmar o artigo. 1.2. O sistema apresenta os lotes sinalizados com desconto para seleção, ignorando a ordem FEFO. 1.3. O funcionário seleciona o lote pretendido; o sistema aplica o preço com desconto e deduz a unidade a esse lote específico, sobrepondo a lógica FEFO. O fluxo retoma no passo 3.
Fluxo de exceção	[Utilizador cancela a venda antes da confirmação] (passo 3) 3.1. O utilizador aciona a opção de anulação da transação em curso. 3.2. O sistema limpa os artigos da memória temporária sem registar movimentos financeiros ou alterações de stock.

Tabela 18: Caso de uso — Registar Venda

O caso de uso que se segue tem origem no requisito RF5. Cobre o registo de uma devolução como transação independente na caixa, com verificação automática de validade do lote devolvido e reposição condicional de stock. A autonomia do operador na decisão de aceitação é salvaguardada pelo fluxo.

Caso de uso	Registar Devolução
Requisitos de Origem	RF5
Ator	Funcionário / Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Processamento de uma devolução de produto, com verificação automática da validade do lote devolvido, reposição condicional de stock e correção do apuramento de caixa.
Pré-condição	O utilizador tem sessão iniciada e decidiu aceitar a devolução.

Pós Condição	O valor do produto é subtraído ao apuramento de caixa. Se o lote estiver dentro do prazo de validade, o stock é incrementado; caso contrário, o stock permanece inalterado.
Fluxo normal	1. O utilizador acede à funcionalidade de devolução, insere o número da fatura original, identifica o artigo devolvido e introduz a data de validade impressa fisicamente na embalagem. 2. O sistema valida a fatura, confirma que o artigo foi lá vendido e que não foi previamente devolvido, resgatando o seu valor de compra exato. 3. O sistema utiliza a data inserida para mapear a unidade ao lote correto e verifica se este ainda se encontra dentro do prazo. 4. O sistema incrementa em uma unidade o stock desse lote. 5. O sistema subtrai o valor resgatado ao apuramento atual de caixa e regista a devolução.
Fluxo alternativo	[Lote fora do prazo] (passo 3) 3.1. O sistema deteta que o lote se encontra fora da data de validade. 3.2. O sistema não repõe a unidade em stock e avança para o passo 5.
Fluxo de exceção	[Fatura inválida, emitida noutra loja, ou artigo não pertencente à fatura] (passo 2) 2.1. O sistema rejeita a operação, não altera o stock nem a caixa, e apresenta mensagem de erro descritiva indicando a irregularidade.

Tabela 19: Caso de uso — Registar Devolução

O caso de uso que se segue tem origem nos requisitos RF20 e RF21. Agrega a consulta de painéis de controlo com métricas globais e de relatórios analíticos avançados, dado que ambos operam exclusivamente sobre os dados consolidados recebidos via fecho de dia e servem o mesmo propósito de apoio à tomada de decisão estratégica.

Caso de uso	Consultar Dashboards e Relatórios Analíticos
Requisitos de Origem	RF20, RF21
Ator	CEO
Área do Sistema	Terminal de Servidor Central
Descrição	Consulta de painéis de controlo com métricas globais de vendas e stocks, e de relatórios analíticos avançados que identificam produtos em risco de rutura iminente ou com excesso de stock parado, com base nos dados consolidados recebidos via fecho de dia.
Pré-condição	O CEO tem sessão iniciada com perfil CEO e existem dados consolidados de pelo menos um fecho de dia recebido.
Pós Condição	O CEO visualiza as métricas e relatórios.
Fluxo normal	1. A Cátia acede ao módulo de análise e seleciona a visualização pretendida (<i>dashboards</i> ou relatórios analíticos). 2. Consoante a seleção, o sistema compila estatísticas macro para apresentação (<i>dashboard</i>), aplica algoritmos para gerar listas de produtos em risco (relatório analítico), ou gera um relatório estruturado exportável com os dados filtrados. Em qualquer das vistas, o sistema disponibiliza filtros para refinar os dados por loja, período temporal e categoria de produto.
Fluxo de exceção	[Ausência de dados consolidados suficientes] (passo 2) 2.1. O sistema apresenta mensagem informativa indicando a ausência de dados históricos suficientes para a análise solicitada.

Tabela 20: Caso de uso — Consultar Dashboards e Relatórios Analíticos

Os restantes catorze casos de uso encontram-se especificados na íntegra no Anexo 7, mantendo a estrutura e as referências aos requisitos funcionais utilizadas nos exemplos acima.

A especificação textual dos casos de uso apresentada, embora rigorosa e detalhada, beneficia de uma representação visual complementar que permita uma leitura mais imediata das fronteiras do sistema e das relações entre atores e funcionalidades²³. De acordo com Sommerville (Sommerville, 2015), os casos de uso constituem uma forma de descrever as interações entre utilizadores e o sistema através de um modelo gráfico e texto estruturado, sendo cada caso de uso identificado pelos atores envolvidos e pelo tipo de interação que desencadeiam. Esta representação visual facilita a validação dos requisitos com os *stakeholders*, na medida em que torna explícito, de forma intuitiva, quem faz o quê e em que contexto do sistema.

A opção por organizar os diagramas em áreas operacionais distintas, em vez de apresentar um único diagrama monolítico, decorre diretamente da arquitetura federada do sistema TrasmUM. Dado que o sistema assenta em duas fronteiras bem delimitadas — o Terminal de Loja e o Terminal de Servidor Central — e que os perfis de utilizador possuem responsabilidades e âmbitos de acesso claramente diferenciados, a decomposição dos diagramas por área operacional permite uma leitura mais clara e focada de cada contexto funcional. Esta abordagem evita a sobrecarga visual de um diagrama único com catorze casos de uso e três atores, que dificultaria a identificação das relações relevantes e prejudicaria a legibilidade do modelo. Nos diagramas que se seguem, a relação de generalização entre atores — em que o Manager herda todas as funcionalidades do Funcionário, e o CEO herda todas as funcionalidades do Manager — reflete a hierarquia operacional da organização e evita a redundância na representação das associações.

O primeiro diagrama representa o conjunto de funcionalidades disponíveis a todos os perfis de utilizador no terminal local de cada loja durante o horário de funcionamento. Estão aqui incluídos os casos de uso nucleares da operação de frente de loja — o início de sessão, o registo de vendas e devoluções, a consulta de produtos e a monitorização da caixa — que constituem o fluxo de trabalho diário dos operadores. A relação de generalização entre os três atores torna explícito que o Manager e o CEO operam com as mesmas capacidades que o Funcionário neste contexto, acrescentando-lhes as responsabilidades definidas nas áreas seguintes.

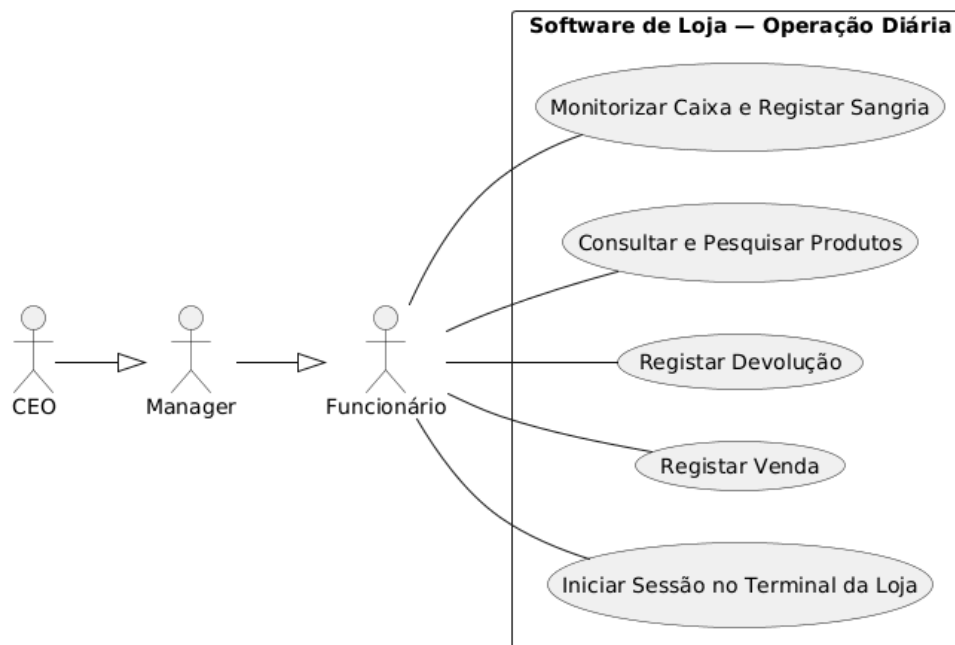


Figura 10: Diagrama de Casos de Uso: Terminal de Loja — Operação Diária.

O segundo diagrama agrupa as funcionalidades de responsabilidade exclusiva do Manager ao nível da gestão operacional do estabelecimento. Estas incluem a criação de pedidos de remessas, o registo de remessas, a gestão do catálogo de produtos, o tratamento documental de fornecedores, a gestão de paga-

²³Diagramas de casos de uso gerados com apoio de IA — ver [Anexo de IA, Entrada 32](#).

mentos pendentes e a aplicação de descontos por aproximação do prazo de validade. Trata-se do conjunto de funcionalidades que substitui diretamente os processos analógicos mais problemáticos identificados na fase de levantamento de requisitos — nomeadamente o arquivo em caixas de sapatos e a gestão manual de validades, dotando cada loja de autonomia operacional plena sobre o seu próprio inventário e cadeia de abastecimento.

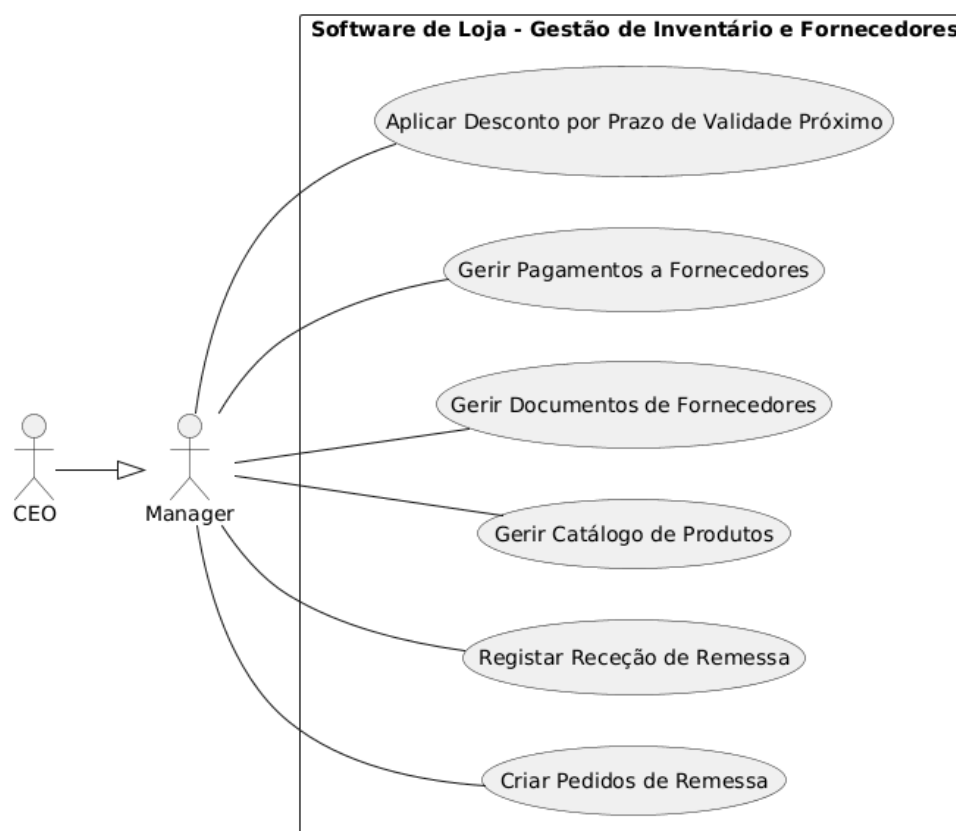


Figura 11: Diagrama de Casos de Uso: Gestão de Inventário e Fornecedores.

O terceiro diagrama representa as funcionalidades de carácter administrativo e de encerramento operacional diário, acessíveis ao Manager e à CEO. A gestão de utilizadores do sistema — onde o Manager gere os funcionários da sua loja e o CEO gere os managers do estabelecimento cujo terminal está a utilizar no momento — e o fecho de dia com envio de dados para o Servidor Central são aqui representados. Este diagrama evidencia o mecanismo central de responsabilização hierárquica do sistema TrasmUM, bem como o único momento de comunicação entre cada loja e o Servidor Central em toda a jornada operacional.

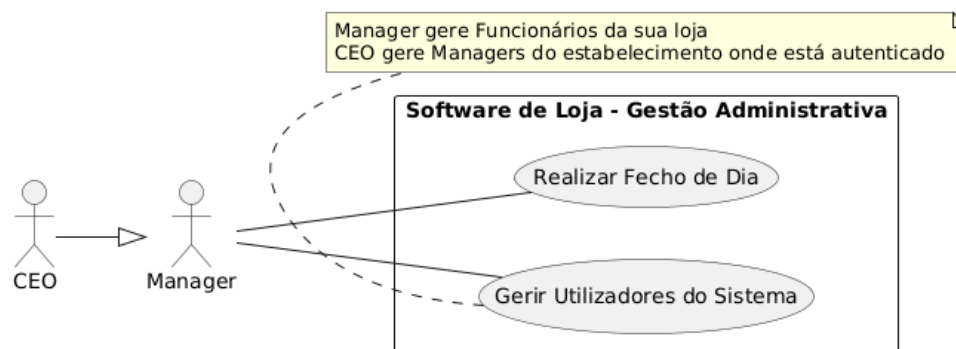


Figura 12: Diagrama de Casos de Uso: Gestão Administrativa.

O quarto diagrama delimita as funcionalidades exclusivas do perfil CEO, executadas no Servidor Central. Este sistema disponibiliza quatro casos de uso: o início de sessão no Servidor Central, que autentica a CEO

com credenciais próprias da base de dados central; a consulta do estado de consolidação da rede, que permite verificar quais as lojas cujo pacote de fecho de dia foi recebido para a data corrente; a consulta de *dashboards* e relatórios analíticos, com métricas globais de vendas e stocks; e a consulta de remessas da rede, com o estado consolidado dos pagamentos a fornecedores de todos os estabelecimentos. Este diagrama ilustra a assimetria intencional da arquitetura federada do TrasmUM: o Servidor Central recebe dados das lojas para fins de análise, mas não emite instruções nem interfere na operação autónoma de cada estabelecimento.

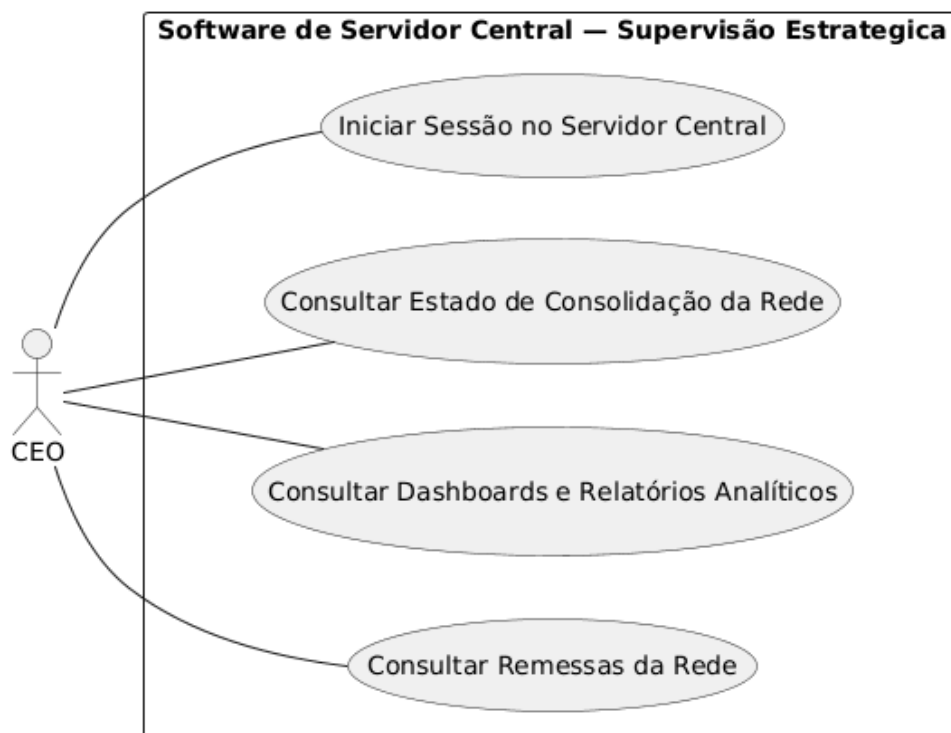


Figura 13: Diagrama de Casos de Uso: Servidor Central — Supervisão Estratégica.

Por fim, apresenta-se o diagrama global do sistema, que agrega numa única representação as duas fronteiras de sistema — o Terminal de Loja e o Terminal de Servidor Central —, os três perfis de utilizador e a totalidade dos catorze casos de uso especificados. Este diagrama não substitui os diagramas por área operacional, mas complementa-os ao oferecer uma visão de conjunto que permite compreender, de forma imediata, a distribuição de responsabilidades entre os atores e a separação entre os dois Ambientes da rede TrasmUM. É especialmente útil como instrumento de comunicação com os *stakeholders*, permitindo a Hélder e à Cátia validar, numa única leitura, que o âmbito do sistema cobre todas as necessidades operacionais e estratégicas identificadas durante o processo de levantamento de requisitos.²⁴

²⁴Texto gerado com apoio de IA — ver [Anexo de IA, Entrada 33](#).

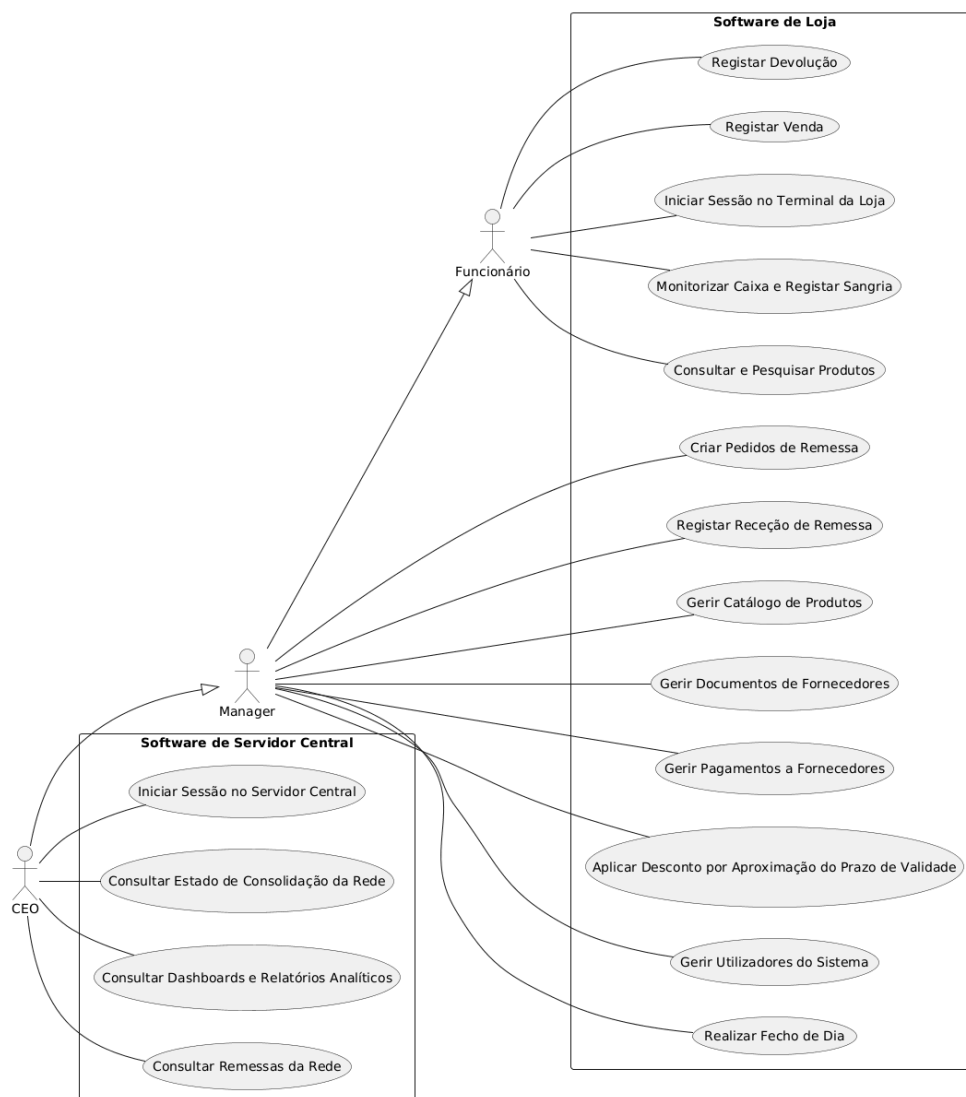


Figura 14: Diagrama de Casos de Uso: Visão Global do Sistema TrasmUM.

3.8. Diagramas de Máquinas de Estado

Os diagramas de máquinas de estado modelam explicitamente os estados possíveis de cada entidade relevante do sistema e as transições que ocorrem em resposta a eventos ou condições. De acordo com Ian Sommerville (Sommerville, 2015), as máquinas de estado são particularmente adequadas para descrever o comportamento de sistemas reativos, nos quais a resposta a eventos e a gestão de estados internos desempenham um papel central. Cada diagrama permite identificar não só os estados possíveis de uma entidade, mas também as regras que governam a sua evolução, tornando explícitas restrições, dependências e pontos críticos do sistema.

Esta representação visual facilita a validação dos requisitos com os *stakeholders* e melhora a compreensão global do sistema, ao evidenciar de forma clara como diferentes componentes — como sessões de utilizador, vendas, inventário ou processos operacionais — evoluem ao longo do seu ciclo de vida e interagem com os restantes elementos da arquitetura.²⁵

A máquina de estados da sessão de utilizador modela o ciclo de vida de uma sessão num terminal do Terminal de Loja da rede TrasmUM, desde o momento em que o terminal se encontra livre até ao encerramento da sessão. Os estados refletem diretamente as restrições impostas pelos RF1 e RF2: a

²⁵Texto e diagramas de máquinas de estado gerados com apoio de IA — ver [Anexo de IA, Entrada 24](#).

determinação das funcionalidades disponíveis com base no perfil autenticado, e o bloqueio temporário do terminal após tentativas de autenticação falhadas consecutivas. O fecho de dia é representado como um estado autónomo dado que bloqueia o encerramento da sessão até à confirmação da receção dos dados pelo Servidor Central, constituindo uma transição com semântica própria e não um simples encerramento de sessão.

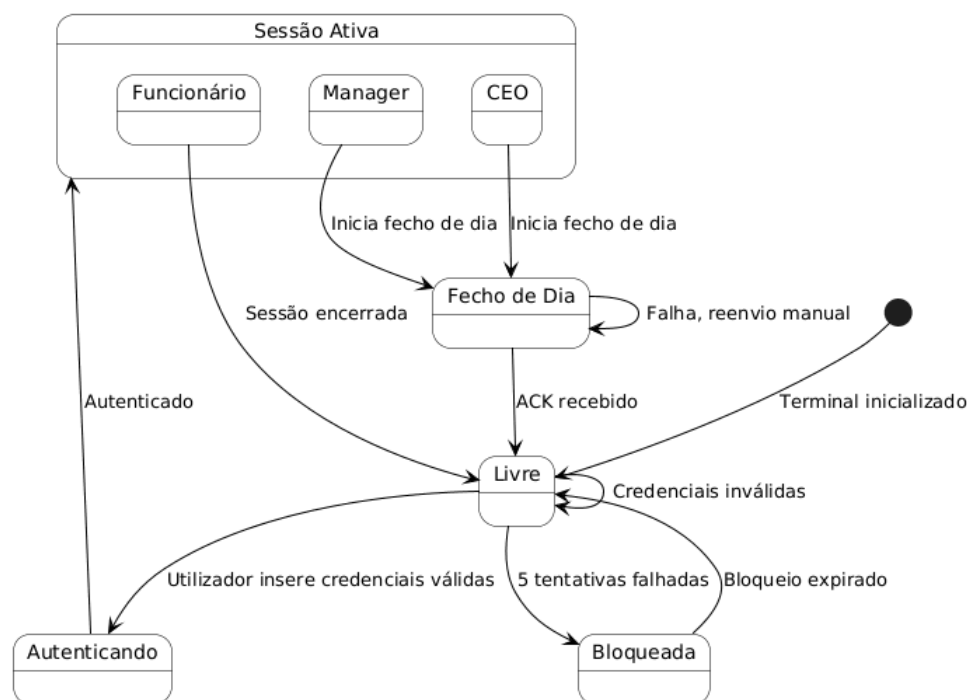


Figura 15: Diagrama de Máquina de Estado: Ciclo de Vida da Sessão do Utilizador no Terminal de uma Loja.

A máquina de estados da venda modela o processo transacional mais frequente do sistema, desde a abertura de uma nova transação até ao seu encerramento por anulação, conclusão com emissão de fatura, ou devolução posterior. Os estados intermédios refletem os momentos críticos especificados nos RF3, RF4, RF5 e RF11: a distinção entre uma transação em curso e uma transação aguardando pagamento determina quando o stock é afetado e quando a caixa é atualizada. No que respeita às devoluções, a decisão de aceitar ou recusar é sempre da responsabilidade do funcionário — a validade do produto não determina automaticamente o desfecho da devolução, mas condiciona o comportamento do sistema em caso de aceitação: se o produto estiver dentro da validade o stock é repostado, caso contrário a devolução é concluída sem reposição de stock. Uma devolução recusada pelo funcionário é igualmente tratada como estado final sem qualquer alteração ao inventário ou à caixa.

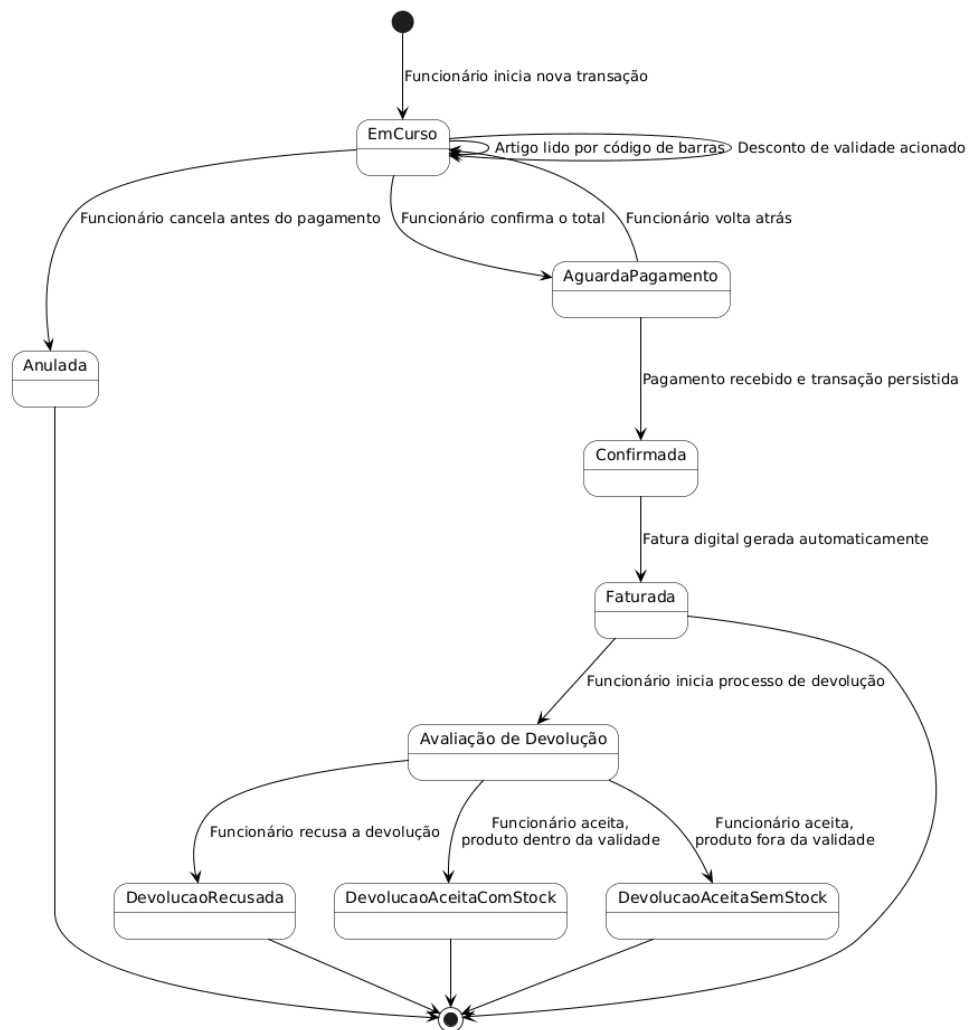


Figura 16: Diagrama de Máquina de Estado: Ciclo de Vida de uma Venda.

A máquina de estados do lote de produto modela o comportamento da entidade central da gestão de inventário do sistema TrasmUM, derivando diretamente dos requisitos RF6, RF10 e RF22. O RF6 estabelece que cada receção de remessa origina lotes independentes com código único e data de validade própria, sem sobrepôr as validades dos lotes pré-existentes — é este mecanismo que dá origem ao estado inicial Ativo. O RF10 determina que o sistema avalia a quantidade total do produto após cada venda e gera um alerta visual quando o stock desce ao limiar mínimo configurado, correspondendo ao estado EmAlertaStock. O RF22 especifica que uma rotina diária identifica os lotes cuja data de validade é inferior ou igual ao limiar configurável pelo manager, desencadeando o estado EmAlertaValidade e permitindo a transição para ComDesconto mediante ação do manager. Um lote pode percorrer trajetórias distintas consoante seja consumido pelas vendas antes de expirar, receba um desconto e seja depois escoado, ou expire com unidades ainda em stock.

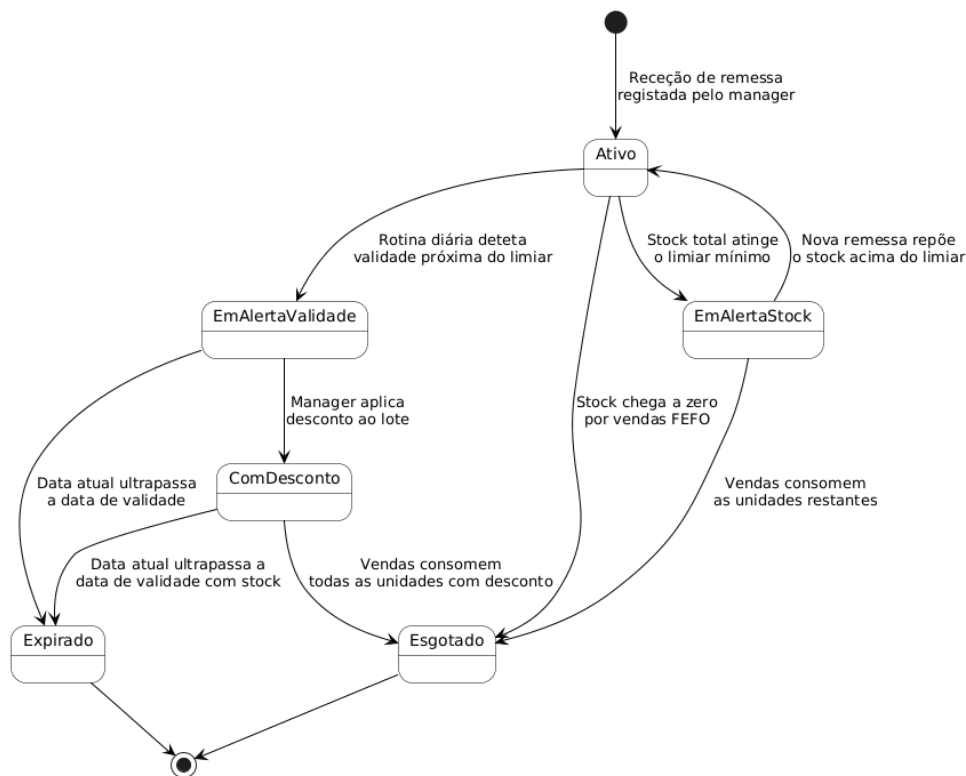


Figura 17: Diagrama de Máquina de Estado: Ciclo de Vida de um Lote de Produto.

A máquina de estados do pedido de remessa modela o processo de aprovisionamento formal de cada loja, conforme especificado no RF7. A introdução do estado SemPedidoPrevio reflete a decisão tomada durante a revisão de requisitos de cobrir explicitamente o cenário em que o manager regista a receção de uma remessa sem ter criado previamente um pedido no sistema — situação operacionalmente válida que não deve gerar erros nem impedir o registo da remessa. Os dois pontos de entrada distintos no diagrama ilustram precisamente esta bifurcação: o fluxo normal com pedido prévio e o fluxo de receção direta sem pedido associado.

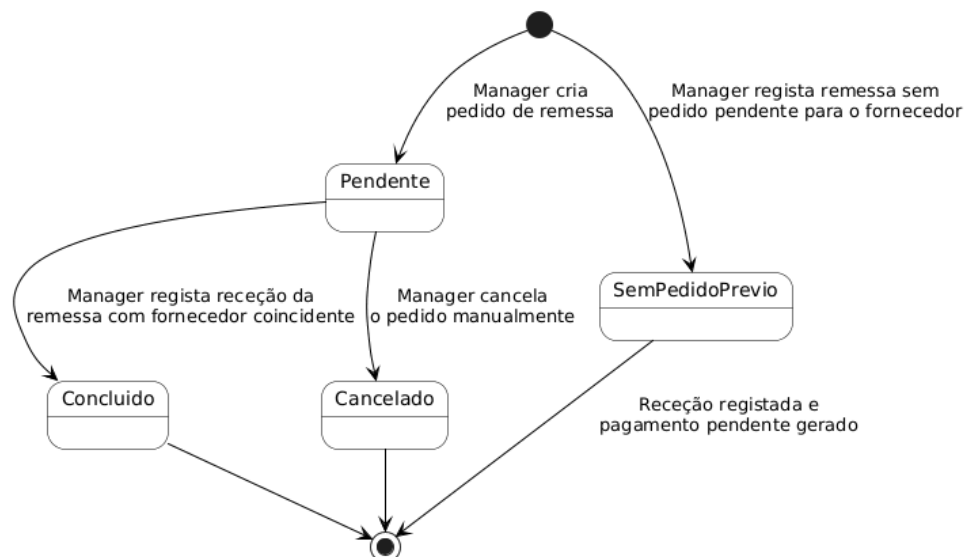


Figura 18: Diagrama de Máquina de Estado: Ciclo de Vida de um Pedido de Remessa.

A máquina de estados do pagamento a fornecedor modela a trajetória de um registo de pagamento desde a sua criação no terminal de loja até à sua visibilidade na base de dados central, derivando diretamente dos requisitos RF6, RF18 e RF19. O RF6 estabelece que o registo de uma remessa origina automaticamente um pagamento pendente associado ao fornecedor, para inclusão no pacote de fecho de

dia — correspondendo ao estado inicial PendenteLoja. O RF18 especifica que o manager pode seleccionar um pagamento pendente e registar a sua liquidação, atualizando o estado para “pago” na base de dados local, com obrigatoriedade de inclusão dessa alteração no pacote de fecho de dia — correspondendo à transição para PagoLoja. O RF19 determina que a CEO deve ter acesso a uma vista consolidada do estado de todas as remessas da rede, recebida via pacotes de fecho de dia — o que explica a distinção entre os estados do contexto da loja e os estados do contexto da rede, tornando explícito que a visibilidade central depende sempre da transmissão via fecho de dia, nunca de atualização em tempo real.

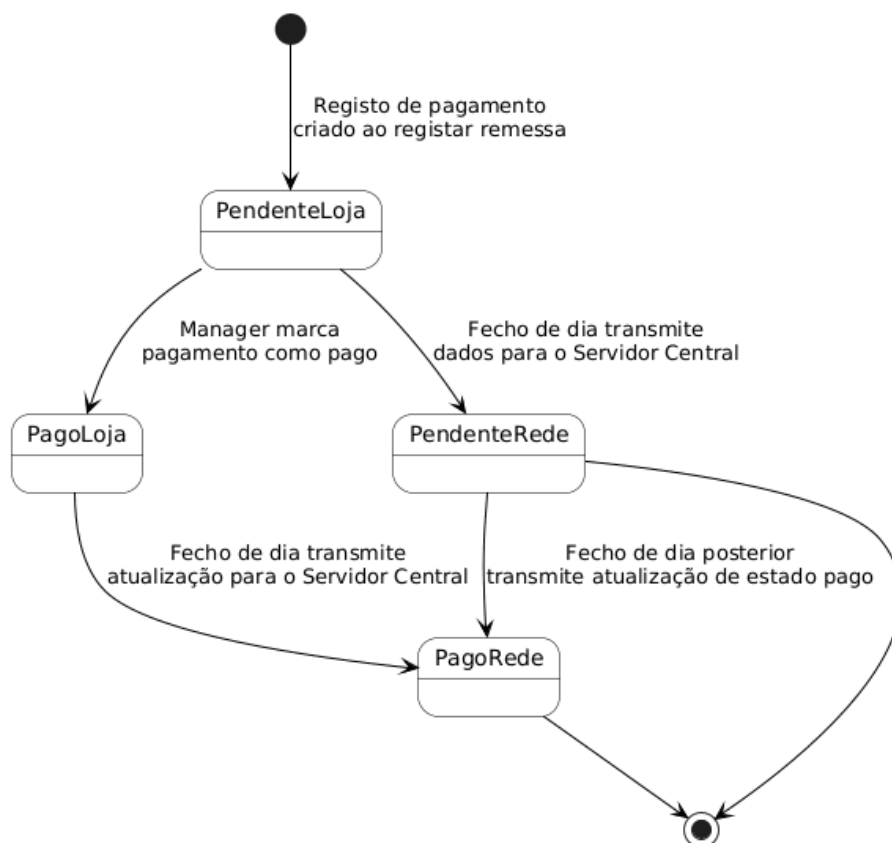


Figura 19: Diagrama de Máquina de Estado: Ciclo de Vida de um Pagamento a Fornecedor.

A máquina de estados do fecho de dia modela o processo de encerramento operacional diário, derivando diretamente dos requisitos RF15 e RF16. O RF16 estabelece que o manager ou CEO aciona o fecho de dia desencadeando o envio automático para o Servidor Central de todos os dados registados durante a jornada — vendas, devoluções, remessas e pagamentos pendentes —, agregados num pacote que cobre todos os registos desde o último fecho bem-sucedido. O RF15 determina que o fecho de dia só é concluído após confirmação de receção dos dados pelo Servidor Central, com possibilidade de reenvio manual em caso de falha de comunicação, e que o encerramento de sessão de um funcionário não desencadeia qualquer envio de dados, ficando estes retidos localmente até à ação do manager ou CEO. Este fluxo é simétrico para todas as lojas da rede.

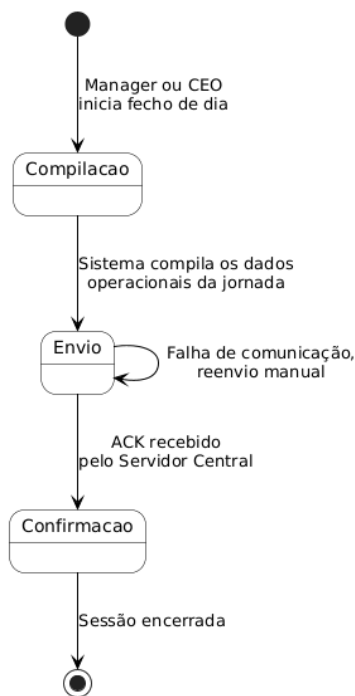


Figura 20: Diagrama de Máquina de Estado: Ciclo de Vida do Fecho de Dia.

3.9. Revisão e Aprovação do Projeto

A fase de oficialização desta segunda etapa culminou numa reunião formal com Cátia das Unhas Pintadas, Diretora-Geral da rede TrasmUM, destinada à revisão exaustiva e aprovação do trabalho desenvolvido. Durante esta sessão, procedeu-se à validação do levantamento de requisitos efetuado, da especificação formal dos requisitos funcionais e não funcionais, dos casos de uso e dos diagramas de máquinas de estado, garantindo que o conjunto de artefactos produzidos reflete com exatidão as necessidades operacionais e estratégicas da organização. A anuência da Diretora-Geral resultou na aprovação formal do projeto, conferindo a autorização necessária para que a equipa avance para a fase de Arquitetura e Design do Software.²⁶

²⁶Texto gerado com apoio de IA — ver [Anexo de IA, Entrada 34](#).



TrasmUM – Reunião 4

Data: 2026/03/20

Duração prevista: 30 minutos

Horário de início: 10h00

Horário de término: 10h30

Participantes: Cátia (unhasPintadas@trasmum.pt)
Gabriel Dantas (a107291@uminho.pt)
José Fernandes (a106937@uminho.pt)
Luís Ferreira (a98286@uminho.pt)
Simão Oliveira (a107322@uminho.pt)

Pauta da reunião:

- Revisão do levantamento de requisitos efetuado, incluindo a especificação formal dos requisitos funcionais e não funcionais.
- Revisão dos casos de uso e diagramas de máquinas de estado produzidos.
- Aprovação formal do trabalho desenvolvido na fase de Levantamento e Análise de Requisitos.

Figura 21: Ata da reunião de revisão e aprovação do levantamento de requisitos.

4. Arquitetura e Design do Software utilizando LLM

4.1. Utilização de LLMs na Fase de Arquitetura

À semelhança do que foi estabelecido nas fases anteriores, a utilização de LLMs nesta fase seguiu uma abordagem estruturada e iterativa, documentada através do *template* de registo de *prompts* definido na secção de Prompts Utilizados em anexo.

A interação com o LLM organizou-se em três momentos progressivos. Numa primeira fase de exploração aberta, foi fornecido ao modelo o relatório integral da fase anterior como contexto e solicitada a identificação dos padrões e estilos arquiteturais mais adequados ao sistema TrasmUM — produzindo uma análise fundamentada de alternativas que serviu de base para a decisão da equipa. Numa segunda fase de aprofundamento técnico, foram comunicadas ao modelo as decisões preliminares e solicitada a sua concretização em termos práticos: estrutura de *packages*, organização das camadas de cada software, mecanismo de *Delta Sync*, comportamentos reativos com JavaFX e integração do servidor HTTP Javalin. Numa terceira fase de revisão crítica, o modelo foi convidado a atuar como arquiteto revisor independente, identificando vulnerabilidades e cenários de falha não cobertos — algumas das observações resultantes conduziram a ajustes documentados nos ADRs.

4.2. Estrutura dos Softwares

O sistema TrasmUM é composto por dois softwares distintos e independentes, correspondentes aos dois contextos operacionais do sistema: o contexto de operação local de cada loja e o contexto de supervisão central da rede. A Figura 22 ilustra um esboço da topologia global do sistema.²⁷

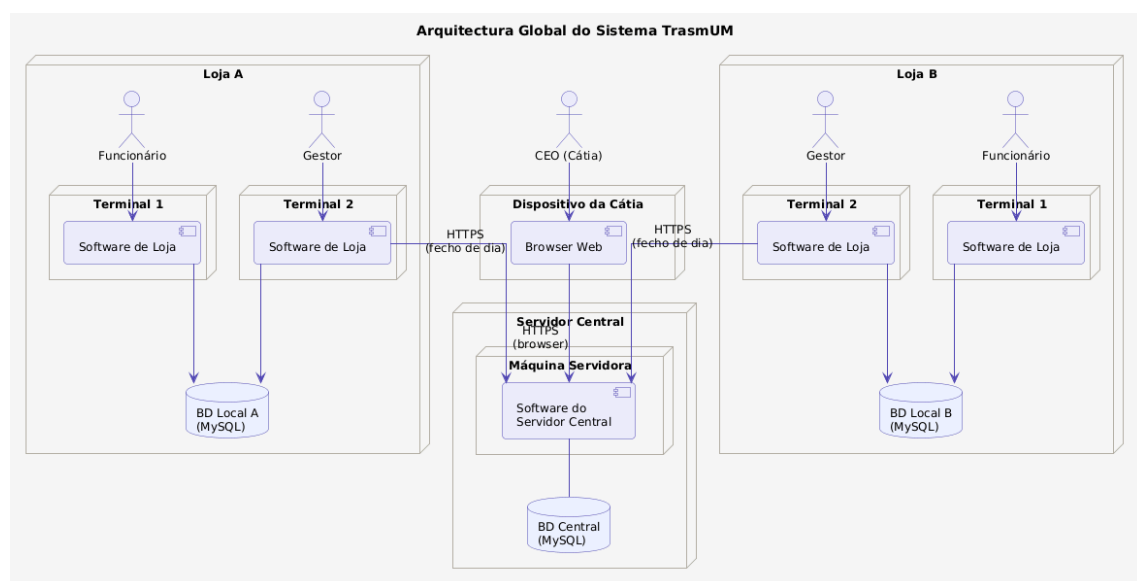


Figura 22: Arquitetura Global do Sistema TrasmUM

²⁷Diagrama gerado com o apoio de IA — ver [Anexo de IA, Entrada 45](#).

O Software de Loja é instalado em todos os terminais de cada estabelecimento e é responsável pela totalidade da operação diária. Este software disponibiliza a interface de caixa e suporta todas as funcionalidades operacionais, incluindo gestão de stock e lotes, processamento de vendas e devoluções, gestão de remessas e fornecedores, gestão do catálogo de produtos e utilizadores, e execução do fecho de dia com geração do respetivo pacote de sincronização. Cada instância do Software de Loja comunica exclusivamente com a base de dados local da respetiva loja, instalada numa máquina designada acessível através da rede interna do estabelecimento. Durante o horário de funcionamento, esta base de dados constitui a única fonte de verdade, não existindo qualquer dependência de conectividade externa.

O Software de Servidor Central é instalado exclusivamente na máquina que desempenha o papel de servidor central da rede. Este software é responsável pela receção dos pacotes de fecho de dia enviados pelas lojas, pela persistência dos dados na base de dados central agregada, e pela disponibilização de funcionalidades de consulta e análise, incluindo *dashboards* e relatórios consolidados. A interface de consulta e análise é disponibilizada ao CEO através de um browser web, tal como descrito no RNF18, sendo as páginas servidas diretamente pelo Javalin. A comunicação entre os dois softwares ocorre exclusivamente no momento do fecho de dia, sendo sempre iniciada pelo Software de Loja. O Servidor Central não inicia comunicação com as lojas em qualquer circunstância. Esta separação em dois softwares independentes permite que os seus ciclos de vida sejam completamente desacoplados, possibilitando atualizações e manutenção de cada componente sem impacto direto no outro, e estabelece uma fronteira clara entre os contextos operacionais do sistema.

A escolha de uma aplicação *desktop* para o Software de Loja decorre da sua natureza *offline-first*: o terminal tem de continuar operacional mesmo sem conectividade externa, o que torna uma solução web inviável sem mecanismos de sincronização adicionais. O Software de Servidor Central, por sua vez, está permanentemente online e serve um único utilizador com necessidades de consulta e análise — contexto em que uma interface web, acessível via browser sem instalação de software adicional, é a solução mais adequada e vai ao encontro do requisito RNF18.²⁸

4.3. Registos de Decisão Arquitetural

As decisões que sustentam a arquitetura do sistema são formalizadas através de registos de decisão arquitetural (ADR) (Nygard, 2011), um instrumento de documentação técnica que visa preservar não apenas as decisões tomadas, mas também o contexto, as alternativas consideradas e as consequências esperadas de cada escolha. Cada ADR segue a estrutura: contexto, decisão, alternativas consideradas, justificação e consequências. A totalidade dos registos encontra-se disponível no Anexo 8.

O ADR-01 estabelece a arquitetura de distribuição global do sistema. Perante a exigência de autonomia operacional plena de cada loja durante o horário de funcionamento, a equipa decidiu adotar uma arquitetura federada com cliente-servidor assimétrico, em que cada loja constitui uma unidade computacional completamente autónoma e a comunicação com o Servidor Central ocorre exclusivamente no momento do fecho de dia, sempre iniciada pela loja. As alternativas de arquitetura centralizada e *peer-to-peer* foram rejeitadas por violarem a autonomia exigida ou por introduzirem complexidade de sincronização desnecessária.

O ADR-02 decorre diretamente do ADR-01 e formaliza a separação em dois softwares distintos e independentes: o Software de Loja, instalado em todos os terminais da rede, e o Software de Servidor Central, instalado exclusivamente na máquina servidora. Esta separação física elimina o risco de acesso indevido a funcionalidades do servidor a partir de terminais de loja, permite ciclos de vida e atualizações independentes, e reduz a superfície de ataque de cada componente.

O ADR-03 define a estratégia de base de dados local de cada loja. Para suportar o acesso concorrente de múltiplos terminais do mesmo estabelecimento, cada loja dispõe de uma instância MySQL centralizada na rede local, partilhada por todos os seus terminais via TCP/IP nativo. A alternativa de base de dados independente por terminal foi rejeitada por exigir um mecanismo de sincronização interna complexo.

²⁸Texto gerado com apoio de IA — ver [Anexo de IA, Entrada 30](#).

O ADR-04 especifica o mecanismo de *Delta Sync* com três estados de sincronização — PENDENTE, EM_TRANSITO e CONFIRMADO — implementado em todas as tabelas operacionais do Software de Loja. Este mecanismo garante que cada pacote de fecho contém exclusivamente os registos ainda não confirmados pelo Servidor Central, com reversão automática de registos em trânsito em caso de falha de comunicação. A marcação como CONFIRMADO ocorre exclusivamente após receção do ACK do servidor.

O ADR-05 complementa o ADR-04 ao definir a estratégia de idempotência no Servidor Central. Todas as tabelas centrais impõem uma restrição de unicidade composta por idLoja e pelo identificador original do registo na loja de origem, com inserções tratadas via INSERT ... ON DUPLICATE KEY UPDATE. Desta forma, o reenvio do mesmo registo em pacotes distintos não produz duplicação de dados, tornando o sistema tolerante a falhas de comunicação sem exigir coordenação adicional.

O ADR-06 resolve o problema de *bootstrapping* presente em qualquer instalação inicial: sem utilizadores registados, não é possível autenticar para criar o primeiro utilizador. A decisão adotada consiste na criação automática, pelo *script* de instalação, de uma conta temporária com perfil CEO, eliminada após a criação da primeira conta definitiva. Este mecanismo aplica-se de forma independente ao Software de Loja e ao Software de Servidor Central.

O ADR-07 determina que os *logs* de auditoria produzidos em cada loja são incluídos no pacote de fecho de dia e ficam disponíveis na base de dados central para consulta pela CEO. A transmissão em tempo real foi rejeitada por violar o princípio de comunicação exclusiva no fecho de dia, e a retenção exclusivamente local privaria a CEO de visibilidade sobre ações críticas da rede.

O ADR-08 estabelece o uso de HTTPS com certificado auto-assinado para a comunicação do fecho de dia. A cifra da comunicação protege dados financeiros e de auditoria sem custos técnicos significativos, recorrendo exclusivamente a ferramentas nativas da JVM e do Javalin. A consequência operacional relevante é a necessidade de importar o certificado do servidor para o *truststore* de cada terminal durante a instalação, passo integrado no procedimento de *bootstrapping* do ADR-06.

O ADR-09 adota a *Layered Architecture* com MVC como padrão de organização interna de ambos os softwares, com quatro camadas horizontais — Apresentação, Serviço, Repositório e Domínio — e dependências estritamente unidirecionais.

O ADR-10 formaliza a *stack* tecnológica adotada: Java como linguagem em ambos os softwares, JavaFX como *framework* de interface no Software de Loja, Javalin como servidor HTTP no Software de Servidor Central, e MySQL como sistema de gestão de base de dados em ambos os componentes, com acesso via JDBC. O Javalin foi preferido ao *Spring Boot* pela sua leveza e ausência de *container* de inversão de controlo, adequada a um servidor de receção passiva. O MySQL foi preferido ao SQLite pela sua arquitetura cliente-servidor nativa, necessária para o acesso concorrente definido no ADR-03.²⁹

4.4. Diagrama de Componentes - Software de Loja

O Software de Loja segue a arquitetura em camadas definida no ADR-09, com dependências estritamente unidirecionais e interfaces em todas as fronteiras.

A camada de Apresentação é composta pelos *controllers* JavaFX, que capturam eventos da interface gráfica e delegam toda a lógica no serviço correspondente. Cada *controller* depende exclusivamente da interface do serviço que consome — por exemplo, o VendaController requer IVendaServico e ICaixaServico, enquanto o FechoDiaController requer apenas IFechoDiaServico. Os *controllers* não se conhecem entre si nem acedem a qualquer outra camada diretamente.

A camada de Serviços constitui o núcleo da lógica aplicacional. Cada serviço expõe uma interface pública (prefixada com I) e consome interfaces de repositórios da camada inferior para persistir e recuperar dados. Dois serviços transversais — AutorizacaoServico e AuditoriaServico — são consumidos por praticamente todos os restantes serviços para verificação de permissões e registo de ações. Por questões

²⁹Texto e tabelas de ADRs gerados com apoio de IA — ver [Anexo de IA, Entrada 31](#).

de legibilidade, as ligações a estes dois componentes transversais foram omitidas do diagrama, sendo indicadas por uma nota UML; todos os serviços dependem de `IAutorizacaoServico` e `IAuditoriaServico`.

Um caso particular nesta camada é o `FechoDiaServico`, responsável por orquestrar a sincronização com o Servidor Central. Em vez de aceder diretamente aos repositórios de cada módulo para compilar o pacote de dados pendentes, este serviço consome as interfaces dos restantes serviços da mesma camada — `IVendaServico`, `IDevolucaoServico`, `ICaixaServico`, `IRemessaServico` e `IAuditoriaServico` — invocando os respetivos métodos de obtenção de registos pendentes. O único repositório a que acede diretamente é o `FechoDiaRepositorio`, para persistir o próprio registo de fecho.

A camada de Dados segue o padrão Repository (ADR-09). Todos os repositórios dependem de uma conexão JDBC ao MySQL local da loja. As interfaces definem contratos de persistência uniformes — operações de escrita, leitura por critérios específicos e listagem de registos pendentes de sincronização — que são realizados pelas implementações concretas sem expor detalhes de SQL à camada de serviços.

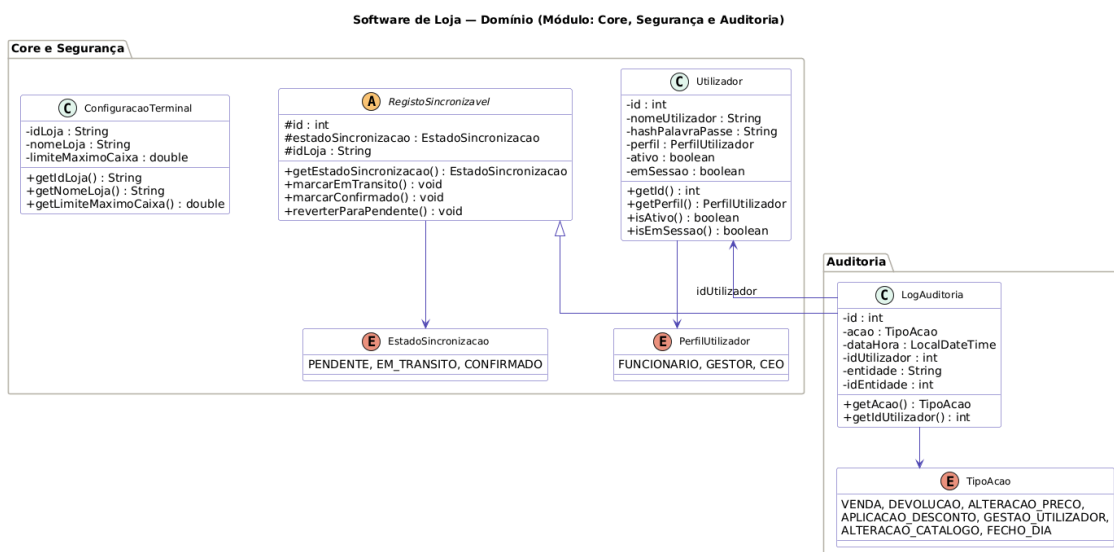
O diagrama completo de componentes encontra-se disponível no Anexo 9, permitindo visualizar a totalidade das dependências entre componentes e a comunicação através de interfaces em todas as fronteiras do sistema.³⁰

4.5. Diagramas de Classe - Software de Loja

Os diagramas de classe são apresentados em abordagem *bottom-up*, do domínio aos *controllers*.³¹

Inicialmente, a camada de domínio estrutura-se em quatro módulos funcionais, partindo das entidades transversais ao sistema até aos módulos operacionais específicos. Um elemento estruturante desta camada é a classe abstrata `RegistoSincronizavel`, da qual herdam todas as entidades transmitidas ao Servidor Central no fecho do dia — vendas, devoluções, remessas, pagamentos, sessões de caixa, fechos de dia e *logs* de auditoria —, adquirindo um campo `estadoSincronizacao` com os três estados definidos no ADR-04.

O módulo de Core e Segurança define as entidades transversais ao sistema. A classe `Utilizador` modela os operadores do terminal, associados a um `PerfilUtilizador` (funcionário, gestor ou CEO) que determina as permissões de acesso. A `ConfiguracaoTerminal` armazena os parâmetros específicos da loja, como o identificador, o nome e o limite máximo de numerário em caixa. O sub-módulo de Auditoria define o `LogAuditoria`, que regista cada ação relevante no sistema — vendas, devoluções, alterações de catálogo, entre outras — com referência ao utilizador que a executou, a entidade afetada e a data/hora.



³⁰ Consultoria arquitetural para diagramas de componentes com apoio de IA — ver [Anexo de IA, Entrada 40](#).

³¹ Geração e revisão dos diagramas de classe do Software de Loja com apoio de IA — ver [Anexo de IA, Entrada 37](#).

Figura 23: Diagrama de classes — Domínio: Core, Segurança e Auditoria.

O módulo de Catálogo, Inventário e Fornecedores modela o ciclo de vida dos produtos comercializados. Um Produto representa um artigo no catálogo, com o respetivo código de barras, preço base e stock mínimo. Cada entrada física em stock é representada por um Lote, que referencia um produto por identificador e contém a quantidade disponível, a data de validade e o preço de venda. Um lote pode ter um Desconto associado, modelado como um *value object* com a percentagem e a data de aplicação. Do lado do aprovisionamento, o Fornecedor agrega a informação de contacto e dados bancários. O PedidoRemessa regista intenções de encomenda com as respetivas linhas, enquanto a Remessa representa a receção efetiva de mercadoria, gerando novos lotes. O Pagamento e a Remessa, que herdam de RegistoSincronizavel, registam operações relevantes sobre fornecedores, respetivamente, os valores pagos e a receção de mercadoria, sendo incluídos no pacote de sincronização do fecho do dia.

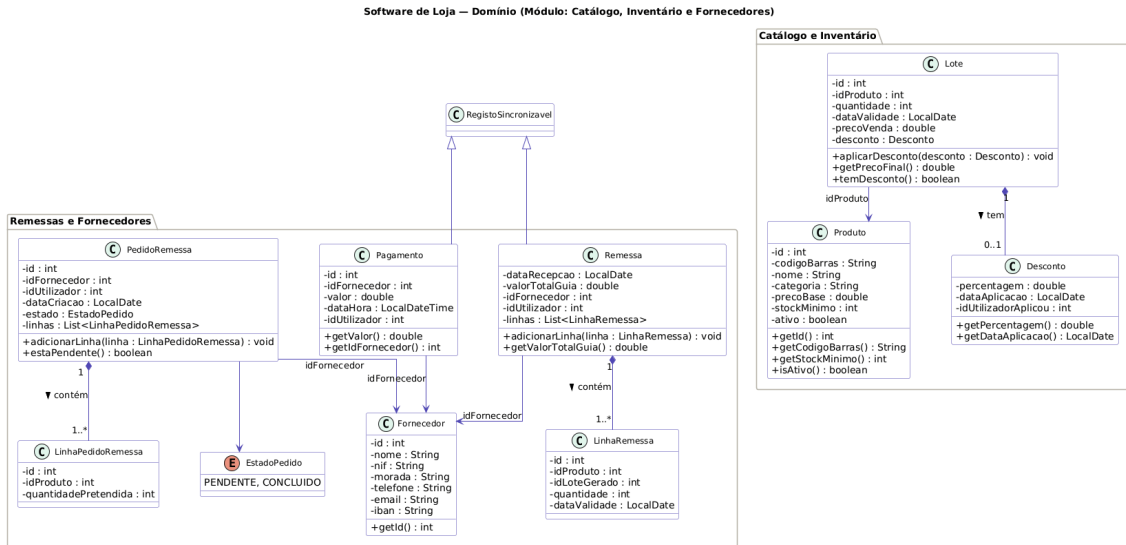


Figura 24: Diagrama de classes — Domínio: Catálogo, Inventário e Fornecedores.

O módulo de Vendas e Faturas modela o processo comercial. A Venda é o *aggregate root* deste módulo, composta por uma ou mais LinhaVenda (cada uma referenciando um lote e a quantidade vendida) e, opcionalmente, uma Fatura emitida com o NIF do cliente. O ciclo de vida da venda é controlado pelo EstadoVenda — em curso, finalizada ou anulada — e o método de pagamento distingue entre numerário e multibanco. A Devolucao referencia uma fatura existente e regista a quantidade devolvida juntamente com a data de validade da embalagem. Tanto Venda como Devolucao herdam de RegistoSincronizavel.

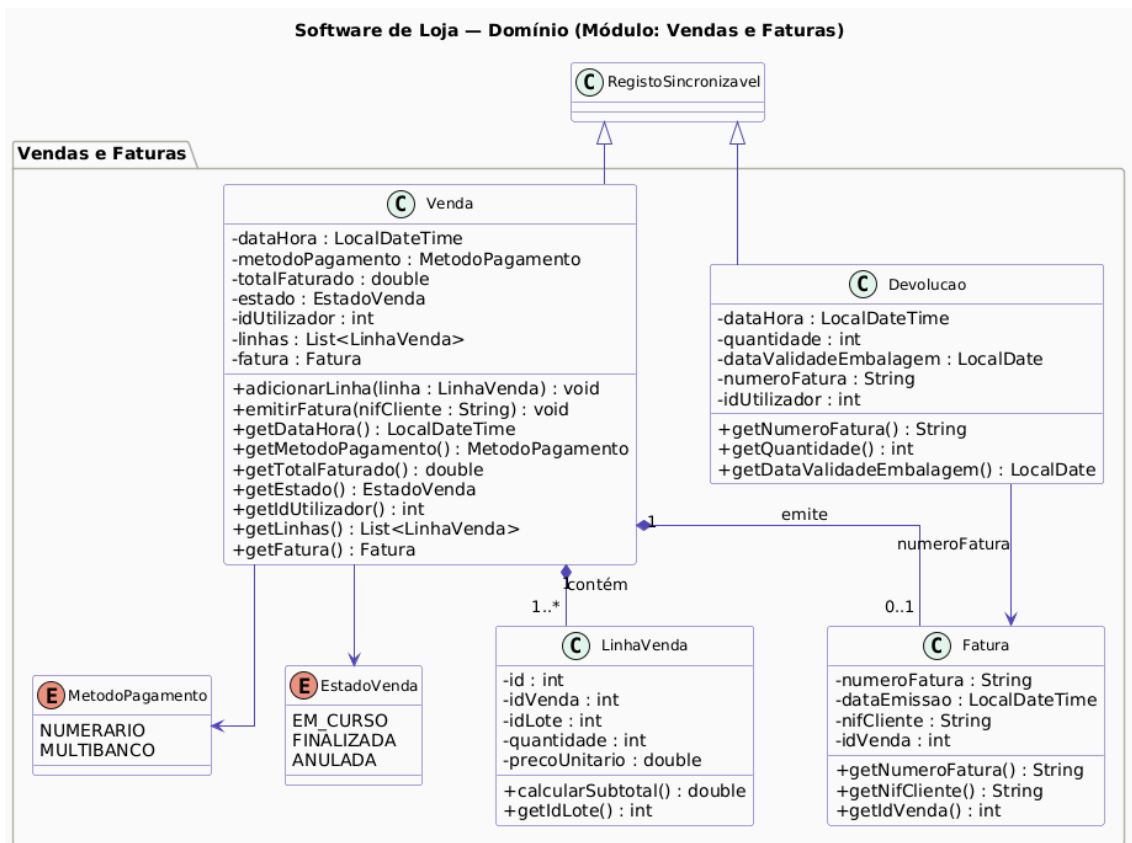


Figura 25: Diagrama de classes — Domínio: Vendas e Faturas.

O módulo de Tesouraria e Caixa modela a gestão de numerário. A *SessaoCaixa* regista a abertura de uma sessão com o respetivo fundo inicial, detalhado por denominação de moeda/nota através de *DetalheNumerario*. Durante a sessão, podem ser efetuadas *Sangrias* para retirar excesso de numerário. A *SessaoCaixa* herda de *RegistoSincronizavel*, sendo incluída no pacote de sincronização. O *FechoDia* marca o encerramento operacional do dia e herda de *RegistoSincronizavel* não por transportar dados operacionais, mas para que o seu estado de envio seja rastreável — permitindo ao sistema distinguir fechos confirmados pelo Servidor Central de fechos pendentes de reenvio.

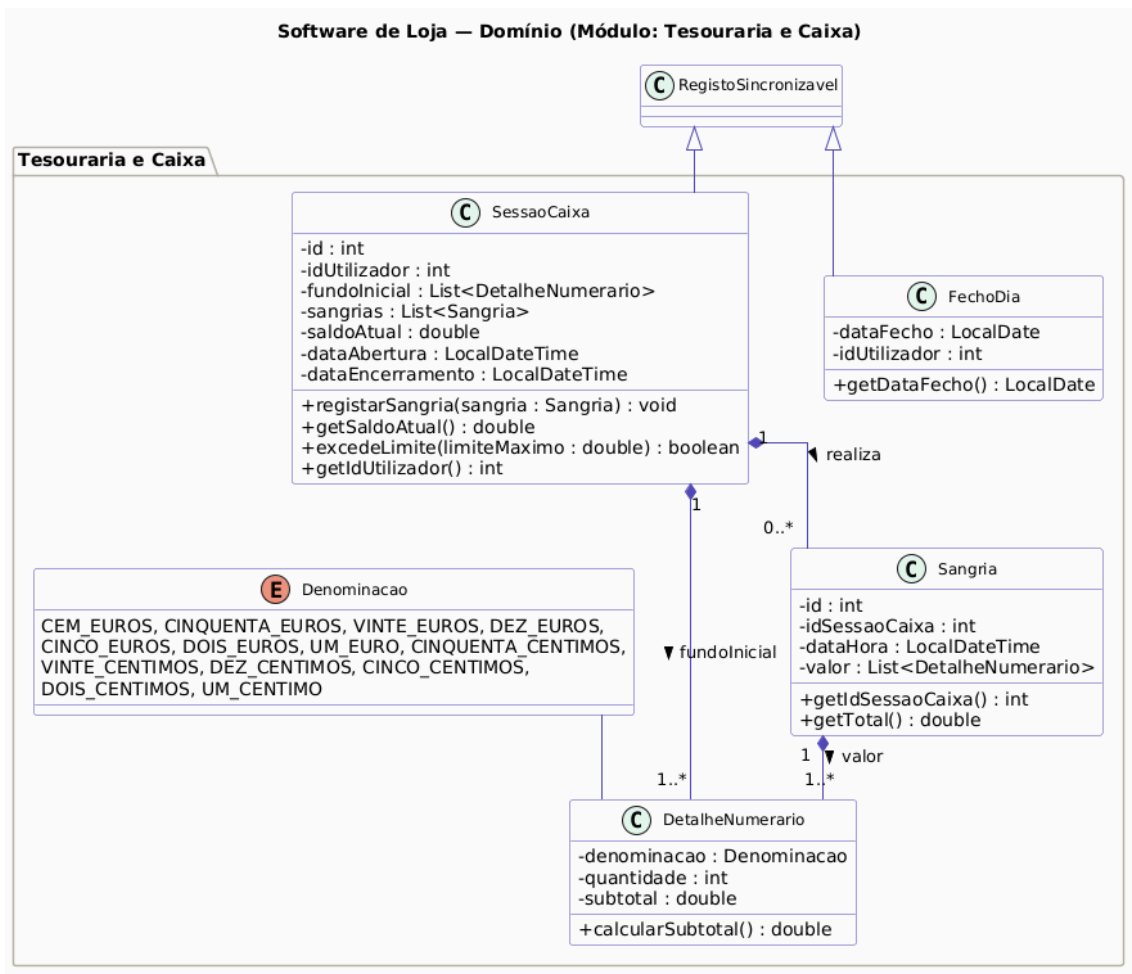


Figura 26: Diagrama de classes — Domínio: Tesouraria e Caixa.

As implementações da camada de Dados são responsáveis por traduzir as operações de domínio em SQL, incluindo a persistência de agregados compostos. As implementações encapsulam a decomposição de agregados compostos (ex: venda em cabeçalho, linhas e fatura), sem expor essa complexidade à camada de serviços. Todos os repositórios de entidades sincronizáveis expõem um método `buscarPendentes()`, que retorna os registos em estado PENDENTE ou EM_TRANSITO, e um método `atualizar()` que permite alterar o estado de sincronização. Esta uniformidade é essencial para o funcionamento do processo de fecho do dia.

O módulo de Core e Segurança disponibiliza o `UtilizadorRepositorio`, com operações de autenticação por credenciais, gestão do ciclo de vida dos utilizadores e verificação de sessões ativas, e o `LogAuditoriaRepositorio`, que persiste e recupera os registos de auditoria pendentes.

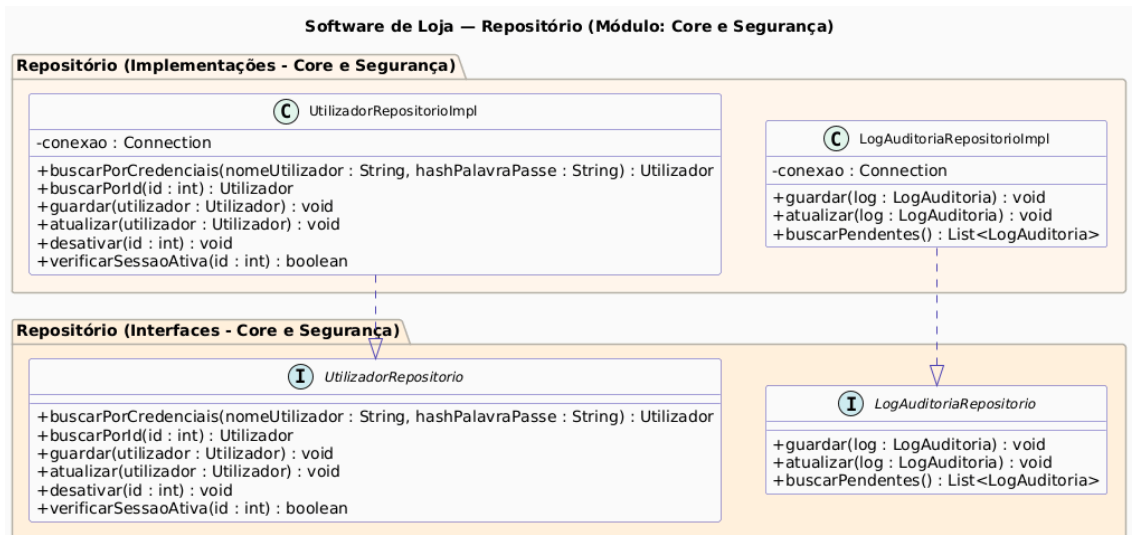


Figura 27: Diagrama de classes — Repositório: Core e Segurança.

O módulo de Catálogo, Inventário e Fornecedores define cinco interfaces de repositório. O ProdutoRepositorio permite pesquisar produtos por código de barras e listar os que se encontram abaixo do stock mínimo. O LoteRepositorio suporta a estratégia FEFO, retornando lotes ordenados por data de validade. Os repositórios de Remessa, PedidoRemessa e Pagamento completam a persistência deste módulo, com as respetivas implementações a gerir internamente as linhas de cada agregado.

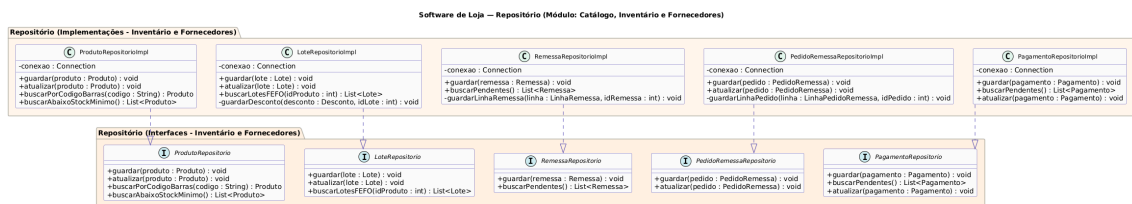


Figura 28: Diagrama de classes — Repositório: Catálogo, Inventário e Fornecedores.

O módulo de Vendas e Devoluções disponibiliza o VendaRepositorio, com pesquisa por identificador e por número de fatura, listagem de pendentes e marcação em lote de vendas confirmadas, e o DevolucaoRepositorio, com pesquisa por fatura e listagem de devoluções pendentes.

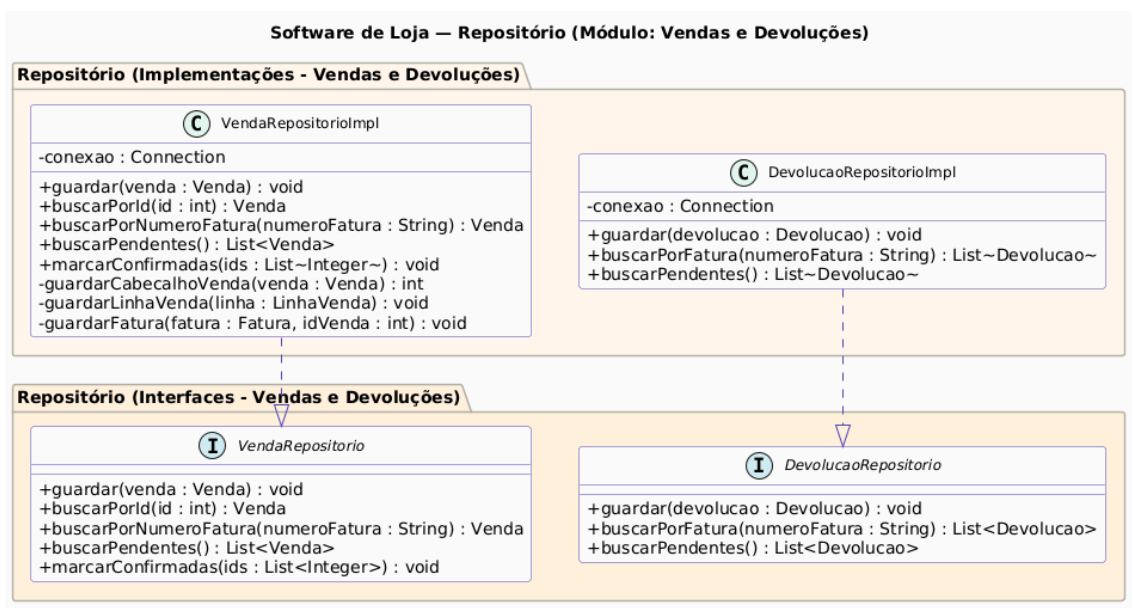


Figura 29: Diagrama de classes — Repositório: Vendas e Devoluções.

O módulo de Tesouraria e Caixa define o *SessaoCaixaRepositorio*, que permite pesquisar a sessão ativa de um utilizador e listar sessões pendentes de sincronização, e o *FechoDiaRepositorio*, com pesquisa do último fecho confirmado e listagem de fechos pendentes para reenvio.

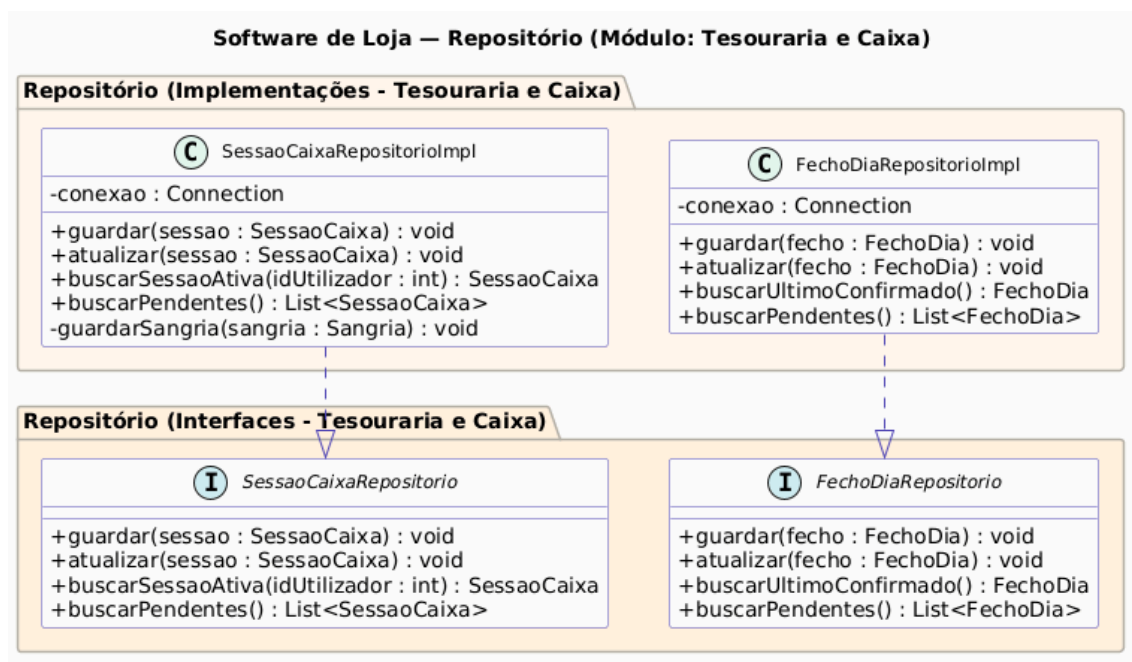


Figura 30: Diagrama de classes — Repositório: Tesouraria e Caixa.

A camada de serviços organiza-se em módulos funcionais alinhados com os da camada de domínio. O *AutorizacaoServico* verifica se o utilizador autenticado possui o perfil necessário para executar uma determinada operação, lançando *AcessoNegadoException* em caso de violação. O *AuditoriaServico* regista cada ação relevante no *log* de auditoria e expõe um método *obterLogsPendentes()* para o processo de sincronização. O *AutenticacaoServico* gere o ciclo de vida das sessões — autenticação por credenciais, encerramento de sessão e deteção de sessões duplicadas — delegando no repositório de utilizadores e registando os eventos no serviço de auditoria.

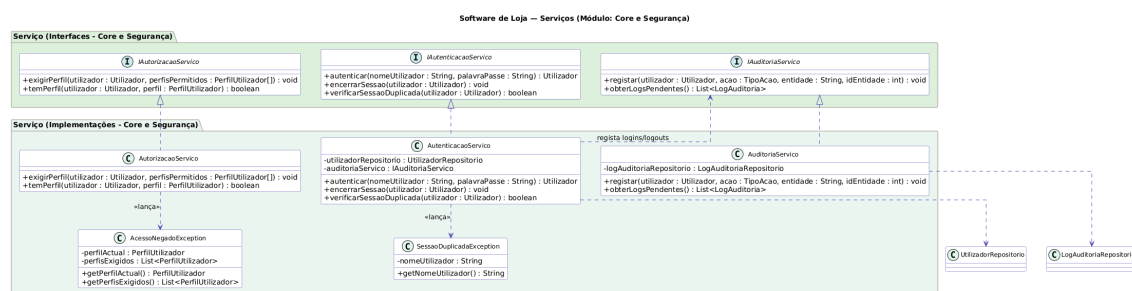


Figura 31: Diagrama de classes — Serviços: Core e Segurança.

O *CatalogoServico* gere o ciclo de vida dos produtos e lotes. Disponibiliza operações de criação, edição e desativação de produtos, aplicação de descontos a lotes, e geração de alertas de stock mínimo e de proximidade de validade. Todas as operações que modificam o catálogo verificam as permissões do utilizador e são registadas na auditoria.

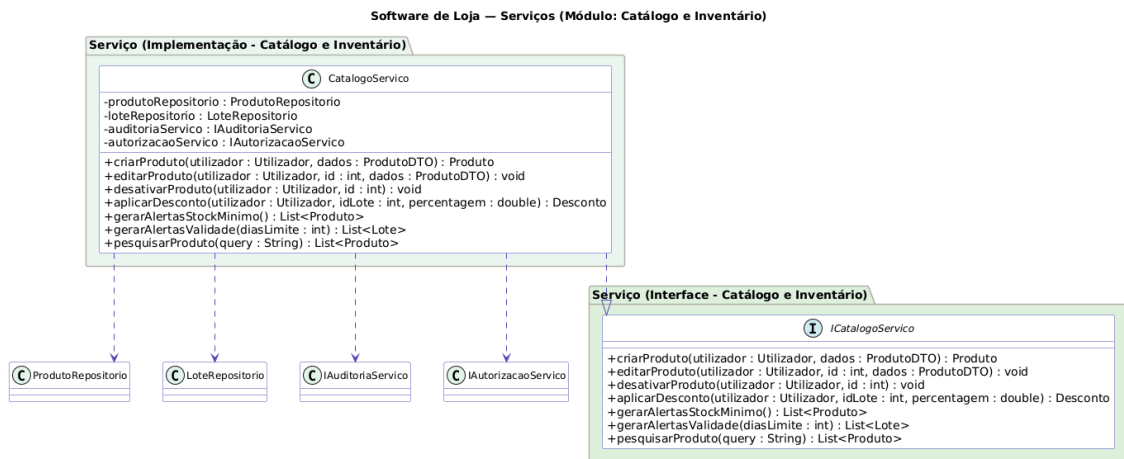


Figura 32: Diagrama de classes — Serviços: Catálogo e Inventário.

Os serviços de Vendas e Devoluções implementam os dois fluxos comerciais centrais. O VendaService conduz o processo de venda desde a iniciação, passando pela adição de linhas (com resolução de lotes via código de barras), até à finalização com emissão de fatura e cálculo de troco, ou anulação. O DevolucaoService processa devoluções validando a existência da fatura original e a quantidade devolvida, repondo o stock no lote correspondente. Ambos expõem métodos obterPendentes() para o processo de sincronização.

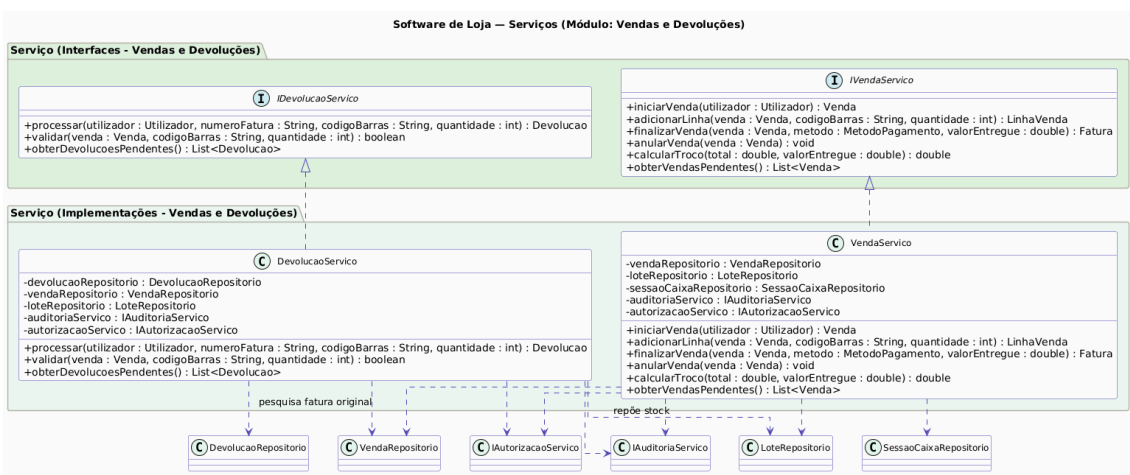


Figura 33: Diagrama de classes — Serviços: Vendas e Devoluções.

O CaixaService gere o ciclo de vida das sessões de caixa — abertura com fundo inicial, registo de sangrias, verificação do limite máximo de numerário e encerramento. Expõe igualmente o método obterSessoesPendentes() para a sincronização.

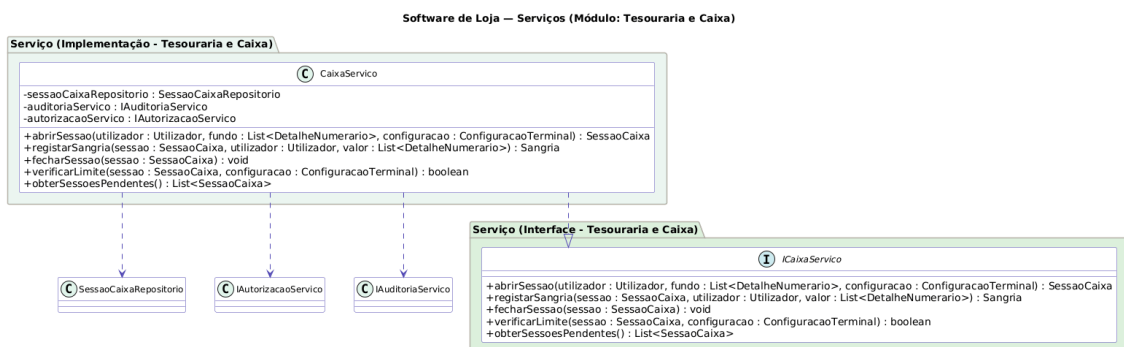


Figura 34: Diagrama de classes — Serviços: Tesouraria e Caixa.

O módulo de Gestão e Backoffice agrupa os serviços de suporte operacional. O RemessaServico coordena o registo de remessas recebidas (criando os lotes correspondentes e agendando o pagamento ao fornecedor), a criação de pedidos de remessa e a listagem por estado. Expõe obterRemessasPendentes() e obterPagamentosPendentes() para a sincronização. O PagamentoServico permite listar e liquidar pagamentos a fornecedores. O UtilizadorServico disponibiliza a gestão de contas — criação, edição e desativação — restrita a utilizadores com perfil de gestor ou CEO.

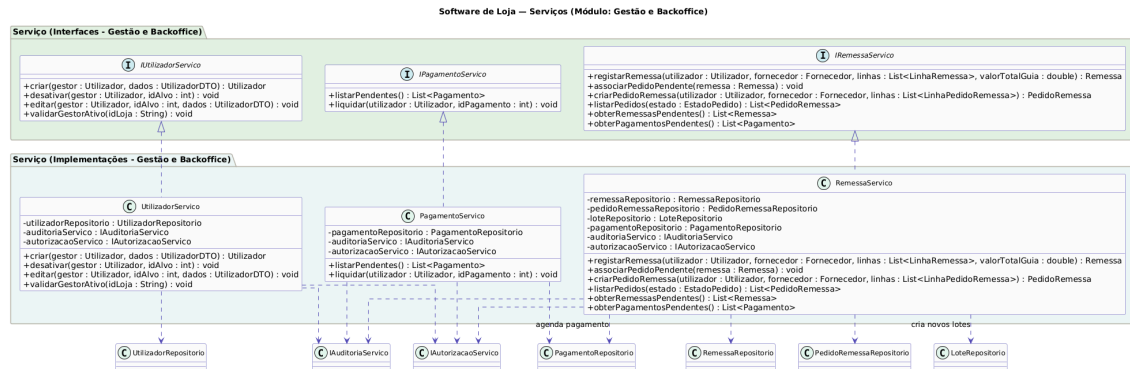


Figura 35: Diagrama de classes — Serviços: Gestão e BackOffice.

O FechoDiaServico orquestra o processo de sincronização com o Servidor Central. Conforme o Diagrama de Componentes, consome os restantes serviços para obter registos pendentes, acedendo diretamente apenas ao FechoDiaRepositorio.

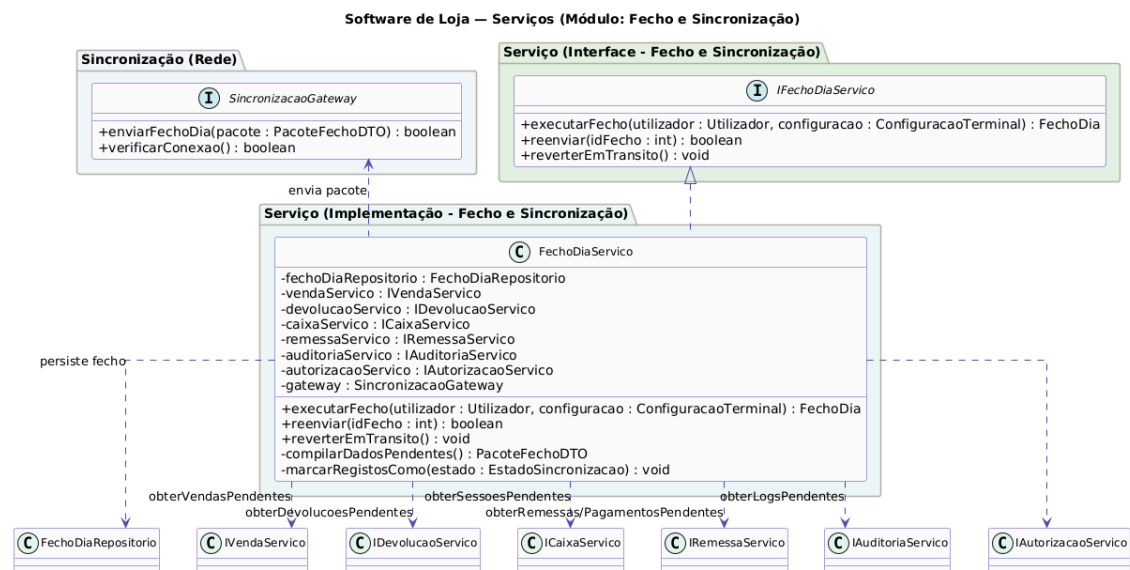


Figura 36: Diagrama de classes — Serviços: Fecho e Sincronização.

A camada de apresentação é composta por dois grupos de *controllers* JavaFX, organizados por perfil de utilização.

Os *controllers core* cobrem as operações do dia-a-dia: o AutenticacaoController gere o fluxo de *login* e *logout*, o VendaController conduz o processo de venda desde a leitura de códigos de barras até à finalização, o DevolucaoController processa devoluções por número de fatura, o CatalogoController disponibiliza a gestão de produtos e descontos, e o CaixaController controla a abertura e encerramento de sessões de caixa.

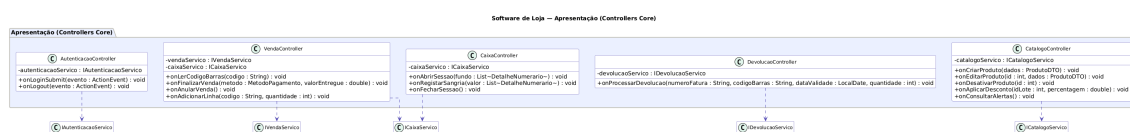


Figura 37: Diagrama de classes — Apresentação: Controllers Core.

Os *controllers* de gestão suportam as operações de *backoffice*: o RemessaController gere o registo de remessas e pedidos, o PagamentoController controla a liquidação de pagamentos a fornecedores, o FechoDiaController permite iniciar o fecho do dia e reenviar pacotes falhados, e o UtilizadorController disponibiliza a gestão de contas de utilizador.

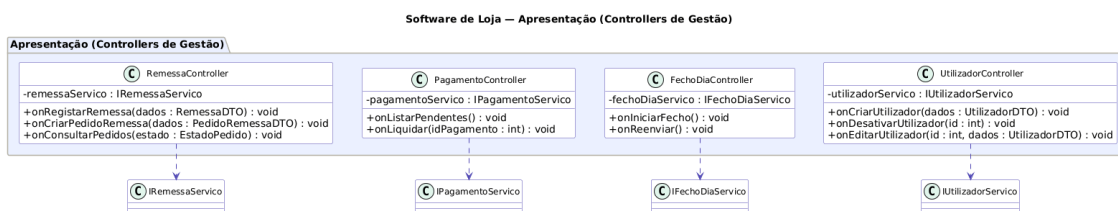


Figura 38: Diagrama de classes — Apresentação: Controllers de Gestão.

4.6. Diagrama de Componentes - Software do Servidor Central

O Servidor Central segue a mesma arquitetura em camadas definida no ADR-09, distinguindo-se do Software de Loja na camada de Apresentação pela substituição dos *controllers* JavaFX por *handlers* HTTP Javalin que respondem a pedidos REST. Cada *handler* mapeia um ou mais *endpoints* a operações do serviço correspondente, recebendo e devolvendo dados serializados em JSON.³²

O *RecepcaoPacoteHandler* é o ponto de entrada para os pacotes de fecho enviados pelas lojas — recebe o *PacoteFechoDTO* via HTTP POST e delega a sua persistência no serviço de receção. Os restantes *handlers* servem o painel do CEO: *dashboard* com métricas globais e por loja, geração de relatórios, consulta de remessas na rede e verificação do estado de consolidação.

A camada de Serviços divide-se em dois grupos funcionais. O primeiro, de Autenticação e Receção, trata da autenticação do CEO (com mecanismo de bloqueio temporário após tentativas falhadas) e da ingestão dos pacotes de fecho. O *RecepcaoPacoteServico* é o componente mais conectado do sistema, dependendo de oito interfaces de repositório para persistir todas as entidades do pacote. Esta concentração de dependências é justificada pela natureza do serviço — é um ingestor que decompõe o pacote e persiste cada tipo de entidade diretamente, sem lógica de negócio intermédia que justificasse a criação de serviços adicionais. O segundo grupo, de Análise, agrega os serviços de consulta: o *DashboardServico* calcula métricas a partir das vendas e devoluções agregadas, o *RelatorioServico* gera relatórios filtrados por loja, período e categoria, o *RemessaRedeServico* expõe a consulta de remessas na rede, e o *ConsolidacaoServico* verifica se todas as lojas enviaram o fecho para uma determinada data.

O diagrama completo de componentes, apresentado no Anexo 10, ilustra estas dependências.

4.7. Diagramas de Classe - Software do Servidor Central

Ao contrário do Software de Loja, cujo domínio modela entidades mutáveis com ciclos de vida complexos, o domínio do Servidor Central é orientado à leitura — as entidades recebem dados já processados e expõem-nos para análise, sem lógica de negócio intermédia que justifique serviços de orquestração ricos. Esta diferença manifesta-se na ausência de hierarquias como *RegistoSincronizavel*, num conjunto de serviços de análise em vez de serviços operacionais, e em *value objects* dedicados à agregação de métricas. A apresentação segue novamente uma abordagem *bottom-up*, partindo do domínio até aos *handlers* HTTP.³³

A camada de domínio do Servidor Central organiza-se em quatro módulos distintos: entidades core, entidades transacionais, *value objects* de análise e DTOs de transferência. As entidades core modelam

³²Consultoria arquitetural para diagramas de componentes com apoio de IA — ver [Anexo de IA, Entrada 40](#).

³³Geração e revisão dos diagramas de classe do Servidor Central com apoio de IA — ver [Anexo de IA, Entrada 39](#).

os conceitos estruturais do servidor. O CEO representa o único perfil com acesso ao sistema, incluindo o mecanismo de bloqueio temporário após tentativas de autenticação falhadas. A Loja é uma entidade leve, criada automaticamente quando o servidor recebe o primeiro pacote de uma loja desconhecida, servindo como chave de agregação para todos os dados recebidos. O FechoDiaCentral registra cada fecho recebido por loja e data, servindo de base para o cálculo dinâmico do estado de completude da rede pelo ConsolidacaoServico.

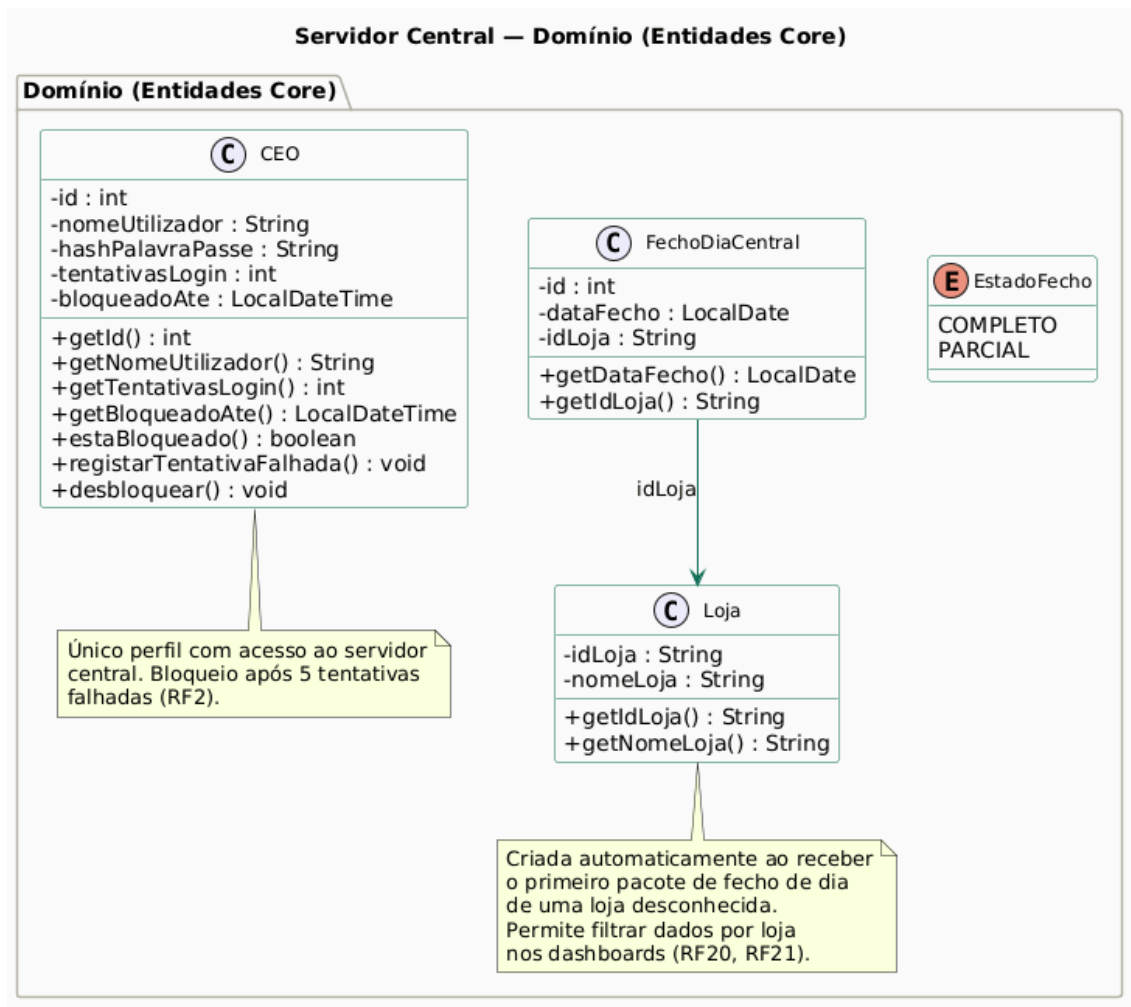


Figura 39: Diagrama de classes — Domínio: Entidades Core.

As entidades transacionais são versões de leitura das entidades operacionais da loja. Cada uma mantém o idOriginalLoja e o idLoja, formando a chave composta de idempotência que previne duplicados em caso de reenvio. A VendaCentral agrega LinhaVendaCentral com informação desnormalizada (nome do produto e categoria em vez de referências a um catálogo local). A DevolucaoCentral mantém o idOriginalVenda para rastreabilidade até à venda de origem. A RemessaCentral inclui o nome do fornecedor e o estado de pagamento. O PagamentoCentral referencia a remessa de origem via idOriginalRemessa. A SessaoCaixaCentral regista o saldo final e os períodos de abertura/encerramento. O LogAuditoriaCentral espelha o TipoAcaoCentral da loja, incluindo FECHO_DIA.

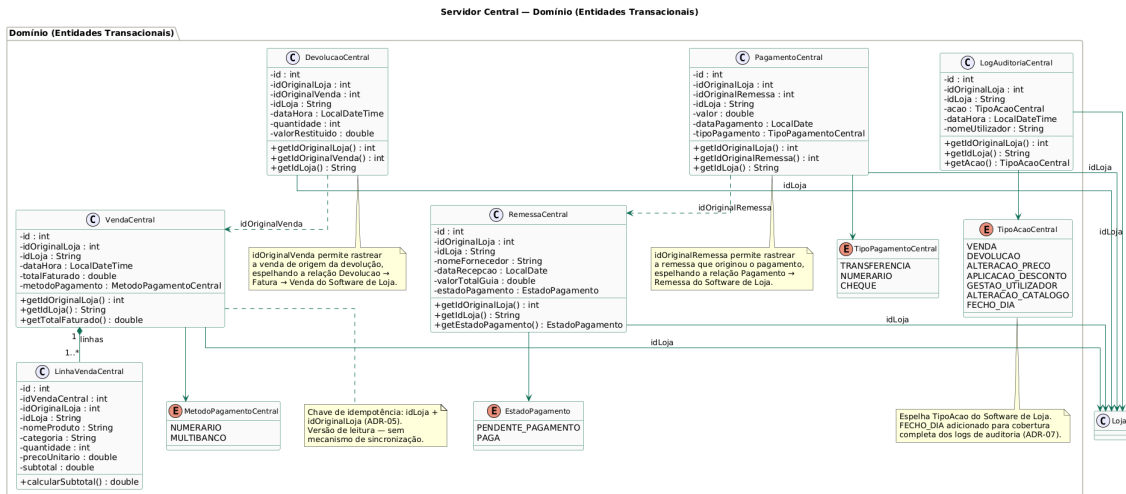


Figura 40: Diagrama de classes — Domínio: Entidades Transacionais.

Os *value objects* de análise suportam as funcionalidades de *dashboard* e relatórios. O *MetricasGlobais* agrega totais de vendas, transações, devoluções e remessas pendentes em toda a rede, com decomposição por loja. O *MetricasLoja* detalha os mesmos indicadores para uma loja específica, incluindo a repartição entre numerário e multibanco. O *Relatorio* encapsula o resultado de uma consulta filtrada por *FiltroRelatorio* (lojas, período e categorias), com capacidade de exportação. O *FiltroRemessa* permite filtrar remessas por estado de pagamento, período e lojas.

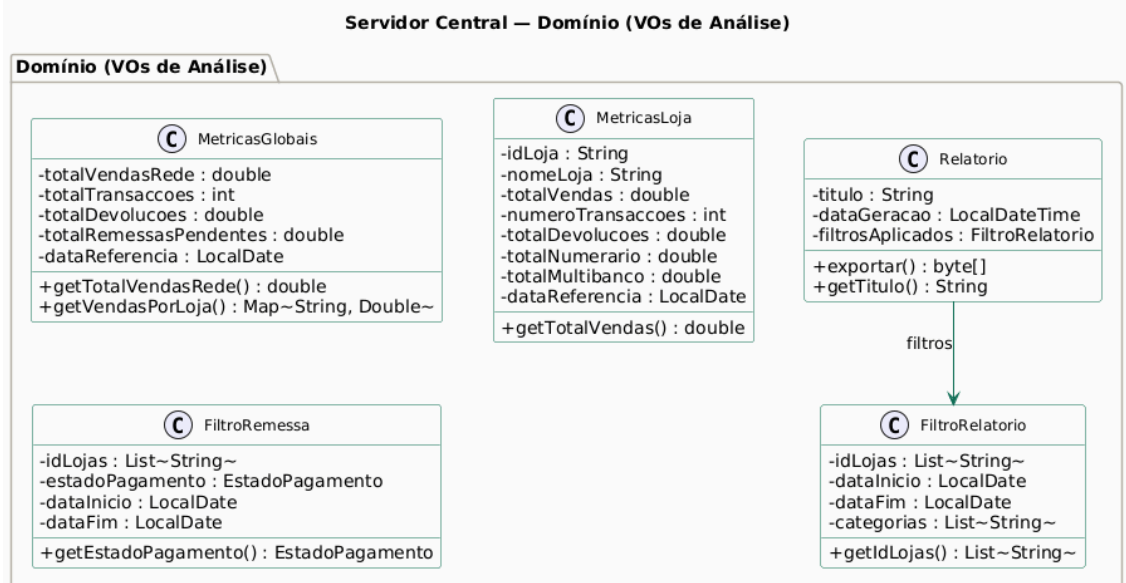


Figura 41: Diagrama de classes — Domínio: Value Objects de Análise.

Os DTOs de transferência definem o contrato de comunicação entre o Software de Loja e o Servidor Central. O *PacoteFechoDTO* é o objeto raiz, serializado em JSON via HTTP, que contém o identificador e nome da loja, a data do fecho, e listas de *VendaDTO*, *DevolucaoDTO*, *RemessaDTO*, *PagamentoDTO*, *SessaoCaixaDTO* e *LogAuditoriaDTO*. Cada DTO transporta o *idOriginalLoja* que permite ao servidor construir a chave de idempotência. A *VendaDTO* é composta por *LinhaVendaDTO* com informação desnormalizada do produto.

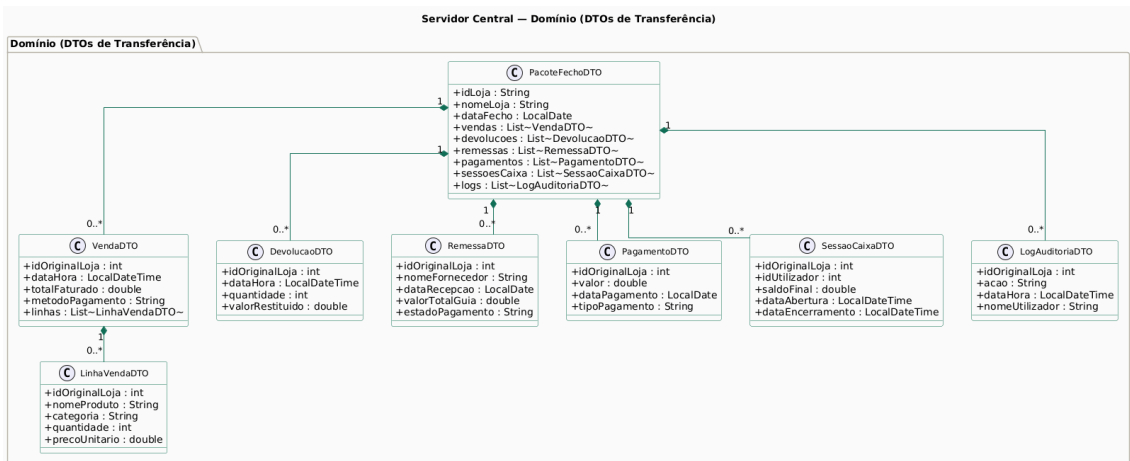


Figura 42: Diagrama de classes — Domínio: DTOs de Transferência.

O módulo de Core e Autenticação define o `CEORepositorio` para autenticação e atualização do estado de bloqueio, o `LojaRepositorio` com registo automático de lojas desconhecidas, e o `FechoDiaCentralRepositorio` que permite consultar fechos por data e identificar lojas que ainda não enviaram dados.

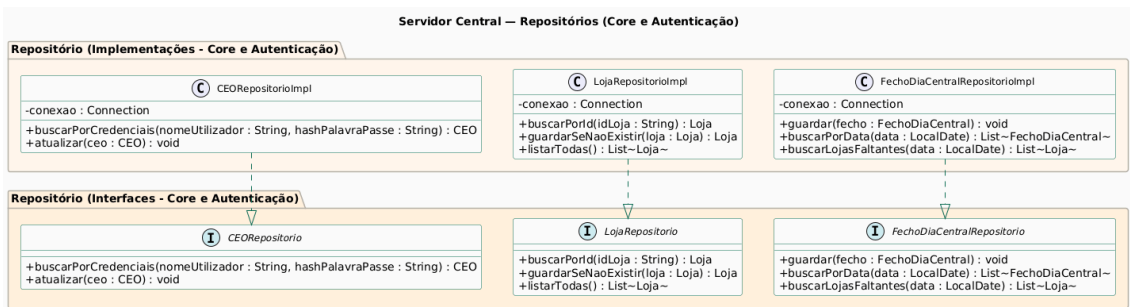


Figura 43: Diagrama de classes — Repositório: Core e Autenticação.

O módulo de Vendas, Devoluções e Sessões de Caixa disponibiliza repositórios com persistência idempotente e consulta por loja e período temporal, suportando as necessidades dos serviços de análise.

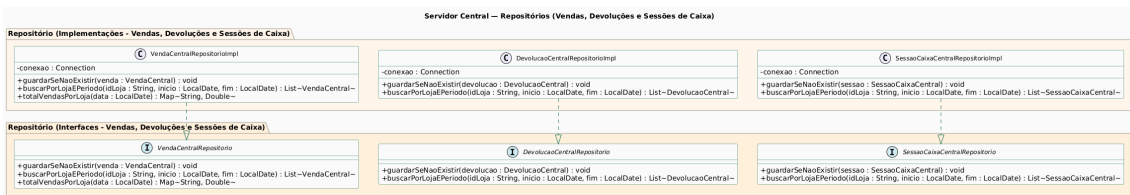


Figura 44: Diagrama de classes — Repositório: Vendas, Devoluções e Sessões de Caixa.

O módulo de Remessas, Pagamentos e Auditoria completa a persistência com repositórios que suportam filtragem por critérios compostos (`FiltroRemessa`) e consulta de *logs* por loja e período.

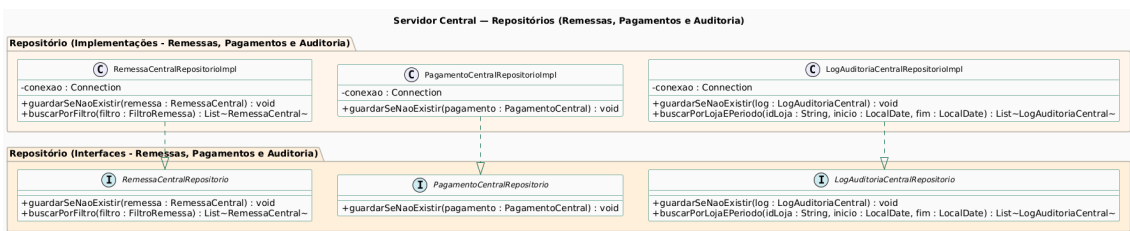


Figura 45: Diagrama de classes — Repositório: Remessas, Pagamentos e Auditoria.

A camada de serviços divide-se em dois módulos funcionais, cada um com as respetivas interfaces e implementações. O módulo de Autenticação e Receção contém o `AutenticacaoCentralServico`, que gere o fluxo de *login* do CEO incluindo o registo de tentativas falhadas e o desbloqueio automático após

expiração do período de bloqueio. O `RecepcaoPacoteServico` é o serviço central de ingestão — recebe o `PacoteFechoDTO`, valida a sua integridade, regista a loja automaticamente se for desconhecida, e persiste cada tipo de entidade no repositório correspondente. A interface `IRecepcaoPacoteServico` expõe apenas `receberPacote()` e `validarIntegridade()`, encapsulando a decomposição interna do pacote como detalhe de implementação.

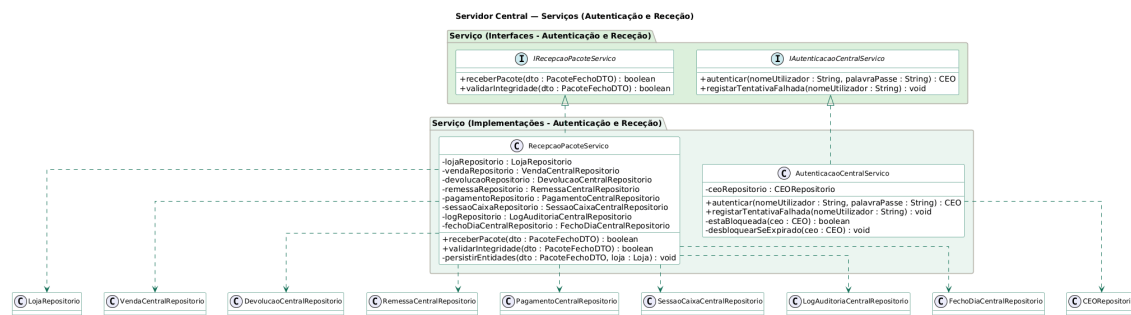


Figura 46: Diagrama de classes — Serviços: Autenticação e Receção.

O módulo de Análise agrupa os serviços de consulta consumidos pelo painel do CEO. O `DashboardServico` calcula métricas globais e por loja, cruzando dados de vendas e devoluções. O `RelatorioServico` gera relatórios parametrizados por filtros de loja, período e categoria. O `RemessaRedeServico` permite consultar remessas na rede filtradas por estado de pagamento. O `ConsolidacaoServico` verifica a completude do fecho diário de forma dinâmica — sem recorrer a nenhum campo persistido, compara os registos de `FechoDiaCentral` recebidos para uma data com a lista de lojas conhecidas no `LojaRepositorio`, devolvendo `EstadoFecho.COMPLETO` se todas enviaram ou `EstadoFecho.PARCIAL` com a lista das lojas em falta caso contrário. As lojas em falta para uma data podem ser obtidas separadamente através de `obterLojasFaltantes()`.

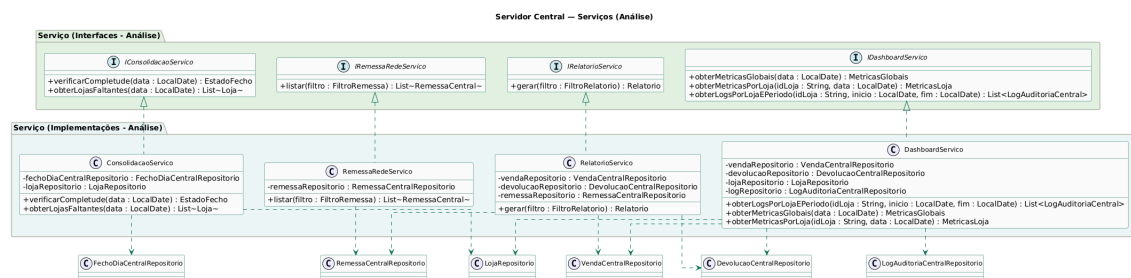


Figura 47: Diagrama de classes — Serviços: Análise.

A camada de apresentação é composta por *handlers* Javalin que mapeiam *endpoints* REST para operações nos serviços. Ao contrário dos *controllers* JavaFX da loja que reagem a eventos de interface gráfica, estes *handlers* recebem pedidos HTTP, extraem os parâmetros do contexto Javalin, invocam o serviço adequado e serializam a resposta em JSON. O `RecepcaoPacoteHandler` expõe o *endpoint* POST que recebe os pacotes das lojas. Os restantes *handlers* servem o painel do CEO com *endpoints* GET para métricas, relatórios, remessas e estado de consolidação. Todos dependem exclusivamente das interfaces de serviço.³⁴

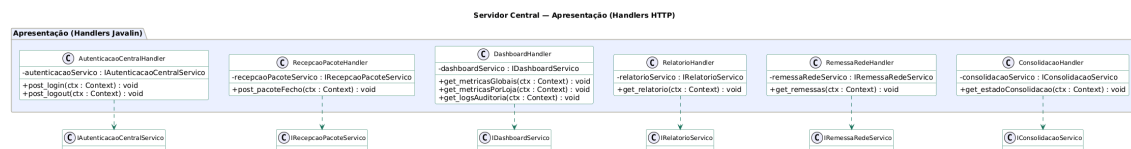


Figura 48: Diagrama de classes — Apresentação: Handlers HTTP.

³⁴Geração do texto das secções de componentes e classes com apoio de IA — ver [Anexo de IA, Entrada 41](#).

4.8. Diagramas de Sequência

Os diagramas de sequência são utilizados para representar a interação entre os diferentes componentes do sistema ao longo de cenários concretos de execução, permitindo explicitar a ordem das chamadas e a distribuição de responsabilidades entre camadas.

No contexto do sistema TrasmUM, sete diagramas de sequência foram produzidos, cobrindo os fluxos mais críticos e representativos de ambos os softwares. A seleção dos cenários a modelar privilegiou os fluxos com maior complexidade comportamental — aqueles onde a ordem das operações, a gestão de exceções e a comunicação entre camadas são mais determinantes para a correção do sistema. Cada diagrama foi associado explicitamente aos requisitos funcionais e às decisões arquiteturais que materializa, garantindo a rastreabilidade entre a especificação e o design dinâmico.

DS1 — Autenticação e Controlo de Sessão

O diagrama DS1 disponível no Anexo 11 modela o fluxo de autenticação no terminal de loja, cobrindo o ciclo de vida completo de uma sessão de utilizador, desde o arranque da aplicação até ao encerramento explícito da sessão. Este diagrama materializa os requisitos RF1 e RF2.

O fluxo inicia-se com o arranque da aplicação, momento em que o `AutenticacaoController` consulta a `ConfiguracaoTerminal` para obter o identificador da loja. Esta leitura ocorre a partir de um ficheiro de configuração local definido no momento da instalação e não alterável através da interface da aplicação — uma restrição arquitetural que garante que cada terminal conhece inequivocamente a loja a que pertence, independentemente do utilizador autenticado.

Após a submissão das credenciais pelo utilizador, o *controller* delega a operação no `IAutenticacaoServico`, que por sua vez consulta o `UtilizadorRepositorio` para validar o par nome de utilizador e palavra-passe. O diagrama distingue dois ramos alternativos neste ponto: se as credenciais forem inválidas, o repositório devolve *null* e o serviço propaga uma `CredenciaisInvalidasException`, que o *controller* traduz num alerta visual para o utilizador; se as credenciais forem válidas, o fluxo avança para a verificação da sessão duplicada.

A verificação de sessão duplicada é um dos elementos mais distintivos deste fluxo e materializa o RF1, que proíbe que o mesmo utilizador esteja autenticado em mais de um terminal simultaneamente. O `IAutenticacaoServico` invoca `verificarSessaoAtiva` no repositório, que devolve um booleano. Se o utilizador já tiver uma sessão ativa noutro terminal, é lançada uma `SessaoDuplicadaException`, impedindo o acesso. Caso contrário, o campo `emSessao` é atualizado para *true* na base de dados local, associando formalmente o utilizador ao terminal, e o sistema navega para o ecrã principal com as funcionalidades correspondentes ao perfil autenticado.

O encerramento de sessão segue um fluxo simétrico: o *controller* invoca `encerrarSessao` no serviço, que atualiza o campo `emSessao` para *false* no repositório, e a *interface* regressa ao ecrã de *login*. Esta simetria entre abertura e fecho de sessão é essencial para garantir que o estado de sessão na base de dados local permanece sempre consistente com o estado real do terminal.

DS2 — Registo de Venda com Lógica FEFO

O diagrama DS2 disponível no Anexo 12 descreve o fluxo de registo de uma venda no terminal de loja, incorporando a lógica FEFO (*First Expired, First Out*) de seleção de lotes, bem como os mecanismos de autorização e auditoria. Este é o fluxo com maior frequência de execução no sistema e materializa os requisitos RF3, RF11 e RF12.

O fluxo inicia-se com uma verificação de autorização: o `VendaController` invoca `exigirPerfil` no `IAutorizacaoServico`, confirmando que o utilizador autenticado possui um dos perfis com acesso à funcionalidade de vendas. Esta verificação prévia a qualquer operação de negócio é um padrão transversal a todos os fluxos do sistema, assegurando que a segregação de responsabilidades é aplicada em tempo de execução e não apenas na configuração da interface.

O ciclo de leitura de artigos é representado como um fragmento de interação repetido (*loop*), onde cada iteração corresponde à leitura de um código de barras. O `IVendaServico` consulta o `ProdutoRepositorio` para identificar o produto correspondente ao código lido, tratando os casos de produto inexistente ou ina-

tivo através de exceções específicas. Quando o produto é válido e ativo, o serviço invoca `buscarLotesFEFO` no `LoteRepositorio`, que devolve os lotes do produto ordenados por data de validade crescente. A nota presente no diagrama esclarece o desvio à lógica FEFO: se o utilizador tiver ativado o botão “Desconto Validade”, o sistema seleciona o lote com desconto configurado pelo gestor, sobrepondo a ordem natural dos lotes. Este comportamento diferenciado, que preserva a regra de negócio sem requerer *hardware* adicional de impressão de etiquetas, é um dos elementos de valor mais relevantes do sistema.

Após a leitura de todos os artigos, o utilizador confirma a venda e seleciona o método de pagamento. No caso de pagamento em numerário, o diagrama inclui uma verificação do valor entregue face ao total a pagar, calculando o troco antes de prosseguir. O `IVendaServico` finaliza a venda persistindo-a no `VendaRepositorio` e atualizando o saldo da sessão de caixa no `SessaoCaixaRepositorio`. Se o saldo atualizado exceder o limite máximo configurado para a caixa da loja, é emitido um alerta de sangria ao utilizador. O fluxo termina com o registo da ação no `IAuditoriaServico` e a devolução da fatura ao ecrã, completando o ciclo transacional.

DS3 — Fecho de Dia e Sincronização Delta Sync

O diagrama DS3 disponível no Anexo 13, modela o processo de fecho de dia, materializando o ADR-01. Este fluxo materializa o RF16, o ADR-04 e o ADR-05, e representa a operacionalização do padrão *Offline-First* com *Delta Sync* adotado na arquitetura do sistema.

O fluxo inicia-se com uma verificação de autorização que restringe esta operação aos perfis de Gestor e CEO, garantindo que um funcionário não pode acidentalmente encerrar o dia operacional da loja.

O primeiro passo substantivo é a recolha dos registos pendentes de sincronização. O `IFechoDiaServico` invoca, em cada serviço do sistema — vendas, devoluções, sessões de caixa, remessas, pagamentos e *logs* de auditoria —, o método `obterPendentes()` correspondente. A nota presente no diagrama clarifica um detalhe crítico: antes de devolver a lista de registos pendentes, cada serviço reverte internamente para o estado `PENDENTE` quaisquer registos que se encontrem no estado `EM_TRANSITO` em resultado de um envio anterior falhado. Este mecanismo de reversão é fundamental para garantir que uma falha de comunicação não resulta em perda silenciosa de dados.

Após a recolha, o `IFechoDiaServico` compila o `PacoteFechoDT0` — um objeto que agrega todos os registos pendentes numa estrutura serializada em JSON, identificada pelo `idLoja` lido da configuração do terminal — e marca todos os registos como `EM_TRANSITO`. Esta transição intermédia é intencional: distingue registos que nunca foram enviados de registos que estão em curso de envio, evitando que um reinício da aplicação durante o envio resulte em dados irre recuperáveis.

O envio é realizado através da interface `SincronizacaoGateway`, que abstrai o protocolo HTTP e isola o serviço dos detalhes de rede. O diagrama representa explicitamente dois ramos de resultado: em caso de falha de rede ou *timeout*, os registos são revertidos para `PENDENTE`, o fecho é persistido localmente com estado `PENDENTE` para reenvio posterior, e a sessão não é encerrada — o utilizador é informado da falha e poderá reenviar manualmente quando a ligação for restaurada; em caso de receção do ACK do Servidor Central com código 200 OK, os registos são marcados como `CONFIRMADO` e a sessão é encerrada, regressando o terminal ao ecrã de *login*. A nota presente no diagrama sublinha que o estado `CONFIRMADO` é marcado exclusivamente após a receção do ACK — nunca antes — o que garante que o fecho é sempre idempotente e que uma falha após o envio mas antes da receção do ACK resulta num reenvio, e não numa perda de dados.

DS4 — Receção do Pacote no Servidor Central

O diagrama DS4, disponível no Anexo 14, descreve o fluxo de receção e processamento de um pacote de fecho de dia no Servidor Central — a perspetiva complementar do DS3, que modela o mesmo evento do ponto de vista da loja. Este diagrama materializa o RF15, o RF16, o ADR-05 e o ADR-08.

O fluxo inicia-se com a receção de um pedido HTTP POST dirigido ao *endpoint* /fecho, enviado pelo Software de Loja. O `RecepcaoPacoteHandler`, implementado com a biblioteca Javalin, recebe o `PacoteFechoDT0` serializado em JSON e delega a sua validação e persistência no `IREcepcaoPacoteServico`. A primeira operação é a verificação da integridade do pacote através de um *hash*: se o *hash* for inválido ou o JSON estiver malformado, o servidor responde com um erro 400 Bad Request e o processo é interrompido.

Verificada a integridade, o serviço resolve a loja de origem consultando o `LojaRepositorio`. Caso a loja seja desconhecida — situação que ocorre na primeira receção de pacote de uma loja —, é criada automaticamente uma entrada para essa loja no servidor central, sem necessidade de qualquer configuração prévia. Esta capacidade de auto-registo é uma decisão deliberada que simplifica o processo de instalação e elimina a necessidade de configuração manual do servidor cada vez que uma nova loja é adicionada à rede.

A fase de persistência é o núcleo do fluxo. Para cada tipo de entidade presente no pacote — vendas, devoluções, sessões de caixa, remessas, pagamentos e *logs* de auditoria —, o serviço invoca o repositório central correspondente com a operação `guardarSeNaoExistir`. Esta operação garante idempotência conforme o ADR-05: um reenvio do mesmo pacote, decorrente de uma falha de comunicação após o envio mas antes da receção do ACK, não resulta em duplicação de registos na base de dados central.

Após a persistência de todas as entidades, o serviço regista o `FechoDiaCentral` e invoca o `IConsolidacaoServico` para verificar se todas as lojas da rede enviaram o fecho para a data em questão. O resultado desta verificação é um estado de consolidação — `COMPLETO` se todos os fechos foram recebidos, `PARCIAL` com a lista das lojas em falta caso contrário — que fica disponível para consulta pela CEO. O fluxo termina com o envio do ACK de confirmação à loja: um código 200 OK com o identificador da loja e a data do fecho, que desencadeará na loja a marcação dos registos como `CONFIRMADO`.

DS5 — Processamento de Devolução

O diagrama DS5, disponível no Anexo 15, descreve o fluxo de processamento de uma devolução no terminal de loja, materializando o RF5. Este fluxo destaca-se pela sequência de validações que o sistema executa antes de aceitar a devolução, cada uma delas correspondendo a uma guarda de negócio distinta.

Após a verificação de autorização inicial, o `IDevolucaoServico` executa quatro guardas em sequência. A primeira verifica a existência e proveniência da fatura: o `VendaRepositorio` é consultado pelo número de fatura, e a operação é rejeitada caso a fatura não exista ou tenha sido emitida numa loja diferente da atual — garantindo que não são aceites devoluções de faturas de outras filiais da rede. A segunda verifica se o artigo consta na fatura, validando a correspondência entre o código de barras apresentado e as linhas da venda original. A terceira guarda avalia a quantidade disponível para devolução, consultando o `DevolucaoRepositorio` para obter as devoluções anteriores associadas àquela fatura e calculando quantas unidades restam por devolver; esta verificação opera ao nível da unidade, bloqueando devoluções que excedam a quantidade originalmente faturada. A quarta e última guarda verifica a validade do lote, utilizando a data de validade impressa na embalagem física para identificar o lote correto no inventário.

O resultado desta última guarda determina o comportamento do sistema relativamente ao stock: se o lote estiver dentro do prazo de validade, a quantidade é repostada no inventário através do `LoteRepositorio`; se o lote estiver expirado, a devolução é aceite financeiramente mas o stock não é repostado, prevenindo a reintrodução no inventário de produtos que não podem ser comercializados. Em qualquer dos casos, a devolução é persistida no `DevolucaoRepositorio` e a ação registada no `IAuditoriaServico`, completando o ciclo com a confirmação ao utilizador do valor a restituir.

DS6 — Registo de Remessa

O diagrama DS6, disponível no Anexo 16, modela o fluxo de registo de receção de uma remessa de mercadoria, materializando os requisitos RF6 e RF7. Este fluxo é da responsabilidade exclusiva do Gestor ou CEO e abrange a criação de novos lotes de stock, a associação opcional a pedidos de remessa preexistentes e a geração automática de um registo de pagamento pendente.

Após a verificação de autorização, o `IRemessaServico` valida se a remessa contém pelo menos uma linha. Para cada linha válida, o serviço consulta o `ProdutoRepositorio` para confirmar que o produto existe e está ativo, e cria um novo lote independente no `LoteRepositorio` com a quantidade e a data de validade da entrega. A nota presente no diagrama sublinha um princípio fundamental da gestão de inventário do sistema: cada receção origina lotes independentes, sem sobrepor as validades dos lotes pré-existentes do mesmo produto. Este mecanismo é o que permite a rastreabilidade individual por lote e a aplicação correta da lógica FEFO nas vendas subsequentes.

Após a criação dos lotes, o serviço verifica se existe um `PedidoRemessa` com estado `PENDENTE` para o mesmo fornecedor. Se existir, o pedido é atualizado para `CONCLUIDO`, estabelecendo a rastreabilidade

entre a intenção de encomenda e a receção efetiva. Se não existir pedido associado, a remessa é registada sem essa ligação — cobrindo explicitamente o cenário operacionalmente válido em que o gestor recebe mercadoria sem ter criado previamente um pedido no sistema.

O fluxo termina com a geração automática de um registo de Pagamento com estado PENDENTE associado ao fornecedor e ao valor total da guia. Este registo será incluído no pacote de fecho de dia para visibilidade da CEO na base de dados central, e poderá ser posteriormente liquidado pelo gestor através do fluxo de gestão de pagamentos.

DS7 — Autenticação do CEO no Servidor Central

O diagrama DS7, disponível no Anexo 17, modela o fluxo de autenticação da CEO no Servidor Central, materializando o RF2. Embora conceptualmente análogo ao DS1, este fluxo apresenta diferenças arquiteturais relevantes que decorrem das especificidades do Servidor Central como plataforma.

A interação não ocorre através de uma interface JavaFX, mas sim através de um *handler* HTTP Javalin que recebe um pedido POST para o *endpoint* / *login*. A autenticação é delegada no `IAutenticacaoCentralServico`, que consulta o `CEORepositorio` com as credenciais recebidas.

O elemento mais distintivo deste fluxo é o mecanismo de bloqueio temporário por tentativas falhadas consecutivas. Quando as credenciais são inválidas, o serviço verifica se a conta existe, avalia se o período de bloqueio já expirou através de `desbloquearSeExpirado`, e incrementa o contador de tentativas falhadas. Se o número de tentativas atingir o limiar configurado de cinco, a conta é bloqueada por quinze minutos, devolvendo ao cliente um erro 403 Forbidden com a indicação do momento em que o bloqueio expirará. Este mecanismo protege a conta da CEO contra ataques de força bruta, sendo particularmente relevante dado que o Servidor Central é acessível em rede local e o perfil CEO tem visibilidade sobre dados consolidados de toda a rede.

Quando as credenciais são válidas e a conta não está bloqueada, o contador de tentativas falhadas é reposto a zero, a sessão HTTP é criada e o cliente é redirecionado para o painel de supervisão estratégica. O encerramento de sessão opera exclusivamente ao nível do estado HTTP, sem alterações na base de dados, dado que o perfil CEO é único no servidor central e não existe o risco de sessões duplicadas que justificou o mecanismo de `emSessao` implementado nos terminais de loja.

4.9. Esboço Das Interfaces do Sistema

Para o esboço das interfaces do sistema foi utilizada a ferramenta, Pencil. Esta permitiu-nos conectar um agente de codificação a uma ferramenta de desenho através de um servidor *Model Context Protocol* (MCP), permitindo que este faça alterações ao esboço acelerando assim o processo de criação das interfaces.

O fluxo de trabalho iniciou-se com o envio do relatório a uma LLM, que gerou as Design Guidelines³⁵ com base nos requisitos funcionais e não funcionais. Posteriormente, o agente conectado ao Pencil recebeu estas diretrizes e, recorrendo ao servidor MCP, executou as instruções para construir automaticamente o esboço no Pencil³⁶.

4.9.1. Estrutura Geral das Interfaces

O sistema TrasmUM, conforme descrito anteriormente, materializa-se em duas aplicações com propósitos distintos. Para garantir a consistência visual e a usabilidade transversal entre ambas, foram desenhadas com recurso ao design system shadcn/ui na sua variante Modo Luz.

A conceção estrutural das interfaces foi orientada por princípios de modularidade e clareza. Do ponto de vista da navegação, os ecrãs para utilizadores autenticados baseiam-se num menu lateral dinâmico (Sidebar), cujos itens se adaptam consoante o perfil do utilizador — Funcionário, Manager ou CEO. A exceção é a Interface de Caixa (ECR-06), que abdica da navegação lateral em prol de um *layout* de ecrã inteiro, maximizando a área de foco do operador durante o atendimento. Do ponto de vista da apresentação de

³⁵Geração das Design Guidelines — ver [Anexo de IA, Entrada 35](#).

³⁶Realização do Esboço no Pencil — ver [Anexo de IA, Entrada 36](#).

dados, a interface privilegia componentes normalizados: informação analítica e de resumo é agrupada em Cards, grandes volumes de dados são estruturados em tabelas, e as interações que requerem introdução de dados ou confirmação de ações críticas são tratadas em janelas modais (Dialogs), prevenindo erros operacionais na linha da frente.

4.9.2. Caracterização das Interfaces - Software de Loja

O ponto de entrada no ecossistema TrasmUM efetua-se através do ecrã de início de sessão. Este ecrã tem como objetivo primordial autenticar o utilizador no terminal e determinar as funcionalidades a que este terá acesso, com base no seu perfil funcional (Funcionário, Manager ou CEO) e no ficheiro de configuração local do terminal. A interface adota uma abordagem minimalista, apresentando apenas os campos estritamente necessários para a submissão das credenciais, de modo a garantir um acesso célere e intuitivo.

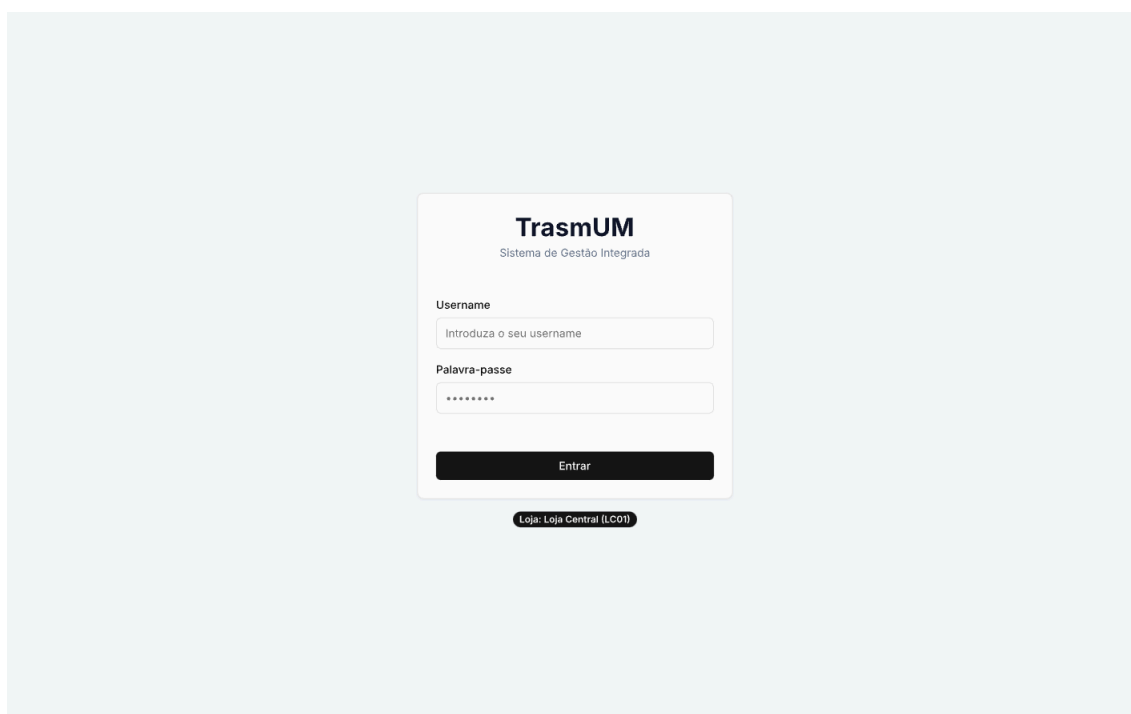


Figura 49: Esboço da “Página de Início de Sessão” (ECR-01)“

De forma a mitigar tentativas de acesso não autorizado e assegurar o cumprimento dos requisitos de segurança (RNF9), o sistema implementa um mecanismo de bloqueio automático. Caso o utilizador falhe a autenticação por cinco vezes consecutivas, a conta é suspensa e o sistema redireciona a navegação para o ecrã de Conta Bloqueada (ECR-01B). Este ecrã exibe um alerta descritivo e apresenta um contador regressivo de 15 minutos para o auto-desbloqueio da conta.

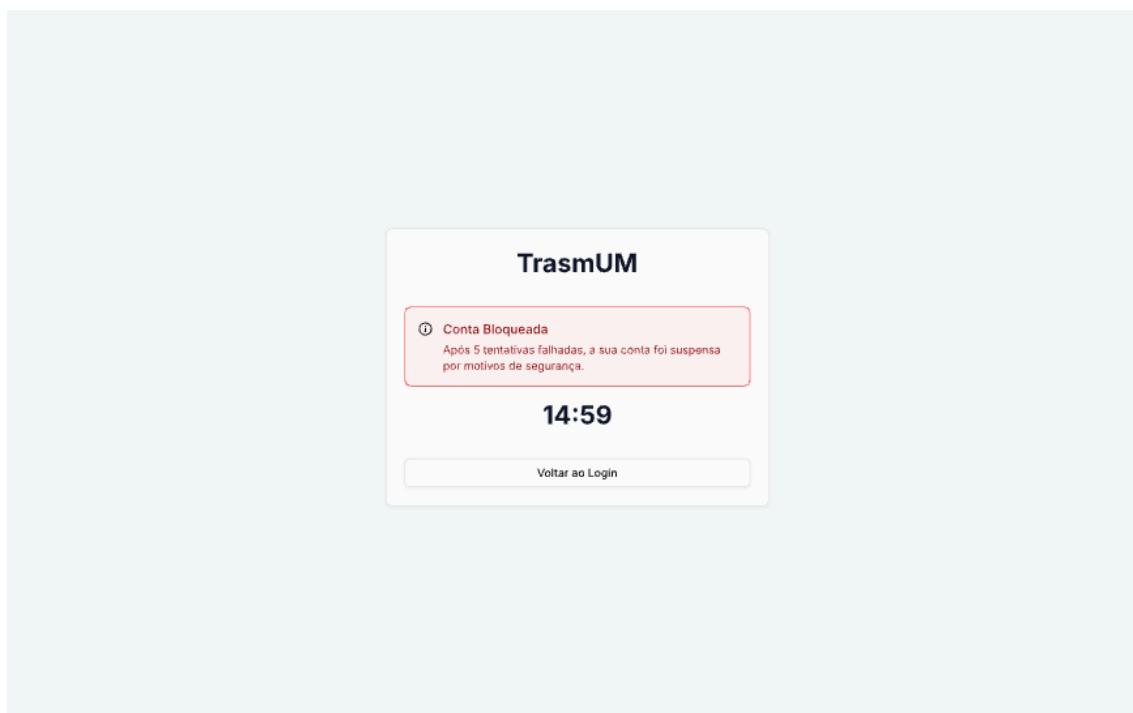


Figura 50: Esboço da “Página de Bloqueio de Início de Sessão” (ECR-01B)

Após uma autenticação bem-sucedida com o perfil de Funcionário, o utilizador acede aos ecrãs dedicados à operação diária da loja, nomeadamente o registo de vendas (ECR-06), o processamento de devoluções (ECR-11) e a consulta de produtos (ECR-12). A Interface de Caixa (ECR-06) constitui o núcleo operacional do estabelecimento. O seu desenho foi otimizado para minimizar o tempo de atendimento ao cliente.

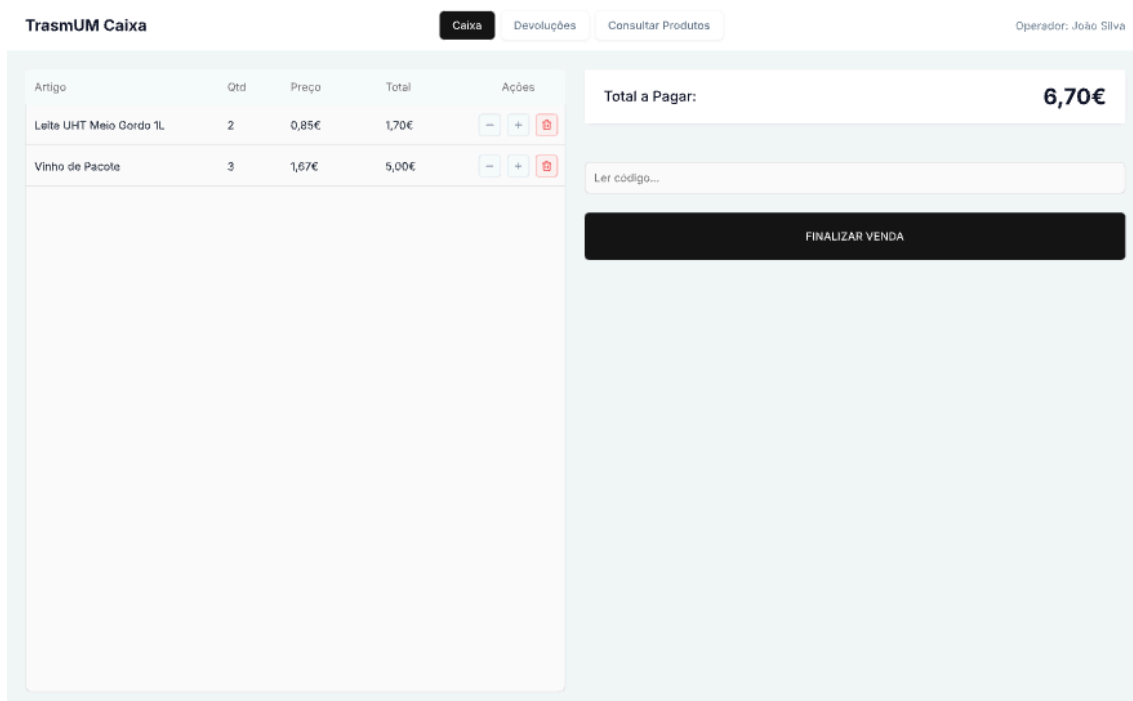


Figura 51: Esboço da “Página de Venda da Caixa” (ECR-06)

O fluxo de pós-venda é suportado pelo ecrã de Registrar Devolução (ECR-11). O processo logístico exige a identificação prévia da fatura original para evitar fraudes. A interface requer que o operador insira a data de validade impressa na embalagem devolvida; é com base nesta informação que o sistema avalia se a unidade se encontra dentro do prazo, determinando automaticamente se o artigo volta a integrar o stock

disponível (reposição) ou se é considerado quebra (não reposição). A interface reflete esta decisão através de emblemas visuais informativos.

TrasmUM Caixa

Caixa Devoluções Consultar Produtos

Operador: João Silva

Fatura #

FT 2026/0452 Procurar

Artigo	Qtd	Preço	Ação
Leite UHT Meio Gordo 1L	2	0,85€	Devolver
Vinho de Pacote	1	1,67€	Devolver

Figura 52: Esboço da “Página de Devoluções da Caixa”

Por fim, o ecrã de Consultar e Pesquisar Produtos (ECR-12) oferece uma ferramenta de busca em tempo real no catálogo local da loja. Através de uma barra de pesquisa com *debounce*, o operador pode consultar rapidamente a disponibilidade de stock, o preço e o estado individual dos lotes.

TrasmUM Caixa

Caixa Devoluções Consultar Produtos

Operador: João Silva

Pesquisar

Nome ou código... Todas as Categorias

Artigo	Categoria	Stock	Preço
Leite UHT Meio Gordo 1L	Laticínios	12	0,85€
Vinho de Pacote	Bebidas	45	1,67€

Figura 53: Esboço da “Página de Consulta da Caixa”

Para além de herdar a totalidade das funcionalidades operacionais de caixa atribuídas ao perfil de Funcionário, o utilizador autenticado com o perfil de Manager dispõe de privilégios alargados para a administração do estabelecimento. A interface de gestão organiza-se em três eixos fundamentais: inventário, administração local e encerramento da jornada.

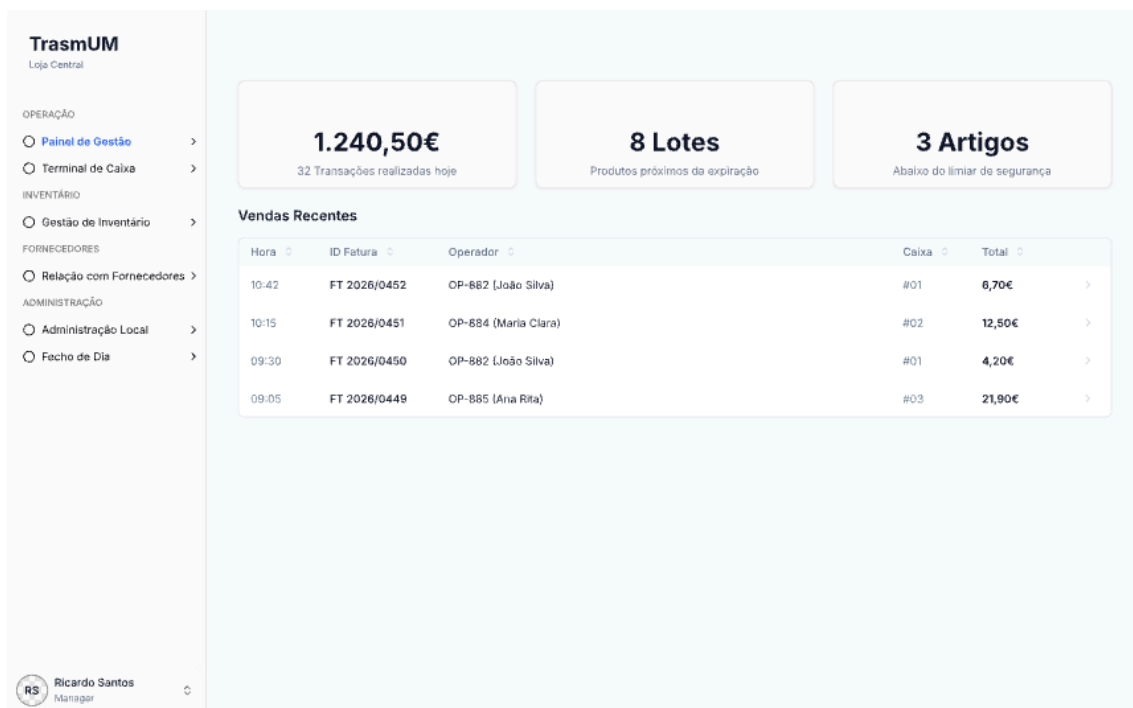


Figura 54: Esboço da “Página principal do Gerente”

A página de Inventário e Stock atua como o painel central de controlo físico e lógico dos produtos da loja. Esta interface agrega a gestão do catálogo (criação e edição de artigos) e a monitorização rigorosa das existências. O design da página permite ao Manager visualizar rapidamente alertas de stock mínimo e sinalizar lotes em risco de expiração do prazo de validade. É a partir desta interface que o gestor pode aplicar descontos a lotes específicos, garantindo que o escoamento de produtos perecíveis não afeta a rentabilidade global do catálogo.

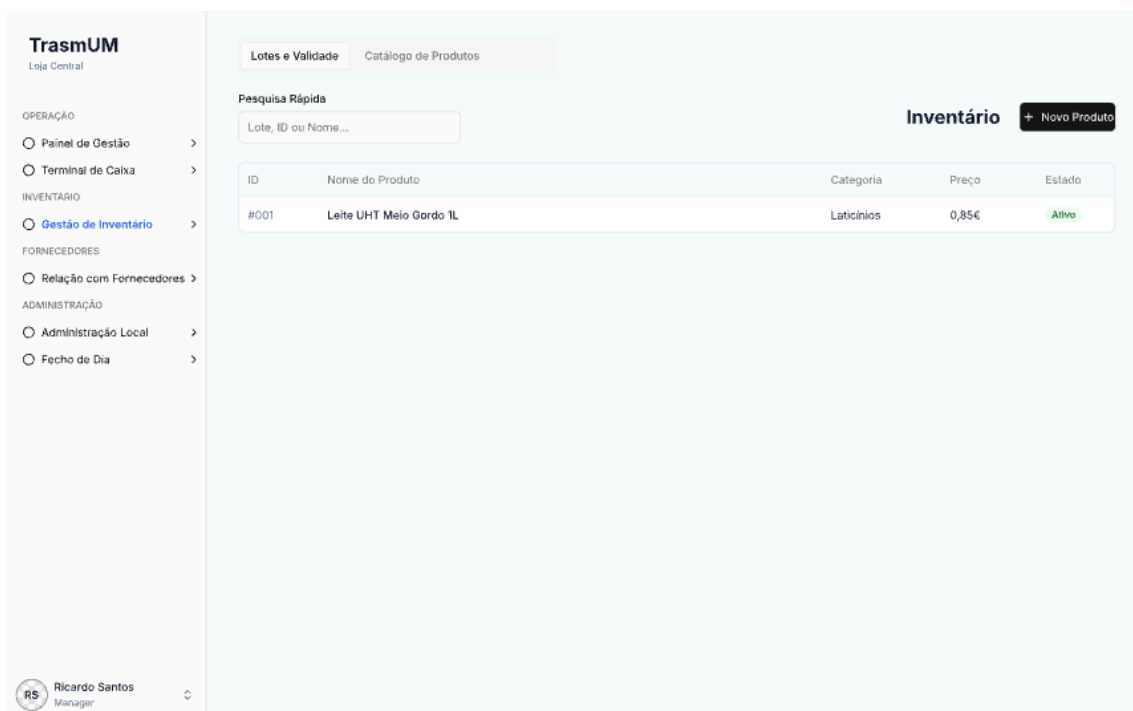


Figura 55: Esboço da “Página de gestão de Inventário do Gerente”

A gestão da cadeia de abastecimento e tesouraria foi consolidada na página de Fornecedores. Esta interface unifica o processamento logístico e administrativo. Nela, o Manager pode registar a receção de remessas mediante a introdução dos dados das guias de transporte. A lógica de negócio assegura a rastre-

abilidade total: ao dar entrada a uma remessa, o sistema cria automaticamente registos independentes de stock para as unidades recebidas, atribuindo-lhes códigos de lote e datas de validade próprias. Em simultâneo, esta operação alimenta a vertente administrativa da mesma página, arquivando digitalmente o documento e gerando de forma automática um registo de pagamento pendente associado ao respetivo fornecedor.

Data	Fornecedor	Valor Total	Estado (Entrega / Pagamento)
17 Mar	Lactogal S.A.	450,00€	Entregue Pendente
15 Mar	Sumol+Compal	1.120,00€	Entregue Pago

Figura 56: Esboço da “Página de gestão dos Fornecedores do Gerente”

A vertente de recursos humanos e parametrização do sistema encontra-se centralizada no ecrã de Gestão de Staff e Parâmetros Locais (ECR-19). Numa ótica de simplificação administrativa, esta interface funde duas necessidades operacionais críticas para a autonomia do estabelecimento. Por um lado, disponibiliza uma tabela de gestão onde o se pode criar contas, editar permissões e desativar credenciais dos funcionários da sua loja, garantindo que todas as operações de caixa permanecem estritamente rastreáveis e individuais. Por outro lado, a mesma interface acolhe o painel de configuração operacional da loja. É através deste formulário que o gestor define e ajusta as variáveis vitais ao funcionamento local e à reatividade do sistema: o limiar (em dias) que despoleta os alertas de aproximação do fim de validade , a percentagem de desconto por defeito a aplicar aos lotes sinalizados , e o limite máximo de numerário tolerado em caixa antes que o sistema emita um alerta obrigatório de sangria para o operador. Esta concentração de parâmetros num único ecrã minimiza a navegação e confere ao Manager um controlo total sobre as regras de negócio aplicadas no seu ponto de venda.

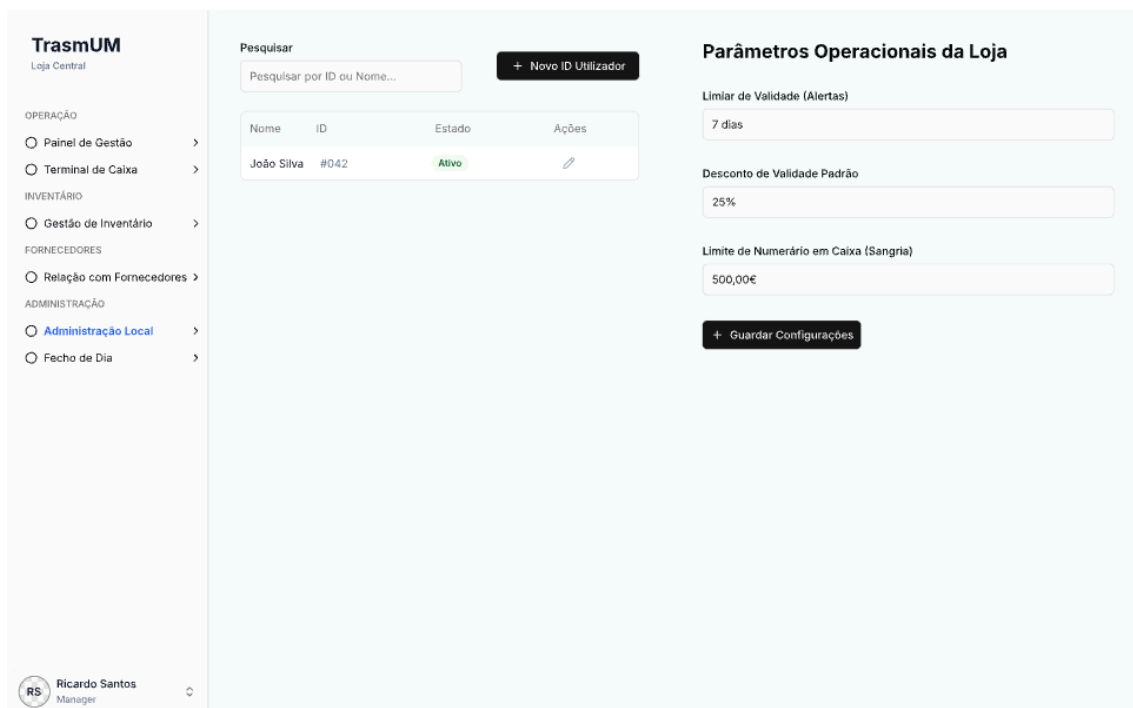


Figura 57: Esboço da “Página de configurações do Gerente”

O culminar da operação diária ocorre no ecrã de Fecho de Dia (ECR-22). Esta interface apresenta um resumo consolidado do dia, agregando o volume de vendas, devoluções, remessas recebidas e pagamentos liquidados. Ao acionar o fecho, o sistema compila os dados transacionais num pacote JSON e submete-o ao Servidor Central via HTTP. O design da interface reflete a restrição técnica de sincronização: o botão de “Encerrar Sessão” permanece obrigatoriamente desativado até que o terminal receba um ACK de sucesso por parte do servidor central. O estado do envio é comunicado ao utilizador em tempo real através de componentes visuais de progresso e badges informativos.

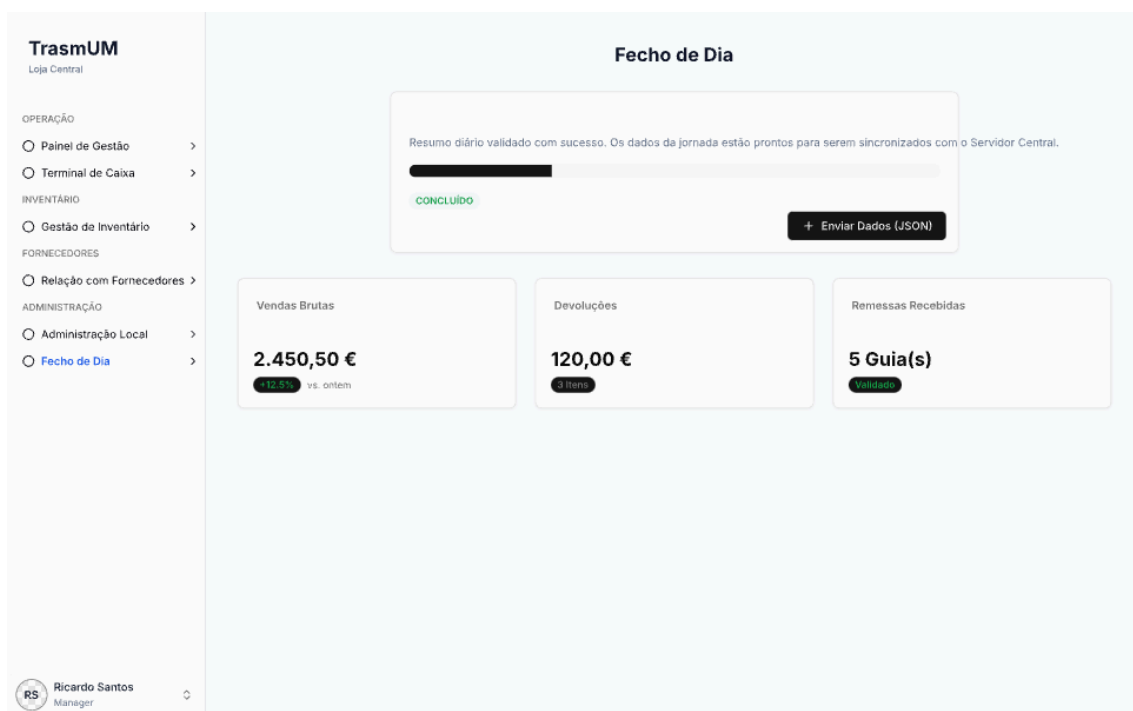


Figura 58: Esboço da “Página de Fecho do dia do Gerente”

4.9.3. Caracterização das Interfaces - Software do Servidor Central

A Aplicação de Analytics (Servidor Central) é dedicada em exclusivo ao perfil de CEO, assumindo um papel passivo de receção e tratamento de dados para fins de supervisão estratégica de toda a rede de três lojas. A integridade desta supervisão depende diretamente do fluxo assíncrono de dados. Para monitorizar este processo crítico, a interface disponibiliza o Monitor de Receção de Dados (ECR-23). Este ecrã atua como um painel de controlo logístico que lista as instâncias locais e exibe em tempo real o estado da receção do pacote JSON diário enviado no momento do fecho de dia de cada loja (identificado visualmente por Badges de estado: “Recebido”, “Em Falta” ou “Erro”). Esta interface permite ao CEO verificar rapidamente o estado da consolidação da rede, sabendo exatamente quais os estabelecimentos que já reportaram a sua jornada operacional e quais apresentam atrasos ou falhas de comunicação.

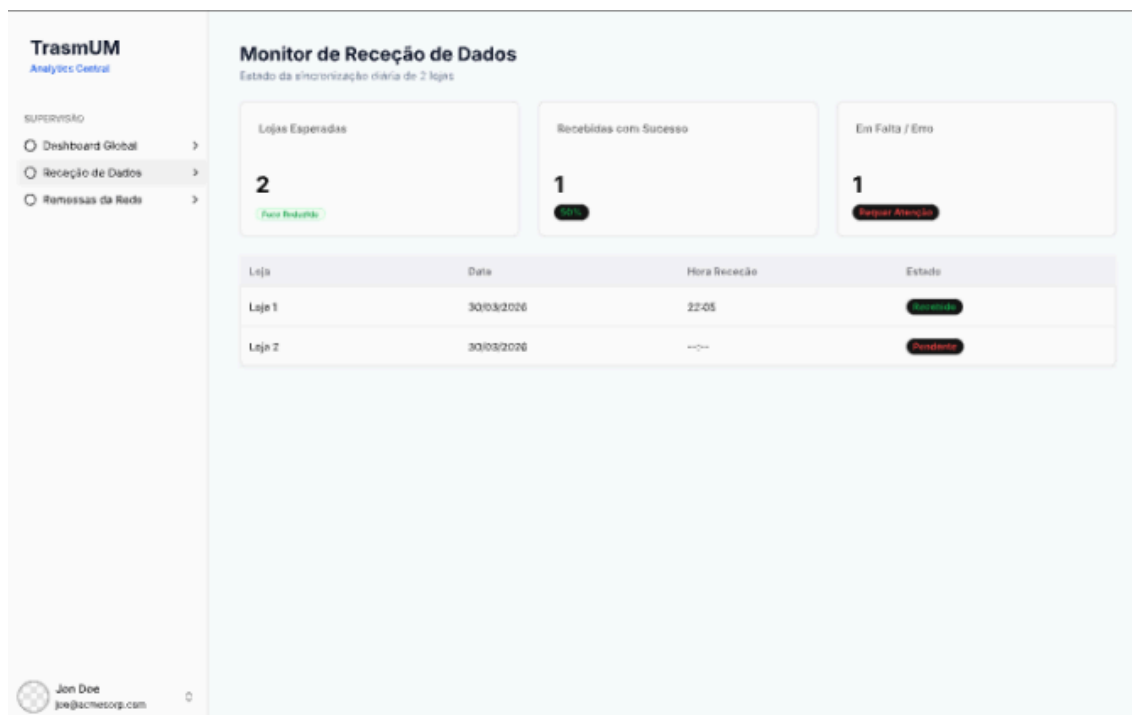


Figura 59: Esboço da “Página de Sincronização de dados da Cátia”

Uma vez garantida a consolidação dos dados, a extração de valor analítico processa-se através do ecrã de Dashboards e Relatórios da Rede (ECR-24). Esta interface de estatísticas globais apresenta os Indicadores-Chave de Desempenho (KPIs) agregados, incluindo o total de vendas da rede, a evolução temporal do volume de negócios por loja e as categorias de produtos mais vendidas. O design faz uso de gráficos interativos e de um indicador estático da última sincronização, reforçando que a análise opera sobre os dados do dia transato.

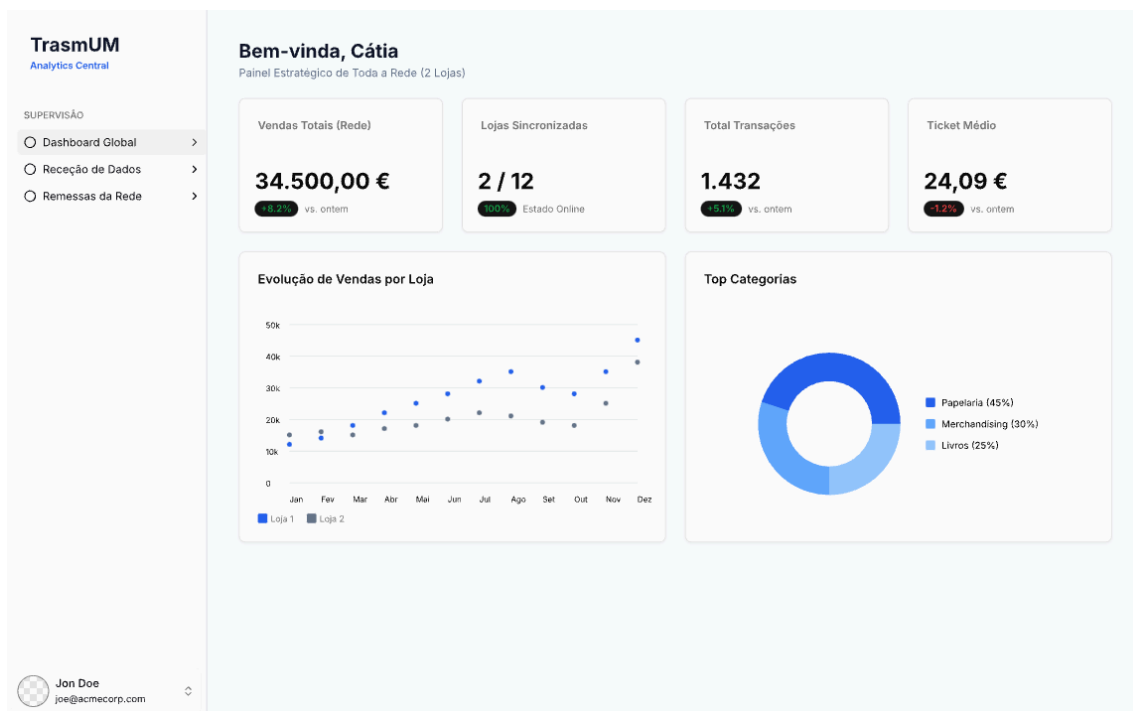


Figura 60: Esboço da “Página de Estatísticas da Cátia”

Por fim, a supervisão financeira e logística estende-se à cadeia de abastecimento através do ecrã de Consultar Remessas da Rede (ECR-25). Esta interface fornece uma vista consolidada, estruturada e auditável de todas as guias de remessa e faturas recebidas pelas lojas. Através de uma tabela de dados central (DataTable) equipada com filtros avançados (por loja, fornecedor, estado de pagamento e intervalo de datas), a CEO ganha a capacidade de cruzar informações e auditar a pontualidade na liquidação de pagamentos a fornecedores. Esta centralização digital elimina definitivamente o risco de perda de faturas físicas e previne conflitos com os parceiros de distribuição.

TrasmUM
Analytics Central

Remessas da Rede

Loja	Nº Doc / Fornecedor	Data Receção	Valor Total	Estado
Loja 1	PT 2026/452 - Staples Portugal	30/03/2026	1.345,50 €	Pagado
Loja 2	GR 882/28 - Porto Editora	29/03/2026	850,00 €	Pendente

Jon Doe
joe@asmcorp.com

Figura 61: Esboço da “Página de Gestão de Remessas da Cátia”

4.10. Conceção do Sistema de Dados - Software de Loja

4.10.1. Modelo Lógico

O modelo lógico do Software de Loja foi construído a partir das entidades de domínio definidas na fase de arquitetura e design. O *schema* resultante organiza-se em cinco módulos distintos: Core e Segurança, Catálogo e Inventário, Remessas e Fornecedores, Vendas e Faturas, e Tesouraria e Caixa. Todas as tabelas operacionais incluem uma coluna *estadoSincronizacao* ENUM('PENDENTE', 'EM_TRANSITO', 'CONFIRMADO') para suporte ao *Delta Sync* (ADR-04): — Venda, Devolucao, Remessa, Pagamento, SessaoCaixa, FechoDia e LogAuditoria. Adicionalmente, todas estas tabelas incluem uma coluna *idLoja* que identifica a loja de origem de cada registo, lida da configuração local do terminal no momento da criação do registo.

A configuração do terminal — nomeadamente o identificador e o nome da loja e o limite máximo de numerário em caixa — é armazenada num ficheiro de configuração local lido no arranque da aplicação, não sendo persistida na base de dados. O *idLoja* é copiado para cada registo sincronizável no momento da sua criação, garantindo que o pacote de fecho de dia identifica inequivocamente a origem de cada registo sem depender de uma ligação à configuração em tempo de execução.

O diagrama EER do modelo lógico encontra-se ilustrado na figura seguinte.³⁷

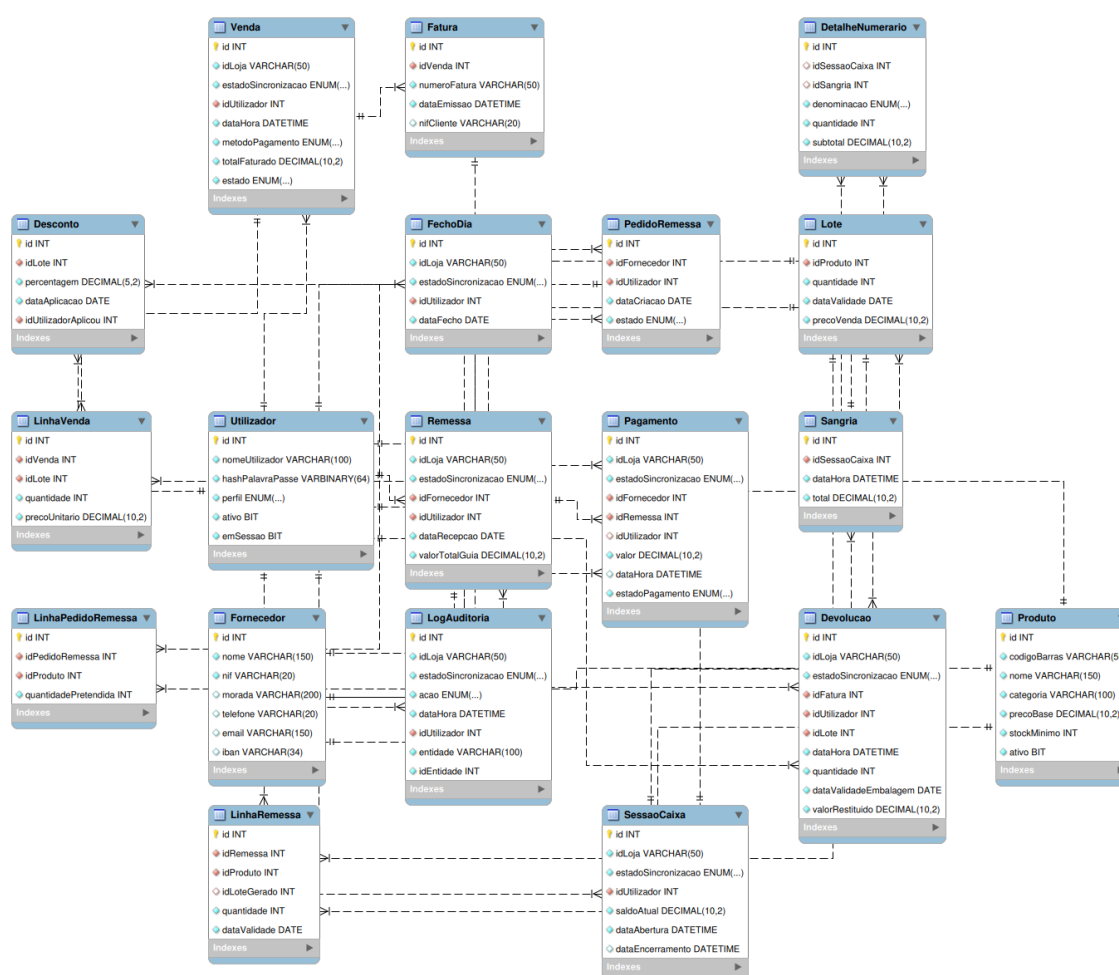


Figura 62: Modelo lógico da base de dados do Software de Loja.

4.10.2. Dicionário de Dados e Decisões de Design

A presente secção documenta as decisões de design tomadas na construção do *schema* do Software de Loja, apresentando para cada decisão relevante o dicionário de dados da tabela que a materializa.

³⁷Schema SQL do Software de Loja gerado com apoio de IA — ver [Anexo de IA, Entrada 42](#).

Por uma questão de concisão, apenas são documentadas as tabelas cujos tipos de dados ou estrutura de colunas resultam de decisões não triviais — as restantes tabelas seguem convenções standard e são suficientemente descritas pelo diagrama EER.

Uma decisão transversal a todo o *schema* diz respeito à representação de valores enumerados. Os campos `estadoSincronizacao`, `perfil`, `metodoPagamento`, `estado em Venda`, `estadoPagamento em Pagamento`, `estado em PedidoRemessa`, `acao em LogAuditoria` e `denominacao em DetalheNumerario` são todos representados como ENUM do MySQL. Esta escolha justifica-se por se tratarem de conjuntos fixos e imutáveis de valores definidos pela lógica de negócio — qualquer valor fora do conjunto representa um erro de aplicação que deve ser rejeitado pela camada de dados e não apenas pela camada de serviços. Para os valores monetários, utilizou-se sistematicamente o tipo DECIMAL (10,2) em vez de FLOAT ou DOUBLE, uma vez que estes últimos são tipos de vírgula flutuante sujeitos a erros de arredondamento que são inaceitáveis em contexto financeiro. O DECIMAL garante precisão exata até duas casas decimais, adequada ao domínio retalhista do sistema TrasmUM.

A tabela `Utilizador` apresenta duas decisões relevantes. A primeira prende-se com o armazenamento da palavra-passe: em conformidade com o RNF8, que proíbe o armazenamento de palavras-passe em texto simples, é armazenada a *hash* produzida pelo algoritmo BCrypt (Provos e Mazières, 1999), um algoritmo adaptativo especificamente concebido para palavras-passe que incorpora um *salt* aleatório em cada *hash*, tornando ataques de dicionário e de *rainbow table* inviáveis. O resultado do BCrypt tem comprimento fixo de 60 caracteres, pelo que se utilizou VARCHAR(60). A segunda decisão refere-se aos campos `ativo` e `emSessao`, ambos do tipo BIT — `ativo` controla se a conta pode ser utilizada para autenticação, permitindo a desativação de contas sem eliminação do registo, e `emSessao` implementa o mecanismo de deteção de sessões duplicadas definido no RF1, sendo atualizado para 1 no momento do *login* e para 0 no momento do *logout*.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador sequencial do utilizador.
nomeUtilizador	VARCHAR(100)	Não	—	Nome único de autenticação no terminal.
hashPalavra-Passe	VARCHAR(60)	Não	—	Hash BCrypt da palavra-passe do utilizador.
perfil	ENUM	Não	—	Perfil de acesso: FUNCIONARIO, GESTOR ou CEO.
ativo	BIT	Não	1	Se a conta está ativa (1) ou desativada (0).
emSessao	BIT	Não	0	Se o utilizador tem sessão ativa (1) num terminal.

Tabela 21: Dicionário de dados — Utilizador

A tabela `Lote` merece atenção pela distinção entre `precoVenda` e o `precoBase` do Produto. O `precoBase` do produto representa o preço de referência do artigo no catálogo, enquanto o `precoVenda` do lote representa o preço efetivo pelo qual as unidades desse lote específico são vendidas — que pode diferir do preço base caso o gestor tenha aplicado um desconto de validade ao lote. Esta separação é o mecanismo que permite ao sistema aplicar descontos a lotes individuais sem alterar o preço de referência do produto, conforme especificado no RF22. A relação com a tabela `Desconto` é de um-para-um opcional — um lote pode ter zero ou um desconto associado, e o desconto pertence exclusivamente a um lote, pelo que a chave estrangeira reside na tabela `Desconto` com uma restrição UNIQUE.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único do lote.
idProduto	INT	Não	—	Referência ao produto a que o lote pertence.

quantidade	INT	Não	0	Número de unidades disponíveis neste lote.
dataValidade	DATE	Não	—	Data de validade das unidades do lote.
precoVenda	DECIMAL(10,2)	Não	—	Preço efetivo de venda, podendo diferir do precoBase do produto caso exista desconto ativo.

Tabela 22: Dicionário de dados — Lote

A tabela Venda contém dois campos ENUM independentes que controlam dimensões distintas do ciclo de vida de uma transação. Uma decisão de design central nesta tabela é que uma venda só é persistida no momento da sua finalização ou anulação — durante o atendimento, o objeto Venda e as suas LinhaVenda vivem exclusivamente em memória, geridos pelo VendaServico. Esta opção elimina a necessidade de gerir registos incompletos na base de dados e simplifica o tratamento de anulações, que passam a ser uma simples limpeza do estado em memória sem qualquer interação com a base de dados. Como consequência direta, os campos metodoPagamento, totalFaturado e estado são sempre definidos no momento da inserção, não sendo necessário admitir valores nulos ou valores por omissão nestes campos. O estado EM_CURSO é um estado puramente de aplicação que nunca é persistido — os únicos valores possíveis na base de dados são FINALIZADA e ANULADA.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único da venda.
idLoja	VARCHAR(50)	Não	—	Identificador da loja de origem, lido da configuração local.
estadoSincronizacao	ENUM	Não	PENDENTE	Estado do ciclo de Delta Sync: PENDENTE, EM_TRANSITO ou CONFIRMADO.
idUtilizador	INT	Não	—	Utilizador que registou a venda.
dataHora	DATETIME	Não	—	Data e hora de finalização ou anulação da transação.
metodoPagamento	ENUM	Não	—	Método de pagamento: NUMERARIO ou MULTIBANCO. Sempre definido na persistência.
totalFaturado	DECIMAL(10,2)	Não	—	Total da venda no momento da finalização.
estado	ENUM	Não	—	Estado da transação persistida: FINALIZADA ou ANULADA.

Tabela 23: Dicionário de dados — Venda

A tabela Pagamento apresenta uma particularidade que merece clarificação explícita: contém dois campos cujo valor PENDENTE tem semânticas completamente independentes. O estadoSincronizacao controla o ciclo de Delta Sync e é gerido pelo mecanismo de fecho de dia — um pagamento está PENDENTE de sincronização quando ainda não foi incluído num pacote enviado ao Servidor Central. O estadoPagamento controla o estado financeiro do pagamento perante o fornecedor — um pagamento está PENDENTE quando o gestor ainda não registou a sua liquidação. Um registo pode estar simultaneamente PENDENTE em ambas as dimensões, ou CONFIRMADO na sincronização e PENDENTE no pagamento, caso o fecho de dia tenha sido enviado antes da liquidação. O campo idUtilizador é nulável porque o pagamento é criado automaticamente no momento do registo da remessa, antes de qualquer ação do gestor, não havendo utilizador associado à sua criação — apenas à sua liquidação.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único do pagamento.

idLoja	VARCHAR(50)	Não	—	Identificador da loja de origem.
estadoSincronizacao	ENUM	Não	PEN-DENTE	Estado do ciclo de Delta Sync, independente do estadoPagamento.
idFornecedor	INT	Não	—	Fornecedor ao qual o pagamento se destina.
idRemessa	INT	Não	—	Remessa que originou este pagamento.
idUtilizador	INT	Sim	NULL	Utilizador que liquidou o pagamento. Nulo até à liquidação.
valor	DECIMAL(10,2)	Não	—	Valor a pagar, baseado no valorTotalGuia da remessa.
dataHora	DATETIME	Sim	NULL	Data e hora da liquidação. Nula até à liquidação.
estadoPagamento	ENUM	Não	PEN-DENTE	Estado financeiro: PENDENTE ou PAGO. Independente do estadoSincronizacao.

Tabela 24: Dicionário de dados — Pagamento

A tabela *DetalheNumerario* serve dois propósitos distintos no sistema: registar a composição do fundo inicial de uma sessão de caixa e registar o valor de uma sangria. Esta dualidade é representada por duas chaves estrangeiras opcionais — *idSessaoCaixa* e *idSangria* — das quais exatamente uma deve estar definida em cada registo. A alternativa de criar duas tabelas separadas — *FundoInicialNumerario* e *SangriaNumerario* — foi considerada mas rejeitada por duplicação de estrutura, dado que os atributos são idênticos em ambos os contextos. O campo *subtotal* é persistido como valor calculado para evitar recálculos repetidos na interface, dado que $\text{denominacao} \times \text{quantidade}$ é uma operação constante e o resultado não varia após a criação do registo.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único do registo.
idSessaoCaixa	INT	Sim	NULL	Sessão de caixa a que pertence o fundo inicial. Mutuamente exclusivo com <i>idSangria</i> .
idSangria	INT	Sim	NULL	Sangria a que pertence este detalhe. Mutuamente exclusivo com <i>idSessaoCaixa</i> .
denominacao	ENUM	Não	—	Denominação da nota ou moeda, das treze denominações do euro.
quantidade	INT	Não	—	Número de unidades desta denominação.
subtotal	DECIMAL(10,2)	Não	—	Valor calculado: $\text{denominacao} \times \text{quantidade}$. Persistido para evitar recálculo.

Tabela 25: Dicionário de dados — DetalheNumerario

A última tabela com decisão de design relevante é *LinhaRemessa*. O campo *idLoteGerado* é definido como nulável para acomodar a ordem de operações no registo de uma remessa: a linha é criada antes do lote correspondente ser persistido, existindo uma dependência sequencial entre os dois registos. O serviço cria primeiro a linha da remessa, persiste o lote com a data de validade e a quantidade recebida, e só então atualiza a linha com o identificador do lote gerado. Esta sequência é necessária porque o identificador do lote é gerado pela base de dados no momento da inserção e só fica disponível após a persistência do registo de Lote.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único da linha.

idRemessa	INT	Não	—	Remessa a que esta linha pertence.
idProduto	INT	Não	—	Produto recebido nesta linha da guia.
idLoteGerado	INT	Sim	NULL	Lote criado para as unidades recebidas. Nulo até à criação do lote.
quantidade	INT	Não	—	Número de unidades recebidas nesta linha.
dataValidade	DATE	Não	—	Data de validade das unidades recebidas, atribuída ao lote gerado.

Tabela 26: Dicionário de dados — LinhaRemessa

4.10.3. Normalização

O modelo lógico do Software de Loja foi construído de modo a respeitar a Terceira Forma Normal (3FN), seguindo os critérios definidos por Connolly e Begg (Connolly e Begg, 2015, pp. 466–471).

A 1FN é satisfeita em todas as tabelas: cada atributo armazena um único valor atômico, não existem grupos de repetição, e todas as tabelas têm chave primária definida. A 2FN é satisfeita trivialmente, uma vez que todas as tabelas utilizam chave primária simples com AUTO_INCREMENT, não existindo subconjuntos próprios da chave que possam originar dependências parciais.

A 3FN exige a ausência de dependências transitivas. A análise confirma que todos os atributos dependem direta e exclusivamente da chave primária da respetiva tabela. A título ilustrativo: em Venda, o totalFaturado e o metodoPagamento dependem do identificador da venda, não do idUtilizador; em Lote, o precoVenda e a quantidade dependem do identificador do lote, não do idProduto. O estado EM_CURSO não é persistido — uma venda só é escrita na base de dados quando finalizada ou anulada — pelo que os únicos valores do campo estado presentes em registos reais são FINALIZADA e ANULADA. Não se verificam dependências transitivas em nenhuma tabela do *schema*.

4.11. Conceção do Sistema de Dados - Software de Servidor Central

4.11.1. Modelo Lógico

O modelo lógico do Software de Servidor Central reflete o seu papel arquitetural como sistema de agregação e leitura — ao contrário do Software de Loja, que opera sobre entidades mutáveis com lógica de negócio rica, o servidor recebe dados já processados e armazena-os para análise estratégica. Esta diferença fundamental manifesta-se em três características estruturais que distinguem este *schema* do anterior.

A primeira característica é a ausência do mecanismo de *Delta Sync*. As tabelas do Servidor Central não têm coluna estadoSincronizacao — o servidor é o destino final dos dados, não um nó intermédio numa cadeia de sincronização.

A segunda característica é a idempotência por chave composta, definida no ADR-05. Todas as entidades transacionais — VendaCentral, LinhaVendaCentral, DevolucaoCentral, RemessaCentral, PagamentoCentral, SessaoCaixaCentral e LogAuditoriaCentral — possuem uma *constraint* UNIQUE (idLoja, idOriginalLoja) que garante que reenvios do mesmo pacote de fecho de dia não criam registos duplicados. Esta *constraint*, combinada com a operação INSERT ... ON DUPLICATE KEY UPDATE do MySQL, torna o processo de ingestão idempotente ao nível do registo individual.

A terceira característica é a desnormalização intencional. Ao contrário do Software de Loja, onde as relações entre entidades são navegadas por chaves estrangeiras, o Servidor Central armazena informação descritiva diretamente nas tabelas transacionais — nomeProduto e categoria em LinhaVendaCentral, e nomeFornecedor em RemessaCentral. Esta desnormalização é justificada pela ausência de catálogo de

produtos e de fornecedores no servidor central — este recebe apenas os dados operacionais agregados nos pacotes de fecho de dia, sem acesso às bases de dados locais de cada loja.

O diagrama EER do modelo lógico encontra-se ilustrado na figura seguinte.³⁸

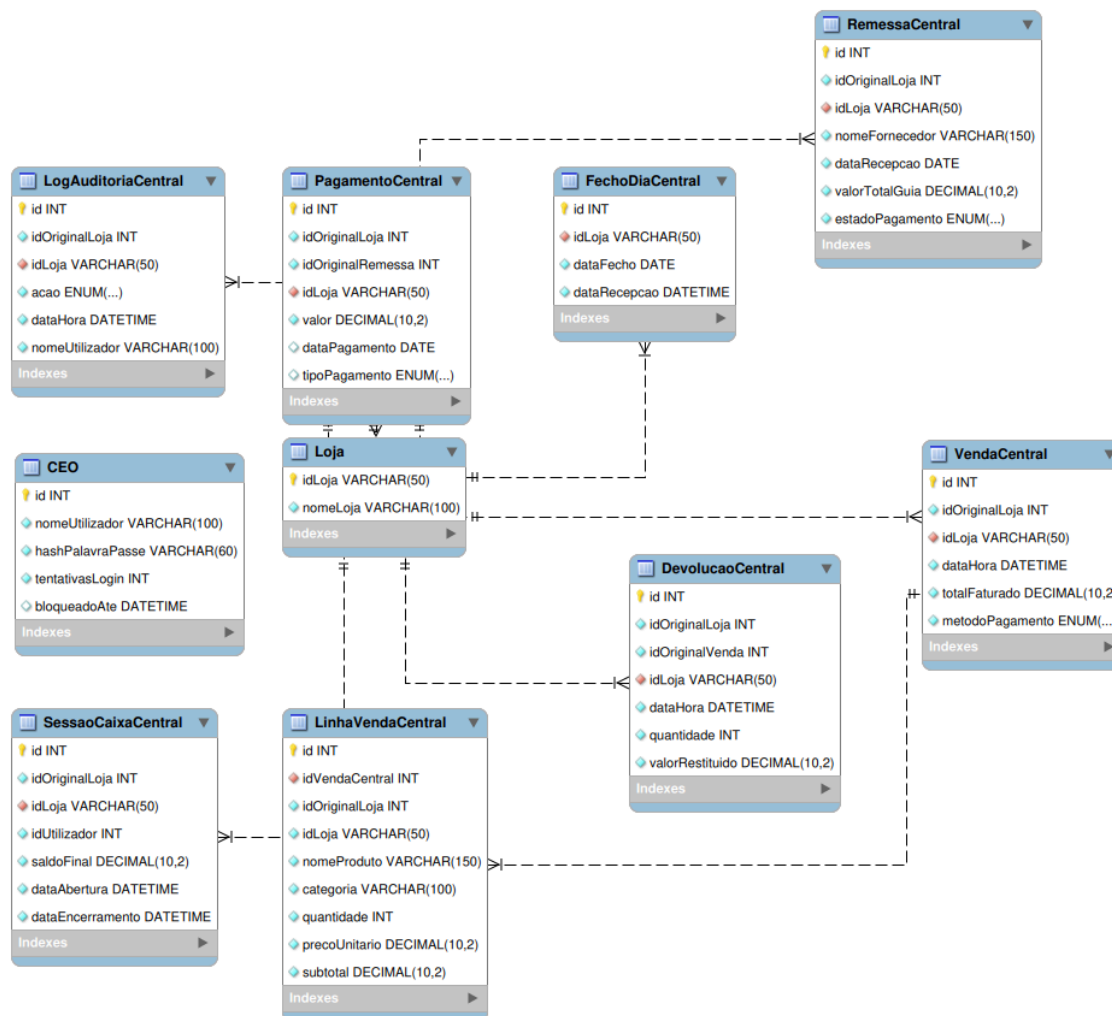


Figura 63: Modelo lógico da base de dados do Software de Servidor Central.

4.11.2. Dicionário de Dados e Decisões de Design

À semelhança do que foi feito para o Software de Loja, documentam-se apenas as tabelas com decisões de design não triviais.

A tabela CEO é a única entidade de autenticação do Servidor Central, ao contrário do Software de Loja que tem uma tabela Utilizador com múltiplos perfis. Esta singularidade justifica uma tabela própria em vez de reutilizar a estrutura de Utilizador — o CEO tem atributos específicos que não existem nos utilizadores de loja, nomeadamente tentativasLogin e bloqueadoAte, que implementam o mecanismo de bloqueio temporário após tentativas de autenticação falhadas definido no RF2. O campo bloqueadoAte é nulável porque só tem valor após o primeiro bloqueio — uma conta nunca bloqueada tem este campo a NULL.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único do CEO.
nomeUtilizador	VARCHAR(100)	Não	—	Nome único de autenticação no servidor central.

³⁸Schema SQL do Software de Servidor Central gerado com apoio de IA — ver [Anexo de IA, Entrada 43](#).

hashPalavra-Passe	VARCHAR(60)	Não	—	Hash BCrypt da palavra-passe.
tentativasLogin	INT	Não	0	Número de tentativas de autenticação falhadas consecutivas. Reposto a 0 após login bem-sucedido.
bloqueadoAte	DATETIME	Sim	NULL	Instante até ao qual a conta está bloqueada. Nulo se nunca foi bloqueada ou se o bloqueio expirou.

Tabela 27: Dicionário de dados — CEO

A tabela Loja é criada automaticamente quando o Servidor Central recebe o primeiro pacote de fecho de dia de uma loja desconhecida, conforme definido no RF15. A chave primária é o próprio idLoja — uma string que identifica a loja — em vez de um inteiro sequencial, porque o identificador já existe no ficheiro de configuração local de cada terminal e é transportado em todos os pacotes de fecho de dia.

Coluna	Tipo	Nulo	Omissão	Descrição
idLoja	VARCHAR(50)	Não	—	Identificador único da loja, lido do ficheiro de configuração local do terminal de origem.
nomeLoja	VARCHAR(100)	Não	—	Nome descritivo da loja, transportado no pacote de fecho de dia.

Tabela 28: Dicionário de dados — Loja

A tabela FechoDiaCentral regista cada pacote de fecho de dia recebido. A *constraint* UNIQUE (idLoja, dataFecho) garante que cada loja só pode ter um fecho registado por data — um reenvio do mesmo pacote é absorvido pela idempotência sem criar um segundo registo. O estado de completude da rede para uma data — se todas as lojas enviaram ou não — é calculado dinamicamente pelo ConsolidacaoServico através da comparação entre os fechos recebidos e a lista de lojas conhecidas, não sendo persistido na base de dados.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único do registo de fecho.
idLoja	VARCHAR(50)	Não	—	Loja que enviou o pacote.
dataFecho	DATE	Não	—	Data operacional do fecho enviado.
dataRecepcao	DATETIME	Não	—	Data e hora em que o pacote foi recebido pelo servidor.

Tabela 29: Dicionário de dados — FechoDiaCentral

A tabela PagamentoCentral tem dois campos nuláveis — dataPagamento e tipoPagamento — porque um pagamento pode chegar ao servidor ainda com estado pendente de liquidação. Neste caso, a data e o tipo de pagamento só ficam definidos quando o gestor liquida o pagamento na loja e o estado atualizado é transportado no pacote do fecho seguinte. Esta situação é distinta da que ocorre com SessaoCaixaCentral, onde a nulabilidade foi deliberadamente eliminada.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único do pagamento central.
idOriginalLoja	INT	Não	—	Identificador do pagamento na base de dados da loja de origem.

idOriginalRemessa	INT	Não	—	Identificador da remessa que originou este pagamento na loja de origem.
idLoja	VARCHAR(50)	Não	—	Loja de origem do pagamento.
valor	DECIMAL(10,2)	Não	—	Valor do pagamento.
dataPagamento	DATE	Sim	NULL	Data de liquidação. Nula enquanto o pagamento está pendente.
tipoPagamento	ENUM	Sim	NULL	Tipo de pagamento: TRANSFERENCIA, NUMERARIO ou CHEQUE. Nulo enquanto pendente.

Tabela 30: Dicionário de dados — PagamentoCentral

A tabela *SessaoCaixaCentral* resulta de uma decisão de design discutida no contexto do mecanismo de *Delta Sync*. Esta invariante decorre da lógica do *ICaixaServico*, que só inclui sessões encerradas nos pendentes de sincronização. — sessões ainda abertas no momento do fecho ficam em estado PENDENTE na loja e são incluídas no pacote do fecho seguinte, quando já estiverem encerradas. Esta invariante garante que o Servidor Central nunca recebe sessões abertas, tornando *dataEncerramento* um campo obrigatório nesta tabela. A *constraint* CHECK (*dataEncerramento* > *dataAbertura*) reforça a consistência temporal a nível da base de dados.

Coluna	Tipo	Nulo	Omissão	Descrição
id	INT	Não	A.I.	Identificador único da sessão central.
idOriginalLoja	INT	Não	—	Identificador da sessão na base de dados da loja de origem.
idLoja	VARCHAR(50)	Não	—	Loja de origem da sessão.
idUtilizador	INT	Não	—	Identificador do utilizador na loja de origem.
saldoFinal	DECIMAL(10,2)	Não	—	Saldo da caixa no momento do encerramento da sessão.
dataAbertura	DATETIME	Não	—	Data e hora de abertura da sessão.
dataEncerramento	DATETIME	Não	—	Data e hora de encerramento. Nunca nulo — só sessões encerradas são enviadas ao servidor.

Tabela 31: Dicionário de dados — SessaoCaixaCentral

4.11.3. Normalização

O modelo lógico do Software de Servidor Central satisfaz igualmente a 3FN, com uma particularidade decorrente da sua natureza de *schema* de agregação. A 1FN e a 2FN são satisfeitas pelos mesmos critérios descritos para o Software de Loja — atributos atómicos, ausência de grupos de repetição, e chaves primárias simples em todas as tabelas.

A nota relevante para a 3FN prende-se com a desnormalização intencional presente em *LinhaVendaCentral* e *RemessaCentral*. O armazenamento de *nomeProduto*, *categoria* e *nomeFornecedor* diretamente nestas tabelas poderia ser interpretado como uma violação da 3FN — estes atributos descrevem entidades externas ao servidor central. No entanto, não constitui uma dependência transitiva no sentido formal, porque não existe no *schema* do servidor nenhuma tabela *Produto* ou *Fornecedor* da qual estes atributos dependessem funcionalmente. Tratam-se de dados históricos desnormalizados intencionalmente para preservar a rastreabilidade dos registos independentemente de alterações futuras nos catálogos locais de cada loja.³⁹

³⁹Texto

das secções de conceção do sistema de dados gerado com apoio de IA — ver [Anexo de IA, Entrada 44](#).

4.12. Especificação da API e Utilização do Swagger

O Swagger, atualmente designado por OpenAPI Specification (OpenAPI Initiative, 2024), é uma ferramenta de documentação, desenho e validação de APIs REST que permite descrever de forma estruturada os contratos de comunicação entre sistemas — *endpoints* disponíveis, métodos HTTP suportados, estrutura dos pedidos e respostas, e códigos de estado esperados. No contexto deste projeto, o Swagger foi utilizado como instrumento de especificação antecipada da API do Servidor Central, permitindo definir e validar o contrato de comunicação antes de qualquer linha de código ser escrita. Na fase de implementação, a integração do *plugin* *java*lin-openapi permitirá gerar automaticamente a especificação OpenAPI a partir das anotações dos *handlers* e disponibilizar a interface Swagger UI em `/swagger-ui`, eliminando a necessidade de ferramentas externas para validar o comportamento dos *endpoints* durante o desenvolvimento. O valor do Swagger neste projeto concentra-se assim em dois momentos distintos: antes da implementação, como ferramenta de alinhamento contratual entre o Software de Loja — enquanto cliente da API — e o Servidor Central — enquanto fornecedor —; e durante o desenvolvimento, como cliente HTTP visual integrado na própria documentação.

Os *endpoints* estão organizados em sete grupos funcionais; alguns exemplos de *payloads* e respostas para cada grupo encontram-se no Anexo 18.⁴⁰

A autenticação cobre o ciclo de vida da sessão da CEO no Servidor Central.

Método	Endpoint	Descrição
POST	/login	Autenticação da CEO com validação de credenciais e controlo de bloqueio por tentativas falhadas
POST	/logout	Encerramento da sessão HTTP ativa

Tabela 32: Endpoints — Autenticação

A gestão de contas CEO materializa o mecanismo de *bootstrapping* definido no ADR-06. Numa instalação inicial o sistema cria automaticamente uma conta temporária com credenciais predefinidas; este *endpoint*, protegido por *token* Bearer, permite criar a primeira conta definitiva enquanto autenticado com essa conta temporária. Após o primeiro *login* com a conta definitiva, a conta temporária é eliminada automaticamente.

Método	Endpoint	Descrição
POST	/ceo	Criação da conta CEO definitiva; requer autenticação com a conta de instalação temporária

Tabela 33: Endpoints — Gestão de Contas CEO

O *endpoint* de sincronização é o ponto de entrada do mecanismo de *Delta Sync* e o mais crítico da API, responsável pela receção dos pacotes de fecho de dia enviados por cada loja. Corresponde ao fluxo descrito no DS3 e DS4 e materializa os requisitos RF15 e RF16. A idempotência é garantida através de uma chave composta por `idLoja` e `idOriginalLoja` — um reenvio do mesmo pacote não produz duplicação de registos na base de dados central. A resposta inclui o campo `estadoConsolidacao`, que complementa a consulta disponível em `/consolidacao/{dataFecho}` e informa a loja sobre o estado global da rede para aquela data. Importa notar que o Servidor Central opera sempre sobre dados do dia anterior: os dados do dia em curso só ficam disponíveis após o fecho de dia da respetiva loja, limitação que deve ser comunicada visualmente nos *dashboards* da CEO conforme o ADR-01.

Método	Endpoint	Descrição
POST	/fecho	Receção, validação e persistência do pacote de fecho de dia enviado pelo Software de Loja

Tabela 34: Endpoints — Sincronização

⁴⁰Texto das secções da especificação de API e utilização do swagger — ver [Anexo de IA, Entrada 45](#).

O painel analítico da CEO é servido por dois *endpoints* de métricas — um global para toda a rede e um por loja —, calculados a partir dos dados consolidados recebidos via fecho de dia.

Método	Endpoint	Descrição
GET	/dashboard/metricas	Métricas globais da rede (total de vendas, devoluções, transações e remessas pendentes)
GET	/dashboard/metricas/{idLoja}	Métricas detalhadas para uma loja específica, com repartição por método de pagamento

Tabela 35: Endpoints — Dashboard e Métricas

Os relatórios analíticos são filtráveis por loja, período temporal e categoria de produto, correspondendo ao caso de uso “Consultar Dashboards e Relatórios Analíticos”.

Método	Endpoint	Descrição
GET	/relatorios	Geração de relatório analítico com suporte a filtros por loja, período e categoria

Tabela 36: Endpoints — Relatórios Analíticos

A consulta consolidada de remessas e pagamentos a fornecedores em toda a rede é disponibilizada com suporte a filtros por estado de pagamento e período.

Método	Endpoint	Descrição
GET	/remessas	Consulta de remessas da rede com filtros por estado de pagamento, período e lojas

Tabela 37: Endpoints — Remessas da Rede

O estado de consolidação da rede permite verificar quais as lojas que já enviaram o fecho de dia para uma determinada data, correspondendo ao caso de uso “Consultar Estado de Consolidação da Rede”.

Método	Endpoint	Descrição
GET	/consolidacao/{dataFecho}	Estado de consolidação da rede para a data indicada, com identificação das lojas em falta

Tabela 38: Endpoints — Estado de Consolidação

4.13. Revisão e Aprovação do Projeto

A fase de arquitetura e design culminou numa reunião de validação com o Hélder e a Cátia, destinada à revisão dos artefactos produzidos e das decisões estruturantes da solução. Durante esta sessão, foram apresentados os esboços das interfaces de ambos os softwares e explicada, em linguagem não técnica, a separação do sistema em dois componentes distintos: o Software de Loja, instalado nos terminais de cada estabelecimento, e o Software de Servidor Central, acessível remotamente pela direção. Foi ainda explicitada a motivação para a escolha de uma aplicação *desktop* no Software de Loja — garantindo operação contínua mesmo na ausência de conectividade externa — e de uma interface web no Software de Servidor Central — permitindo à CEO aceder a *dashboards* e relatórios a partir de qualquer dispositivo sem instalação de software adicional. Ambos validaram as decisões sem reservas. A anuência do investidor conferiu a autorização necessária para que a equipa avançasse para a fase de Implementação.⁴¹

⁴¹Texto gerado com apoio de IA — ver [Anexo de IA, Entrada 52](#).



TrasmUM – Reunião 5

Data: 2026/03/20

Duração prevista: 30 minutos

Horário de início: 10h00

Horário de término: 10h30

Participantes: Hélder Cerqueira (helderCerqueira@trasmum.pt)
Gabriel Dantas (a107291@uminho.pt)
José Fernandes (a106937@uminho.pt)
Luís Ferreira (a98286@uminho.pt)
Simão Oliveira (a107322@uminho.pt)

Pauta da reunião:

- Apresentação da estrutura geral do sistema em linguagem não técnica, nomeadamente a separação em dois softwares distintos: o Software de Loja, instalado nos terminais de cada estabelecimento, e o Software de Servidor Central, acessível remotamente pela direção.
- Apresentação dos esboços das interfaces de ambos os softwares.
- Validação da decisão de adotar uma aplicação desktop no Software de Loja, garantindo operação contínua mesmo na ausência de conectividade externa.
- Validação da decisão de adotar uma interface web no Software de Servidor Central, permitindo à CEO aceder a dashboards e relatórios a partir de qualquer dispositivo sem instalação de software adicional.
- Aprovação formal do trabalho desenvolvido na fase de Arquitetura e Design do Software.

Figura 64: Ata da reunião de revisão e aprovação da arquitetura e design do software.

5. Implementação e Desenvolvimento Assistido por LLM

5.1. Utilização de LLMs na Fase de Implementação

Para a implementação do software, o papel dos modelos alterou-se qualitativamente face às fases anteriores: deixaram de ser interlocutores consultivos e passaram a ser executores de código, com a equipa a assumir o papel de revisora e integradora. Esta mudança motivou duas práticas novas — o recurso a um LLM como arquiteto de *prompts*, e o registo sistemático de desvios de implementação relativamente ao design original.

O Claude Sonnet assumiu o papel de arquiteto de *prompts*: recebeu a totalidade dos artefactos da fase de design — diagramas de classe, requisitos, casos de uso, diagramas de sequência, *schema* SQL e ADRs — e produziu instruções estruturadas destinadas ao Claude Code.⁴² Perante a dimensão do Software de Loja, o Sonnet decidiu autonomamente dividir a instrução em dois *prompts* sequenciais, argumentando que um escopo excessivo induziria degradação de qualidade no agente executor. O primeiro *prompt*⁴³ cobriu a fundação — estrutura Maven, *schema* SQL, classes de domínio, repositórios JDBC e esqueleto do `AppContext` —, validada por compilação antes de se avançar para o segundo⁴⁴, que cobriu serviços, *gateway* de sincronização, *controllers* JavaFX e vistas FXML. O Servidor Central foi objeto de um único *prompt*⁴⁵, justificado pelo escopo menor e pela existência de um *frontend* Vue.js já gerado pelo Gemini a partir dos ficheiros `.pen` do Pencil, cujo contrato de API ficou implicitamente definido pelas chamadas `fetch` presentes no `networkService.ts`. O mesmo *prompt* cobriu ainda a geração do `openapi.yaml` com a especificação completa dos *endpoints*, *schemas* e exemplos de *payload*.⁴⁶

O registo sistemático de desvios acompanhou toda a fase de implementação. Cada interação com o Claude Code era acompanhada de instrução explícita para documentar qualquer alteração com impacto no modelo de domínio, no contrato de serviços ou na arquitetura, tornando explícita a divergência inevitável entre os diagramas e o modelo implementado.⁴⁷

5.2. Processo de Implementação e Divisão de Trabalho

Dado que o sistema TrasmUM é composto por dois softwares distintos e independentes, a equipa dividiu-se em dois pares durante esta fase: um par ficou responsável pelo Software de Loja e outro pelo Software de Servidor Central. Esta divisão permitiu o desenvolvimento paralelo e tirou partido do desacoplamento de ciclos de vida previsto na arquitetura, sendo a integração entre os dois softwares realizada numa fase posterior, quando ambos os componentes tinham atingido um nível de maturidade compatível com testes *end-to-end*.

A distribuição interna de tarefas em cada par seguiu a ordem natural da implementação *bottom-up* — fundação Maven, *schema*, domínio, repositórios, serviços, apresentação — concluindo com a fase de revisão e *refactoring*. A figura seguinte sintetiza as tarefas atribuídas a cada par.

⁴²Prompts de implementação gerados com apoio do Claude Sonnet — ver [Anexo de IA, Entrada 47](#).

⁴³Primeiro prompt fornecido para implementação do Software de Loja — ver [Anexo de IA, Entrada 48](#).

⁴⁴Segundo prompt fornecido para implementação do Software de Loja — ver [Anexo de IA, Entrada 49](#).

⁴⁵Prompt fornecido para implementação do Servidor Central — ver [Anexo de IA, Entrada 50](#).

⁴⁶Prompt fornecido para implementação do Servidor Central — ver [Anexo de IA, Entrada 50](#).

⁴⁷Texto do Capítulo 3 gerado com apoio de IA — ver [Anexo de IA, Entrada 54](#).

Tarefa	Responsáveis	Tarefa	Responsáveis
Configuração Maven e schema SQL	Simão, José	Configuração Maven do backend e schema SQL	Luís, Dantas
Camada de domínio	Simão, José	Camada de domínio e DTOs de ingestão	Luís, Dantas
Repositórios JDBC e infraestrutura de conexão	Simão, José	Repositórios JDBC com idempotência	Luís, Dantas
Composition root inicial e validação de compilação	Simão, José	Serviço de ingestão e verificação de hash	Luís, Dantas
Serviços core (autenticação, autorização, auditoria)	Simão, José	Serviços de autenticação CEO e dashboards	Luís, Dantas
Serviços operacionais (venda, devolução, caixa)	Simão, José	Handlers HTTP Javalin e middleware de autenticação	Luís, Dantas
Serviços de gestão (catálogo, remessa, fornecedor, utilizador)	Simão, José	Reescrita do networkService.ts no frontend	Luís, Dantas
Gateway de sincronização e fecho de dia	Simão, José	LoginView.vue e navigation guard	Luís, Dantas
Controllers JavaFX e ficheiros FXML	Simão, José	Validação end-to-end com a Loja	Luís, Dantas
Estilização CSS e validação end-to-end	Simão, José	Revisão e refactoring	Luís, Dantas
Geração de fatura PDF	Simão, José		
Revisão e refactoring	Simão, José		

Figura 65: Divisão de tarefas de implementação por par da equipa.

5.3. Configuração do Ambiente de Trabalho

A configuração do ambiente de trabalho foi definida tendo por base as decisões tecnológicas do ADR-10 e complementada com ferramentas auxiliares para desenvolvimento, controlo de versões e geração de código. Os dois softwares partilham a mesma *stack* Java: Java 21 como linguagem de desenvolvimento, Maven (Apache Software Foundation, 2024) como sistema de gestão de dependências e *build*, e MySQL 8 (Oracle Corporation, 2024) como sistema de gestão de base de dados. O Software de Loja utiliza adicionalmente JavaFX 21 (Gluon and Oracle, 2024) para a interface gráfica, jBCrypt 0.4 para *hashing* de palavras-passe, Gson 2.10.1 para serialização JSON na comunicação com o Servidor Central, e OpenPDF 1.3.30 para geração de faturas em PDF. O Software de Servidor Central utiliza Javalin 6.3 (Javalin, 2024) como servidor HTTP, Gson 2.10.1 e jBCrypt 0.4, e o *frontend* Vue.js (You e Vue.js Contributors, 2024) recorre a Vite como ferramenta de *build* e TypeScript como linguagem. As escolhas de versão foram pautadas pela estabilidade à data de desenvolvimento: o Java 21 é uma versão LTS com suporte alargado para os próximos anos e o Javalin 6.3 é a última versão estável compatível com Java 21 sem regressões conhecidas.

Cada software tem o seu próprio *script* de schema SQL. Estes *scripts* criam todas as tabelas definidas na secção de Conceção do Sistema de Dados, incluindo as colunas de estadoSincronizacao e idLoja nas tabelas operacionais do Software de Loja e as *constraints* UNIQUE (idLoja, idOriginalLoja) nas tabelas centrais do Servidor Central. O mecanismo de *bootstrapping* previsto no ADR-06 foi implementado como parte da raiz de composição de cada software. No Software de Loja, o AppContext verifica no arranque se existe uma conta com nome de utilizador admin na tabela Utilizador; se não existir, é criada automaticamente uma conta com perfil CEO e credenciais admin/admin123, com a palavra-passe cifrada com BCrypt. No Software de Servidor Central, o AppContext executa a mesma lógica sobre a tabela CEO. Ao contrário da especificação original do ADR-06 — que previa a eliminação automática da conta após a criação da primeira conta definitiva — a implementação atual elimina a conta admin apenas quando é registado um *login* bem-sucedido com uma conta diferente; esta alteração foi necessária para evitar que a conta de *bootstrap* fosse eliminada no seu próprio primeiro *login*, deixando o sistema sem qualquer conta ativa.

Cada instância do Software de Loja lê no arranque um ficheiro de configuração que contém o identificador único da loja, o nome da loja, os dados de contacto usados na geração de faturas (morada, localidade, NIF, email), os parâmetros de ligação à base de dados local, e o URL do Servidor Central para sincronização. No Software de Servidor Central, o ficheiro homónimo contém os parâmetros da base de dados central e a origem permitida para pedidos CORS do *frontend*. O desenvolvimento decorreu em IntelliJ IDEA para o código Java e em VS Code para o *frontend* Vue, com o repositório mantido no GitHub. O controlo de versões foi usado em conformidade com práticas padrão de Git, com *commits* incrementais à medida que cada módulo ou camada era gerado e validado, e mensagens de *commit* a referenciar explicitamente o *prompt* ou a entrada de anexo que tinha motivado a alteração.

5.4. Desenvolvimento da Camada de Domínio - Software de Loja

A camada de domínio é composta por classes Java simples organizadas em quatro sub-pacotes alinhados com os módulos funcionais dos diagramas de classe: core, catalogo, fornecedores, vendas e tesouraria. O elemento estruturante desta camada é a classe abstrata `RegistoSincronizavel`, da qual herdam todas as entidades transmitidas no fecho de dia — Venda, Devolucao, Remessa, Pagamento, SessaoCaixa, FechoDia e LogAuditoria. A classe abstrata expõe os métodos `marcarEmTransito()`, `marcarConfirmado()` e `reverterParaPendente()`, assegurando que o ciclo de vida do estado de sincronização é manipulado uniformemente pelo `FechoDiaServico`.

```
package pt.trasmum.loja.dominio.core;

public abstract class RegistoSincronizavel {

    protected int id;
    protected EstadoSincronizacao estadoSincronizacao;
    protected String idLoja;

    protected RegistoSincronizavel() {
        this.estadoSincronizacao = EstadoSincronizacao.PENDENTE;
    }

    protected RegistoSincronizavel(String idLoja) {
        this.idLoja = idLoja;
        this.estadoSincronizacao = EstadoSincronizacao.PENDENTE;
    }

    public void marcarEmTransito() {
        this.estadoSincronizacao = EstadoSincronizacao.EM_TRANSITO;
    }

    public void marcarConfirmado() {
        this.estadoSincronizacao = EstadoSincronizacao.CONFIRMADO;
    }

    public void reverterParaPendente() {
        this.estadoSincronizacao = EstadoSincronizacao.PENDENTE;
    }
}
```

Listagem 1: Implementação da classe `RegistoSincronizavel`

As exceções de domínio foram organizadas em pacotes por módulo funcional — `ExcecoesCatalogo` para `ProdutoNaoEncontradoException`, `ProdutoInativoException` e `StockInsuficienteException`; `ExcecoesSeguranca` para `CredenciaisInvalidasException`, `SessaoDuplicadaException` e `AcessoNegadoException` — para evitar a proliferação de ficheiros e para comunicar visualmente a pertença de cada exceção a um contexto específico.

```
package pt.trasmum.loja.dominio.exceptions;

public final class ExcecoesCatalogo {
    private ExcecoesCatalogo() {}

    public static class ProdutoInativoException extends RuntimeException {
        public ProdutoInativoException(String message) { super(message); }
    }

    public static class ProdutoNaoEncontradoException extends RuntimeException {
        public ProdutoNaoEncontradoException(String message) { super(message); }
    }

    public static class ProdutoNaoFornecidoException extends RuntimeException {
        public ProdutoNaoFornecidoException(String message) { super(message); }
    }
}
```

```

    }
}

```

Listagem 2: Exemplo de implementação de exceções: exceções de catálogo

5.5. Desenvolvimento da Camada de Repositório - Software de Loja

A camada de repositórios segue o padrão Repository. As implementações utilizam JDBC puro sobre a Connection injetada pelo ApplicationContext. Todos os repositórios de entidades sincronizáveis expõem buscarPendentes(), atualizar() e reverterEmTransito(), conforme o ciclo de vida descrito no ADR-04 e detalhado no DS3.

```

package pt.trasmum.loja.repositorio.impl;

public class VendaRepositorioImpl implements VendaRepositorio {

    private final Connection connection;

    public VendaRepositorioImpl(Connection connection) {
        this.connection = connection;
    }

    public void reverterEmTransito() {
        String sql = "UPDATE Venda SET estadoSincronizacao = 'PENDENTE' WHERE estadoSincronizacao = 'EM_TRANSITO'";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RuntimeException("Erro ao reverter vendas em trânsito", e);
        }
    }

    public List<Venda> buscarPendentes() {
        String sql = "SELECT * FROM Venda WHERE estadoSincronizacao = 'PENDENTE'";
        List<Venda> lista = new ArrayList<>();
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ResultSet rs = ps.executeQuery();
            while (rs.next()) lista.add(mapearComDetalhes(rs));
        } catch (SQLException e) {
            throw new RuntimeException("Erro ao buscar vendas pendentes", e);
        }
        return lista;
    }

    public void atualizarSincronizacao(int id, EstadoSincronizacao estado) {
        String sql = "UPDATE Venda SET estadoSincronizacao = ? WHERE id = ?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, estado.name());
            ps.setInt(2, id);
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RuntimeException("Erro ao atualizar sincronização de venda", e);
        }
    }
}

```

Listagem 3: Implementação de buscarPendentes(), atualizar() e reverterEmTransito()

Particular atenção foi dada à persistência de agregados compostos. O `VendaRepositorioImpl` decompõe o armazenamento de uma venda em três operações internas — `guardarCabecalhoVenda`, `guardarLinhaVenda` e `guardarFatura` — encapsulando a complexidade relacional sem a expor à camada superior. De forma análoga, o `SessaoCaixaRepositorioImpl` gere internamente a persistência das sangrias associadas a uma sessão. Esta decomposição é invisível aos serviços, que invocam um único `guardar(venda)` ou `guardar(sessao)`.

5.6. Desenvolvimento da Camada de Serviço - Software de Loja

A camada de serviços concentra a lógica de negócio do sistema e foi a mais exigente em especificação no *Prompt* 2. Três serviços merecem menção pela complexidade comportamental: o `VendaServico`, o `DevolucaoServico` e o `FechoDiaServico`.

O `VendaServico` implementa a lógica FEFO no método `adicionarLinha`: após validar a existência e o estado ativo do produto, invoca `buscarLotesFEFO` no `LoteRepositorio` para obter os lotes ordenados por data de validade ascendente, verifica o stock total disponível, e decrementa dos lotes mais antigos primeiro.

```
public LinhaVenda adicionarLinha(Venda venda, String codigoBarras, int quantidade) {
    Produto produto = produtoRepo.buscarPorCodigoBarras(codigoBarras);
    if (produto == null) throw new ProdutoNaoEncontradoException("Produto não
encontrado: " + codigoBarras);
    if (!produto.isAtivo()) throw new ProdutoInativoException("Produto inativo: " +
produto.getNome());

    List<Lote> lotesFEFO = loteRepo.buscarLotesFEFO(produto.getId());
    int totalDisponivel = lotesFEFO.stream().mapToInt(Lote::getQuantidade).sum();
    if (totalDisponivel < quantidade) {
        throw new StockInsuficienteException(
            "Stock insuficiente para '" + produto.getNome() + "'. Disponível: "
+ totalDisponivel);
    }
    // FEFO: decrementa dos lotes mais antigos primeiro
    // Cria uma LinhaVenda por lote consumido; retorna a primeira
    LinhaVenda primeira = null;
    int restante = quantidade;
    for (Lote lote : lotesFEFO) {
        if (restante <= 0) break;
        int retirar = Math.min(lote.getQuantidade(), restante);
        lote.setQuantidade(lote.getQuantidade() - retirar);
        loteRepo.atualizar(lote);

        LinhaVenda linha = new LinhaVenda(lote.getId(), retirar,
lote.getPrecoFinal());
        venda.adicionarLinha(linha);
        if (primeira == null) primeira = linha;
        restante -= retirar;
    }
    return primeira;
}
```

Listagem 4: Método responsável por adicionar uma nova linha de venda.

Uma decisão de design que foi tomada durante a geração — e que posteriormente motivou uma correção na camada de apresentação — foi a de criar uma `LinhaVenda` por cada lote consumido, em vez de uma única `LinhaVenda` com agregação. Esta granularidade é necessária para preservar a rastreabilidade ao lote, mas obrigou o `VendaController` a agregar visualmente linhas com o mesmo produto e mesmo preço unitário antes de as apresentar na tabela do carrinho, para evitar confundir o funcionário.

O `DevolucaoServico` implementa as quatro guardas em sequência, conforme especificado no *Prompt 2*. A primeira guarda procura a fatura no `VendaRepositorio` — se o resultado for nulo, a operação é rejeitada com uma `FaturaNaoEncontradaException`; se a fatura existir mas tiver sido emitida noutra loja, a mesma exceção é lançada com mensagem específica. A segunda guarda verifica que o artigo identificado pelo código de barras consta nas linhas da fatura; a terceira guarda calcula quantas unidades desse artigo restam por devolver, consultando as devoluções anteriores associadas à fatura; a quarta guarda utiliza a data de validade impressa na embalagem para identificar o lote correto e valida se esta é posterior à data corrente, determinando se o stock é reposto ou se a devolução é aceite apenas financeiramente.

O `FechoDiaServico` implementa o fluxo de *Delta Sync* descrito no DS3, orquestrando a reversão preventiva, recolha de pendentes, marcação em trânsito, envio via `SincronizacaoGateway`, e confirmação ou reversão consoante o resultado.

```
public FechoDia executarFecho(Utilizador utilizador, ConfiguracaoTerminal config) {
    autorizacaoServico.exigirPerfil(utilizador, PerfilUtilizador.GESTOR,
    PerfilUtilizador.CEO);

    List<Venda> vendas = vendaServico.obterVendasPendentes();
    List<Devolucao> devolucoes = devolucaoServico.obterDevolucoesPendentes();
    List<SessaoCaixa> sessoes = caixaServico.obterSessoesPendentes();
    List<Remessa> remessas = remessaServico.obterRemessasPendentes();
    List<Pagamento> pagamentos = remessaServico.obterPagamentosPendentes();
    List<LogAuditoria> logs = auditoriaServico.obterLogsPendentes();

    vendas.forEach(v -> { v.marcarEmTransito(); atualizarSincVenda(v); });
    devolucoes.forEach(d -> { d.marcarEmTransito(); atualizarSincDevolucao(d); });
    sessoes.forEach(s -> { s.marcarEmTransito(); sessaoCaixaRepo.atualizar(s); });
    remessas.forEach(r -> { r.marcarEmTransito(); atualizarSincRemessa(r); });
    pagamentos.forEach(p -> { p.marcarEmTransito(); pagamentoRepo.atualizar(p); });
    logs.forEach(l -> { l.marcarEmTransito(); logRepo.atualizar(l); });

    PacoteFechoDTO pacote = construirPacote(config, vendas, devolucoes, remessas,
    pagamentos, sessoes, logs);
    boolean sucesso = sincronizacaoGateway.enviarFechoDia(pacote);
    if (!sucesso) {
        reverterEmTransito();
        throw new EnvioFechoFalhouException("Falha no envio do fecho de dia ao
    servidor central.");
    }

    vendas.forEach(v -> { v.marcarConfirmado(); atualizarSincVenda(v); });
    devolucoes.forEach(d -> { d.marcarConfirmado(); atualizarSincDevolucao(d); });
    sessoes.forEach(s -> { s.marcarConfirmado(); sessaoCaixaRepo.atualizar(s); });
    remessas.forEach(r -> { r.marcarConfirmado(); atualizarSincRemessa(r); });
    pagamentos.forEach(p -> { p.marcarConfirmado(); pagamentoRepo.atualizar(p); });
    logs.forEach(l -> { l.marcarConfirmado(); logRepo.atualizar(l); });

    FechoDia fecho = new FechoDia(config.getIdLoja(), utilizador.getId(),
    LocalDate.now());
    fecho.marcarConfirmado();
    fechoDiaRepo.guardar(fecho);

    auditoriaServico.registar(utilizador, TipoAcao.FECHO_DIA, "FechoDia",
    fecho.getId(),
        "Fecho de dia executado para " + fecho.getDataFecho());

    return fecho;
}
```

Listagem 5: Método responsável por executar o fecho de dia.

Um elemento transversal à camada é o `AutorizacaoServico`, consumido por praticamente todos os restantes serviços para verificação de permissões. A sua invocação via `exigirPerfil` no início de cada

operação sensível traduz em código a restrição documentada no RF23 e materializa o princípio de que a segregação de responsabilidades é aplicada em tempo de execução, não apenas na configuração da interface.

5.7. Desenvolvimento da Camada de Apresentação - Software de Loja

A camada de apresentação é composta por *controllers* JavaFX, ficheiros FXML e uma folha de estilo CSS. Cada vista tem um *controller* dedicado, coordenados por um *MainController* que gere o menu lateral adaptativo ao perfil do utilizador autenticado e a navegação por substituição do centro do *BorderPane* principal. Os *controllers* capturam os eventos da interface, invocam o serviço correspondente através das interfaces expostas pelo *AppContext*, e atualizam a vista com o resultado retornado, mantendo a separação rigorosa entre apresentação e lógica de negócio.

A geração da interface apresentou desafios específicos. Os esboços produzidos no Pencil na fase de design não podem ser exportados diretamente para FXML, pelo que o *Prompt 2* recebeu apenas uma descrição textual da estrutura de cada ecrã. Decisões como o uso de *TabPane* para agrupar fluxos funcionalmente relacionados em vez de janelas modais, a adoção de *TableView* em detrimento de *ListView* para coleções com múltiplas colunas informativas, ou o uso de *FlowPane* na grelha de produtos do ecrã de venda foram tomadas pelo modelo na geração inicial e posteriormente validadas pela equipa por se alinharem com as convenções idiomáticas do JavaFX e com o padrão de utilização de cada ecrã. O resultado é uma interface funcional que cumpre todos os fluxos especificados, ainda que com aparência distinta dos esboços iniciais — divergência registada e validada com o investidor e a CEO conforme descrito na secção de Desvios ao Design Original.

Um aspeto transversal a toda a camada de apresentação é o *GestorEscala*, um componente *singleton* que centraliza o fator de escala da interface e expõe-no como propriedade observável. Cada vista relevante regista um *listener* neste fator e ajusta dinamicamente os tamanhos de fonte, espaçamentos e dimensões dos painéis, permitindo que o operador adapte a interface ao tamanho do monitor do terminal sem reiniciar a aplicação.

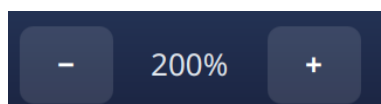


Figura 66: Botões de controlo de escala da interface.

Complementarmente, o menu lateral do *MainController* é construído de forma adaptativa em função do perfil do utilizador autenticado: o Funcionário acede apenas às operações de frente de loja — venda, catálogo de consulta e caixa —, ao passo que o Manager e o CEO veem ainda os botões de Aprovisionamento, Pagamento, Administração e Fecho de Dia. Esta lógica é implementada no método `initialize()` do *MainController*, que torna invisíveis e não-geridos os componentes de gestão para perfis sem permissão, em vez de simplesmente os desativar — abordagem que evita a exposição visual de funcionalidades indisponíveis.



Figura 67: Menu lateral adaptativo: à esquerda, sessão de Funcionário com acesso restrito às operações de frente de loja; à direita, sessão de CEO com a totalidade dos acessos de gestão.

O ecrã de autenticação concretiza o requisito de segurança RNF6 ao bloquear a conta após cinco tentativas falhadas consecutivas. O `LoginController` mantém um contador de tentativas e um instante de desbloqueio, exibindo um cronómetro regressivo que se atualiza em tempo real através de uma `Timeline` JavaFX. Durante o bloqueio, os campos de credenciais e o botão de submissão ficam desativados, garantindo a continuidade operacional sem necessidade de intervenção hierárquica imediata.

Bem-vindo

Inicie sessão para continuar

Nome de utilizador

admin

Palavra-passe



Conta bloqueada por demasiadas tentativas. Tente novamente em 14:57.

Entrar

Figura 68: Ecrã de autenticação após cinco tentativas falhadas, com cronómetro regressivo e bloqueio dos campos de credenciais.

A vista de Venda agrupa o registo de venda e o processamento de devolução num mesmo `TabPane`, com o separador “Devolução” incluído via `fx:include` da `DevolucaoView`. Esta integração — desvio relativamente ao desenho original, que previa uma entrada de menu autónoma para devoluções — fundamenta-se na proximidade funcional entre os dois fluxos: ambos são iniciados pelo Funcionário no contexto de uma transação de caixa, ambos consultam a mesma base de produtos e lotes, e ambos terminam com efeitos no saldo da sessão de caixa. O painel esquerdo apresenta a grelha de produtos disponíveis com filtragem em tempo real por nome ou categoria, e o painel direito contém o registo da venda em curso, com cálculo automático de subtotal, total e troco e indicação visível do método de pagamento selecionado.

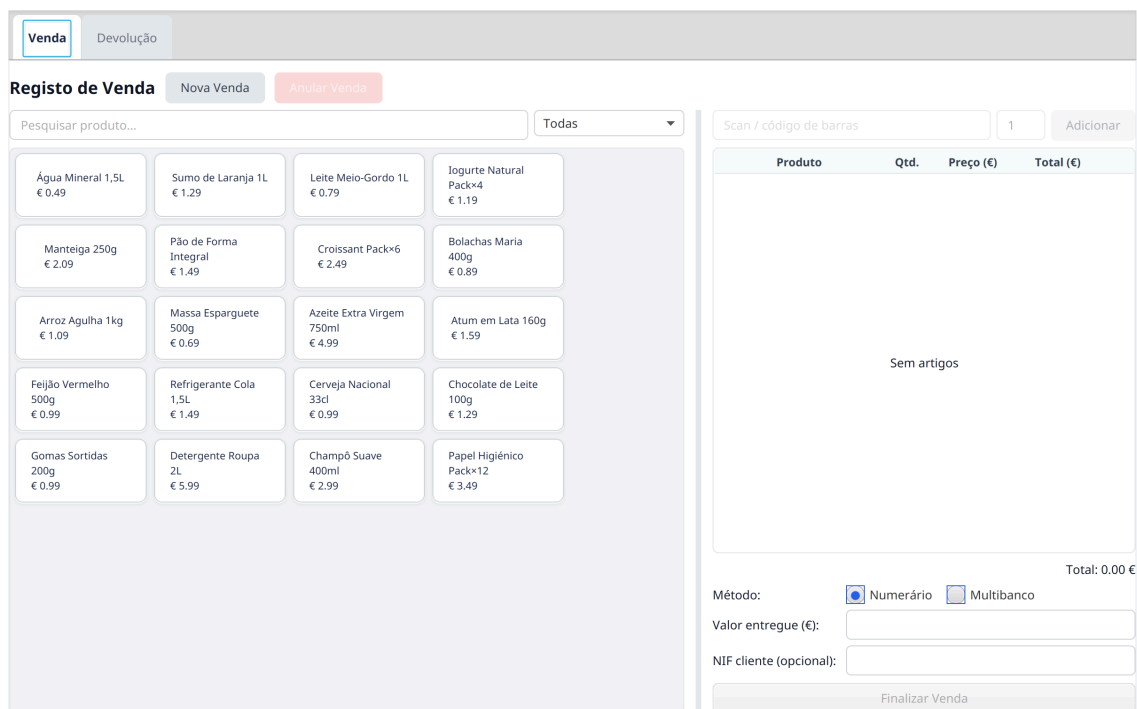


Figura 69: Vista de Venda com TabPane Venda/Devolução: integração dos dois fluxos funcionalmente relacionados num único contexto de operação.

O ecrã de Catálogo combina a listagem completa de produtos com um painel lateral de alertas, gerados em tempo real pelo `CatalogoServico` a partir de duas fontes distintas: o `stockTotal` agregado por produto, comparado com o `stockMinimo` configurado, e a lista de lotes cuja `dataValidade` está abaixo do limiar de dias de alerta definido em `ConfiguracaoLoja`. Os alertas são ordenados por criticidade — primeiro os produtos com stock zero, depois os lotes já expirados, e por fim os lotes em alerta de validade — e cada entrada inclui uma ação de navegação direta para o produto ou lote em causa, agilizando a resposta operacional do gestor.

Alertas

▲ Stock mínimo:

- Pão de Forma Integral
- Croissant Pack×6

▲ A vencer (7 dias):

- Gomas Sortidas 200g — val. 2026-05-08 (5 un.) [Desconto](#)

Figura 70: Vista de Catálogo com painel de alertas ativos: ruturas de stock e lotes em alerta de validade, ordenados por criticidade.

Na vista de Aprovisionamento ao iniciar o registo de uma remessa ou a criação de um pedido, o operador seleciona primeiro o fornecedor, e a grelha de produtos é então restringida apenas aos artigos que esse fornecedor efetivamente fornece.

Figura 71: Vista de Aprovisionamento com filtragem dinâmica de produtos por fornecedor, refletindo a relação FornecedorProduto introduzida na fase de implementação.

5.8. Gateway de Sincronização e raiz de composição - Software de Loja

O `HttpSincronizacaoGateway` encapsula a comunicação HTTPS com o Servidor Central. O seu método `enviarFechoDia` serializa o `PacoteFechoDT0` com o `Gson` — configurado com serializadores para `LocalDate` e `LocalDateTime` via `toString` —, calcula o hash SHA-256 do JSON sem o campo `hashIntegridade` preenchido, adiciona o *hash* calculado ao pacote, e envia o pacote completo via HTTP POST. O cliente HTTP é construído com um *truststore* personalizado para aceitar o certificado auto-assinado do servidor, conforme especificado no ADR-08.

```
public boolean enviarFechoDia(PacoteFechoDT0 pacote) {
    try {
        // Calcula hash antes de serializar o pacote completo
        String jsonSemHash = gson.toJson(pacote);
        pacote.hashIntegridade = sha256(jsonSemHash);

        String json = gson.toJson(pacote);
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(urlServidor + "/fecho"))
            .header("Content-Type", "application/json")
            .timeout(Duration.ofMinutes(5))
            .POST(HttpRequest.BodyPublishers.ofString(json,
                StandardCharsets.UTF_8))
            .build();

        HttpResponse<String> response = httpClient.send(request,
            HttpResponse.BodyHandlers.ofString());
        return response.statusCode() == 200;
    } catch (Exception e) {
        return false;
    }
}
```

Listagem 6: Método responsável por enviar fecho de dia ao servidor.

O `AppContext` é a raiz de composição da aplicação, implementado como um *singleton* que instancia e injeta manualmente todos os componentes no arranque. O construtor do `AppContext` invoca três operações de recuperação após a instanciação dos serviços: `limparSessoesOrfas` coloca todos os utilizadores fora de sessão — necessário porque o encerramento abrupto da aplicação deixa o *flag* em Sessão a 1 e bloquearia novos *logins*; `garantirContaAdmin` implementa o *bootstrapping* do ADR-06; e `reverterEstadosEmTransito` invoca a operação homónima em todos os repositórios sincronizáveis.

```
private AppContext() {
    this.configuracao = ConfiguracaoTerminal.carregar();
    this.conexao = DatabaseConnection.getInstance().getConnection();

    // Repositórios
```

```

*INICIALIZAÇÃO DE TODOS OS REPOSITÓRIOS*

// Gateway
Properties p = carregarProps();
this.sincronizacaoGateway = new HttpSincronizacaoGateway(
    configuracao.getUrlServidor(),
    p.getProperty("servidor.truststore.path", ""),
    p.getProperty("servidor.truststore.password", ""));

// Serviços (por ordem de dependência)
*INICIALIZAÇÃO DE TODOS OS SERVIÇOS*

limparSessoesOrfas();
garantirContaAdmin();
reverterEstadosEmTransito();
bootstrapLoja();
}

```

Listagem 7: Construtor da classe AppContext.

5.9. Desenvolvimento do Backend - Servidor Central

O *backend* segue a mesma arquitetura em camadas do Software de Loja, com a camada de apresentação substituída por *handlers* HTTP Javalin. A organização de pacotes — domínio, dto, repositório, serviço, handler, infra, app — espelha a do Software de Loja e facilita a leitura cruzada por quem já conhece o primeiro. A composição é feita imperativamente no *ServerMain*, que instancia repositórios, serviços e *handlers* e registra as rotas Javalin, mantendo a mesma abordagem sem *framework* de injeção de dependências adotada na Loja.

A fronteira de autenticação foi materializada via *middleware* Javalin. O *ServerMain* registra um `app.before("/api/*")` que invoca `authHandler.verificarToken`, rejeitando com HTTP 401 qualquer pedido sem *token* Bearer válido. Os *endpoints* `/api/auth/login` e `/api/auth/logout` estão explicitamente isentos da verificação dentro do próprio *middleware* — caso contrário, o *login* seria impossível por falta de *token* —, bem como os pedidos *OPTIONS* necessários ao mecanismo CORS de *pre-flight* do navegador. O *endpoint* `/api/ceo`, que implementa o *bootstrapping* do ADR-06, está sob o prefixo `/api/*` e portanto exige *token* Bearer — o único momento em que é invocado é enquanto a conta de instalação temporária está ativa, pelo que o *token* é o da sessão dessa conta. O *endpoint* `/fecho`, por sua vez, está deliberadamente fora do prefixo `/api/*` por duas razões: não requer autenticação CEO — é chamado pelas lojas, não pelo *frontend* —, e a sua exposição direta em `/fecho` simplifica o URL usado pelos clientes HTTP das lojas. A tabela seguinte sintetiza o conjunto de *endpoints* expostos pelo servidor e o seu modelo de autenticação.

Endpoint	Método	Autenticação	Consumidor
/fecho	POST	Pública	Software de Loja
/ping	GET	Pública	Healthcheck
/api/auth/login	POST	Pública	Frontend
/api/auth/logout	POST	Pública	Frontend
/api/ceo	POST	Bearer	Frontend
/api/dashboard/global	GET	Bearer	Frontend
/api/dashboard/loja/{idLoja}	GET	Bearer	Frontend
/api/monitor	GET	Bearer	Frontend
/api/remessas	GET	Bearer	Frontend
/api/relatorios	GET	Bearer	Frontend
/api/lojas	GET	Bearer	Frontend

Tabela 39: Endpoints HTTP expostos pelo Servidor Central

O `IngestaoServico` é o componente central do *backend* e merece descrição detalhada. O método `processarPacote` recebe um `PacoteFechoDT0` deserializado pelo *handler* e executa três fases em sequência. Na primeira fase, verifica o *hash* de integridade: limpa o campo `hashIntegridade` do pacote recebido, serializa-o com o mesmo Gson configurado de forma idêntica ao do Software de Loja, calcula o SHA-256, e compara com o *hash* recebido — uma divergência provoca o lançamento de uma `IntegridadeInvalidaException` que o Javalin traduz numa resposta HTTP 400. Na segunda fase, obtém uma `Connection` do *pool*, desativa o *auto-commit*, e executa todas as operações de persistência na mesma transação: criação da loja se for desconhecida, inserção do `FechoDiaCentral`, e iteração sobre cada lista do pacote — vendas, devoluções, remessas, pagamentos, sessões de caixa e *logs* —, invocando o respetivo repositório com a operação `guardar(conn, entidade)`. Na terceira fase, se todas as operações concluírem sem exceção, a transação é confirmada com `commit`; qualquer exceção despoleta um `rollback` integral, garantindo a atomicidade exigida pelo ADR-05.

A idempotência é garantida ao nível do repositório. Cada implementação usa `INSERT ... ON DUPLICATE KEY UPDATE` sobre a chave composta (`idLoja`, `idOriginalLoja`), absorvendo reenvios do mesmo pacote sem duplicação. Esta operação é atomicamente segura em MySQL, tornando a ingestão tolerante a falhas de comunicação sem exigir coordenação adicional entre o cliente e o servidor.

A documentação interativa da API foi exposta através de dois *endpoints* registados no `ServerMain`: `/openapi.yaml`, que serve o ficheiro de especificação OpenAPI, e `/swagger-ui`, que serve a interface Swagger UI alimentada por esse ficheiro. Esta abordagem substitui o *plugin* `javalin-openapi` previsto na fase de arquitetura — que geraria a especificação automaticamente a partir de anotações nos *handlers* —, optando-se por um ficheiro `openapi.yaml` estático, produzido pelo assistente LLM e versionado no repositório.

5.10. Desenvolvimento do Frontend - Servidor Central

O *frontend* Vue.js tinha sido previamente gerado pelo Gemini a partir dos ficheiros `.pen` do Pencil. A integração com o *backend* foi realizada através de duas intervenções mínimas no código existente, preservando a estrutura de componentes, a folha de estilos Tailwind, e a arquitetura de rotas. A figura seguinte apresenta o resultado consolidado da integração: o *dashboard* global do CEO, com as métricas agregadas da rede e a decomposição por loja.

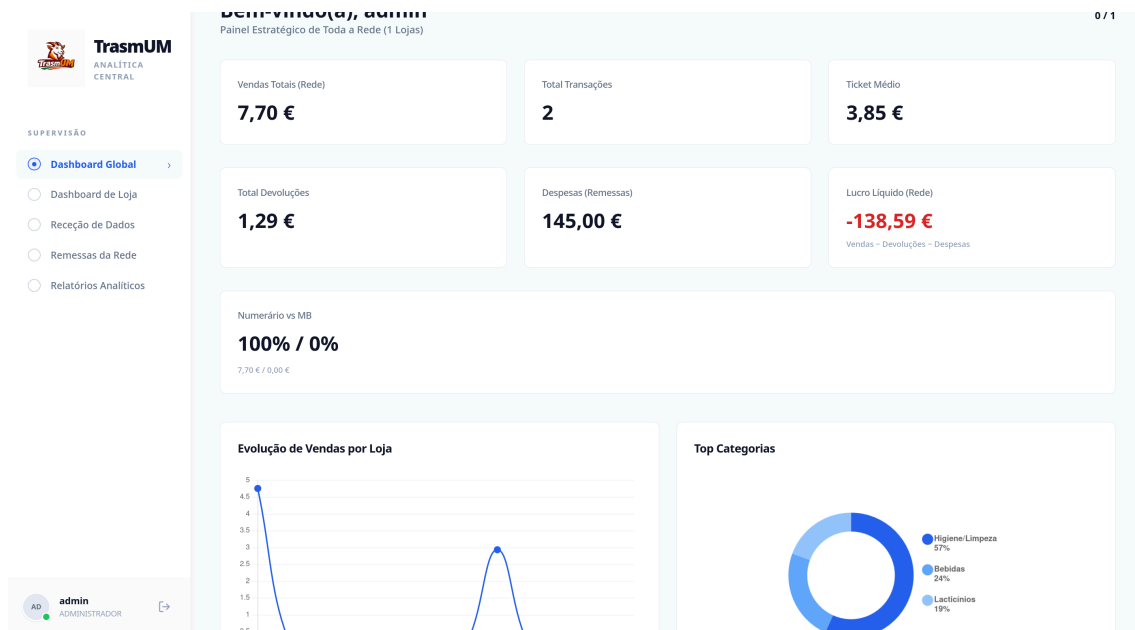


Figura 72: Dashboard global do Servidor Central: métricas agregadas da rede, decomposição por loja, totais de despesas e lucro líquido — resultado final da integração entre o *frontend* Vue e o *backend* Javalin.

A primeira intervenção foi a reescrita do ficheiro `src/api/networkService.ts`, originalmente um repositório de dados *mock*. A nova versão chama a API real via `fetch`, mas preserva os tipos `Store` e `Shipment` que os componentes consumiam: onde o servidor central não produz dados equivalentes — por exemplo nos campos `dailySales` e `stockAlerts` —, os campos são preenchidos com valores por defeito a zero, mantendo a forma dos objetos esperada pelos componentes. Esta abordagem de adaptador evita alterações intrusivas na camada de apresentação, pagando o preço de manter campos sem significado real que poderão ser removidos numa revisão futura. O Monitor de Receção de Dados (ECR-23) é um dos consumidores mais diretos desta integração, lendo o *endpoint* `/api/monitor` em tempo real e materializando-o como uma tabela de estados por loja para a data corrente.

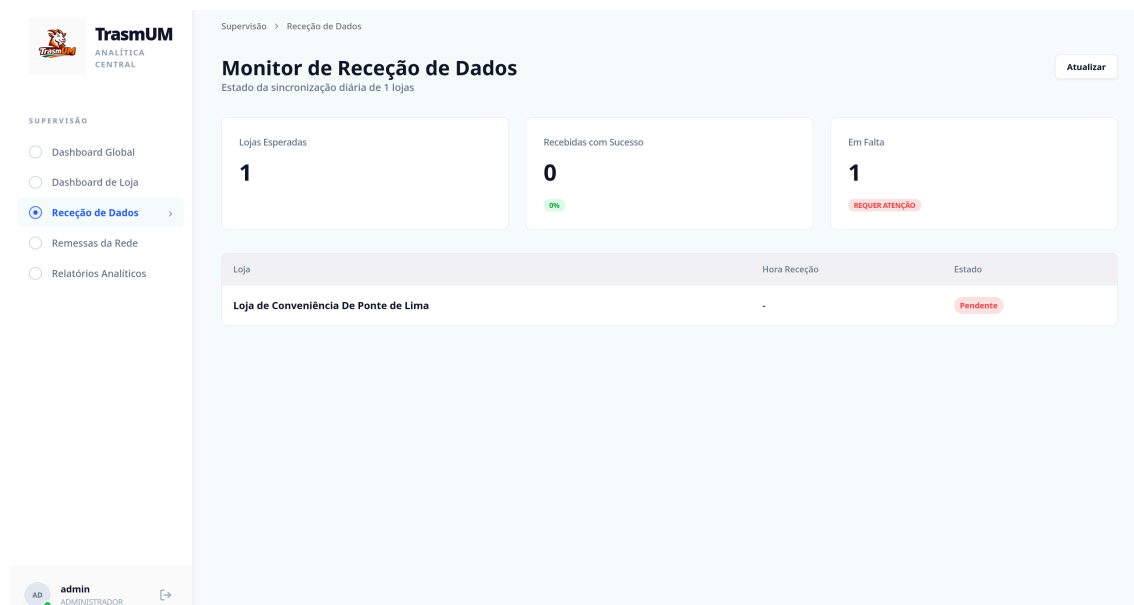


Figura 73: Monitor de Receção de Dados: estado consolidado da rede no momento corrente, com indicação visual por loja sobre o pacote diário de fecho — Recebido, Em Falta ou Erro.

A segunda intervenção introduziu autenticação no *frontend*, que não existia na versão original gerada a partir dos esboços. Foi criado um módulo `src/api/http.ts` com uma função `request` que adiciona automaticamente o cabeçalho `Authorization: Bearer <token>` — lido de `localStorage` — em todos os pedidos, e que redireciona o utilizador para `/login` em caso de resposta HTTP 401. Foi criada uma rota `/login` com um componente `LoginView.vue` minimalista, e foi adicionado um *navigation guard* no Vue Router que redireciona para `/login` qualquer rota sem *meta-flag* publica quando não há *token* armazenado. O botão “Terminar Sessão” da barra lateral foi ligado ao `authService.logout()`, que limpa o *token* do `localStorage`, invoca o *endpoint* `/api/auth/logout` e redireciona para `/login`. Esta intervenção foi cirúrgica: nenhum componente de visualização foi alterado, sendo todo o controlo de acesso concentrado nas duas camadas transversais (cliente HTTP e *router*).

A consulta detalhada por loja completa o conjunto de vistas analíticas do servidor, permitindo à CEO isolar o desempenho de um estabelecimento individual sem perder o contexto agregado. As mesmas métricas do *dashboard* global são recalculadas para o âmbito da loja selecionada — vendas totais, número de transações, devoluções, despesas, lucro líquido e ticket médio —, complementadas pelo histórico de fechos diários recebidos e pelos *logs* de auditoria sincronizados a partir do terminal local.

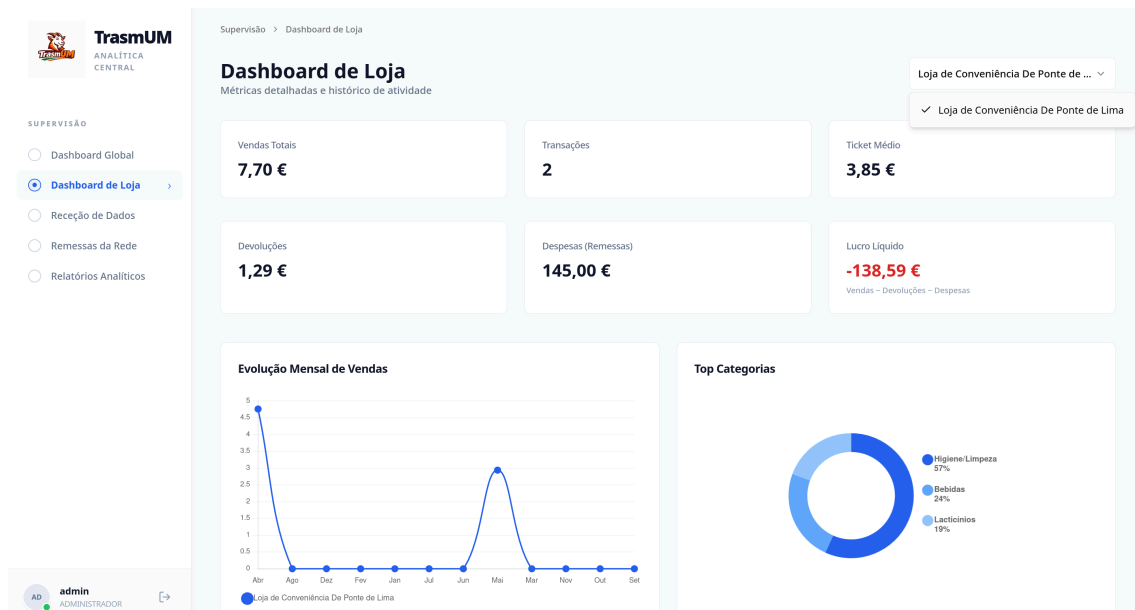


Figura 74: Dashboard por loja: decomposição das métricas analíticas para um estabelecimento individual.

5.11. Revisão de Código, Deteção de *Smells* e Refactoring

A revisão subsequente seguiu uma abordagem estritamente humana: não foi utilizado nenhum mecanismo automatizado de análise estática nem *linters* configurados, e a equipa dividiu-se entre revisão de código e teste manual do comportamento da aplicação em funcionamento — ambas as atividades executadas em paralelo à medida que os módulos iam sendo gerados. As alterações decorrentes desta revisão foram, à semelhança da geração inicial, executadas pelo Claude Code: a equipa identificava o problema ou a melhoria pretendida e instruía o modelo a aplicá-la.

Uma parte significativa dos problemas detetados não decorreu de erros do modelo nem de inadvertências da equipa, mas de uma propriedade estrutural do processo: o design produzido na fase anterior, embora coerente no nível de abstração apropriado a diagramas e ADRs, deixava espaço por preencher quando confrontado com a especificidade exigida pela implementação. Perante zonas omissas — operações de repositório não listadas, comportamentos de borda não cobertos pelos diagramas de sequência, estruturas de dados subespecificadas — o modelo tendia a produzir código que fazia exatamente o que estava especificado, sem procurar oportunidades de decomposição que um programador humano teria naturalmente explorado.

O *smell* mais frequente foi métodos longos. Vários métodos de serviço apresentavam comprimentos na ordem das cinquenta a cem linhas, misturando validação, invocação de repositórios, manipulação de estado de domínio e registo de auditoria sem extração de métodos auxiliares. O `FechoDiaServico.executarFecho` é o exemplo mais paradigmático: a primeira versão concentrava num único método a reversão de em-trânsito, a recolha de pendentes, a marcação como em-trânsito, a construção do pacote, o envio e a confirmação. Foi parcialmente refactorizado com a extração de um método privado `construirPacote`, mas permanece acima do ideal em termos de responsabilidades. A correção foi aplicada pelo Claude Code após identificação humana do problema, padrão que se repetiu em todas as intervenções subsequentes.

Um segundo *smell* foi a repetição estrutural entre repositórios. Os métodos `reverterEmTransito()`, `buscarPendentes()` e `atualizar()` têm implementações quase idênticas nos seis repositórios de entidades sincronizáveis, variando apenas o nome da tabela e as colunas. Uma extração para uma classe abstrata genérica `RepositorioSincronizavelBase<T>` teria eliminado a duplicação, mas exigiria *reflection* ou funções de mapeamento injetadas em construtor, complicando a leitura. A equipa optou por manter a duplicação em nome da clareza, reconhecendo explicitamente o *trade-off*.

Um terceiro *smell* emergiu no `DevolucaoServico`: o método `lotesPertencemAoProduto` invoca `buscarAbaixoDaValidade(36500)`, passando um prazo de cem anos para obter todos os lotes do produto. Esta escolha foi consequência direta da ausência no `LoteRepositorio` de um método genérico `buscarPorProduto(int idProduto)` — o modelo, confrontado com a necessidade de obter todos os lotes incluindo os esgotados, optou por reaproveitar um método existente com um parâmetro exagerado em vez de acrescentar uma operação ao repositório.

Os *refactorings* aplicados foram pontuais, limitados a situações com impacto funcional ou complexidade de alteração contida. No `VendaController`, a primeira versão apresentava na tabela do carrinho uma linha por cada `LinhaVenda` retornada pelo serviço — como o `VendaServico` cria uma `LinhaVenda` por lote consumido em FEFO, o funcionário via linhas duplicadas sempre que um produto requisitasse unidades de múltiplos lotes. A revisão introduziu um método privado `atualizarTabela` que agrega as linhas por combinação (nome do produto, preço unitário), preservando a distinção entre lotes com preço diferente. Na interface do Software de Loja, as vistas `VendaView.fxml` e `DevolucaoView.fxml` foram fundidas num único ecrã em `TabPane`, dado que o fluxo de devolução se inicia naturalmente a partir do contexto de venda. No *backend* do Servidor Central, o `IngestaoServico` foi objeto de um *refactoring* de assinatura nas interfaces de repositório: a primeira versão não permitia executar toda a ingestão numa única transação JDBC porque cada repositório abria a sua própria `Connection`; a revisão introduziu variantes com `Connection` explícita, mantendo as originais para uso fora de transações agregadas.

5.12. Desvios ao Design Original

Após a implementação estar concluída, a equipa decidiu fazer o registo sistemático de desvios de implementação relativamente ao design original. Cada vez que uma alteração com impacto no modelo de domínio, no contrato de serviços ou na arquitetura era introduzida — quer durante a geração inicial de código, quer na revisão subsequente, quer em iterações posteriores —, a equipa instruiu o Claude Code a documentar o desvio de forma estruturada, com indicação da área afetada, do que foi alterado e da motivação subjacente. Alguns destes desvios têm impacto que extravasa a camada técnica e afeta aspetos visuais ou funcionais perceptíveis pelo cliente. A alteração do *frontend* da Loja relativamente aos esboços originais, por razões referidas na secção de implementação, é o exemplo mais significativo. Embora as funcionalidades subjacentes não tenham sido afetadas, a aparência e a organização dos ecrãs divergem dos esboços validados com os *stakeholders* na fase de design. Esta divergência foi apresentada ao Hélder e Cátia, que aprovaram o novo design após revisão, pelo que a alteração é considerada validada. Em linha com esta prática, qualquer alteração futura que afete elementos visuais ou funcionais perceptíveis pelo cliente deverá ser objeto de nova validação com o investidor ou com a CEO, conforme a natureza do desvio.

A tabela seguinte apresenta cinco entradas selecionadas de desvios do design original do software da loja; a lista exaustiva de todas as entradas encontra-se no Anexo 18.

Área	O que foi alterado	Porquê
Domínio, Serviço, Repositório, UI	Introduzida a relação <code>FornecedorProduto</code> (N:N) com atributo <code>precoFornecedor</code> , nova exceção <code>ProdutoNaoFornecidoException</code> , novo <code>FornecedorProdutoRepositorio</code> , novo <code>IFornecedorServico</code> , validação adicional em <code>RemessaServico.registarRemessa</code> e <code>criarPedidoRemessa</code> , nova vista <code>FornecedorView</code> , e filtro de produtos por fornecedor no fluxo de pedido.	O modelo original permitia que qualquer fornecedor fornecesse qualquer produto, o que não reflete a realidade de uma loja e permitia gerar pedidos inválidos.
Apresentação / Arquitetura	Removida a vista autónoma <code>DevolucaoView.fxml</code> / <code>DevolucaoController</code> . A lógica de devolução foi integrada em <code>VendaController</code> como	O fluxo de devolução está intimamente ligado ao contexto de venda. Ter uma página separada

	segundo separador num TabPane. O botão “Devolução” do menu lateral foi eliminado.	fragmentava desnecessariamente a navegação.
Apresentação / Serviço	Os DTOs em PacoteFechoDT0 foram reescritos para o contrato denormalizado esperado pelo Servidor Central. O FechoDiaServico ganhou quatro novas dependências de repositório usadas exclusivamente para denormalização no envio.	O servidor central usa um modelo de <i>analytics</i> denormalizado e não mantém tabelas de catálogo. Receber IDs locais seria inútil — precisa dos nomes resolvidos no momento do envio.
Domínio / Repositório / Schema	Introduzido o atributo boolean <code>temValidade</code> em <code>Produto</code> ; <code>Lote.dataValidade</code> e <code>LinhaRemessa.dataValidade</code> passaram a NULL-aceitáveis. FEFO ordenado com lotes sem validade no fim. Diálogos de catálogo e remessa adaptados.	Nem todos os artigos da loja têm prazo de validade. Forçar uma data artificial corrompia os lotes e gerava alertas falsos.
Domínio / Serviço / Repositório / UI / Schema	Introduzida a entidade Loja (<code>id</code> , <code>nome</code> , <code>morada</code> , <code>localidade</code> , <code>nif</code> , <code>email</code> , <code>limiteMaximoCaixa</code> , <code>diasAlertaValidade</code>) com tabela <code>ConfiguracaoLoja</code> , novo <code>LojaRepositorio</code> e <code>ILojaServico</code> , separador “Loja” na Administração. <code>ConfiguracaoTerminal</code> tornou-se parcialmente mutável.	Os dados de identidade da loja e os parâmetros operacionais estavam apenas em <code>config.properties</code> , sem interface de edição. A entidade em BD permite editá-los via UI, auditá-los, e garantir efeito imediato sem reiniciar a aplicação.

Tabela 40: Desvios ao design original — Software de Loja (selecção)

Complementarmente, a tabela seguinte apresenta quatro entradas seleccionadas de desvios de design do software do servidor central; a lista de todas as entradas encontra-se no Anexo 19.

Área	O que foi alterado	Porquê
Backend	Cada interface de repositório envolvida na ingestão expõe duas variantes: a descrita no <i>prompt</i> e uma adicional <code>guardar(Connection conn, Entity) / criarSeNaoExistir(Connection conn, Loja)</code> .	O processamento de um <code>PacoteFechoDT0</code> é executado numa única transação JDBC. Sem aceitar a <code>Connection</code> externa em cada repositório, teria de se abrir uma ligação nova por tabela, quebrando a transação.
Apresentação	<code>networkService.ts</code> foi reescrito para chamar a API real e adaptar as respostas ao formato esperado pelos componentes. Campos sem equivalente no servidor (<code>dailySales</code> , <code>stockAlerts</code>) ficam a 0.	Não alterar a estrutura visual nem a lógica dos componentes Vue. Manter os tipos legados com um adaptador garante isso, ao custo de manter campos sem significado real.
Backend / Frontend / Schema	Os <i>dashboards</i> global e por loja expõem <code>totalDespesas</code> e <code>lucroLiquido</code> (vendas menos devoluções menos despesas). A decomposição por loja foi estendida com despesas e lucro. Novos <i>cards</i> no <i>frontend</i> com colocação dinâmica.	Os dados de remessas eram recebidos e guardados mas nunca usados analiticamente. A visibilidade do lucro líquido é necessária para suporte à tomada de decisão pelo CEO.
Backend / Infraestrutura	A especificação OpenAPI não foi gerada via <i>plugin</i> <code>java11-openapi</code> com anotações nos <i>handlers</i> . Em vez disso, foi criado um <code>openapi.yaml</code> estático servido por duas rotas <i>custom</i> em <code>ServerMain.java</code> — GET /	O <code>java11-openapi</code> exige configuração Maven adicional, anotações em todos os <i>handlers</i> e um passo de <i>build</i> separado. A abordagem estática atinge o mesmo resultado

	openapi.yaml e GET /swagger-ui —, com a interface Swagger UI carregada a partir de CDN.	— documentação navegável e interativa da API — com menor complexidade de infraestrutura.
--	---	--

Tabela 41: Desvios ao design original — Software de Servidor Central (selecção)

5.13. Containerização do Software do Servidor Central

Por fim, a equipa de desenvolvimento resolveu criar uma containerização do Software do Servidor Central. Esta containerização é composta por três serviços orquestrados via Docker Compose (Merkel, 2014): a base de dados MySQL, o *backend* Javalin e o *frontend* Vue.js servido pelo Nginx. A separação em serviços distintos permite que cada componente seja construído, atualizado e substituído de forma independente, sem impacto nos restantes.

O serviço de base de dados utiliza a imagem oficial `mysql:8.0`, com o *plugin* de autenticação configurado para `mysql_native_password` para compatibilidade com o conector JDBC utilizado no *backend*. O *schema* da base de dados central é injetado automaticamente no arranque via o mecanismo `docker-entrypoint-initdb.d` do MySQL, garantindo que a estrutura de tabelas é criada sem intervenção manual. A persistência dos dados é assegurada por um volume Docker nomeado, que sobrevive ao ciclo de vida dos *containers*.

O *backend* é construído a partir do JAR produzido pelo Maven e empacotado numa imagem baseada em `eclipse-temurin:21-jre`. A decisão de utilizar a variante JRE em vez da JDK na imagem final reduz a superfície da imagem de produção, excluindo ferramentas de compilação desnecessárias em *runtime*. As configurações de ligação à base de dados e os parâmetros operacionais são injetados via variáveis de ambiente, sobrepostas às propriedades do `config.properties` pelo mecanismo `overrideWithEnv` já existente no `AppContext`, sem qualquer alteração ao código de produção. O *frontend* é construído através de um processo multi-estágio: numa primeira fase, a imagem `node` instala as dependências e executa `npm run build`, produzindo os artefactos estáticos do Vite; numa segunda fase, esses artefactos são copiados para uma imagem `nginx:alpine`, que os serve em produção. O URL base da API é injetado em tempo de *build* através da variável `VITE_API_URL`, que o Vite embebe nos ficheiros JavaScript gerados — comportamento consistente com a implementação existente no `http.ts`, que lê `import.meta.env.VITE_API_URL` para determinar o destino das chamadas à API.

Um problema operacional relevante na orquestração de serviços é a janela de tempo entre o arranque do *container* MySQL e a disponibilidade efetiva do motor para aceitar ligações. Esta limitação é mitigada por dois mecanismos complementares: um *healthcheck* configurado no serviço MySQL, que declara o serviço como saudável apenas quando o motor está operacional, condicionando o arranque do *backend* a esse estado; e o *retry loop* já existente no `DatabaseConnection`, que tenta restabelecer a ligação até ao número máximo de tentativas configurado via `DB_RETRY_MAX_TENTATIVAS`, com o intervalo definido por `DB_RETRY_DELAY_MS`, atuando como rede de segurança para variações no tempo de inicialização que o *healthcheck* não consiga antecipar com precisão.

A configuração do sistema containerizado é centralizada num ficheiro `.env` na raiz do projeto, que define as credenciais da base de dados, a origem permitida pelo mecanismo CORS do *backend*, o URL da API para o *frontend* e os parâmetros de *retry* da ligação à base de dados. Este ficheiro não é incluído no controlo de versão, sendo disponibilizado separadamente no processo de instalação. As variáveis são consumidas pelo Docker Compose e injetadas nos serviços relevantes, mantendo o código de produção independente de qualquer detalhe de infraestrutura.

O Software de Loja não foi containerizado. A presença de uma interface JavaFX e a integração com periféricos físicos do terminal tornariam o processo significativamente mais complexo do que o do servidor, complexidade considerada injustificada num contexto em que cada terminal corre uma única instância dedicada da aplicação.

5.14. Apresentação do Protótipo Funcional

Concluída a fase de implementação, o protótipo funcional do sistema TrasmUM foi apresentado à cliente Cátia e ao investidor Hélder Cerqueira numa sessão realizada a 6 de maio de 2026. A demonstração percorreu os fluxos principais do Software de Loja — abertura de sessão de caixa, registo de venda com rotação FEFO e descontos por lote, emissão de fatura em PDF, devolução de artigo e fecho de dia — bem como o Servidor Central, com receção do pacote de sincronização e consulta dos *dashboards* analíticos globais e por loja.

Os *stakeholders* validaram que os requisitos funcionais levantados na fase inicial se encontram cobertos pelo protótipo, com particular destaque para a visibilidade centralizada dos indicadores financeiros de cada estabelecimento — vendas, devoluções, despesas de aprovisionamento e lucro líquido — acessível em tempo real através do *dashboard*. A sessão culminou com a aprovação formal do trabalho desenvolvido na fase de implementação.

Foram ainda identificadas, neste contexto, funcionalidades fora do âmbito do presente ciclo de desenvolvimento: a gestão centralizada de utilizadores ao nível da rede e a emissão de notificações automáticas em caso de atraso no fecho de dia. Ambas constituem candidatos naturais a iterações futuras do sistema.



TrasmUM – Reunião 6

Data: 2026/05/06

Duração prevista: 45 minutos

Horário de início: 14h00

Horário de término: 14h45

Participantes: Hélder Cerqueira (helderCerqueira@trasmum.pt)
Cátia (unhasPintadas@trasmum.pt)
Gabriel Dantas (a107291@uminho.pt)
José Fernandes (a106937@uminho.pt)
Luís Ferreira (a98286@uminho.pt)
Simão Oliveira (a107322@uminho.pt)

Pauta da reunião:

- Demonstração do protótipo funcional do Software de Loja, com percurso completo dos fluxos principais: abertura de sessão de caixa, registo de venda com aplicação automática de rotação FEFO e descontos por lote, emissão de fatura em PDF, devolução de artigo e fecho de dia com sincronização para o Servidor Central.
- Demonstração do Servidor Central, incluindo receção e validação do pacote de fecho de dia enviado pela loja, e consulta dos dashboards analíticos globais e por loja (vendas totais, devoluções, despesas de aprovisionamento e lucro líquido).
- Validação por parte do investidor de que os requisitos funcionais identificados na fase de levantamento se encontram cobertos pelo protótipo apresentado.
- Identificação de melhorias e funcionalidades fora do âmbito do presente ciclo de desenvolvimento, nomeadamente a gestão centralizada de utilizadores ao nível da rede e a emissão de notificações automáticas em caso de atraso no fecho de dia.
- Aprovação formal do trabalho desenvolvido na fase de Implementação do Software.

Figura 75: Ata da reunião da apresentação do protótipo Funcional.

6. Verificação, Validação e Avaliação da Qualidade do Software Produzido

6.1. Utilização de LLMs na Fase de Verificação e Validação

O papel do LLM nesta fase diferiu qualitativamente do das fases anteriores: em vez de consultor ou executor generalista, o Claude Code foi utilizado como gerador dirigido de código de teste, a partir de um *prompt* de especificação produzido pela equipa. O *prompt* #51⁴⁸ definiu de forma estruturada o âmbito completo da suíte: *stack* tecnológico (JUnit 5.11 (Bechtold *et al.*, 2024) com junit-jupiter, Mockito 5.12 (Mockito Contributors, 2024) com mockito-core e mockito-junit-jupiter, maven-surefire-plugin 3.2.5), as seis classes de serviço a testar, as convenções de nomenclatura em português descritivo, o uso obrigatório de `@ExtendWith(MockitoExtension.class)`, `@Mock` e `@InjectMocks`, e a proibição explícita de acesso real à base de dados.

A geração inicial produziu uma suíte funcional que passou a compilação sem erros e executou sem falhas. O refinamento posterior, conduzido pela equipa, incidiu em três áreas: (1) correção de *stubs* desnecessários (UnnecessaryStubbing) assinalados pela extensão Mockito JUnit 5, substituídos por `lenient().when()` ou eliminados; (2) adição de importações em falta identificadas apenas em compilação — nomeadamente `java.util.List` em `IngestaoServicoTest` e a eliminação de importações duplicadas em `VendaServicoTest`; e (3) reforço de asserções em testes de delegação pura, onde `assertNotNull` sobre listas estruturalmente impossíveis de ser nulas foi substituído por `assertSame` com referência concreta, tornando as asserções genuinamente falsificáveis. Estes refinamentos ilustram o padrão observado ao longo do projeto: o modelo produz código correto para o caso típico, mas requer revisão humana para garantir que as asserções verificam o que afirmam. A redação desta secção foi igualmente apoiada por LLM.⁴⁹

6.2. Estratégia de Testes

A estratégia adotada concentra os testes automáticos na camada de serviço de ambos os softwares, complementada por verificação manual dos fluxos de interface e inspeção estática do código produzido.

A camada de serviço concentra a totalidade da lógica de negócio do sistema: cálculos FEFO, verificação de integridade por *hash* SHA-256, controlo de acesso por perfil, ciclo de vida dos estados de sincronização, regras de devolução, bloqueio de conta após tentativas de autenticação falhadas e criação de conta com validação de unicidade e armazenamento seguro de palavras-passe. Trata-se da camada com maior densidade de decisões verificáveis de forma isolada, sem dependência de infraestrutura. Todas as dependências externas — repositórios, *gateway* de sincronização e configuração do terminal — foram substituídas por *mocks* Mockito, garantindo testes rápidos, deterministas e repetíveis.

Os testes de integração automáticos não foram implementados devido às características da infraestrutura necessária. Os controladores JavaFX dependem do motor de renderização da plataforma e requerem um ambiente *headless* configurado, acrescentando complexidade sem cobertura adicional de lógica de

⁴⁸Prompt de geração de testes unitários para ambos os subprojetos — ver [Anexo de IA, Entrada 51](#).

⁴⁹Prompt de redação da secção de Verificação, Validação e Avaliação da Qualidade do Software Produzido — ver [Anexo de IA, Entrada 53](#).

negócio, uma vez que essa lógica já se encontra integralmente coberta pelos serviços. Os repositórios JDBC requerem uma instância MySQL ativa, tendo o seu comportamento sido verificado manualmente durante o desenvolvimento. Para os *endpoints* HTTP do Servidor Central, a autorização por *token* e a serialização Gson das respostas foram testadas manualmente através da interface Swagger UI, permitindo enviar pedidos reais ao servidor em execução e observar as respostas produzidas. A atomicidade das transações JDBC na ingestão foi verificada por inspeção estática do código, confirmando o uso correto das primitivas de transação JDBC no `RecepcaoPacoteServico`.

Todos os fluxos funcionais descritos nos casos de uso foram verificados manualmente numa instância completa do sistema em execução, incluindo o fluxo de venda com FEFO, devolução com validação de fatura, registo de remessa e pagamento, fecho de dia com confirmação do servidor, gestão de catálogo e *dashboards* da CEO. A verificação manual cobriu ainda os requisitos de desempenho (RNF1–RNF4) e os requisitos de usabilidade (RNF12 e RNF13) através de observação direta durante o desenvolvimento.

A medição de cobertura foi realizada com JaCoCo 0.8.12, configurado em ambos os `pom.xml` com as execuções `prepare-agent`, `report` e `check`. O *threshold* imposto automaticamente no `mvn verify` corresponde a 80% de cobertura de instrução sobre o pacote `servico/impl` de cada subprojeto. A métrica de instrução foi privilegiada relativamente à métrica de linha por ser mais granular e independente da formatação do código. A cobertura de ramo foi igualmente medida, embora não tenha sido definida como requisito mínimo obrigatório. Os ramos não cobertos correspondem maioritariamente a tratadores de exceção associados a falhas de infraestrutura, como `SQLException` e `NoSuchAlgorithmException`, que não são exercitáveis em testes unitários sem injeção artificial de falhas da JVM, bem como a variantes de estado cuja execução realista exigiria testes de integração.

6.3. Testes Unitários

A suíte de testes unitários do sistema TrasmUM é composta por 54 testes distribuídos por seis classes de teste. O Software de Loja contribui com 34 testes distribuídos por quatro classes pertencentes ao pacote `pt.trasmum.loja.servico.impl`, enquanto o Servidor Central contribui com 20 testes distribuídos por duas classes pertencentes ao pacote `pt.trasmum.servidor.servico.impl`.

A listagem completa dos testes, incluindo o comportamento verificado e o resultado obtido em execução, encontra-se nos anexos Anexo 21 e Anexo 22.

6.4. Testes de Integração — Endpoints HTTP

Não foram produzidos testes de integração automáticos para os *endpoints* HTTP do Servidor Central. Esta decisão foi tomada deliberadamente por duas razões complementares. Em primeiro lugar, os *handlers* Javalin têm estrutura mínima: recebem o pedido HTTP, invocam o serviço correspondente e serializam a resposta com Gson. A lógica de negócio reside integralmente na camada de serviço, já testada unitariamente com cobertura de instrução entre 88% e 100%. Em segundo lugar, os testes de integração Javalin requerem a disponibilidade de uma instância MySQL ativa e de um ambiente de rede configurado, introduzindo dependências de infraestrutura incompatíveis com o objetivo de testes rápidos e reproduzíveis.

A verificação do comportamento dos *endpoints* foi realizada manualmente com recurso à interface Swagger UI disponibilizada em `/swagger-ui`, que documenta interativamente todos os *endpoints*, os seus parâmetros e os esquemas de resposta. Esta abordagem permitiu verificar o comportamento de autenticação (rejeição de credenciais inválidas, retorno do *token* em autenticação bem-sucedida), o comportamento de ingestão (aceitação de pacotes válidos, rejeição por *hash* inválido) e as respostas dos *endpoints* de análise (*dashboards*, relatórios e consolidação por data). O *endpoint* `/api/ceo`, por exigir uma sessão com a conta de instalação temporária ativa, foi verificado através do fluxo de primeiro acesso do *frontend*: após autenticação com as credenciais de instalação, o *frontend* redireciona automaticamente para a página de criação de conta, onde foram confirmadas manualmente as respostas 201 (criação bem-sucedida), 400 (campos em branco) e 409 (conta já existente).

6.5. Verificação dos Requisitos (SRS)

As tabelas seguintes cruzam cada requisito funcional e não funcional com o mecanismo de verificação utilizado e o estado de conformidade apurado. A coluna “Verificação” indica se o requisito foi coberto por testes unitários automáticos (UT), por verificação manual sobre o sistema em execução (M), ou por ambos (UT+M). Requisitos classificados como “Não implementado” ou “Não testado” correspondem a restrições de qualidade para as quais não foram conduzidos testes formais no âmbito deste projeto.

Req.	Descrição Resumida	Verificação	Estado
RF1	Sessão única por utilizador — bloqueio de login duplicado.	M	Passou
RF2	Autenticação por credenciais com controlo de acesso por perfil.	UT + M	Passou
RF3	Registo de vendas com lógica FEFO de consumo de lotes.	UT + M	Passou
RF4	Anulação de venda em curso com reposição de stock.	UT + M	Passou
RF5	Processamento de devoluções com validação de fatura e reposição condicional de stock.	UT + M	Passou
RF6	Registo de remessa com criação de lotes e pagamento pendente automático.	M	Passou
RF7	Criação e gestão de pedidos de remessa a fornecedores.	M	Passou
RF8	Gestão local do catálogo de produtos pelo manager.	M	Passou
RF9	Monitorização de caixa com alerta de sangria.	M	Passou
RF10	Alertas visuais de stock mínimo após cada venda.	M	Passou
RF11	Geração de fatura digital após finalização da venda.	M	Passou
RF12	Cálculo automático do troco em pagamentos por numerário.	UT + M	Passou
RF13	Pesquisa de produtos por nome e categoria.	M	Passou
RF14	Consulta de documentos de fornecedores na base de dados local.	M	Passou
RF15	Fecho de dia confirmado apenas após ACK do servidor; reenvio manual em caso de falha.	UT + M	Passou
RF16	Acionamento do fecho de dia pelo manager ou CEO com agregação de todos os dados pendentes.	UT + M	Passou
RF17	Gestão autónoma do catálogo por loja, sem dependência de sincronização externa.	M	Passou
RF18	Consulta e liquidação de pagamentos a fornecedores pelo manager.	M	Passou
RF19	Vista consolidada de remessas da rede para a CEO.	M	Passou
RF20	<i>Dashboards</i> de vendas e stocks globais para a CEO.	M	Passou

RF21	Alertas de validade e aplicação de descontos a lotes pelo manager.	M	Passou
RF22	Receção, validação e persistência atómica de pacotes de fecho de dia no Servidor Central.	UT + M	Passou
RF23	Gestão hierárquica de contas de utilizador por perfil (CEO gere managers; manager gere funcionários).	UT + M	Passou

Tabela 42: Verificação dos Requisitos Funcionais

Req.	Descrição Resumida	Verificação	Estado
RNF1	Persistência de venda em ≤ 2 s.	M	Passou
RNF2	Geração de fatura em ≤ 3 s.	M	Passou
RNF3	Envio do pacote de fecho em ≤ 5 min.	M	Passou
RNF4	<i>Dashboards</i> carregam em ≤ 3 s.	M	Passou
RNF5	<i>Backups</i> automáticos diários com retenção de 30 dias.	—	Não implementado
RNF6	Autenticação individual obrigatória para todos os perfis.	UT + M	Passou
RNF7	Bloqueio após 5 tentativas falhadas consecutivas; desbloqueio automático configurável.	UT + M	Passou
RNF8	Palavras-passe armazenadas com BCrypt (hash irreversível).	UT + M	Passou
RNF9	Acesso a dados financeiros consolidados restrito ao perfil CEO.	UT + M	Passou
RNF10	Registo de logs de auditoria para todas as ações críticas.	UT + M	Passou
RNF11	Idempotência da transmissão — reenvio não duplica registos na base de dados central.	M	Passou
RNF12	Fluxo de registo de venda em ≤ 5 interações.	M	Passou
RNF13	Mensagens de erro descritivas com indicação de ação corretiva.	M	Passou
RNF14	Suporte a crescimento de 50% de volume sem degradação superior a 10%.	—	Não testado
RNF15	Registo de novas lojas sem reconfiguração manual do servidor central.	UT + M	Passou
RNF16	Parâmetros operacionais configuráveis via interface sem recompilação.	M	Passou
RNF17	Base de dados local autónoma; comunicação com servidor exclusivamente no fecho de dia.	M	Passou
RNF18	Interface do servidor acessível via <i>browser</i> sem instalação de software adicional.	M	Passou

Tabela 43: Verificação dos Requisitos Não Funcionais

O requisito RNF5 — relativo a *backups* automáticos diários — não foi implementado no âmbito deste projeto académico, por estar fora do perímetro técnico da aplicação desenvolvida e ser da responsabilidade da infraestrutura de *hosting*. O requisito RNF14 — escalabilidade para 50% de crescimento de volume — não foi objeto de testes de carga formais; a avaliação realizada foi estritamente qualitativa, baseada na arquitetura (JDBC direto sem ORM, *schema* com índices sobre as colunas de filtragem mais frequentes) e na ausência de gargalos identificados durante a verificação manual.

6.6. Métricas de Qualidade

A avaliação da qualidade do software TrasmUM foi estruturada em torno das quatro características da norma ISO/IEC 25010 (International Organization for Standardization, 2011) mais diretamente verificáveis com os meios disponíveis: *adequação funcional*, *fiabilidade*, *segurança* e *manutenibilidade*.

A adequação funcional foi avaliada através do cruzamento entre os 23 requisitos funcionais especificados e o estado de verificação apurado na secção anterior. Todos os 23 requisitos funcionais foram verificados, 9 deles por testes unitários automáticos e os restantes por verificação manual sobre o sistema em execução. A correção funcional é suportada pelos 54 testes unitários que verificam os comportamentos esperados da camada de serviço, com 0 falhas em execução e 0 erros de compilação.

A fiabilidade é evidenciada pelos testes que verificam o comportamento do sistema perante entradas inválidas e condições de falha. `IngestaoServicoTest` verifica que pacotes com *hash* SHA-256 inválido são rejeitados antes de qualquer escrita na base de dados. `FechoDiaServicoTest` verifica que uma falha no envio ao servidor reverte todos os registos para o estado PENDENTE, prevenindo perda de dados. `AutenticacaoCEOServicoTest` verifica o comportamento correto perante credenciais inválidas, contas bloqueadas e criação de conta CEO, incluindo a rejeição por conta já existente, garantindo a restrição de conta única. `VendaServicoTest` e `DevolucaoServicoTest` verificam o comportamento perante diferentes condições de guarda do domínio, como produto inexistente, stock insuficiente, fatura não encontrada e quantidade excedida.

A capacidade de recuperação é documentada pelo mecanismo de reversão dos registos em estado EM_TRANSITO: no arranque, quaisquer registos nesse estado, resultantes de uma interrupção abrupta durante um fecho de dia anterior, são automaticamente revertidos para PENDENTE, garantindo que nenhum dado se perde permanentemente, conforme verificado em `reverterEmTransito_invocaReverterEmTodosOsRepositorios`.

A segurança é suportada pelo uso de BCrypt para armazenamento de palavras-passe e pelo controlo de acesso baseado em perfis. O teste `criar_caminhoFeliz_persisteCeoComHashBcrypt`, presente em `AutenticacaoCEOServicoTest`, captura o objeto CEO persistido e confirma com `BCrypt.checkpw` que o *hash* armazenado corresponde à palavra-passe fornecida. O controlo de acesso é validado em `AutorizacaoServicoTest`.

A integridade dos dados é assegurada através do mecanismo de *hash* SHA-256 utilizado nos pacotes de fecho de dia, existindo testes que verificam a rejeição de pacotes com *hash* ausente, em branco ou incorreto. A autenticidade é suportada pelo mecanismo de bloqueio de conta testado em `AutenticacaoCEOServicoTest`, incluindo cenários que verificam o limiar de tentativas falhadas, a definição de bloqueadoAte no futuro e o desbloqueio automático após expiração do período de bloqueio.

A manutenibilidade é avaliada através da sub-característica de testabilidade, quantificada pelas métricas de cobertura obtidas com JaCoCo 0.8.12 (EclEmma, 2024). As tabelas seguintes apresentam os resultados por classe testada, para as métricas de cobertura de instruções e de ramos.

Classe	Instruções Cobertas	Instru. (%)	Ramos Cobertos	Ramos (%)
AutorizacaoServico	40/40	100%	6/6	100%
VendaServico	320/321	≈100%	25/30	83%
DevolucaoServico	265/275	96%	20/26	77%
FechoDiaServico	847/873	97%	35/50	70%

Tabela 44: Cobertura de código por classe testada — Software de Loja

Classe	Instruções Cobertas	Instru. (%)	Ramos Cobertos	Ramos (%)
AutenticacaoCEOServico	170/175	97%	23/26	88%
IngestaoServico	473/535	88%	41/53	77%

Tabela 45: Cobertura de código por classe testada — Servidor Central

A cobertura de instrução nas classes testadas varia entre 88% (*IngestaoServico*) e 100% (*AutorizacaoServico*), com todas as classes acima do *threshold* de 80% imposto automaticamente no `mvn verify`. *AutenticacaoCE0Servico* atingia 100% antes da adição do método `criar`; com os três testes introduzidos a cobertura passou para 97% de instrução e 88% de ramo — a instrução e o ramo não cobertos correspondem à validação de palavra-passe em branco, cujo teste não foi incluído por já estar demonstrado o padrão de validação no teste de nome de utilizador em branco. A diferença entre cobertura de instrução e cobertura de ramo nas restantes classes resulta de dois fatores: (1) ramos de exceção para falhas de infraestrutura estruturalmente impossíveis de reproduzir em testes unitários sem injeção artificial de falhas de JVM — como o `catch NoSuchElementException` em *IngestaoServico*, que não pode ser alcançado em *runtime* normal porque SHA-256 é garantidamente disponível em qualquer JVM; e (2) variantes de estado nos mapeadores de DTO — como a *branch* do método `reenviar` que retorna `true` em caso de sucesso — que exigiriam testes adicionais para ser completamente exercitadas. A cobertura global dos projetos, incluindo as camadas de apresentação, repositório e infraestrutura não testadas unitariamente, é de 11,3% no Software de Loja e 15,4% no Servidor Central — valores esperados numa arquitetura em camadas em que apenas a camada de serviço foi objeto de testes unitários.

A sub-característica de *modularity* é evidenciada pela arquitetura em camadas adotada — *apresentacao*, *servico*, *repositorio*, *dominio* no Software de Loja; *handler*, *servico*, *repositorio*, *dominio* no Servidor Central —, que permite substituir qualquer camada por mock sem afetar as restantes, conforme demonstrado pela suíte de testes. A sub-característica de *analysability* é suportada pelo relatório HTML gerado pelo JaCoCo após cada execução de `mvn verify`, disponível em `target/site/jacoco/index.html`, que permite identificar visualmente as linhas e ramos não cobertos em cada classe.

7. Guia de Instalação, Operação e Manutenção

O presente guia descreve os procedimentos necessários para instalar, configurar, operar e manter o sistema TrasmUM em ambiente operacional. O sistema é composto por dois softwares distintos — o Software de Loja e o Software do Servidor Central —, que comunicam entre si via HTTP. O Software do Servidor Central agrupa um *backend* Javalin e um *frontend* Vue.js, e pode ser implantado via Docker Compose ou localmente. O Software de Loja é sempre executado localmente em cada terminal de ponto de venda.

7.1. Pré-requisitos

A tabela seguinte lista os componentes necessários consoante o modo de implantação escolhido.

Componente	Versão mínima	Finalidade	Modo
Java (JDK)	21 LTS	Executar o Software de Loja e o <i>backend</i> local	Docker e Local
Maven	3.9	Compilar e executar os projetos Java	Docker e Local
MySQL	8.0	Base de dados local (Loja e Servidor Central sem Docker)	Local
Node.js	20 LTS	Compilar e executar o <i>frontend</i> Vue.js sem Docker	Local
Docker	24.0	Orquestrar MySQL, <i>backend</i> e <i>frontend</i> em containers	Docker
Docker Compose	2.20	Orquestrar os serviços do Servidor Central	Docker

Tabela 46: Pré-requisitos por modo de implantação

7.2. Instalação e Arranque via Docker Compose

Esta via é recomendada para o Servidor Central em ambiente operacional. Pressupõe que Docker e Docker Compose estão instalados e que as portas 3307 (MySQL), 8080 (*backend*) e 3000 (*frontend*) estão disponíveis na máquina anfitriã.

- **1. Clonar o repositório e aceder ao directório do Servidor Central:**

```
git clone https://github.com/Simao0liveira05/LI4
cd Codigo/Servidor_Central
```

- **2. Criar o ficheiro de configuração de ambiente a partir do modelo:**

```
cp .env.example .env
```

Editar o ficheiro `.env` conforme necessário — ver a secção de Configuração. Os valores por omissão são funcionais para uma instalação local típica.

- **3. Construir e arrancar os serviços:**

```
docker compose up --build
```

O Docker Compose arranca três serviços em sequência: MySQL, *backend* Javalin e *frontend* Nginx. O *backend* aguarda automaticamente que o MySQL esteja operacional antes de tentar estabelecer ligação. No final do primeiro arranque, o *frontend* fica disponível em <http://localhost:3000> e o *backend* em <http://localhost:8080>.

- **4. Parar os serviços:**

```
# Parar sem remover dados
```

```
docker compose down
```

```
# Parar e remover volumes (apaga todos os dados da base de dados central)
```

```
docker compose down -v
```

7.3. Instalação e Arranque Local (sem Docker)

Esta via é obrigatória para o Software de Loja e pode também ser usada para o Servidor Central em ambiente de desenvolvimento.

7.3.1. Preparação da Base de Dados

Criar um utilizador MySQL *trasmum* com palavra-passe *trasmum* e conceder-lhe permissões sobre as bases de dados do sistema:

```
CREATE USER 'trasmum'@'localhost' IDENTIFIED BY 'trasmum';
GRANT ALL PRIVILEGES ON trasmum_loja.* TO 'trasmum'@'localhost';
GRANT ALL PRIVILEGES ON trasmum_servidor.* TO 'trasmum'@'localhost';
FLUSH PRIVILEGES;
```

O *script* `db-reset.sh` cria ou recria as bases de dados e carrega os *schemas* e dados de exemplo:

```
# Ambas as bases de dados
```

```
Codigo/scripts/db-reset.sh
```

```
# Apenas a base de dados da Loja
```

```
Codigo/scripts/db-reset.sh loja
```

```
# Apenas a base de dados do Servidor Central
```

```
Codigo/scripts/db-reset.sh servidor
```

O *script* aceita as variáveis de ambiente `DB_USER`, `DB_PASS` e `DB_HOST` para substituir os valores por omissão (*trasmum*, *trasmum*, *localhost*).

7.3.2. Arranque do Servidor Central

```
# Backend (:8080) e frontend (:5173) em paralelo – terminar com Ctrl-C
```

```
Codigo/scripts/dev-up.sh
```

```
# Ou individualmente:
```

```
cd Codigo/Servidor_Central/backend && mvn compile && mvn exec:java
```

```
cd Codigo/Servidor_Central/frontend && npm install && npm run dev
```

7.3.3. Arranque do Software de Loja

```
Codigo/scripts/loja-up.sh
```

```
# Ou directamente:
```

```
cd Codigo/Loja && mvn javafx:run
```

A sequência correta de arranque é: base de dados → *backend* → *frontend* → Software de Loja. O Software de Loja pode funcionar em modo de venda sem ligação ao servidor; a ligação só é necessária no momento do fecho de dia.

7.4. Configuração

7.4.1. Software de Loja

O ficheiro de configuração do Software de Loja encontra-se em `Codigo/Loja/src/main/resources/config.properties`. Os parâmetros configuráveis estão descritos na tabela seguinte.

Parâmetro	Descrição	Valor por omissão
<code>loja.id</code>	Identificador único da loja, incluído em todos os pacotes de fecho	LOJA_002
<code>loja.nome</code>	Nome comercial da loja (aparece nas faturas geradas)	—
<code>loja.morada</code>	Morada da loja (aparece nas faturas geradas)	—
<code>loja.localidade</code>	Localidade da loja (aparece nas faturas geradas)	—
<code>loja.nif</code>	NIF da loja (aparece nas faturas geradas)	—
<code>loja.email</code>	Endereço de correio eletrónico de contacto da loja	—
<code>loja.limiteMaximoCaixa</code>	Limite máximo de numerário em caixa, em euros	500.00
<code>servidor.url</code>	URL base do <i>backend</i> do Servidor Central	<code>http://localhost:8080</code>
<code>servidor.alertaValidade.dias</code>	Dias de antecedência para alertas de validade de produtos	7
<code>db.url</code>	URL de ligação à base de dados local da Loja	MySQL em <code>localhost:3306</code>
<code>db.user</code>	Utilizador da base de dados local da Loja	trasmum
<code>db.password</code>	Palavra-passe da base de dados local da Loja	trasmum

Tabela 47: Parâmetros de configuração do Software de Loja

Cada instância de loja deve ter um `loja.id` único na cadeia, e o `servidor.url` deve apontar para o endereço acessível a partir do terminal de ponto de venda.

7.4.2. Software do Servidor Central

- **Via Docker:** a configuração é feita no ficheiro `.env` em `Codigo/Servidor_Central/`. O ficheiro `.env.example` contém todos os parâmetros disponíveis com os respetivos valores por omissão.
- **Via execução local:** a configuração é lida de `Codigo/Servidor_Central/backend/src/main/resources/config.properties`. As variáveis de ambiente do sistema sobrepõem-se sempre aos valores do ficheiro — o mesmo binário pode assim ser configurado externamente sem recompilação.

Parâmetro / Variável de ambiente	Descrição	Valor por omissão
----------------------------------	-----------	-------------------

db.url / DB_URL	URL de ligação à base de dados central	MySQL em localhost:3306
db.user / DB_USER	Utilizador da base de dados central	trasmum
db.password / DB_PASSWORD	Palavra-passe da base de dados central	trasmum
frontend.origem / FRONTEND_ORIGEM	Origem permitida para pedidos CORS do <i>frontend</i>	http://localhost:5173
servidor.porta / SERVER_PORT	Porta de escuta do <i>backend</i>	8080
VITE_API_URL	URL do <i>backend</i> visível ao navegador, injetada em tempo de <i>build</i> do <i>frontend</i>	http://localhost:8080
db.retry.maxTentativas / DB_RETRY_MAX_TENTATIVAS	Número máximo de tentativas de ligação à BD no arranque	10
db.retry.delayMs / DB_RETRY_DELAY_MS	Intervalo entre tentativas de ligação à BD, em milissegundos	2000

Tabela 48: Parâmetros de configuração do Servidor Central

7.5. Credenciais e Primeiro Acesso

Ambos os softwares criam automaticamente, no primeiro arranque, uma conta de administração com as credenciais `admin / admin123`. Esta conta é eliminada assim que um utilizador com nome de utilizador diferente efetua o primeiro *login* com sucesso — comportamento implementado para garantir que o sistema nunca fica sem nenhuma conta ativa após a configuração inicial.

No Software de Loja, a conta de *bootstrap* tem perfil CEO e permite aceder a todas as funcionalidades de gestão. No Software do Servidor Central, a mesma conta permite aceder ao *dashboard* de monitorização e gestão central.

7.6. Operação Corrente

7.6.1. Ciclo Diário

O sistema opera em ciclos de um dia. Cada terminal de Loja acumula as transações do dia — vendas, devoluções, remessas e pagamentos — e envia-as para o Servidor Central no fecho de dia:

1. O gestor autentica-se no terminal de Loja com perfil de Gestor ou CEO.
2. Acede ao menu de fecho de dia e inicia o processo.
3. O sistema agrega todas as transações do dia num pacote, calcula o *hash* de integridade SHA-256 e envia-o para o Servidor Central via HTTP POST em `<servidor.url>/fecho`.
4. O Servidor Central valida o *hash*, persiste os dados na base de dados central e devolve uma confirmação.
5. O terminal de Loja atualiza o estado de todas as transações enviadas para *confirmado* e regista o fecho.

O Software de Loja pode continuar a registar vendas durante a sessão seguinte independentemente da disponibilidade do Servidor Central; o pacote de fecho pode ser reenviado se a ligação estiver indisponível no momento do fecho.

7.6.2. Monitorização

O *dashboard* do Servidor Central, acessível em `http://localhost:3000` (Docker) ou `http://localhost:5173` (execução local), disponibiliza:

- Visão global das vendas e devoluções por loja e por período;
- Estado de sincronização de cada loja, incluindo a data do último fecho confirmado;
- Gestão de remessas de produtos para as lojas;
- Relatórios de vendas por categoria e por período.

7.7. Manutenção

7.7.1. Cópia de Segurança da Base de Dados

```
# Base de dados da Loja
mysqldump -u trasmum -ptrasmum trasmum_loja > backup_loja_$(date +%Y%m%d).sql

# Base de dados do Servidor Central (execução local)
mysqldump -u trasmum -ptrasmum trasmum_servidor > backup_servidor_$(date +%Y%m%d).sql

# Base de dados do Servidor Central (via Docker, com os serviços a correr)
docker exec mysql_data mysqldump -u trasmum -ptrasmum trasmum_servidor \
  > backup_servidor_$(date +%Y%m%d).sql
```

7.7.2. Restauro da Base de Dados

```
mysql -u trasmum -ptrasmum trasmum_loja < backup_loja_YYYYMMDD.sql
mysql -u trasmum -ptrasmum trasmum_servidor < backup_servidor_YYYYMMDD.sql
```

7.7.3. Atualização do Software

```
git pull

# Servidor Central via Docker – recompila e substitui os containers
cdCodigo/Servidor_Central && docker compose up --build

# Execução local – recompilar cada componente
cdCodigo/Servidor_Central/backend && mvn compile
cdCodigo/Servidor_Central/frontend && npm install && npm run build
cdCodigo/Loja && mvn clean package
```

7.8. Resolução de Problemas Comuns

Sintoma	Causa provável	Solução
<i>Backend</i> não arranca: Config obrigatória em falta: db.url	Variáveis de ambiente não definidas e config.properties sem valores de base de dados	Verificar o ficheiro .env (Docker) ou config.properties (local) e garantir que db.url, db.user e db.password estão preenchidos
<i>Backend</i> não arranca: Port already in use: 8080	Outra instância ou container já a utilizar a porta 8080	Terminar o processo em conflito ou parar o container Docker correspondente
Fecho de dia falha com Connection refused	<i>Backend</i> não acessível no endereço configurado em servidor.url	Verificar se o <i>backend</i> está em execução e se o valor de servidor.url em config.properties da Loja está correcto

Fecho de dia falha com Hash inválido	Corrupção ou alteração dos dados durante o envio	Repetir o fecho de dia; se o erro persistir, verificar a integridade da base de dados local da Loja
<i>Frontend</i> Docker inacessível imediatamente após docker compose down	Container ainda em processo de paragem	Aguardar alguns segundos; para paragem forçada usar <code>docker compose down --timeout 5</code>
Software de Loja não arranca: Erro ao ligar à base de dados	MySQL não está em execução ou as credenciais estão incorretas	Verificar se o MySQL está ativo e se os parâmetros <code>db.*</code> em <code>config.properties</code> da Loja estão corretos
<i>Dashboard</i> não apresenta dados após fecho de dia confirmado	<i>Frontend</i> a apontar para URL do <i>backend</i> incorreto	Verificar <code>VITE_API_URL</code> no <code>.env</code> e reconstruir o <i>frontend</i> com <code>docker compose up --build</code>

Tabela 49: Problemas frequentes e respectivas soluções

8. Conclusões e Trabalho Futuro

Ao longo dos últimos meses, a equipa de desenvolvimento foi capaz de especificar, desenhar e implementar o sistema TrasmUM, uma solução de gestão integrada para uma rede de três lojas de conveniência no distrito de Bragança e a respetiva sede central. O percurso atravessou as quatro etapas previstas para o projeto — definição do domínio, levantamento e análise de requisitos, arquitetura e *design*, e implementação — e culminou na verificação e validação do software produzido, com todas as etapas sistematicamente apoiadas pelo uso de modelos de linguagem de grande dimensão, com registo integral dos *prompts* e reflexão crítica sobre as interações. Sendo a Utilização de LLMs no Desenvolvimento de Software Aplicacional o tema central do trabalho, esta secção dedica especial atenção à reflexão sobre as vantagens e desvantagens efetivamente sentidas no recurso a estas ferramentas, complementando-a com uma síntese dos resultados alcançados e das linhas de trabalho futuro identificadas.

8.1. Resultados Alcançados

A totalidade dos 23 requisitos funcionais especificados foi verificada e a totalidade dos 18 requisitos não funcionais foi acomodada pela solução final, validada formalmente pelo investidor Hélder Cerqueira e pela CEO Cátia na sessão de apresentação do protótipo realizada a 6 de maio de 2026. A solução materializou-se em dois softwares independentes — o Software de Loja, suportado em JavaFX com persistência local em SQLite, e o Software do Servidor Central, composto por um *backend* Javalin e um *frontend* Vue.js sobre MySQL — unidos por um protocolo de sincronização assíncrona com verificação de integridade por *hash* SHA-256 e reversão automática em caso de falha. A arquitetura federada adotada cumpre o requisito central do cliente, ao dotar cada loja de autonomia operacional plena e ao centralizar no Servidor Central apenas a informação necessária à tomada de decisão estratégica. A camada de serviço de ambos os softwares foi coberta por 54 testes unitários sem falhas, com cobertura de instrução superior a 80% em todas as classes testadas, suportada por uma análise de qualidade segundo a norma ISO/IEC 25010. A documentação do processo constitui também um resultado relevante: oito registos de decisão arquitetural, um conjunto exaustivo de diagramas UML, uma especificação OpenAPI da API do Servidor Central e o registo integral das interações com LLMs compõem um corpo de artefactos que torna o trabalho auditável e reproduzível.

8.2. Reflexão sobre a Utilização de LLMs

A reflexão crítica sobre o uso de modelos de linguagem ao longo do projeto constitui, dada a natureza temática deste trabalho, a contribuição central desta secção. A equipa procurou ao longo de todas as fases adotar uma postura genuinamente avaliadora, registando não apenas o que correu bem mas também as limitações e os riscos que sentiu em primeira mão. A apreciação que se segue distingue entre o que se revelou efetivamente um ganho e o que se demonstrou ser, pelo contrário, um custo associado a esta forma de trabalhar.

Do lado das vantagens, quatro destacaram-se. A primeira foi a aceleração do trabalho. A produção de artefactos extensos — especificações de requisitos, diagramas UML, código-fonte, casos de teste, secções do relatório — beneficiou de forma muito visível da capacidade dos modelos para gerar rascunhos competentes em escassos minutos. A fase de implementação ilustra-o de forma particularmente clara: o uso do Claude Sonnet como arquiteto de *prompts*, encarregue de transformar os artefactos de *design* em instru-

ções estruturadas, em conjunto com o Claude Code como executor das instruções produzidas, permitiu à equipa gerar a fundação de ambos os softwares em poucas iterações, libertando tempo para tarefas de revisão e refinamento. A segunda vantagem foi a qualidade da documentação produzida. O presente relatório, em particular, beneficiou da capacidade dos modelos para produzir texto consistente em registo académico, com terminologia uniforme e formatação coerente entre secções, sendo o resultado um documento mais rigoroso e completo do que o que a equipa teria produzido sem este apoio nas mesmas restrições de tempo. A terceira vantagem foi o apoio na deteção de inconsistências e ambiguidades. A revisão crítica do documento integral conduzida por LLM ao final da fase de requisitos, bem como o uso do modelo como arquiteto revisor independente na fase de *design*, identificaram contradições internas e cenários de falha não cobertos que de outra forma teriam permanecido por endereçar. A quarta vantagem — e talvez a mais estruturante — foi a possibilidade de focar o tempo da equipa na revisão crítica em vez da redação ou da codificação mecânica. Boa parte do esforço passou de produzir conteúdo a avaliar, corrigir e integrar conteúdo gerado, uma transição que aproxima o trabalho do que se espera de um engenheiro em ambiente profissional.

Do lado das desvantagens, quatro merecem igualmente destaque, sendo a última a mais preocupante na perspetiva da equipa. A primeira foi o risco de aceitar código ou texto sem o sujeitar a uma revisão crítica suficientemente aprofundada. O fenómeno é conhecido como complacência automatizada, e manifestou-se de forma subtil: *outputs* plausíveis, bem estruturados e superficialmente alinhados com o contexto do projeto podem ser aceites com leituras menos atentas do que aquelas que a equipa aplicaria a conteúdo produzido por um colega. A equipa contrapôs este risco com a regra de que todo o conteúdo gerado por LLM seria revisto por pelo menos um membro antes de integrado no relatório ou na base de código, mas reconhece que a vigilância requerida é cognitivamente exigente e não é sustentável indefinidamente sem disciplina explícita. A segunda desvantagem foi a necessidade de fornecer um volume significativo de contexto para obter resultados verdadeiramente úteis. Os *prompts* que produziram bons resultados foram, em regra, os que incluíam artefactos prévios, decisões já tomadas e restrições do projeto, e a sua preparação representou um investimento de tempo não negligenciável. A terceira desvantagem foi a dificuldade em manter coerência ao longo de sessões longas e ao longo do documento. À medida que o relatório crescia, tornou-se progressivamente mais trabalhoso garantir que decisões referidas numa secção eram refletidas corretamente nas seguintes, e diversas iterações foram dedicadas exclusivamente à reconciliação de inconsistências introduzidas pelo modelo em interações sucessivas.

A quarta desvantagem — e aquela que a equipa considera mais relevante para projetos semelhantes — foi a tendência clara dos modelos para sobre-engenharia. *Outputs* propondo abstrações adicionais, camadas intermédias, padrões de *design* não justificados pelos requisitos e estruturas modulares mais ricas do que o necessário surgiram com frequência ao longo de todas as fases. A equipa procurou ativamente mitigar este viés — solicitando explicitamente nas instruções a opção pela solução mais simples, recusando abstrações sem justificação no contexto do projeto e simplificando *outputs* na fase de revisão —, mas reconhece com franqueza que, apesar destes esforços, o sistema final é materialmente mais complexo do que aquele que a equipa teria desenhado sem o apoio destas ferramentas. Esta observação levanta uma questão incontornável: o ganho de produtividade que os LLMs proporcionam vem acompanhado de um custo de manutenibilidade de longo prazo que apenas se manifesta plenamente em fases que extravasam o âmbito do presente trabalho. É, no entender da equipa, o aspeto mais merecedor de atenção em utilizações futuras destas ferramentas em contexto de engenharia de software.

Em síntese, a equipa recomendaria esta abordagem a outras equipas em projetos semelhantes, mas com reservas. Os modelos demonstraram-se verdadeiramente úteis nas fases de redação de documentação extensa, de exploração de alternativas arquiteturais, de revisão crítica em busca de inconsistências e de geração dirigida de código de teste com contrato bem definido. Em contrapartida, mostraram-se contraproducentes em momentos em que se exigia simplicidade radical, em decisões que beneficiavam de profundo conhecimento tácito do domínio, e em qualquer contexto onde o esforço de preparação do *prompt* excedesse o esforço de realizar a tarefa diretamente. A utilização criteriosa destas ferramentas — fase a fase, tarefa a tarefa, e sempre acompanhada de revisão humana atenta — afigura-se à equipa como o modelo de utilização mais responsável, e foi o que se procurou praticar ao longo do projeto.

8.3. Trabalho Futuro

A apresentação do protótipo funcional aos *stakeholders* permitiu identificar duas linhas de trabalho futuro que extravasam o âmbito do presente ciclo de desenvolvimento.

A primeira é a introdução de notificações automáticas em caso de atraso no fecho de dia. Atualmente, o Servidor Central recebe os pacotes de fecho à medida que cada loja os envia, mas não dispõe de qualquer mecanismo ativo de alerta quando uma loja não cumpre a janela temporal esperada para o envio. A introdução de uma rotina de verificação no Servidor Central, capaz de detetar a ausência de pacote de fecho dentro do intervalo previsto e notificar a CEO via *email* ou através do próprio *dashboard*, completaria o ciclo de supervisão remoto e reforçaria a capacidade da CEO de identificar precocemente situações de falha operacional, indisponibilidade de loja ou eventual fraude.

A segunda linha de trabalho futuro é a gestão centralizada de utilizadores ao nível da rede. Na solução atual, cada Software de Loja gere autonomamente os seus utilizadores e respetivos perfis, sem visibilidade ou controlo a partir do Servidor Central. Embora esta opção seja coerente com a arquitetura federada adotada e com o princípio de autonomia operacional de cada loja, dificulta operações como a transferência de funcionários entre estabelecimentos, a aplicação uniforme de políticas de palavra-passe ou o bloqueio centralizado de contas em caso de cessação de funções. A introdução de um modelo de sincronização bidirecional restrito à entidade *Utilizador*, com a gestão a ser realizada a partir do Servidor Central e propagada às lojas, responderia a esta limitação preservando a autonomia operacional das lojas no plano transacional.

Em suma, a equipa de desenvolvimento foi capaz de especificar e implementar uma solução de gestão integrada para a rede TrasmUM, entregue dentro do prazo previsto e validada pelos *stakeholders*. Para lá do produto final, o presente trabalho deixa também um conjunto de aprendizagens sobre a utilização de LLMs em contexto de engenharia de software, traduzidas numa apreciação ponderada das suas reais vantagens e dos seus riscos efetivos, que se espera vir a constituir uma contribuição útil para futuros projetos da mesma natureza.

Referências

- Anthropic (2025) *Claude — Model Documentation*. Disponível em: <https://docs.claude.com/>.
- Apache Software Foundation (2024) *Apache Maven Project Documentation*. Disponível em: <https://maven.apache.org/>.
- Bechtold, S. et al. (2024) *JUnit 5 User Guide*. Disponível em: <https://junit.org/junit5/docs/current/user-guide/>.
- Connolly, T. e Begg, C. (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6.ª ed. Harlow: Pearson Education.
- EclEmma (2024) *JaCoCo Java Code Coverage Library*. Disponível em: <https://www.jacoco.org/jacoco/>.
- Gluon and Oracle (2024) *OpenJFX — JavaFX Documentation*. Disponível em: <https://openjfx.io/>.
- Google (2025) *Gemini API Documentation*. Disponível em: <https://ai.google.dev/gemini-api/docs>.
- International Organization for Standardization (2011) «ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models».
- Javalin (2024) *Javalin — A Simple Web Framework for Java and Kotlin*. Disponível em: <https://javalin.io/>.
- Madje, M. e Haug, L. (2024) *Typst Documentation*. Disponível em: <https://typst.app/docs>.
- Merkel, D. (2014) «Docker: Lightweight Linux Containers for Consistent Development and Deployment», *Linux Journal*, 2014(239), p. 2:2.
- Mockito Contributors (2024) *Mockito Framework Documentation*. Disponível em: <https://site.mockito.org/>.
- Nygard, M.T. (2011) *Documenting Architecture Decisions*. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>.
- OpenAPI Initiative (2024) *OpenAPI Specification 3.1.0*. Disponível em: <https://spec.openapis.org/oas/v3.1.0>.
- Oracle Corporation (2024) *MySQL 8.0 Reference Manual*. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>.
- Provos, N. e Mazières, D. (1999) «A Future-Adaptable Password Scheme», em *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. Monterey, CA: USENIX Association, pp. 81–91.
- Sommerville, I. (2015) *Software Engineering*. 10.ª ed. Boston: Pearson Education.
- You, E. e Vue.js Contributors (2024) *Vue.js — The Progressive JavaScript Framework*. Disponível em: <https://vuejs.org/>.

Lista de Siglas e Acrônimos

IA	Inteligência Artificial
LLM	Large Language Model
ADR	Architectural Decision Record
ACK	Acknowledgement
API	Application Programming Interface
CEO	Chief Executive Officer
CQRS	Command Query Responsibility Segregation
CRUD	Create, Read, Update, Delete
EDI	Electronic Data Interchange
FEFO	First Expired First Out
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IBAN	International Bank Account Number
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JDBC	Java Database Connectivity
KPI	Key Performance Indicator
MVC	Model-View-Controller
NACK	Negative Acknowledgement
NIF	Número de Identificação Fiscal
POS	Point of Sale
RF	Requisito Funcional
RNF	Requisito Não Funcional
SIBS	Sociedade Interbancária de Serviços
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language

Anexos

Anexo 1: Logo da Universidade do Minho



Registo de Entrevista - Elicitação de Requisitos

Entrevistado(a):	Hélder Cerqueira (Administração)
Equipa/Entrevistador:	Equipa de Análise de Sistemas
Data e Hora:	02 de Março de 2026, 16:00
Local/Plataforma:	Instalações da Sede - TrasmUM
Objetivo Principal:	Compreender as ineficiências do modelo atual de gestão de fornecedores e apuramento de requisitos.

Transcrição da Entrevista

P: Boa tarde, Hélder. Para começar, gostaríamos de saber quais são as suas principais expectativas para o novo sistema que iremos desenvolver.

R: Boa tarde! Olhem, acima de tudo quero que o sistema nos traga controlo e confiança nos números, porque hoje em dia isso simplesmente não existe. Quero conseguir ver as vendas e o stock das lojas sem depender de telefonemas nem de papelada perdida. Também é importante que ajude a equipa nas lojas a trabalhar mais rápido e com menos erros — os operadores perdem demasiado tempo com coisas que deviam ser automáticas. No fundo, procuro uma solução simples de usar, mas robusta o suficiente para suportar o crescimento da rede.

P: No que diz respeito ao funcionário, como é que este deve, num dia normal de funcionamento, interagir com o sistema?

R: Bom, o funcionário chega, autentica-se no sistema — e aqui é importante que seja apenas um por loja, por turno, para sabermos sempre quem é responsável pelo quê — e durante o dia basicamente regista as vendas e trata de eventuais devoluções. No final do dia, o manager faz o fecho de dia e o sistema encarrega-se de enviar tudo automaticamente para a sede. Sem telefonemas, sem papelada — é isso que queremos.

P: Como deve o manager registar uma remessa recém-chegada?

R: Quando chega uma remessa, o manager deve conseguir indicar no sistema quais os produtos que chegaram e em que quantidades, e o sistema adiciona imediatamente essas quantidades ao stock da loja. Ah, e uma coisa importante — cada remessa vem com uma data de validade associada ao lote, e essa data deve ficar registada no sistema. Assim sabemos sempre o que está mais perto de expirar. E como há um pagamento associado a cada remessa, o sistema deve também registar esse valor como pendente, para o manager tratar depois.

P: Quem é responsável pela gestão do catálogo de produtos de cada loja?

R: Isso é da responsabilidade do manager de cada loja. Faz sentido que seja assim — cada loja tem a sua realidade, os seus fornecedores, os seus produtos. O manager deve poder criar, editar e desativar produtos diretamente no terminal da sua loja, e essas alterações devem ter efeito imediato, sem esperar por nada. Não faz sentido centralizar isso na sede.

P: Relativamente ao stock, que funcionalidades espera que o sistema implemente?

R: Há algumas coisas que para mim são essenciais. Primeiro, o stock tem de ser atualizado automaticamente a cada venda e a cada devolução — sem intervenção manual. Depois, quero que o sistema avise o funcionário quando um produto está a ficar com pouco stock, abaixo de um certo limite que o manager possa configurar. Assim evitamos aquelas situações em que o cliente chega à prateleira e não há nada. E do lado da sede, a Cátia precisa de ter acesso a relatórios que identifiquem produtos com excesso de stock ou em risco de rutura, com base nos dados que chegam via fecho de dia. Nada de surpresas no final do mês.

P: No que toca à segurança do sistema, que funcionalidades espera que o mesmo suporte?

R: A segurança é algo que me preocupa bastante. Cada funcionário deve ter as suas próprias credenciais — utilizador e palavra-passe — nada de partilhar acessos. E se alguém errar a palavra-passe várias vezes seguidas, o sistema deve bloquear a conta. As palavras-passe também não devem ser guardadas em texto simples, devem estar cifradas — isso é básico nos dias de hoje. E há níveis de acesso completamente diferentes: um funcionário não deve ver o que um manager vê, e um manager não deve ver os dados consolidados de toda a rede que a Cátia consulta. Os dados financeiros agregados, por exemplo, só ela é que deve conseguir aceder.

P: Tem falado bastante no controlo das datas de validade dos produtos. Como pretende que o sistema suporte a aplicação de descontos a artigos cuja validade está próxima do fim?

R: É algo que vejo muito nos supermercados e que faz todo o sentido aplicar aqui também. Perder produtos porque passaram o prazo é dinheiro deitado ao lixo. O que pretendo é que cada terminal de loja analise diariamente as validades dos seus produtos. Quando um lote estiver abaixo de um certo número de dias para expirar — um limiar que o manager possa configurar — o sistema alerta o manager, e este pode então aplicar um desconto manualmente com um simples clique. Esse desconto incide apenas sobre esse lote específico, não sobre os outros lotes do mesmo produto que ainda estão bem. E a percentagem de desconto também deve ser configurável pelo manager.

P: Só para esclarecer: quem pode criar e gerir utilizadores no sistema?

R: A gestão de utilizadores segue a hierarquia natural da empresa. A Cátia gere os managers de toda a rede — cria, edita, desativa. Cada manager gere os funcionários da sua própria loja. Um funcionário de loja não pode criar nem mexer em contas de ninguém. É importante que cada nível só consiga gerir quem está abaixo de si e na sua própria esfera.

P: Já abordámos algumas otimizações do sistema na reunião que realizámos consigo e com a Cátia. Tem mais alguma que pretenda acrescentar?

R: Sim, há algumas que me parecem importantes. Uma delas é a questão do dinheiro em caixa — o sistema deve monitorizar o valor em numerário, e quando atingir um certo limite, avisar o funcionário para retirar dinheiro. Não convém ter muito dinheiro na caixa por questões de segurança. Outra coisa importante é que o sistema da loja deve conseguir funcionar de forma completamente autónoma durante o dia, sem depender de comunicação com o servidor central. Se a ligação cair a meio do dia, a loja não pode parar — a base de dados local é a fonte de verdade da loja. E do lado da escalabilidade, quero que o sistema esteja preparado para crescer — se amanhã quisermos adicionar mais lojas à rede, não quero que seja preciso andar a reconfigurar tudo manualmente. Ah, e os backups — tanto o servidor central como os terminais locais devem fazer backups automáticos diariamente. Perder dados é algo que simplesmente não posso aceitar.

P: Quanto aos tempos de resposta do sistema, tem algum limite que pretenda impor?

R: Não sou muito técnico nessas coisas, mas há algumas que me parecem óbvias do ponto de vista do negócio. O registo de uma venda tem de ser rápido — dois segundos no máximo. A geração da fatura também deve ser quase imediata, três segundos no máximo. A pesquisa de produtos também — digamos num segundo. As dashboards da Cátia têm de carregar em três segundos. E o envio do pacote de fecho de dia deve estar concluído em cinco minutos por loja, para que de manhã tudo esteja disponível para análise.

P: Ao longo da entrevista falámos de vários parâmetros configuráveis. Pretende que seja o manager de cada loja a gerir estes valores diretamente no terminal?

R: Sim, exatamente. O limiar de validade para aplicar descontos, a percentagem de desconto, o valor máximo de dinheiro na caixa — tudo isso deve ser configurável pelo manager diretamente no terminal da sua loja, sem precisar de chamar ninguém técnico. Cada loja tem as suas especificidades, e os managers devem ter essa autonomia.

P: Da nossa parte, obtivemos todas as respostas de que necessitávamos. Tem algo a acrescentar antes de terminarmos?

R: Só quero reforçar que, relativamente à usabilidade, confio que o vosso trabalho reflita a qualidade que já demonstraram no projeto das limousines. Qualquer dúvida que surja ao longo do desenvolvimento, não hesitem em contactar-me. Boa sorte!

P: Por acaso, essas questões já foram abordadas nos questionários que realizámos aos seus funcionários! Muito obrigado pela atenção, Hélder, até à próxima!

R: Ainda bem! Fico descansado por saber que também ouviram a opinião deles — são quem está na linha da frente no dia a dia. De nada, foi um prazer! Até à próxima!

Registo de Entrevista - Elicitação de Requisitos

Entrevistado(a):	Cátia (CEO da rede TrasmUM e Manager de Loja)
Equipa/Entrevistador:	Equipa de Análise de Sistemas
Data e Hora:	03 de Março de 2026, 10:00
Local/Plataforma:	Instalações da Sede - TrasmUM
Objetivo Principal:	Compreender as necessidades operacionais da Cátia enquanto manager de loja no Software de Loja e enquanto CEO no Software de Servidor Central, nomeadamente a supervisão estratégica da rede, a gestão hierárquica de utilizadores e o controlo de pagamentos a fornecedores.

Transcrição da Entrevista

P: Bom dia, Cátia. Para começar, gostaríamos de perceber melhor o seu papel na rede TrasmUM — acumula a gestão da Loja Central com a supervisão de toda a rede?

R: Bom dia! Sim, exatamente. Sou ao mesmo tempo a manager da Loja Central e a CEO de toda a rede. No dia a dia, a Loja Central funciona como qualquer outra loja — tenho os meus funcionários, as minhas remessas, o meu catálogo. Mas tenho também uma visão que os outros managers não têm: consigo ver o que está a acontecer em todas as lojas, com base nos dados que chegam no final de cada dia. É essa dupla função que me dá o pulso da empresa.

P: Quando fala em “ver o que está a acontecer em todas as lojas”, o que entende exatamente por isso?

R: Preciso de saber como estão as vendas globais da rede e qual é o estado dos pagamentos a fornecedores em toda a rede. Não para interferir nas decisões de cada manager — cada um gere a sua loja — mas para ter uma visão estratégica e detetar problemas antes que se tornem graves.

P: De que forma espera que essa informação lhe chegue? Precisa de a consultar em tempo real?

R: Não, não é necessário em tempo real. O que me interessa é ter essa informação disponível de manhã, consolidada com os dados do dia anterior de cada loja. Cada loja envia os seus dados no fecho de dia, e eu consulto tudo numa dashboard quando chego. Painéis de controlo com gráficos e números — volumes de vendas por loja, evolução de stocks, esse tipo de coisas. E quero poder filtrar por loja, por período, por categoria de produto. Não me serve uma coisa estática que me mostre sempre o mesmo.

P: Falou em pagamentos a fornecedores. Como funciona esse processo atualmente e o que pretende do sistema?

R: Hoje é um caos total. As faturas perdem-se, os prazos falham, e depois recebo chamadas dos fornecedores furiosos. O que quero é que cada manager registe as remessas que chegam à sua loja e marque os pagamentos como pagos quando os liquidar. Eu, da minha parte, quero poder consultar o

estado de todas as remessas de toda a rede — o que está pendente, o que já foi pago, em que loja, para que fornecedor. Não para gerir os pagamentos das outras lojas — isso é responsabilidade de cada manager — mas para ter visibilidade e detetar atrasos.

P: No que toca à sua própria loja, como gere atualmente a equipa de funcionários? O que espera do sistema nessa área?

R: Tenho vários funcionários a trabalhar simultaneamente em cada loja. O problema é que quando deteto uma quebra de caixa ou uma anulação estranha, ninguém assume a responsabilidade. O funcionário da tarde culpa o da manhã e vice-versa. Para acabar com isso, o sistema tem de exigir autenticação individual de cada operador e registar tudo o que cada um faz durante o seu turno. Quero poder ver exatamente quem fez o quê e quando. E só deve poder estar uma sessão ativa de cada vez no terminal — assim não há desculpas.

P: Relativamente à gestão dos managers da rede, como pretende que o sistema a suporte?

R: Sou eu que crio e giro as contas dos managers de todas as lojas — essa responsabilidade é minha e não deve ser delegável. Se um manager sai, desativo a conta imediatamente e não quero que ele consiga entrar mais no sistema. Cada manager, por sua vez, gere os funcionários da sua própria loja. É uma hierarquia clara: eu giro os managers, os managers girem os funcionários deles, e ninguém tem acesso ao que não é da sua responsabilidade.

P: Há alguma preocupação específica em relação à segurança da informação financeira?

R: Sim, e muito. Os dados financeiros consolidados da rede — as dashboards, os relatórios de toda a rede — são informação sensível que só eu devo poder ver. Um manager de loja não tem nem deve ter acesso a isso. Cada um deve ver apenas o que diz respeito à sua loja. É uma questão de sigilo comercial, mas também de evitar que informação sobre o desempenho das outras lojas circule entre managers.

P: Voltando à sua função de manager da Loja Central — há funcionalidades operacionais que considera especialmente críticas para o dia a dia?

R: O fecho de dia é a mais crítica de todas. No final de cada jornada, quero fazer o fecho aqui na Loja Central tal como qualquer outro manager faz na sua loja — confirmar os valores, enviar os dados para o servidor. Mas o sistema só me deve deixar concluir o fecho se os dados forem efetivamente recebidos pelo servidor. Se houver uma falha, quero saber e poder tentar de novo, não quero que o sistema feche sem confirmar que a informação chegou. Além disso, a gestão dos pagamentos aos fornecedores da minha loja funciona exatamente como nas outras — registo as remessas, marco os pagamentos como pagos. Só que eu tenho também a visão agregada das outras lojas, que os outros managers não têm.

P: Tem alguma sugestão adicional que considere relevante antes de terminarmos?

R: Só quero reforçar que o sistema tem de ser fiável acima de tudo. Se eu não puder confiar nos números que vejo nas dashboards, de nada serve tê-las. E do ponto de vista da usabilidade, tem de ser simples — os meus funcionários não são técnicos informáticos, e não posso estar a formar pessoas durante dias sempre que entra alguém novo. Duas horas de formação e a pessoa deve conseguir trabalhar — é esse o objetivo.

P: Da nossa parte, ficou muito claro. Muito obrigado pela disponibilidade, Cátia.

R: De nada! Estou à disposição para qualquer dúvida que surja. Tenho muito interesse em que isto corra bem — afinal, sou eu que vou usar o sistema todos os dias!

Anexo 4: Lista completa de requisitos levantados

TrasmUM — Levantamento de Requisitos (ordem de elicitação) Versão PRÉ-refinamento					
● Fundo vermelho = requisito removido (redundante, inviável ou reformulado) ○ Fundo amarelo = texto corrigido na fase de refinamento Texto original mantido nesta folha					
ID	Tipo	Requisito (texto original)	Analista	Fonte	Estado
R1	Funcional	O sistema deve permitir a operação e autenticação de apenas um funcionário por loja, associando-lhe todas as operações do turno.	José Fernandes	Entrevista — Cátia das Unhas Pintadas	Mantido
R2	Funcional	O sistema deve monitorizar o valor em numerário na caixa, assumindo um fundo inicial de 100€, e sugerir a retirada de dinheiro por questões de segurança.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Questionário — Operadores de Caixa	Mantido
R3	Funcional	O operador deve poder realizar o fecho de caixa diário, confirmando o valor apurado e despoletando o envio dos dados do dia para o servidor central.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira	Mantido
R4	Funcional	A administração deve poder criar pedidos de remessas e direcioná-los especificamente para reabastecimento de lojas selecionadas.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira	Corrigido
R5	Funcional	No final do dia, a loja deve remeter todas as operações de vendas, devoluções e remessas para a sede de forma automática.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas	Corrigido
R6	Funcional	A administração deve poder redigir e enviar mensagens e anúncios informativos para os terminais dos funcionários nas lojas.	Gabriel Dantas	User Story — Cátia das Unhas Pintadas	Removido — Funcionalidade fora do âmbito definido; ausência de requisito técnico correspondente e sem caso de uso associado
R7	Não Funcional	O sistema da loja deve garantir disponibilidade mínima de 99% mensal durante o horário de funcionamento definido.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Hélder Cerqueira	Mantido
R8	Não Funcional	O servidor central deve realizar backups automáticos diários com retenção mínima de 30 dias.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Hélder Cerqueira	Corrigido
R9	Funcional	O funcionário deve poder anular uma venda que ainda se encontre em curso, antes da sua finalização, sem impacto no stock nem no apuramento de caixa.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Mantido
R10	Funcional	O funcionário deve poder registar as vendas dos produtos na loja, atualizando o sistema automaticamente.	Gabriel Dantas	Questionário — Operadores de Caixa	Corrigido
R11	Funcional	O funcionário deve poder processar a devolução de produtos, corrigindo o valor em caixa e repondo o stock disponível, estritamente caso o artigo se encontre dentro do prazo de validade.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Corrigido
R12	Funcional	O manager deve poder registar a entrada de uma remessa de fornecedores, adicionando imediatamente as novas quantidades ao stock da loja e associando-lhes a data de validade correspondente ao lote recebido.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Corrigido
R13	Não Funcional	O registo de uma venda deve ser concluído em ≤ 2 segundos, medido desde a confirmação da operação até à persistência da transação.	José Fernandes	Questionário — Operadores de Caixa	Mantido
R14	Não Funcional	Um novo funcionário deve conseguir executar o registo de venda após ≤ 2 horas de formação.	José Fernandes	Questionário — Operadores de Caixa	Mantido
R15	Não Funcional	O fluxo de registo de venda não deve exigir mais de 5 interações principais.	Gabriel Dantas	Questionário — Operadores de Caixa	Mantido
R16	Não Funcional	O sistema deve apresentar mensagens de erro descritivas e indicar ação corretiva sempre que aplicável.	Luís Ferreira	Questionário — Operadores de Caixa	Mantido

R17	Funcional	O sistema deve calcular automaticamente o troco a devolver ao cliente quando o pagamento é feito em numerário.	José Fernandes	User Story — Funcionário	Mantido
R18	Funcional	O funcionário deve poder pesquisar por produtos na loja para consultar rapidamente o stock e os preços disponíveis.	Gabriel Dantas	User Story — Funcionário	Mantido
R19	Não Funcional	A pesquisa de produtos deve apresentar resultados em ≤ 1 segundo para catálogos até 2.000 produtos.	Luís Ferreira	User Story — Funcionário	Mantido
R20	Funcional	O sistema deve armazenar e apresentar os detalhes atualizados de cada produto vendido na loja, como preço, stock atual e prazo de validade.	Luís Ferreira	User Story — Funcionário	Mantido
R21	Funcional	O manager deve poder registar a receção de remessas de fornecedores, para que o stock da sua loja seja atualizado automaticamente.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	User Story — Manager de Loja	Mantido
R22	Funcional	O manager deve poder criar e atualizar os produtos e preços do catálogo da sua loja diretamente no terminal local.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	User Story — Manager de Loja	Mantido
R23	Funcional	O manager deve poder criar e gerir as contas dos funcionários da sua loja, para que cada operador aceda com credenciais próprias.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	User Story — Manager de Loja	Mantido
R24	Funcional	O manager deve poder realizar o fecho de dia e enviar automaticamente os dados para a Loja Central.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	User Story — Manager de Loja	Corrigido
R25	Funcional	O sistema deve gerar uma fatura em formato digital no final de cada venda para entrega ou visualização.	Simão Oliveira	Análise Documental — Fatura física TrasmUM	Mantido
R26	Funcional	O manager deve poder introduzir e consultar digitalmente os documentos dos fornecedores (guias de remessa e faturas) na base de dados local da loja.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Hélder Cerqueira / Análise Documental — Guia de Remessa	Mantido
R27	Não Funcional	O sistema deve exigir autenticação individual baseada em credenciais únicas para todos os perfis de utilizador.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Mantido
R28	Não Funcional	O sistema deve bloquear a conta após 5 tentativas falhadas consecutivas.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Mantido
R29	Não Funcional	As palavras-passe dos utilizadores devem ser armazenadas de forma cifrada e irreversível, não sendo possível a sua recuperação em texto simples.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Mantido
R30	Não Funcional	O sistema deve garantir que apenas utilizadores com perfil CEO possam aceder aos dados financeiros consolidados da rede.	Simão Oliveira	Entrevista — Cátia	Mantido
R31	Não Funcional	Todas as ações críticas devem ser registadas em logs de auditoria (vendas, devoluções, alterações de preços, aplicação de descontos, gestão de utilizadores e alterações ao catálogo).	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira	Mantido
R32	Não Funcional	O envio do pacote de fecho de dia entre a loja e o servidor da Loja Central deve garantir idempotência, impedindo a duplicação de transações em caso de reenvio.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Hélder Cerqueira	Corrigido
R33	Não Funcional	O sistema deve suportar crescimento de até 50% no volume anual de transações sem degradação superior a 10% nos tempos de resposta definidos.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Hélder Cerqueira	Mantido
R34	Não Funcional	A arquitetura deve permitir o registo de novas lojas no servidor da Loja Central sem necessidade de reconfiguração manual do sistema central, bastando o aprovisionamento local do novo terminal.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Corrigido

R35	Não Funcional	Parâmetros operacionais locais (limiar de validade para alertas, percentagem de desconto, valor máximo de caixa) devem ser configuráveis pelo manager de cada loja através da interface do terminal, sem necessidade de recompilação do software.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Mantido
R36	Não Funcional	O sistema de cada loja deve manter uma base de dados local autónoma que suporte a totalidade das operações diárias — vendas, devoluções, registo de remessas e gestão de catálogo — sem necessidade de qualquer comunicação com o servidor da Loja Central durante o horário de funcionamento.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Hélder Cerqueira / Cátia das Unhas Pintadas	Corrigido
R37	Funcional	As lojas devem analisar diariamente as validades dos produtos e o sistema sugerir/alertar os produtos em risco, cabendo à administração clicar num botão para “Aplicar Desconto” de forma manual.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Corrigido
R38	Funcional	O fecho de caixa só deve ser considerado concluído após confirmação explícita de receção bem-sucedida dos dados pelo servidor central.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira	Mantido
R39	Funcional	O sistema deve emitir um alerta visual para o operador de loja quando a quantidade em stock de um produto atingir ou descer abaixo de um limiar mínimo configurável.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Mantido
R40	Não Funcional	As dashboards devem carregar em ≤ 3 segundos para consultas padrão, sobre os dados consolidados recebidos via fecho de dia.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Hélder Cerqueira	Mantido
R41	Funcional	A gestão de contas de utilizador deve seguir a hierarquia de autenticação do sistema: a Cátia gere as contas de manager de cada loja; cada manager gere exclusivamente as contas de funcionários da sua própria loja.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Entrevista — Cátia / User Story — Manager de Loja	Mantido
R42	Funcional	O servidor da Loja Central deve receber os pacotes de fecho de dia enviados por cada loja, inserindo-os na base de dados central para análise estratégica.	Simão Oliveira	Entrevista — Hélder Cerqueira	Corrigido
R43	Funcional	A Cátia deve poder consultar dados de análise avançada indicando os artigos de alto stock parado ou de rutura iminente, com base nos dados consolidados da rede.	Simão Oliveira	Entrevista — Cátia	Mantido
R44	Funcional	A Cátia deve ter acesso a uma vista consolidada do estado de todas as remessas da rede (pendentes e pagas), recebidas via pacotes de fecho de dia.	Luís Ferreira	Entrevista — Cátia	Mantido
R45	Não Funcional	O envio e processamento do pacote de fecho de dia de cada loja deve ser concluído em ≤ 5 minutos.	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira	Mantido
R46	Não Funcional	A geração de fatura deve ocorrer em ≤ 3 segundos após confirmação da venda.	Gabriel Dantas	Análise Documental — Fatura física TrasmUM	Mantido
R47	Funcional	O sistema deve permitir ao manager exportar o catálogo de produtos da loja para um ficheiro CSV.	Luís Ferreira	User Story — Manager de Loja (rascunho)	Removido — Funcionalidade não confirmada por nenhum stakeholder; ausência de fonte válida e sem caso de uso correspondente
R48	Funcional	O funcionário deve poder consultar o histórico completo de vendas da loja nos últimos 12 meses a partir do terminal de caixa.	Simão Oliveira	Rascunho interno — equipa	Removido — Redundante face a R43 e R44; visibilidade histórica agregada é exclusiva do Servidor Central e da CEO
R49	Não Funcional	O sistema deve enviar notificações automáticas por email ao manager quando o stock de um produto atingir o limiar mínimo.	Gabriel Dantas	Rascunho interno — equipa	Removido — Restrição de âmbito: o sistema não realiza comunicações externas automáticas (cf. Análise de Contexto e Restrições); funcionalidade coberta pelo alerta visual interno do RF10
R50	Funcional	O sistema deve permitir à CEO definir centralmente os preços dos produtos de todas as lojas a partir do Servidor Central.	José Fernandes	Rascunho interno — equipa	Removido — Contradiz a decisão arquitetural de gestão local autónoma do catálogo por cada manager (RF17); viola o princípio de autonomia operacional por loja

Anexo 5: Especificação Completa dos Requisitos Funcionais

Requisito de Utilizador	O sistema deve impedir que o mesmo utilizador esteja autenticado em mais do que um terminal simultaneamente, independentemente do número de terminais disponíveis na loja, independentemente do seu perfil de autenticação, associando-lhe todas as operações realizadas durante o turno. Numa loja com dois terminais, podem coexistir duas sessões simultâneas — uma por terminal —, sem que tal constitua violação desta restrição.
Analista	José Fernandes
Fonte	Entrevista — Cátia
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve verificar, no momento da autenticação, se o utilizador já possui uma sessão ativa noutro terminal, bloqueando o login caso isso se verifique. 2. O sistema deve registar internamente o identificador único do utilizador autenticado e associá-lo a todas as operações de escrita na base de dados local durante a sessão. 3. A restrição aplica-se ao utilizador, não ao terminal: dois utilizadores distintos podem ter sessões ativas em terminais diferentes da mesma loja simultaneamente.
Relevância para a Aplicação	Resolve a ineficiência de responsabilização (o “jogo do empurra” entre turnos), assegurando a integridade da autoria das transações financeiras independentemente do nível hierárquico do utilizador em funções. A existência de dois terminais por loja garante ainda a continuidade operacional da caixa, eliminando a necessidade de interromper o atendimento ao cliente para a realização de tarefas de gestão.

Tabela 50: Especificação do Requisito Funcional 1

Requisito de Utilizador	O sistema deve exigir credenciais únicas (utilizador e palavra-passe) para acesso ao terminal, desbloqueando funcionalidades distintas consoante o perfil autenticado — Funcionário, Manager ou CEO.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira

Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve apresentar um único ecrã de início de sessão que exija a inserção de utilizador e palavra-passe, comum a todos os perfis. 2. O sistema deve validar as credenciais contra a base de dados local de utilizadores ativos antes de permitir o acesso. 3. O sistema deve ler, no arranque, um ficheiro de configuração local que contenha o identificador único da loja a que o terminal pertence. Este ficheiro é definido no momento da instalação e não é alterável através da interface da aplicação. 4. Após autenticação bem-sucedida, o sistema determina as funcionalidades disponíveis com base no perfil do utilizador, de acordo com as seguintes regras: • O perfil Funcionário acede ao registo de vendas, devoluções e consulta de produtos. • O perfil Manager acede adicionalmente à gestão de remessas, catálogo, preços, funcionários e fecho de dia. • O perfil CEO acede a todas as funcionalidades do Manager, acrescentando a gestão de contas de manager do mesmo estabelecimento.
Relevância para a Aplicação	Garante a segregação de responsabilidades e o sigilo da informação financeira e operacional, impedindo que utilizadores acedam a funcionalidades ou dados fora do âmbito do seu cargo. A autenticação individual substitui o acesso livre e partilhado, prevenindo a manipulação indevida dos dados de faturação e acessos não autorizados.

Tabela 51: Especificação do Requisito Funcional 2

Requisito de Utilizador	O funcionário deve poder registar as vendas dos produtos identificando o código de barras do produto, para atualização automática de stock e obtenção do seu preço. Esta funcionalidade está disponível a todos os perfis de utilizador.
Analista	Gabriel Dantas
Fonte	Questionário — Operadores de Caixa
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. A interface de caixa deve manter o campo de inserção automaticamente em foco, processando de imediato a entrada do código de barras genérico do produto. 2. O sistema deve deduzir automaticamente uma unidade ao lote desse produto com a data de validade mais próxima a expirar (FEFO) na base de dados local, exceto quando o funcionário aciona o botão “Desconto Validade”, caso em que o sistema sobrepõe a lógica FEFO e abate a unidade ao lote sinalizado com desconto selecionado pelo funcionário. 3. O sistema deve apresentar o preço de venda atualizado, aplicando automaticamente eventuais descontos de validade já configurados para esse lote.
Relevância para a Aplicação	A lógica FEFO garante que os lotes mais antigos são sempre os primeiros a ser vendidos. O botão ‘Desconto Validade’ permite ao funcionário desviar-se desta lógica de forma controlada, assegurando que os lotes sinalizados pelo manager são escoados sem necessidade de hardware adicional de impressão de etiquetas.

Tabela 52: Especificação do Requisito Funcional 3

Requisito de Utilizador	O funcionário deve poder anular uma venda em curso antes de a finalizar, sem afetar o apuramento final ou os stocks.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. A interface deve disponibilizar uma opção para cancelar a transação corrente antes da emissão do comprovativo. 2. O sistema deve limpar os produtos selecionados da memória temporária sem registar movimentos financeiros ou alterações no inventário local.
Relevância para a Aplicação	Aumenta a eficiência da linha de frente e previne a contaminação dos dados contabilísticos com vendas não efetivadas.

Tabela 53: Especificação do Requisito Funcional 4

Requisito de Utilizador	O funcionário deve poder registar devoluções associadas a uma fatura de venda prévia, deduzindo o valor exato em caixa e repondo o stock de um artigo apenas se este estiver dentro da validade. O sistema local deve validar se a fatura inserida foi emitida na base de dados dessa mesma loja. Apenas são aceites devoluções de transações originadas localmente, rejeitando faturas de outras filiais.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira

Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve exigir a inserção do identificador da fatura original, do artigo e da data de validade física impressa na embalagem, para mapeamento automático ao código de lote correspondente. 2. O sistema deve validar a existência do artigo na fatura e bloquear a operação se este já tiver sido alvo de devolução prévia. A validação opera ao nível da unidade: o sistema deve verificar quantas unidades do artigo constam na fatura e quantas já foram devolvidas, bloqueando quando a quantidade devolvida igualar a quantidade faturada. 3. O sistema deve resgatar o valor exato pago na altura da compra e subtrair-lo ao apuramento atual de caixa. 4. O sistema deve comparar a data de validade do lote com a data atual e, se válida, incrementar em um a quantidade de stock local para esse lote; caso contrário, não deve alterar a quantidade em stock.
Relevância para a Aplicação	Assegura a exatidão financeira do apuramento diário e previne a comercialização indevida de artigos perecíveis expirados.

Tabela 54: Especificação do Requisito Funcional 5

Requisito de Utilizador	O manager deve registar a receção de remessas introduzindo a respetiva guia, de forma a atualizar o stock local com lotes independentes e gerar um registo de pagamento pendente associado ao fornecedor.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve permitir ao manager a introdução dos dados da guia de remessa (fornecedor, produtos, quantidades, data, valor) e a sua submissão local. 2. O sistema deve criar registos independentes de stock (novos lotes) para as unidades recebidas, atribuindo-lhes um código único de lote e a data de validade da entrega, sem sobrepor as validades dos lotes pré-existentes. 3. O sistema deve gerar localmente um registo de pagamento pendente associado ao fornecedor, com base no valor financeiro registado na guia, para inclusão no pacote de fecho de dia. 4. O sistema deve permitir ao manager atualizar o estado de um registo de pagamento pendente para “pago”, ficando essa alteração incluída no próximo pacote de fecho de dia.
Relevância para a Aplicação	Elimina o arquivo físico caótico em caixas de sapatos, assegura a rastreabilidade individual dos lotes e garante que as obrigações de tesouraria ficam registadas localmente para posterior consolidação no Servidor Central.

Tabela 55: Especificação do Requisito Funcional 6

Requisito de Utilizador	O manager deve poder criar pedidos de remessa dirigidos a fornecedores, registando os produtos e quantidades pretendidas, ficando o pedido com estado “pendente” até à receção da mercadoria.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve permitir ao manager criar um pedido de remessa, selecionando o fornecedor, os produtos do catálogo local e as quantidades pretendidas. 2. O sistema deve guardar o pedido com estado “pendente” na base de dados local. 3. Quando o manager regista a receção de uma remessa, o sistema deve verificar se existe um pedido pendente para esse fornecedor e, caso exista, atualizar automaticamente o seu estado para “concluído”. Se não existir nenhum pedido pendente para o fornecedor da remessa recebida, o sistema regista a remessa sem associação a pedido, gerando apenas o registo de pagamento pendente. 4. O manager deve poder consultar a listagem de pedidos de remessa, filtrável por estado (pendente ou concluído) e por fornecedor.
Relevância para a Aplicação	Formaliza o processo de aprovisionamento de cada loja, criando um rasto documental dos pedidos efetuados e eliminando a dependência de comunicação informal com fornecedores.

Tabela 56: Especificação do Requisito Funcional 7

Requisito de Utilizador	O sistema deve manter e exibir os detalhes de cada produto da loja (preço, stock atual, prazo de validade), sendo o catálogo local gerido diretamente pelo manager ou CEO.
Analista	Luís Ferreira
Fonte	User Story — Funcionário

Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve armazenar numa base de dados local os atributos identificadores, preços, e listagem de lotes (com stocks e validades correspondentes) para cada produto. 2. O sistema deve expor esta informação na interface da frente de loja aquando da seleção de um artigo.
Relevância para a Aplicação	Elimina as discrepâncias causadas pelas anotações em papel e agiliza a resposta do funcionário perante o cliente, com base em dados geridos autonomamente pelo manager de cada loja.

Tabela 57: Especificação do Requisito Funcional 8

Requisito de Utilizador	O sistema deve monitorizar o numerário em caixa e emitir alertas para a retirada de dinheiro (sangria) por questões de segurança.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Cátia das Unhas Pintadas / Questionário — Operadores de Caixa
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve inicializar a sessão diária de caixa com um valor base de 100€. 2. O sistema deve monitorizar cumulativamente os pagamentos em numerário e despoletar um alerta na interface quando um limite superior configurado for excedido. 3. O sistema deve permitir o registo da operação de “sangria”, subtraindo o valor declarado ao total em caixa e desativando o alerta em conformidade.
Relevância para a Aplicação	Garante práticas seguras de tesouraria em ambiente de loja de conveniência e previne erros de contagem diária.

Tabela 58: Especificação do Requisito Funcional 9

Requisito de Utilizador	O sistema deve alertar visualmente o manager quando o stock de um produto baixar do limiar mínimo.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve avaliar a quantidade total (soma de todos os lotes) de um produto após cada venda. 2. O sistema deve gerar um indicador visual (notificação ou destaque na interface) se o valor resultante for menor ou igual ao valor mínimo configurado para esse produto.
Relevância para a Aplicação	Reduz falhas no serviço ao cliente e atua como medida preventiva contra perdas de vendas diretas.

Tabela 59: Especificação do Requisito Funcional 10

Requisito de Utilizador	O sistema deve gerar uma fatura em formato digital para consulta ou entrega após cada venda.
Analista	Simão Oliveira
Fonte	Análise Documental — Fatura física TrasmUM
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve compilar os dados da transação finalizada (produtos, valores, data e método de pagamento) num formato documental digital. 2. O sistema deve permitir a visualização dessa fatura simulada na interface de caixa, dispensando comunicação com o portal da Autoridade Tributária.
Relevância para a Aplicação	Permite aos funcionários concluir a operação comercial com normalidade e regista evidências da transação no sistema.

Tabela 60: Especificação do Requisito Funcional 11

Requisito de Utilizador	O sistema deve calcular o troco a devolver ao cliente nas transações feitas em numerário.
Analista	José Fernandes
Fonte	User Story — Funcionário

Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve apresentar um campo de preenchimento para o valor monetário entregue pelo cliente quando a opção “numerário” for selecionada. 2. O sistema deve subtrair o total a pagar ao valor entregue e apresentar instantaneamente a diferença como “Troco” no ecrã.
Relevância para a Aplicação	Reduz a carga mental da linha da frente e assegura a integridade das contas de tesouraria diárias da loja.

Tabela 61: Especificação do Requisito Funcional 12

Requisito de Utilizador	O funcionário deve poder pesquisar de forma célere por produtos para verificar o inventário e os preços da loja.
Analista	Gabriel Dantas
Fonte	User Story — Funcionário
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve fornecer uma barra de texto com suporte para queries dinâmicas (nome ou categoria) sobre o catálogo local. 2. O sistema deve devolver como resultado de pesquisa as informações relativas ao stock global do produto e respetivos preços.
Relevância para a Aplicação	Otimiza exponencialmente a velocidade de serviço de balcão, permitindo que o funcionário responda de imediato às dúvidas dos clientes sobre disponibilidade e preços, sem interrupções na operação da caixa.

Tabela 62: Especificação do Requisito Funcional 13

Requisito de Utilizador	O manager deve poder consultar digitalmente os documentos dos fornecedores (guias de remessa e faturas) na base de dados local da loja.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Hélder Cerqueira / Análise Documental — Guia de Remessa
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve permitir ao manager o preenchimento de metadados classificativos para cada documento recebido (fornecedor, data, tipo, valor). 2. O sistema deve permitir pesquisar os registos armazenados baseando-se nesses mesmos filtros e categorias.
Relevância para a Aplicação	Substitui os arquivos caóticos, assegurando que toda a operação se torna fiscal e administrativamente rastreável, com gestão autónoma por parte de cada Manager.

Tabela 63: Especificação do Requisito Funcional 14

Requisito de Utilizador	O fecho de dia, da responsabilidade exclusiva do manager ou CEO, só é concluído após confirmação de receção dos dados pelo Servidor Central, permitindo novas tentativas em caso de erro. O Servidor Central recebe os pacotes de forma passiva e contínua, disponibilizando em qualquer momento o estado de receção de cada loja da rede para consulta pelo CEO.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira
Área do Sistema	Terminal de Loja e Terminal de Servidor Central
Requisitos de Sistema	Terminal de Loja: 1. O sistema deve bloquear o encerramento do fecho de dia até confirmar que o pacote foi corretamente recebido e persistido pelo Servidor Central. 2. Em caso de falha de comunicação, o sistema deve apresentar o motivo do erro e fornecer uma opção manual para reexecutar o processo de envio. 3. O encerramento de sessão de um funcionário não despoleta qualquer envio de dados — os dados ficam retidos localmente até ao fecho de dia realizado pelo manager ou CEO. Terminal de Servidor Central: 4. O Servidor Central deve receber e persistir automaticamente cada pacote de fecho de dia submetido por qualquer loja da rede, associando-o à loja e à data correspondente, e confirmar a receção com um ACK ao terminal de origem.

	5. O Servidor Central deve disponibilizar em qualquer momento o estado de receção dos pacotes de cada loja da rede para a data corrente. 6. O Servidor Central deve registar automaticamente qualquer loja cujo identificador não exista ainda na sua base de dados, criando um novo registo de loja sem intervenção manual.
Relevância para a Aplicação	Garante a integridade dos dados transacionais de cada loja e assegura que o Terminal de Loja nunca encerra o fecho sem confirmação explícita de receção pelo Servidor Central.

Tabela 64: Especificação do Requisito Funcional 15

Requisito de Utilizador	O manager ou o CEO deve poder acionar o fecho de dia, despoletando o envio automático para o Servidor Central de todos os dados registados na loja durante a jornada (vendas, devoluções, remessas e pagamentos pendentes).
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve agregar todos os registos gerados desde o último fecho de dia bem-sucedido para transmissão ao Servidor Central. 2. O sistema deve submeter os dados agregados ao Servidor Central exclusivamente quando o manager acionar o processo de Fecho de Dia.
Relevância para a Aplicação	Substitui a morosa recolha telefónica de dados e é o mecanismo central de consolidação de informação operacional da rede TrasmUM.

Tabela 65: Especificação do Requisito Funcional 16

Requisito de Utilizador	O manager deve poder gerir o catálogo de produtos da sua loja, criando, atualizando e desativando produtos de forma autónoma.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira / User Story — Manager
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve fornecer ao manager ecrãs de operações de CRUD sobre os produtos do catálogo local da loja. 2. As alterações efetuadas ao catálogo produzem efeito imediato na base de dados local, ficando disponíveis para os funcionários sem necessidade de sincronização externa.
Relevância para a Aplicação	Concede a cada loja autonomia operacional plena sobre o seu próprio inventário, eliminando a dependência de sistemas externos para atualização de produtos e preços.

Tabela 66: Especificação do Requisito Funcional 17

Requisito de Utilizador	O manager deve poder consultar e gerir os pagamentos associados às remessas da sua loja, registando a liquidação de pagamentos pendentes a fornecedores.
Analista	Luís Ferreira
Fonte	Entrevista — Cátia / User Story — Manager
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve apresentar ao manager a listagem de todas as remessas registadas na loja com o respetivo estado (pendente ou pago), ordenável por fornecedor, montante e data. 2. O sistema deve permitir ao manager selecionar um pagamento pendente e registar a sua liquidação, atualizando o estado para “pago” na base de dados local. 3. A alteração de estado deve ser incluída no pacote de fecho de dia para transmissão ao Servidor Central.
Relevância para a Aplicação	Dota cada manager de controlo autónomo sobre as obrigações financeiras da sua loja perante os fornecedores, eliminando o risco de pagamentos em atraso por falta de rastreabilidade.

Tabela 67: Especificação do Requisito Funcional 18

Requisito de Utilizador	A Cátia deve ter acesso a uma vista consolidada do estado de todas as remessas da rede (pendentes e pagas), recebidas via pacotes de fecho de dia, para fins de supervisão e auditoria.
Analista	Luís Ferreira
Fonte	Entrevista — Cátia
Área do Sistema	Terminal de Servidor Central

Requisitos de Sistema	1. O Servidor Central deve agregar os registos de remessas e respetivos estados recebidos nos pacotes de fecho de dia de todas as lojas. 2. O Servidor Central deve apresentar a listagem consolidada filtrável por loja e estado de pagamento.
Relevância para a Aplicação	Fornece à Cátia visibilidade estratégica sobre o estado das obrigações financeiras de toda a rede, sem interferir na gestão autónoma de cada loja.

Tabela 68: Especificação do Requisito Funcional 19

Requisito de Utilizador	A Cátia deve poder visualizar através de painéis de controlo os volumes globais de vendas e a evolução dos stocks de toda a rede, com base nos dados consolidados recebidos via fecho de dia.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Servidor Central
Requisitos de Sistema	1. O Servidor Central deve compilar estatísticas macro a partir da sua base de dados, construída a partir dos pacotes de fecho de dia recebidos das lojas da rede. 2. A interface deve traduzir estes dados em gráficos visuais e mostradores numéricos consolidados. 3. A interface deve indicar claramente a data e hora da última sincronização de dados recebida. 4. O sistema deve permitir à CEO exportar os dados apresentados nos dashboards sob a forma de relatório estruturado, filtrável por loja, por período temporal e por categoria de produto.
Relevância para a Aplicação	Concede à Cátia as ferramentas necessárias para medir o sucesso global da rede com base nos dados do dia anterior de cada estabelecimento.

Tabela 69: Especificação do Requisito Funcional 20

Requisito de Utilizador	O manager deve poder aplicar descontos a lotes isolados cuja validade esteja em vias de expirar, com base em alertas gerados automaticamente pelo terminal local da sua loja.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O terminal local de cada loja deve executar diariamente uma rotina de verificação que identifique todos os lotes cuja data de validade seja inferior ou igual ao limiar configurável pelo manager. 2. O sistema deve apresentar ao manager um alerta com os lotes identificados e um botão “Aplicar Desconto” para cada um, permitindo-lhe introduzir a percentagem de desconto pretendida. 3. O sistema deve reduzir o preço exclusivamente do lote sinalizado, sem afetar o preço dos restantes lotes do mesmo produto na base de dados local. 4. A interface de caixa deve disponibilizar um botão de ação rápida “Desconto Validade” que, ao ser acionado no momento da venda, apresenta ao funcionário a lista de lotes com desconto ativo para que este selecione o correto, aplicando a redução de preço configurada e abatendo a unidade ao lote selecionado.
Relevância para a Aplicação	Instrumento de otimização de quebra zero gerido localmente por cada manager, impulsionando as vendas dos bens prestes a perecer antes que o descarte seja forçado.

Tabela 70: Especificação do Requisito Funcional 21

Requisito de Utilizador	O Servidor Central deve receber os pacotes de fecho de dia enviados por todas as lojas da rede, inserindo-os na base de dados central para análise estratégica.
Analista	Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Servidor Central
Requisitos de Sistema	1. O Servidor Central deve disponibilizar um mecanismo de receção de pacotes de dados operacionais submetidos por todas as lojas da rede no fecho de dia, inserindo-os na base de dados central local. 2. O Servidor Central deve assegurar as transações de dados na base de dados central garantindo relacionamentos atómicos. 3. O sistema deve confirmar o recebimento emitindo uma resposta afirmativa (ACK) ao terminal de origem, simetricamente para todas as lojas da rede.
Relevância para a Aplicação	É o mecanismo recetor da arquitetura federada, construindo a base de dados central com os dados financeiros e operacionais da rede.

Tabela 71: Especificação do Requisito Funcional 22

Requisito de Utilizador	A gestão de contas de utilizador deve seguir a hierarquia de autenticação do sistema: a Cátia gere as contas de manager de cada loja deslocando-se ao terminal dessa loja e autenticando-se com perfil CEO; cada manager gere exclusivamente as contas de funcionários da sua própria loja a partir do terminal local.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Cátia / User Story — Manager
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O perfil CEO deve ter acesso a operações de CRUD sobre contas de perfil Manager na base de dados local do terminal onde está autenticado. 2. O perfil Manager deve ter acesso a operações de CRUD sobre contas de perfil Funcionário, geridas localmente no terminal da sua loja. 3. Um manager não deve ter acesso à gestão de contas de outras lojas nem de contas de nível Manager ou superior. 4. A desativação de uma conta de funcionário deve ter efeito imediato no terminal local, impedindo novos inícios de sessão com essas credenciais.
Relevância para a Aplicação	Implementa a segregação hierárquica de responsabilidades na gestão de acessos, garantindo que cada nível só gere os utilizadores sob a sua alçada direta.

Tabela 72: Especificação do Requisito Funcional 23

Anexo 6: Especificação Completa dos Requisitos Não Funcionais

Requisito de Utilizador	O registo de uma venda deve ser concluído em ≤ 2 segundos, medido desde a confirmação da operação até à persistência da transação.
Analista	José Fernandes
Fonte	Questionário — Operadores de Caixa
Relevância para a Aplicação	Garante um fluxo de atendimento contínuo e ininterrupto, reduzindo a pressão sobre os funcionários na linha da frente e assegurando que as limitações do software não prejudicam a capacidade de resposta da loja.

Tabela 73: Especificação do Requisito Não Funcional 1

Requisito de Utilizador	A geração de fatura deve ocorrer em ≤ 3 segundos após confirmação da venda.
Analista	Gabriel Dantas
Fonte	Análise Documental — Fatura física TrasmUM
Relevância para a Aplicação	Evita estrangulamentos na conclusão do processo de compra, assegurando que o cliente não aguarda desnecessariamente pelo talão e que o operador fica rapidamente livre para o próximo atendimento.

Tabela 74: Especificação do Requisito Não Funcional 2

Requisito de Utilizador	O envio e processamento do pacote de fecho de dia de cada loja deve ser concluído em ≤ 5 minutos, considerando o volume médio diário de transações.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira
Relevância para a Aplicação	Garante a consolidação ágil de todos os dados financeiros e logísticos diários no Servidor Central, assegurando que os dados de todas as lojas estão disponíveis para análise pela Cátia na manhã seguinte.

Tabela 75: Especificação do Requisito Não Funcional 3

Requisito de Utilizador	As dashboards devem carregar em ≤ 3 segundos para consultas padrão, sobre os dados consolidados recebidos via fecho de dia.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Relevância para a Aplicação	Proporciona à Cátia uma experiência analítica reativa e eficiente, essencial para o acompanhamento contínuo da rentabilidade da rede e planeamento tático imediato.

Tabela 76: Especificação do Requisito Não Funcional 4

Requisito de Utilizador	O sistema deve realizar backups automáticos diários com retenção mínima de 30 dias, abrangendo tanto o Servidor Central como as bases de dados locais de cada loja.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira

Fonte	Reunião — Hélder Cerqueira
Relevância para a Aplicação	Assegura a recuperação total do sistema em caso de incidente informático grave, protegendo a integridade dos dados tanto a nível central como local, dado que cada loja opera de forma autónoma e os dados locais são críticos até ao momento do fecho de dia.

Tabela 77: Especificação do Requisito Não Funcional 5

Requisito de Utilizador	O sistema deve exigir autenticação individual baseada em credenciais únicas para todos os perfis de utilizador (Funcionário, Manager e CEO).
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Relevância para a Aplicação	Impede o uso não autorizado dos terminais e garante que todas as transações, anulações e operações de gestão fiquem inequivocamente ligadas à identidade do utilizador em funções.

Tabela 78: Especificação do Requisito Não Funcional 6

Requisito de Utilizador	O sistema deve bloquear temporariamente o acesso ao terminal após 5 tentativas de autenticação falhadas consecutivas, desbloqueando-o automaticamente após um período de espera configurável, sem necessidade de intervenção de outro utilizador.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Relevância para a Aplicação	Previne invasões por tentativa e erro nas interfaces de login, salvaguardando o sistema contra acessos indevidos que pudessem comprometer o normal funcionamento da operação.

Tabela 79: Especificação do Requisito Não Funcional 7

Requisito de Utilizador	As palavras-passe dos utilizadores devem ser armazenadas de forma cifrada e irreversível, não sendo possível a sua recuperação em texto simples.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Relevância para a Aplicação	Assegura a confidencialidade das credenciais de acesso mesmo perante fugas de dados, respeitando as boas práticas de cibersegurança e protegendo a integridade da identidade dos utilizadores.

Tabela 80: Especificação do Requisito Não Funcional 8

Requisito de Utilizador	O sistema deve garantir que apenas utilizadores com perfil CEO possam aceder aos dados financeiros consolidados da rede.
Analista	Simão Oliveira
Fonte	Entrevista — Cátia
Relevância para a Aplicação	Protege o sigilo comercial e os dados financeiros agregados da rede TrasmUM, assegurando que apenas a Cátia tem acesso ao núcleo económico consolidado do negócio.

Tabela 81: Especificação do Requisito Não Funcional 9

Requisito de Utilizador	Todas as ações críticas devem ser registadas em logs de auditoria, incluindo vendas, devoluções, alterações de preços, aplicação de descontos, gestão de utilizadores e alterações ao catálogo local.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Cátia das Unhas Pintadas / Hélder Cerqueira
Relevância para a Aplicação	Fornecer uma base documental informática rigorosa para a resolução de discrepâncias e prevenção de fraudes, permitindo uma auditoria completa de qualquer evento relevante ocorrido no sistema, quer ao nível local quer ao nível central.

Tabela 82: Especificação do Requisito Não Funcional 10

Requisito de Utilizador	A transmissão dos dados de fecho de dia entre o terminal de loja e o Servidor Central deve garantir idempotência, impedindo a duplicação de registos na base de dados central em caso de reenvio.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Hélder Cerqueira
Relevância para a Aplicação	Garante a exatidão contabilística e do inventário ao impedir que falhas de rede causem o registo múltiplo da mesma fatura ou devolução na base de dados central.

Tabela 83: Especificação do Requisito Não Funcional 11

Requisito de Utilizador	O fluxo de registo de venda não deve exigir mais de cinco interações principais.
Analista	Gabriel Dantas
Fonte	Questionário — Operadores de Caixa
Relevância para a Aplicação	Garante que a principal tarefa do funcionário ocorre sem atritos na interface, acelerando os fechos de compras e atenuando o stress originado por um software demasiadamente intrincado.

Tabela 84: Especificação do Requisito Não Funcional 12

Requisito de Utilizador	O sistema deve apresentar mensagens de erro descritivas e indicar ação corretiva sempre que aplicável.
Analista	Luís Ferreira
Fonte	Questionário — Operadores de Caixa
Relevância para a Aplicação	Evita a paralisação do serviço por dúvidas perante bloqueios do sistema, guiando o utilizador para a rápida resolução de falhas processuais ou de conexão.

Tabela 85: Especificação do Requisito Não Funcional 13

Requisito de Utilizador	O sistema deve suportar crescimento de até 50% no volume anual de transações sem degradação superior a 10% nos tempos de resposta definidos.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Hélder Cerqueira
Relevância para a Aplicação	Assegura que o sistema não se tornará obsoleto ou inutilizável a curto prazo com o aumento da clientela, acompanhando pacificamente o crescimento da faturação da rede TrasmUM.

Tabela 86: Especificação do Requisito Não Funcional 14

Requisito de Utilizador	A arquitetura deve permitir o registo de novas lojas no Servidor Central sem necessidade de reconfiguração manual do sistema central, bastando o aprovisionamento local do novo terminal.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Relevância para a Aplicação	Prepara a rede para futuras expansões de forma sustentável, libertando a equipa de TI de constrangimentos técnicos ao abrir um novo estabelecimento.

Tabela 87: Especificação do Requisito Não Funcional 15

Requisito de Utilizador	Parâmetros operacionais locais (limiar de validade para alertas, percentagem de desconto, valor máximo de caixa) devem ser configuráveis pelo manager de cada loja através da interface do terminal, sem necessidade de recompilação do software.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Relevância para a Aplicação	Concede a cada manager a agilidade necessária para adaptar as estratégias comerciais da sua loja de forma autónoma, sem constrangimentos técnicos.

Tabela 88: Especificação do Requisito Não Funcional 16

Requisito de Utilizador	O sistema de cada loja deve manter uma única base de dados local autónoma, partilhada entre todos os terminais do estabelecimento via rede local, que suporte a totalidade das operações diárias sem necessidade de qualquer comunicação com o Servidor Central durante o horário de funcionamento. A comunicação com o Servidor Central ocorre exclusivamente no momento do fecho de dia, para transmissão unidirecional dos dados da jornada.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Reunião — Hélder Cerqueira / Cátia das Unhas Pintadas
Relevância para a Aplicação	Assegura a total imunidade da operação de loja a problemas de conectividade externa, garantindo que o registo de transações flui normalmente independentemente do estado da ligação ao Servidor Central. Reflete a arquitetura federada central do sistema TrasmUM.

Tabela 89: Especificação do Requisito Não Funcional 17

Requisito de Utilizador	A interface do Software de Servidor Central deve ser acessível através de um browser web standard, sem necessidade de instalação de software adicional na máquina do CEO, permitindo o acesso aos dashboards e relatórios a partir de qualquer dispositivo com acesso ao Servidor Central.
Analista	Simão Oliveira

Fonte	Entrevista — Cátia das Unhas Pintadas
Relevância para a Aplicação	Permite à Cátia consultar os dados consolidados da rede a partir de qualquer dispositivo sem dependência de uma instalação de software específica.

Tabela 90: Especificação do Requisito Não Funcional 18

Anexo 7: Especificação Completa dos Casos de Uso

Caso de uso	Iniciar Sessão no Terminal da Loja
Requisitos de Origem	RF1, RF2
Ator	Funcionário / Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Autenticação individual no terminal com bloqueio de sessões simultâneas, desbloqueando as funcionalidades correspondentes ao perfil autenticado e associando todas as operações do turno à identidade do utilizador.
Pré-condição	O terminal da loja está disponível e o utilizador não possui sessão ativa noutro terminal.
Pós Condição	O utilizador tem sessão iniciada e o sistema disponibiliza exclusivamente as funcionalidades correspondentes ao seu perfil, associando o seu identificador único a todas as operações subsequentes da sessão.
Fluxo normal	- O sistema apresenta o ecrã de início de sessão solicitando utilizador e palavra-passe. - O utilizador insere as suas credenciais únicas. - O sistema valida as credenciais. - O sistema determina o perfil do utilizador e disponibiliza as funcionalidades correspondentes.
Fluxo Alternativo 1	[Credenciais inválidas] (passo 3) 3.1. O sistema apresenta mensagem de erro e volta ao passo 1.
Fluxo Alternativo 2	[Limite de 5 tentativas excedido] (passo 3) 3.1. O sistema bloqueia temporariamente a conta e apresenta mensagem descritiva indicando o tempo restante até ao desbloqueio automático. 3.2. O sistema reativa automaticamente a conta findo o período de bloqueio configurado. 3.3. O fluxo normal retoma no passo 1 após o desbloqueio automático.
Fluxo de Exceção	[Sessão já ativa noutro terminal] (passo 3) 3.1. O sistema bloqueia o início de sessão e notifica o utilizador com mensagem descritiva indicando que já existe uma sessão ativa associada às credenciais introduzidas.

Tabela 91: Caso de uso — Iniciar Sessão no Terminal da Loja

Caso de uso	Consultar e Pesquisar Produtos
Requisitos de Origem	RF8, RF13
Ator	Funcionário / Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Pesquisa dinâmica de artigos no catálogo local para consulta de stock, preço e prazo de validade, sem registo de qualquer transação.
Pré-condição	O utilizador tem sessão iniciada e o catálogo local está disponível na base de dados da loja.
Pós Condição	O utilizador visualiza a informação do produto consultado. Nenhuma alteração é efetuada no sistema.
Fluxo normal	- O utilizador acede à barra de pesquisa e introduz o nome ou categoria do produto. - O sistema executa uma query dinâmica sobre o catálogo local e apresenta os resultados com stock, preços e validades dos lotes disponíveis.
Fluxo de Exceção	[Produto não encontrado no catálogo] (passo 2) 2.1. O sistema apresenta mensagem informativa indicando que não foram encontrados resultados para os termos introduzidos.

Tabela 92: Caso de uso — Consultar e Pesquisar Produtos

Caso de uso	Monitorizar Caixa e Registar Sangria
--------------------	--------------------------------------

Requisitos de Origem	RF9
Ator	Funcionário / Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Monitorização automática do valor em numerário na caixa com emissão de alerta ao exceder o limite de segurança configurado, e registo manual da operação de sangria pelo utilizador em funções.
Pré-condição	O utilizador tem sessão iniciada e a caixa foi inicializada com o fundo base de 100€.
Pós Condição	O montante declarado na sangria é subtraído logicamente ao saldo de caixa e o alerta é desativado.
Fluxo normal	1. O utilizador acede à funcionalidade de sangria e declara o valor a retirar. 2. O sistema subtrai o valor declarado ao saldo de caixa e desativa o alerta.
Fluxo de Exceção	[Valor declarado superior ao saldo disponível] (passo 1) 1.1. O sistema rejeita a operação e apresenta mensagem de erro descritiva.

Tabela 93: Caso de uso — Monitorizar Caixa e Registar Sangria

Caso de uso	Criar Pedidos de Remessa
Requisitos de Origem	RF7
Ator	Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Criação e consulta de pedidos de remessa a fornecedores.
Pré-condição	O utilizador tem sessão iniciada com perfil Manager ou CEO.
Pós Condição	O pedido de remessa fica registado na base de dados local com estado pendente, ficando disponível para consulta e para ser associado a uma receção de remessa futura.
Fluxo normal	1. O manager acede à área de gestão de pedidos de remessa e seleciona a opção de criar novo pedido. 2. O manager introduz os dados do pedido (fornecedor, produtos solicitados e respetivas quantidades). 3. O sistema regista o pedido na base de dados local com estado pendente e apresenta confirmação.
Fluxo de Exceção	[Produto solicitado não existe no catálogo local] (passo 2) 2.1. O sistema apresenta mensagem de erro descritiva. O manager deve primeiro criar o produto no catálogo local antes de o incluir no pedido.

Tabela 94: Caso de uso — Criar Pedidos de Remessa

Caso de uso	Registar Receção de Remessa
Requisitos de Origem	RF6
Ator	Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Registo da entrada de mercadoria na loja mediante a introdução da guia de remessa, com criação de lotes independentes em stock e geração local de pagamento pendente associado ao fornecedor.
Pré-condição	O utilizador tem sessão iniciada com perfil Manager ou CEO.
Pós Condição	As novas unidades são adicionadas ao stock local em lotes independentes e é gerado um registo de pagamento pendente associado ao fornecedor.
Fluxo normal	1. O manager acede à funcionalidade de receção de remessa e introduz os dados da guia de remessa (fornecedor, produtos, quantidades, data, valor). 2. O sistema verifica se existe um pedido de remessa pendente para o mesmo fornecedor e, caso exista, atualiza automaticamente o seu estado para “concluído”. 3. O sistema cria registos independentes de stock (novos lotes) para as unidades recebidas, atribuindo-lhes o código de lote e a data de validade da entrega, sem sobrepor as validades dos lotes pré-existent. 4. O sistema gera localmente um registo de pagamento pendente associado ao fornecedor, com base no valor financeiro registado na guia.
Fluxo de Exceção	[Produto da guia não existe no catálogo local] (passo 3) 3.1. O sistema apresenta mensagem de erro descritiva. O manager deve primeiro criar o produto no catálogo local antes de registar a remessa.

Tabela 95: Caso de uso — Registar Receção de Remessa

Caso de uso	Gerir Documentos de Fornecedores
Requisitos de Origem	RF14
Ator	Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Pesquisa e consulta de documentos de fornecedores arquivados automaticamente durante o registo de receção de remessas.
Pré-condição	O utilizador tem sessão iniciada com perfil Manager ou CEO.
Pós Condição	O manager visualiza a informação do documento consultado. Nenhuma alteração é efetuada no sistema.
Fluxo normal	- O manager acede à funcionalidade de gestão documental. - O manager introduz os filtros pretendidos (fornecedor, data, valor) e o sistema apresenta os registos correspondentes.
Fluxo de Exceção	[Ausência de documentos para os filtros introduzidos] (passo 2) O sistema apresenta mensagem informativa indicando que não foram encontrados registos para os critérios selecionados.

Tabela 96: Caso de uso — Gerir Documentos de Fornecedores

Caso de uso	Gerir Pagamentos a Fornecedores
Requisitos de Origem	RF18
Ator	Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Consulta e gestão dos pagamentos associados às remessas recebidas na loja, com possibilidade de registar a liquidação de um pagamento pendente.
Pré-condição	O utilizador tem sessão iniciada com perfil Manager ou CEO e existem registos de remessas na base de dados local.
Pós Condição	O estado do pagamento é atualizado na base de dados local e a alteração é incluída no próximo pacote de fecho de dia.
Fluxo normal	- O manager acede à área de pagamentos a fornecedores. - O sistema apresenta a listagem de remessas com o respetivo estado (pendente ou pago), ordenável por fornecedor, montante e data. - O manager seleciona um pagamento pendente e regista a sua liquidação. - O sistema atualiza o estado do registo para “pago” na base de dados local.
Fluxo de Exceção	[Tentativa de liquidar um pagamento já marcado como pago] (passo 3) O sistema rejeita a operação e apresenta mensagem de erro descritiva.

Tabela 97: Caso de uso — Gerir Pagamentos a Fornecedores

Caso de uso	Aplicar Desconto por Aproximação do Prazo de Validade
Requisitos de Origem	RF22
Ator	Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Consulta dos alertas de lotes em risco de expiração gerados automaticamente pelo terminal local e aplicação manual de desconto exclusivamente sobre os lotes sinalizados, sem afetar os restantes lotes do mesmo produto.
Pré-condição	O utilizador tem sessão iniciada com perfil Manager ou CEO e o terminal executou a rotina diária local de verificação de validade.
Pós Condição	O desconto configurado é aplicado exclusivamente ao lote sinalizado. O preço base dos restantes lotes do mesmo produto permanece inalterado.
Fluxo normal	- O terminal executa diariamente a rotina local de verificação e identifica todos os lotes cuja data de validade é inferior ou igual ao limiar configurado pelo manager. - O sistema apresenta na interface do manager os alertas com os lotes identificados e o botão “Aplicar Desconto” para cada um. - O manager analisa os alertas e aciona o botão “Aplicar Desconto” para os lotes que pretende descontar. - O manager introduz a percentagem de desconto que pretende aplicar ao lote.

	5. O sistema reduz o preço do lote sinalizado de acordo com a percentagem configurada, sem alterar o preço dos restantes lotes do mesmo produto na base de dados local.
Fluxo de Exceção	[Ausência de lotes em risco para a data corrente] (passo 2) 2.1. O sistema apresenta mensagem informativa indicando que não existem lotes a requerer atenção para os limiares configurados.

Tabela 98: Caso de uso — Aplicar Desconto por Aproximação do Prazo de Validade

Caso de uso	Gerir Catálogo de Produtos
Requisitos de Origem	RF17
Ator	Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Gestão do catálogo de produtos da loja (criar, consultar, editar e desativar), com efeito imediato na base de dados local.
Pré-condição	O utilizador tem sessão iniciada com perfil Manager ou CEO.
Pós Condição	O catálogo local reflete as alterações efetuadas com efeito imediato, ficando disponíveis para os funcionários sem necessidade de sincronização externa.
Fluxo normal	1. O manager acede à área de gestão de catálogo e seleciona a operação pretendida (Criar, Consultar, Editar ou Desativar produto). 2. Consoante a operação selecionada, o manager preenche os atributos de um novo produto (nome, preço, categoria), ou seleciona um produto existente para visualizar, alterar os seus dados ou acionar a desativação. 3. O sistema executa a operação solicitada e persiste as alterações na base de dados local.
Fluxo de Exceção	[Tentativa de desativar produto com lotes em stock] (passo 2) 2.1. O sistema apresenta mensagem de aviso descritiva e solicita confirmação explícita antes de prosseguir com a desativação.

Tabela 99: Caso de uso — Gerir Catálogo de Produtos

Caso de uso	Realizar Fecho de Dia
Requisitos de Origem	RF15, RF16
Ator	Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Processo de fecho de dia de qualquer loja da rede, com agregação dos dados da jornada num pacote e envio obrigatório e confirmado para o Servidor Central. A sessão só pode ser encerrada após confirmação de receção pelo Servidor Central.
Pré-condição	O utilizador tem sessão iniciada com perfil Manager ou CEO e existem dados da jornada a enviar desde o último fecho de dia bem-sucedido.
Pós Condição	O Servidor Central confirma a receção dos dados, os registos são marcados como confirmados na base de dados local, a sessão local é encerrada e os dados ficam disponíveis para consulta pelo CEO.
Fluxo normal	1. O manager aciona o processo de fecho de dia. 2. O sistema agrega todos os registos gerados desde o último fecho de dia bem-sucedido — vendas, devoluções, remessas e pagamentos pendentes — num pacote de dados e marca-os como em trânsito. 3. O sistema transmite o pacote de dados ao Servidor Central. 4. O Servidor Central processa os dados recebidos e confirma a persistência bem-sucedida na base de dados central, enviando um ACK ao terminal de origem. 5. O sistema marca os registos como confirmados e encerra a sessão local.
Fluxo Alternativo	[Falha no envio dos dados ao Servidor Central] (passo 3) 3.1. O sistema reverte os registos para estado pendente, notifica o manager com mensagem de erro descritiva e mantém o estado de fecho pendente, impedindo o encerramento da sessão. 3.2. O manager pode acionar manualmente o reenvio. O fluxo retoma no passo 2.

Tabela 100: Caso de uso — Realizar Fecho de Dia

Caso de uso	Gerir Utilizadores do Sistema
Requisitos de Origem	RF23

Ator	Manager / CEO
Área do Sistema	Terminal de Loja
Descrição	Gestão do ciclo de vida das contas de utilizador sob a alçada do perfil autenticado: o Manager gere contas de Funcionário da sua loja; o CEO gere contas de Manager da loja onde está autenticado.
Pré-condição	O utilizador tem sessão iniciada com perfil Manager ou CEO. Se o ator for CEO e pretender criar ou gerir uma conta de Manager, deve estar autenticado no terminal da loja em questão.
Pós Condição	As alterações à conta do utilizador produzem efeito imediato na base de dados local, impedindo novos inícios de sessão com credenciais desativadas.
Fluxo normal	1. O utilizador acede à área de gestão de utilizadores e seleciona a operação pretendida (Criar, Editar, Consultar ou Desativar). 2. Consoante a operação, o utilizador preenche os dados do novo perfil, ou seleciona um utilizador existente e efetua as alterações desejadas ou aciona a desativação. 3. O sistema executa a operação e persiste as modificações na base de dados local com efeito imediato.
Fluxo de Exceção	[Tentativa de desativação do único manager ativo na loja] (passo 3) 3.1. O sistema rejeita a operação e apresenta mensagem de erro descritiva.

Tabela 101: Caso de uso — Gerir Utilizadores do Sistema

Caso de uso	Consultar Remessas da Rede
Requisitos de Origem	RF19
Ator	CEO
Área do Sistema	Servidor Central
Descrição	Consulta agregada do estado de todas as remessas da rede (pendentes e pagas), recebidas via pacotes de fecho de dia, para fins de supervisão e auditoria.
Pré-condição	A Cátia tem sessão iniciada com perfil CEO no Terminal de Servidor Central e existem dados consolidados de pelo menos um fecho de dia recebido.
Pós Condição	A Cátia visualiza o estado das remessas. Nenhuma alteração é efetuada no sistema.
Fluxo normal	1. A Cátia acede à área de remessas da rede. 2. O sistema apresenta a listagem consolidada de todas as remessas de todas as lojas, com estado (pendente ou pago), ordenável por loja, fornecedor, montante e data, com indicação da data de última atualização dos dados.
Fluxo de Exceção	[Ausência de dados consolidados] (passo 2) 2.1. O sistema apresenta mensagem informativa indicando que ainda não foram recebidos pacotes de fecho de dia.

Tabela 102: Caso de uso — Consultar Remessas da Rede

Caso de uso	Consultar Estado de Consolidação da Rede
Requisitos de Origem	RF15
Ator	CEO
Área do Sistema	Terminal de Servidor Central
Descrição	Consulta do estado de receção dos pacotes de fecho de dia de cada loja da rede, permitindo à CEO verificar quais os estabelecimentos cujos dados já foram recebidos e quais se encontram ainda em falta para a data corrente.
Pré-condição	A Cátia tem sessão iniciada com perfil CEO no Terminal de Servidor Central.
Pós Condição	A Cátia visualiza o estado de consolidação da rede. Nenhuma alteração é efetuada no sistema.
Fluxo normal	1. A Cátia acede à área de estado de consolidação da rede. 2. O sistema apresenta a listagem de todas as lojas registadas, com indicação do estado de receção do pacote de fecho de dia para a data corrente — recebido ou em falta — e a data e hora de receção para as lojas já consolidadas.
Fluxo de Exceção	[Ausência de pacotes recebidos para a data corrente] (passo 2) 2.1. O sistema apresenta mensagem informativa indicando que ainda não foram recebidos pacotes de fecho de dia para a data corrente.

Tabela 103: Caso de uso — Consultar Estado de Consolidação da Rede

Caso de uso	Iniciar Sessão no Servidor Central
--------------------	------------------------------------

Requisitos de Origem	RF2
Ator	CEO
Área do Sistema	Terminal de Servidor Central
Descrição	Autenticação da CEO na interface do Servidor Central, desbloqueando exclusivamente as funcionalidades de supervisão estratégica da rede — dashboards, relatórios analíticos, consulta consolidada de remessas e estado de consolidação da rede.
Pré-condição	O Servidor Central está operacional e a CEO não possui sessão ativa.
Pós Condição	A CEO tem sessão iniciada no Servidor Central e o sistema disponibiliza as funcionalidades correspondentes ao perfil CEO neste terminal.
Fluxo normal	1. O sistema apresenta o ecrã de início de sessão solicitando utilizador e palavra-passe. 2. A CEO insere as suas credenciais únicas. 3. O sistema valida as credenciais contra a base de dados central. 4. O sistema confirma o perfil CEO e disponibiliza as funcionalidades de supervisão estratégica da rede.
Fluxo Alternativo	[Credenciais inválidas] (passo 3) 3.1. O sistema apresenta mensagem de erro descritiva e retorna ao passo 1.
Fluxo de Exceção	[Limite de 5 tentativas excedido] (passo 3) 3.1. O sistema bloqueia temporariamente o acesso e apresenta mensagem descritiva indicando o tempo restante até ao desbloqueio automático. 3.2. O sistema reativa automaticamente o acesso findo o período de bloqueio configurado. O fluxo retoma no passo 1.

Tabela 104: Caso de uso — Iniciar Sessão no Servidor Central

Anexo 8: Registos de Decisão Arquitetural

Contexto	O sistema TrasmUM é composto por três lojas geograficamente dispersas e um Servidor Central. Os requisitos impõem autonomia operacional plena de cada loja durante o horário de funcionamento, comunicação unidirecional exclusivamente no momento do fecho de dia, e ausência de dependência de conectividade externa durante o dia.
Decisão	Adotar uma arquitetura federada com cliente-servidor assimétrico. Cada loja é uma unidade computacional completamente autónoma. A comunicação com o Servidor Central é sempre iniciada pela loja, nunca pelo servidor, e ocorre exclusivamente no momento do fecho de dia.
Alternativas Consideradas	Arquitetura centralizada com servidor sempre ativo; arquitetura peer-to-peer entre lojas.
Justificação	A arquitetura centralizada violaria diretamente o RNF20, que exige autonomia operacional plena sem dependência de comunicação externa durante o horário de funcionamento. A arquitetura peer-to-peer introduziria complexidade de sincronização desnecessária e sem benefício para os requisitos identificados.
Consequências	A Cátia trabalha sempre com dados do dia anterior — comportamento explicitamente aceite pelos requisitos e que deve ser comunicado visualmente nos dashboards. Não existe mecanismo de correção retroativa de dados já enviados — documentado como limitação conhecida.

Tabela 105: ADR-01 — Arquitetura de Distribuição Global

Contexto	O sistema TrasmUM requer dois contextos operacionais completamente distintos: a operação diária de cada loja, com interface de caixa, gestão de stock e fecho de dia, e a supervisão estratégica da rede, com receção de pacotes, dashboards e relatórios analíticos. A questão arquitetural é se estes dois contextos devem coexistir numa única aplicação ou ser desenvolvidos como softwares independentes.
Decisão	Desenvolver dois softwares distintos e independentes: o Software de Loja, instalado em todos os terminais da rede, e o Software de Servidor Central, instalado exclusivamente na máquina que serve de servidor central. Cada software tem o seu próprio projeto Java, processo de build, base de dados e ciclo de deployment.
Alternativas Consideradas	Uma única aplicação Java com dois módulos ativados condicionalmente por ficheiro de configuração; parametrização por feature flags sem separação modular explícita.
Justificação	A separação em dois softwares distintos estabelece uma fronteira física entre os dois contextos operacionais, eliminando o risco de acesso indevido a funcionalidades do Servidor Central a partir de um terminal de loja. Os ciclos de vida dos dois softwares são completamente independentes, permitindo atualizações isoladas sem impacto na restante rede. A ausência de código do Servidor Central nos terminais de loja reduz a superfície de ataque e simplifica a auditoria de segurança.

Consequências	A equipa mantém dois projetos Java distintos, com os respetivos processos de build e deployment. Este custo adicional de manutenção é considerado justificado pelas vantagens de separação física, independência operacional e segurança entre os dois softwares.
----------------------	---

Tabela 106: ADR-02 — Dois Softwares Distintos e Independentes

Contexto	Cada loja pode ter múltiplos terminais físicos que precisam de aceder aos mesmos dados operacionais em tempo real. A arquitetura precisa de garantir consistência sem introduzir dependências entre terminais que violem a autonomia operacional.
Decisão	Cada loja dispõe de uma instância MySQL instalada na máquina designada que funciona como servidor de base de dados local da loja. Todos os terminais do estabelecimento acedem a esta instância via rede local através do protocolo TCP/IP nativo do MySQL.
Alternativas Consideradas	Base de dados independente em cada terminal com sincronização interna; SQLite com acesso via sistema de ficheiros de rede.
Justificação	A arquitetura cliente-servidor nativa do MySQL suporta de forma robusta o acesso concorrente de múltiplos terminais, eliminando os riscos de corrupção de dados associados ao acesso em rede a ficheiros de base de dados como o SQLite. Uma base de dados independente por terminal exigiria um mecanismo de sincronização interna entre terminais da mesma loja, introduzindo complexidade não prevista nos requisitos.
Consequências	Se a máquina que aloja a instância MySQL local estiver indisponível, todos os terminais da loja ficam sem acesso aos dados. Esta dependência é uma limitação conhecida e deve ser documentada no guia de operação. A instalação do MySQL na máquina designada de cada loja requer configuração inicial por parte da equipa técnica. Em instalações de pequena dimensão, é admissível que a máquina designada para alojar a instância MySQL seja um dos terminais operacionais da loja. Neste caso, esse terminal acumula o papel de servidor de base de dados local, o que pode introduzir degradação de desempenho sob carga elevada. Esta limitação é considerada aceitável para o volume expectável de uma loja de conveniência. Em instalações de maior dimensão, recomenda-se uma máquina dedicada exclusivamente ao papel de servidor de base de dados.

Tabela 107: ADR-03 — Base de Dados Partilhada por Loja

Contexto	O pacote de fecho de dia deve conter apenas os registos gerados desde o último fecho bem-sucedido. O sistema precisa de saber exatamente quais registos ainda não foram enviados, distinguindo entre registos nunca enviados, registos em trânsito e registos confirmados pelo Servidor Central.
Decisão	Todas as tabelas operacionais do Software de Loja incluem um campo sincronizado com três estados possíveis: pendente (registo criado, nunca enviado), em_transito (pacote construído e enviado, aguarda ACK), e confirmado (ACK recebido com sucesso). A marcação como confirmado ocorre exclusivamente após receção do ACK. Em caso de reinício da aplicação com registos em estado em_transito, estes são tratados como pendente e incluídos no próximo pacote.
Alternativas Consideradas	Campo binário 0/1; registo de último timestamp de sincronização por tabela.
Justificação	O campo binário não representa adequadamente o estado intermédio de um envio em curso, criando ambiguidade em caso de falha durante o envio. O mecanismo de idempotência por chave composta no Servidor Central garante que reenvios de registos já confirmados são ignorados sem efeitos secundários.
Consequências	Requer que o Servidor Central implemente idempotência por chave composta loja_id + id_registo em todas as tabelas centrais, conforme definido no ADR-05. O mecanismo de reversão de em_transito para pendente cobre igualmente o cenário de falha após receção do ACK mas antes da marcação como confirmado. Neste caso, os registos são reenviados no fecho seguinte sem efeitos secundários, dado que o Servidor Central garante idempotência por chave composta conforme definido no ADR-05.

Tabela 108: ADR-04 — Delta Sync com Três Estados de Sincronização

Contexto	O mecanismo de Delta Sync pode resultar no reenvio de registos já processados pelo Servidor Central, seja por falha de comunicação, seja por acumulação de dados de dias anteriores num único pacote. O Servidor Central precisa de garantir que registos duplicados não são inseridos.
Decisão	Todas as tabelas centrais utilizam uma chave primária composta por loja_id e pelo identificador original do registo na loja de origem. A inserção de registos duplicados é tratada com INSERT ... ON DUPLICATE KEY UPDATE, garantindo deduplicação automática independentemente do contexto do pacote.
Alternativas Consideradas	Idempotência ao nível do pacote por lojaId + dataFecho; verificação prévia de existência antes de cada inserção.
Justificação	A idempotência ao nível do pacote é insuficiente quando um pacote contém dados de múltiplos dias, situação possível quando fechos anteriores falharam. A chave composta por registo individual é a solução mais robusta e mais simples de implementar corretamente.

Consequências	O esquema da base de dados do Servidor Central difere do esquema das bases de dados locais das lojas — as tabelas centrais têm chaves primárias compostas enquanto as locais usam AUTOINCREMENT. Esta diferença deve ser tida em conta na fase de implementação. O processamento de cada pacote de fecho no Servidor Central é executado numa única transação de base de dados — ou todos os registos do pacote são persistidos, ou nenhum. Em caso de falha a meio do processamento, o rollback automático garante que o estado da base de dados central permanece consistente. O reenvio do pacote no fecho seguinte insere os registos em falta sem duplicação, garantido pela chave composta.
----------------------	---

Tabela 109: ADR-05 — Idempotência por Chave Composta nas Tabelas Centrais

Contexto	Numa instalação inicial do Software de Loja, a base de dados local não contém nenhuma conta de utilizador. De forma análoga, numa instalação inicial do Software de Servidor Central, a base de dados central não contém nenhuma conta CEO. É necessário um mecanismo que permita a criação inicial das contas sem depender de um utilizador já autenticado.
Decisão	O script de instalação de cada software cria automaticamente uma conta de instalação temporária com credenciais predefinidas: no Software de Loja, uma conta com perfil CEO na base de dados local do terminal instalado; no Software de Servidor Central, uma conta com perfil CEO na base de dados central. Estas são contas em bases de dados distintas e independentes entre si.
Alternativas Consideradas	Script SQL de seed manual executado pelo técnico de instalação; criação de conta via linha de comandos.
Justificação	A conta temporária integra o processo de instalação sem exigir intervenção técnica adicional após o deployment, reduzindo o risco de erros de configuração inicial em ambos os softwares.
Consequências	Após a criação da primeira conta definitiva com perfil CEO, a conta de instalação temporária é automaticamente eliminada da base de dados, não sendo possível reutilizá-la. Este comportamento deve ser implementado como parte do fluxo de primeiro login em ambos os softwares.

Tabela 110: ADR-06 — Bootstrapping por Conta de Instalação Temporária

Contexto	O RNF12 exige o registo de todas as ações críticas em logs de auditoria. A questão arquitetural é se estes logs ficam exclusivamente nos terminais locais ou se são transmitidos ao Servidor Central para visibilidade da Cátia.
Decisão	Os logs de auditoria são armazenados localmente em cada terminal e incluídos no pacote de fecho de dia, ficando disponíveis na base de dados do Servidor Central para consulta pela Cátia. Os logs cobrem vendas, devoluções, alterações de preços, aplicação de descontos, gestão de utilizadores e alterações ao catálogo local.
Alternativas Consideradas	Logs exclusivamente locais; logs transmitidos em tempo real via API separada.
Justificação	A transmissão em tempo real violaria o princípio de comunicação exclusiva no fecho de dia. Os logs exclusivamente locais privariam a Cátia de visibilidade sobre ações críticas nas lojas, limitando a capacidade de auditoria central.
Consequências	O pacote de fecho de dia aumenta em volume proporcional ao número de ações críticas registadas durante o dia. A título indicativo, assumindo uma loja com 200 transações diárias e uma média de 3 ações auditáveis por transação, o volume adicional de logs por pacote é da ordem dos 600 registos. A um tamanho médio de 200 bytes por registo, o acréscimo é inferior a 120 KB por pacote — negligenciável face ao RNF4 (5 minutos por loja).

Tabela 111: ADR-07 — Logs de Auditoria Incluídos no Pacote de Fecho

Contexto	A comunicação entre os terminais de loja e o Servidor Central transporta dados financeiros, de auditoria e de stock via HTTP. A adoção de cifra garante a confidencialidade destes dados mesmo em redes locais não completamente controladas.
Decisão	Adotar HTTPS com certificado auto-assinado para a comunicação do fecho de dia entre cada loja e o Servidor Central.
Alternativas Consideradas	HTTP simples; HTTPS com autoridade certificadora interna.
Justificação	A implementação de HTTPS com certificado auto-assinado recorre exclusivamente a ferramentas nativas do Java e do Javalin, sem bibliotecas adicionais, tornando o custo técnico negligenciável. A alternativa de HTTP simples foi rejeitada por expor dados sensíveis a interceptação sem custo justificável.
Consequências	A gestão do certificado auto-assinado — geração, distribuição e renovação — constitui uma responsabilidade operacional adicional em cada instalação. O certificado deve ser renovado antes da sua expiração para evitar falhas de comunicação no fecho de dia. O processo de instalação do Software de Loja inclui a importação do certificado do Servidor Central para o truststore local da JVM de cada terminal, conforme o procedimento de bootstrapping descrito no ADR-06. Sem este passo, o terminal rejeita a ligação HTTPS ao Servidor Central.

Tabela 112: ADR-08 — Comunicação Segura com HTTPS

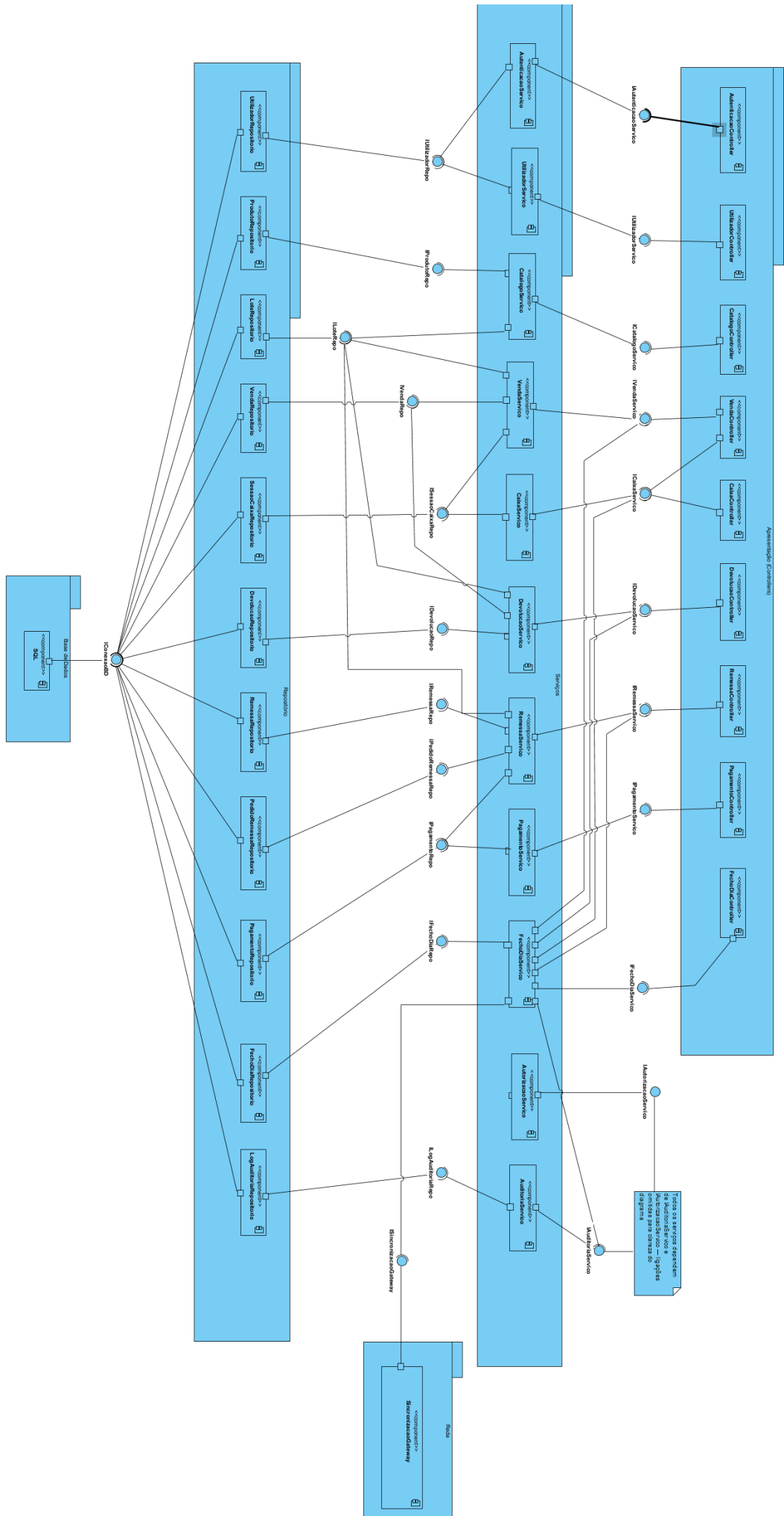
Contexto	Ambos os softwares requerem uma organização interna que estabeleça fronteiras claras de responsabilidade, facilite a divisão de trabalho entre os membros da equipa, e permita a testabilidade isolada da lógica de negócio. A questão arquitetural é qual o padrão de organização interna mais adequado ao contexto tecnológico e ao prazo do projeto.
Decisão	Adotar a Layered Architecture com MVC em ambos os softwares, organizando cada aplicação em quatro camadas horizontais com dependências estritamente unidirecionais: Apresentação, Serviço, Repositório e Domínio. A regra de dependência é verificável: um Controller depende exclusivamente de interfaces de serviço, nunca de implementações concretas nem de repositórios; um Repository nunca importa nada da camada de Apresentação.
Alternativas Consideradas	Hexagonal Architecture; arquitetura sem camadas explícitas com separação apenas por módulo funcional.
Justificação	A Layered Architecture é familiar à equipa e bem documentada para Java com JavaFX, reduzindo a curva de aprendizagem. A Hexagonal Architecture, embora tecnicamente mais elegante, foi considerada de risco elevado para o prazo disponível pela complexidade adicional dos adaptadores. A separação por módulo funcional sem camadas explícitas não oferece garantias verificáveis de separação de responsabilidades.
Consequências	A regra de dependência unidirecional obriga a que todas as fronteiras entre camadas sejam definidas por interfaces, com injeção de dependências explícita em cada ponto de entrada. Um Controller nunca instancia diretamente uma implementação de repositório — depende exclusivamente de interfaces de serviço. Esta restrição é verificável durante a revisão de código e deve ser aplicada consistentemente em ambos os softwares.

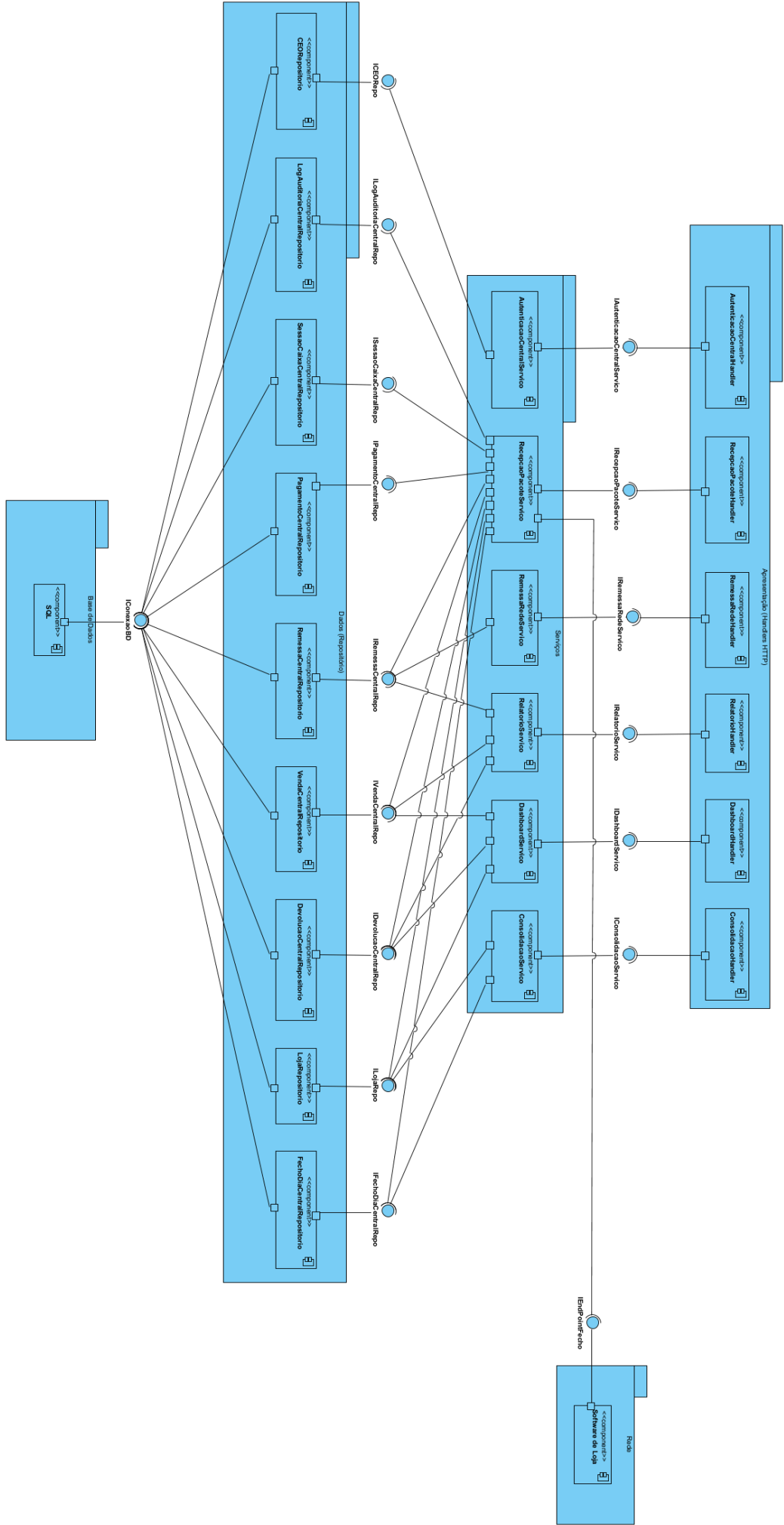
Tabela 113: ADR-09 — Layered Architecture com MVC como Padrão de Organização Interna

Contexto	O desenvolvimento de dois softwares distintos requer a escolha de uma linguagem de programação, frameworks de interface e comunicação HTTP, e um sistema de gestão de base de dados. As escolhas devem ser coerentes entre os dois softwares, adequadas ao padrão arquitetural definido no ADR-09, e compatíveis com o prazo e as competências da equipa.
Decisão	Adotar Java como linguagem de desenvolvimento em ambos os softwares. No Software de Loja, utilizar JavaFX como framework de interface gráfica, com acesso à base de dados via JDBC. No Software de Servidor Central, utilizar Javalin como servidor HTTP leve. Em ambos os softwares, utilizar MySQL como sistema de gestão de base de dados, com acesso via JDBC.
Alternativas Consideradas	Spring Boot em substituição do Javalin; SQLite em substituição do MySQL; Swing em substituição do JavaFX.
Justificação	Java é a linguagem de maior familiaridade da equipa e integra-se naturalmente com o padrão MVC definido no ADR-09. O JavaFX oferece suporte nativo ao padrão MVC e comportamentos reativos via propriedades observáveis, sem bibliotecas adicionais. O Javalin foi preferido ao Spring Boot pela sua leveza e ausência de IoC container, o que reduz o overhead de arranque e elimina complexidade de configuração desnecessária para um servidor de receção passiva sem lógica de negócio complexa. O MySQL foi preferido ao SQLite pela sua arquitetura cliente-servidor nativa, que suporta acesso concorrente de múltiplos terminais conforme definido no ADR-03.
Consequências	A equipa utiliza um único ecossistema tecnológico em ambos os softwares, simplificando a partilha de código utilitário e a revisão cruzada entre membros. A dependência do JavaFX implica que o Software de Loja requer uma JVM com módulos JavaFX instalados em cada terminal.

Tabela 114: ADR-10 — Stack Tecnológica: Java, JavaFX, Javalin e MySQL

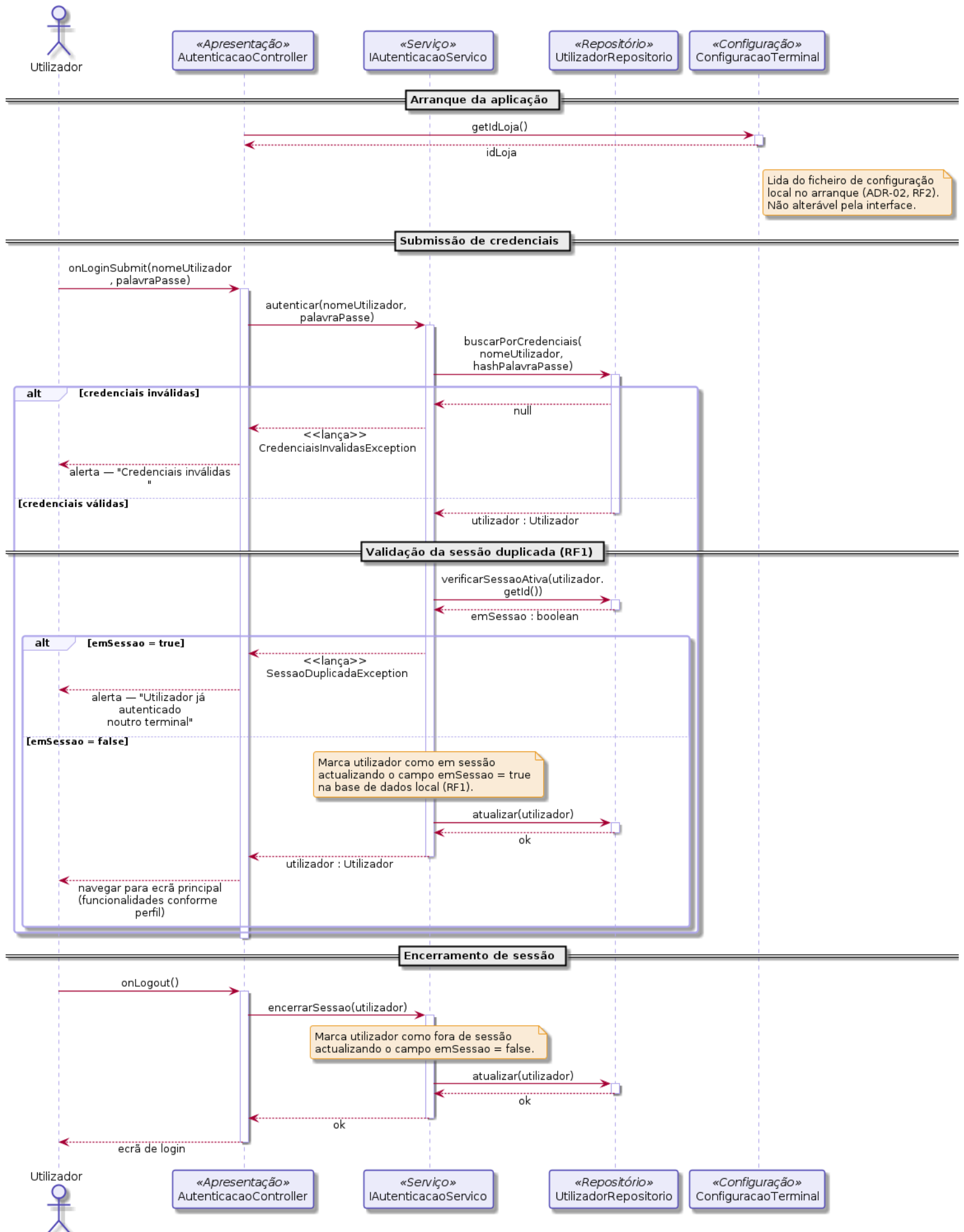
Anexo 9: Diagrama de componentes — Software de Loja





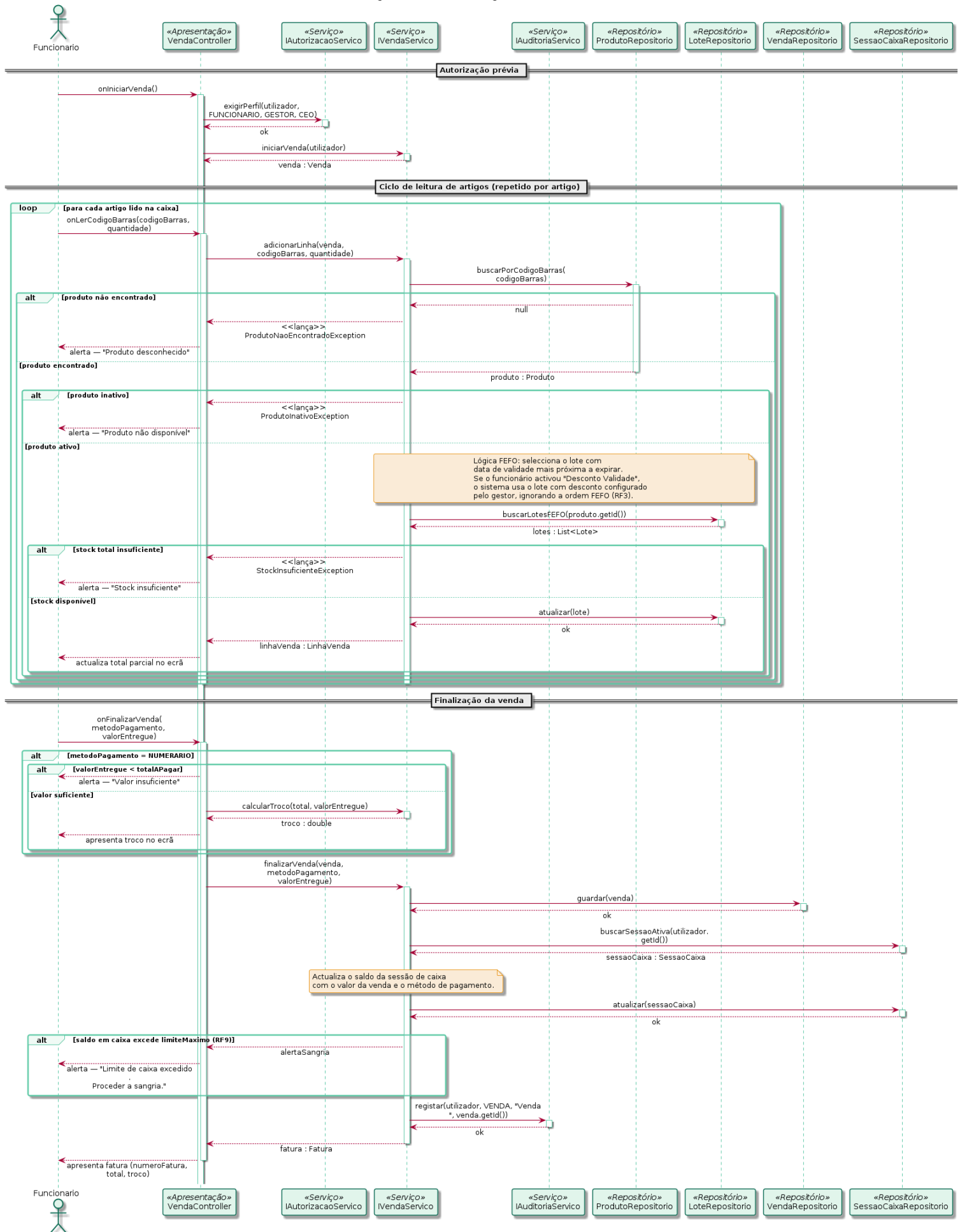
Anexo 11: Diagrama de Sequência — Autenticação Loja

DS1 — Autenticação e Controlo de Sessão (RF1, RF2)



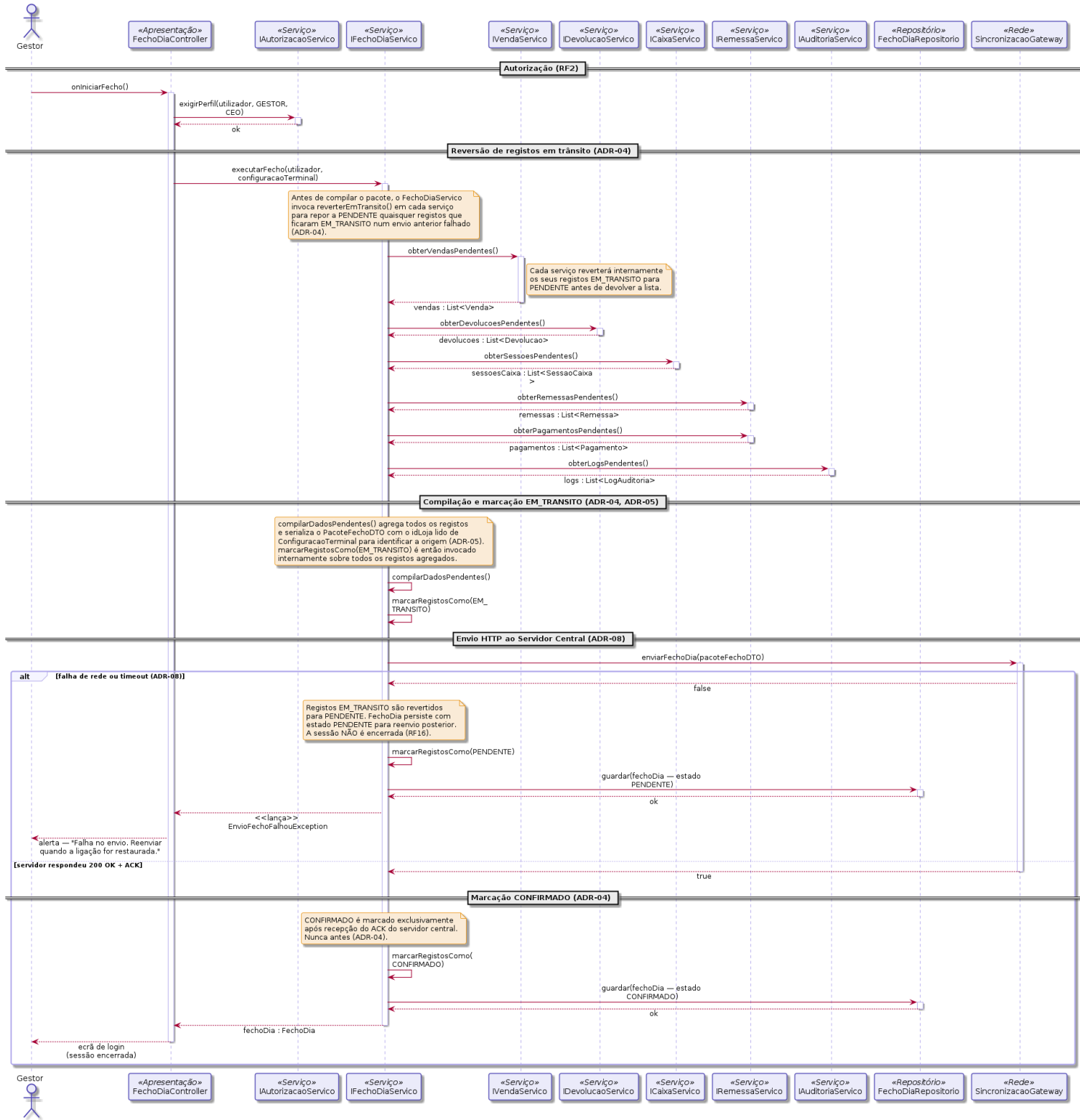
Anexo 12: Diagrama de Sequência — Registo de Venda

DS2 — Registo de Venda com Lógica FEFO (RF3, RF11, RF12)

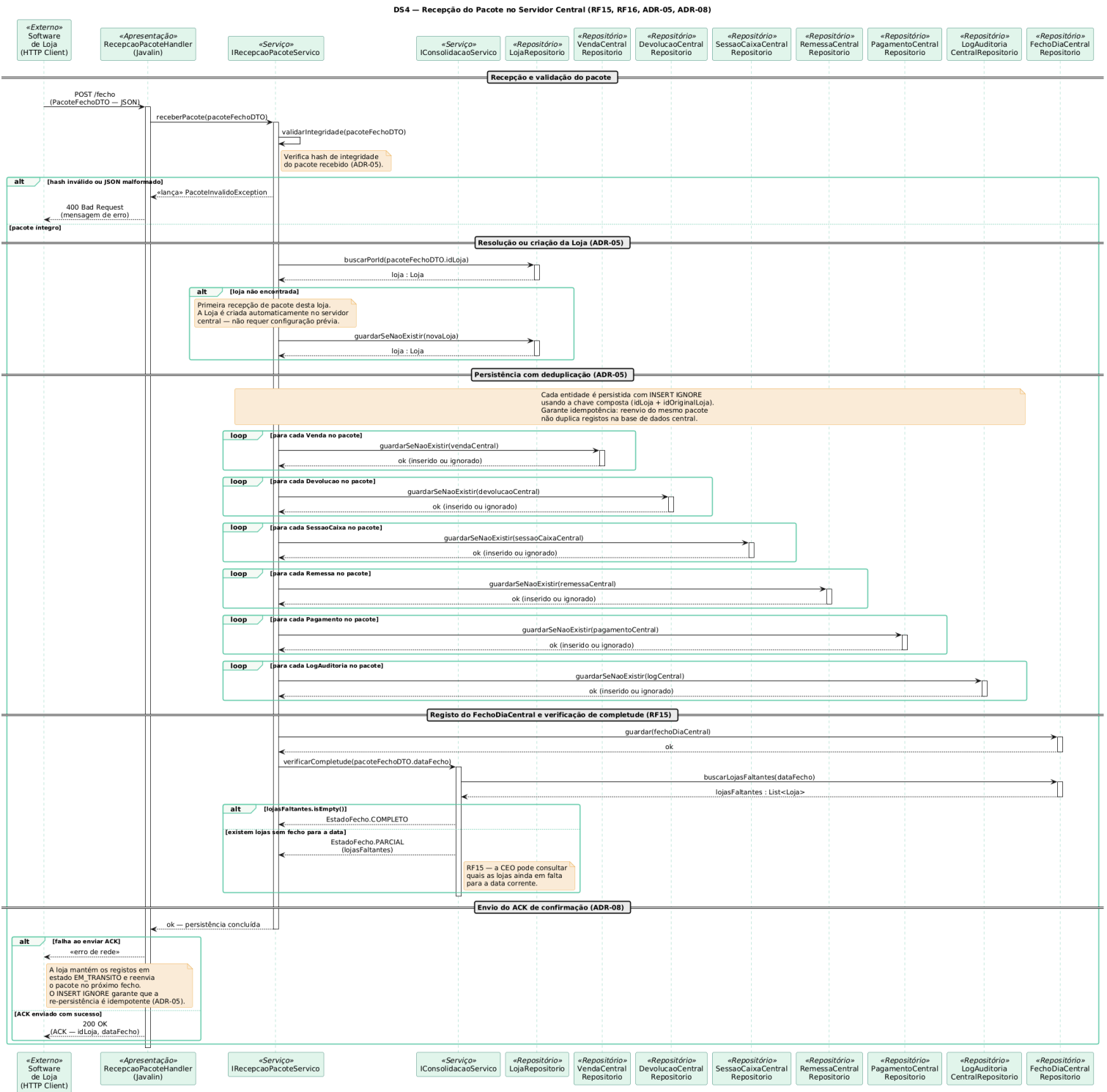


Anexo 13: Diagrama de Sequência — Fecho de dia e Sincronização

DS3 — Fecho de Dia e Sincronização Delta Sync (RF16, ADR-04, ADR-05, ADR-08)

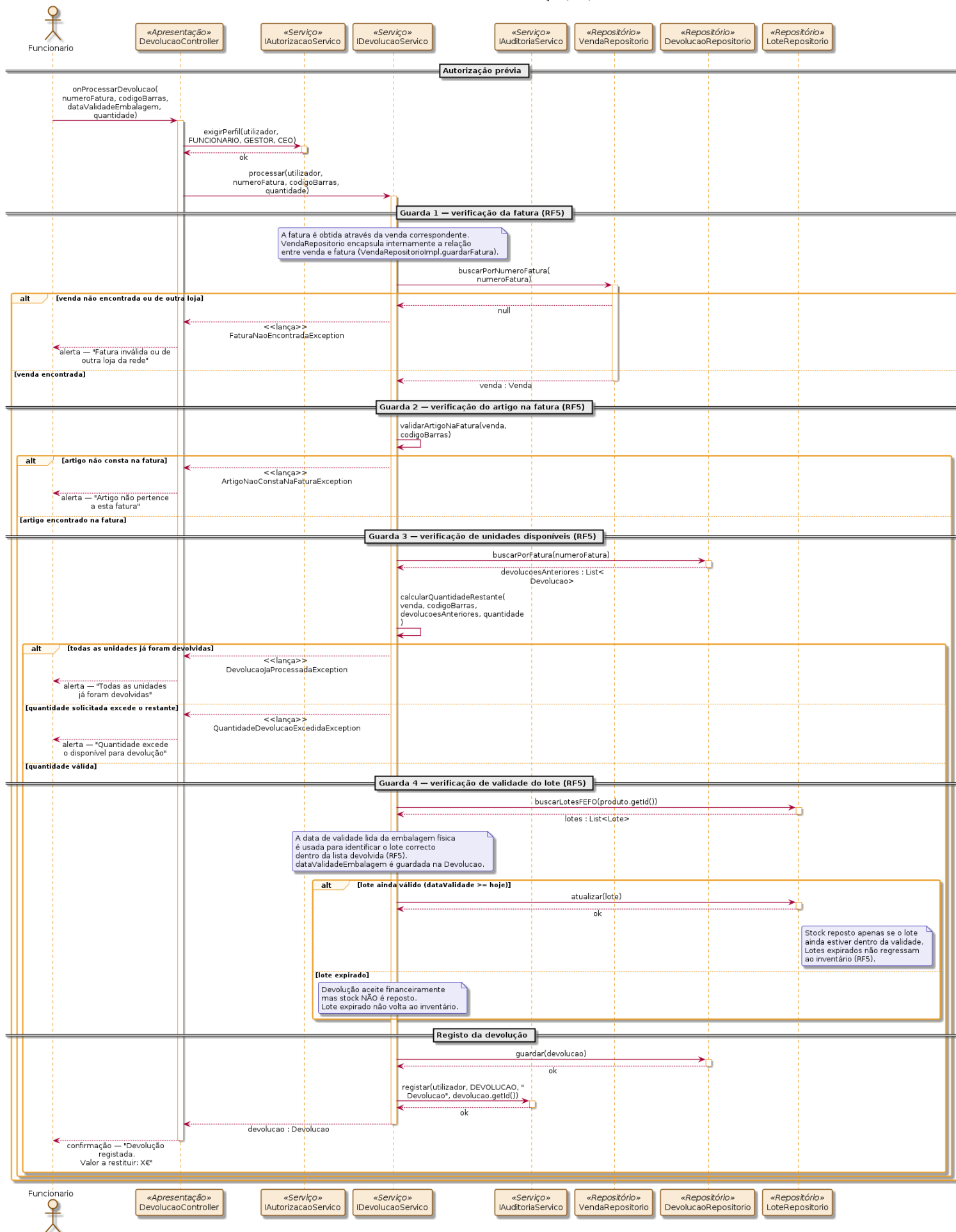


Anexo 14: Diagrama de Sequência — Receção de Pacotes no Servidor Central



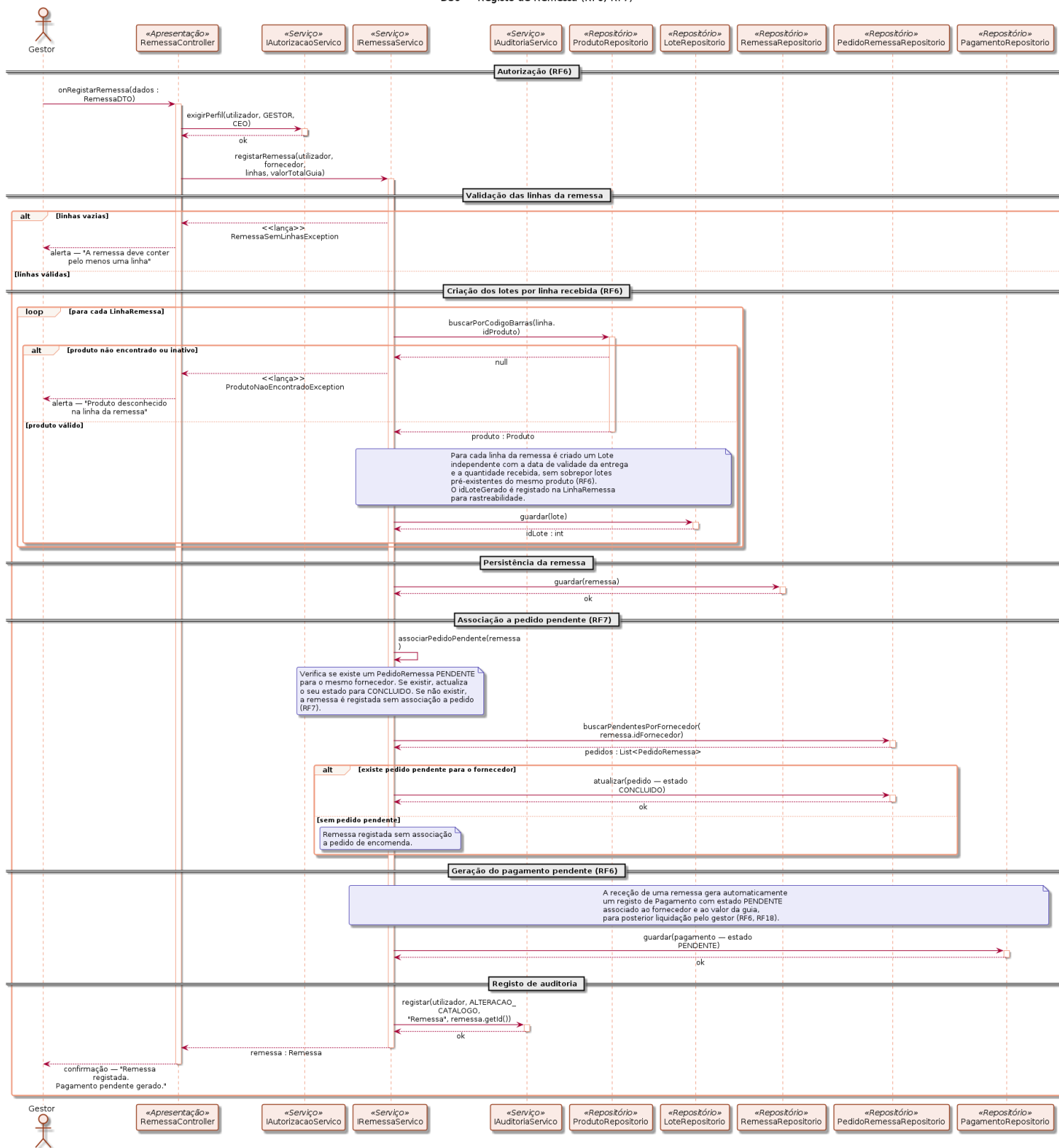
Anexo 15: Diagrama de Sequência — Processamento da Devolução

DS5 — Processamento de Devolução (RF5)

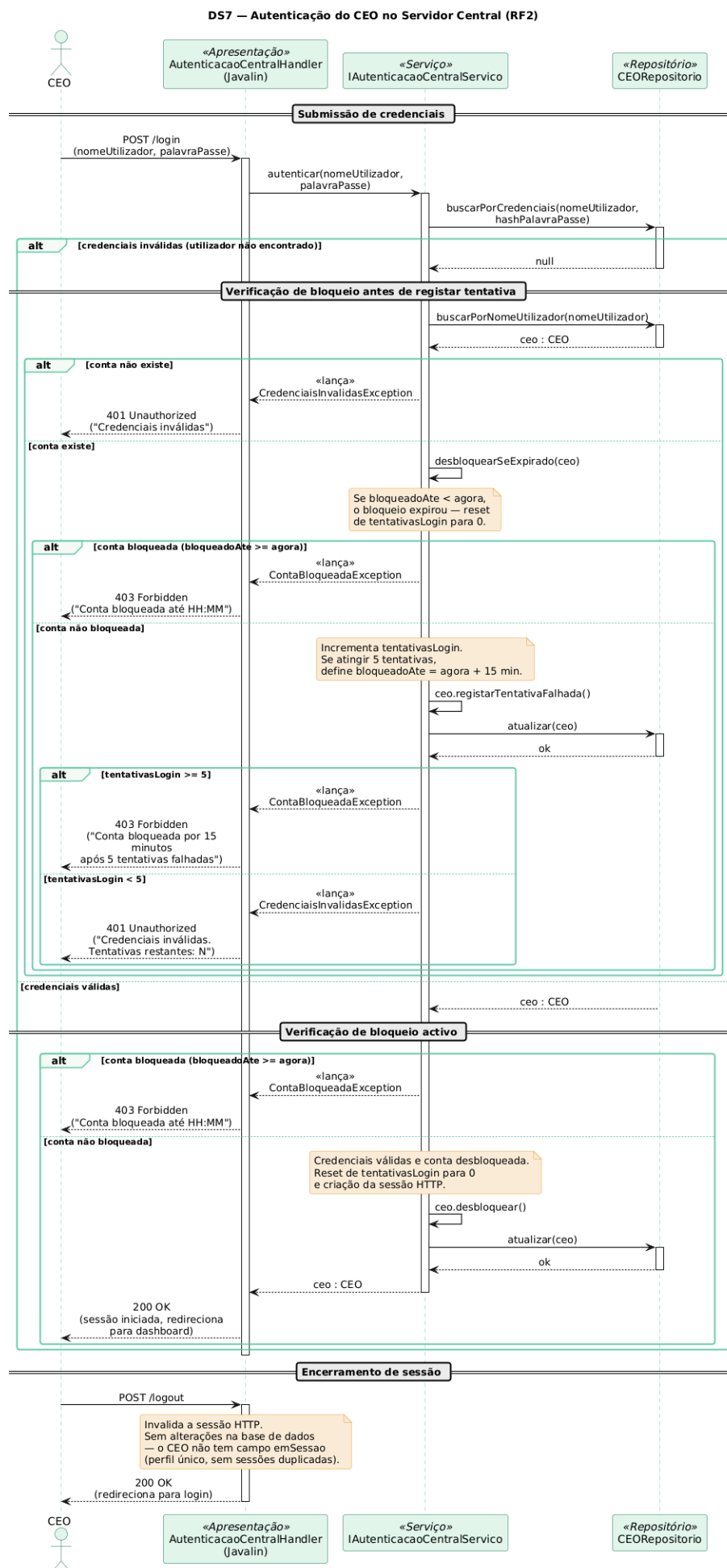


Anexo 16: Diagrama de Sequência — Registo de Remessa.

DS6 — Registo de Remessa (RF6, RF7)



Anexo 17: Diagrama de Sequência — Autenticação Servidor Central



Anexo 18: Exemplo de payloads e respostas da API

Cenário	Payload / Resposta
Pedido POST /login	<pre>{ "nomeUtilizador": "catia.trasmum", "palavraPasse": "hashed_password" }</pre>
200 OK Autenticação bem-sucedida	<pre>{ "mensagem": "Sessão iniciada com sucesso.", "perfil": "CEO" }</pre>
403 Forbidden Conta bloqueada	<pre>{ "erro": "Conta bloqueada.", "bloqueadaAte": "2026-04-30T11:15:00" }</pre>

Tabela 115: Exemplos de pedido e resposta — Autenticação

Cenário	Payload / Resposta
Pedido POST /fecho (abreviado)	<pre>{ "idLoja": "loja-braga-01", "nomeLoja": "TrasmUM Braga", "dataFecho": "2026-04-30", "vendas": [{ "idOriginalLoja": 1042, "totalFaturado": 12.75, "..." }], "devolucoes": [{ "idOriginalLoja": 201, "..." }], "sessoesCaixa": [{ "idOriginalLoja": 301, "..." }], "remessas": [{ "idOriginalLoja": 401, "..." }], "pagamentos": [{ "idOriginalLoja": 501, "..." }], "logs": [{ "idOriginalLoja": 601, "..." }] }</pre>
200 OK Receção bem-sucedida	<pre>{ "idLoja": "loja-braga-01", "dataFecho": "2026-04-30", "estadoConsolidacao": "PARCIAL", "lojasFaltantes": ["loja-guimaraes-01"] }</pre>
400 Bad Request Pacote inválido	<pre>{ "erro": "Integridade do pacote comprometida. Hash inválido." }</pre>

Tabela 116: Exemplos de pedido e resposta — Sincronização

Cenário	Resposta
200 OK Métricas globais	<pre>{ "dataReferencia": "2026-04-30", "totalVendasRede": 15420.80, "totalTransacoes": 1284, "totalDevolucoes": 310.50, "totalRemessasPendentes": 2, "vendasPorLoja": { "loja-braga-01": 6200.30, "loja-guimaraes-01": 4890.10, "loja-viana-01": 4330.40 } }</pre>

Tabela 117: Exemplos de resposta — Dashboard e Métricas

Cenário	Pedido / Resposta
Pedido com filtros	<pre>GET /relatorios?idLojas=loja-braga-01,loja-viana-01 &dataInicio=2026-04-01 &dataFim=2026-04-30 &categorias=Bebidas,Snacks</pre>
200 OK	<pre>{ "titulo": "Relatório de Vendas – Abril 2026", </pre>

Cenário	Pedido / Resposta
	<pre> "totalVendas": 8430.20, "totalDevolucoes": 124.00, "totalTransacoes": 701 } </pre>

Tabela 118: Exemplos de pedido e resposta — Relatórios Analíticos

Cenário	Pedido / Resposta
Pedido com filtros	<pre> GET /remessas?estadoPagamento=PENDENTE &dataInicio=2026-04-01 &dataFim=2026-04-30 </pre>
200 OK	<pre> { "remessas": [{ "idLoja": "loja-braga-01", "nomeFornecedor": "Distribuições Minho Lda.", "valorTotalGuia": 340.00, "estadoPagamento": "PENDENTE" }], "total": 1 } </pre>

Tabela 119: Exemplos de pedido e resposta — Remessas da Rede

Cenário	Resposta
200 OK Consolidação parcial	<pre> { "dataFecho": "2026-04-30", "estadoConsolidacao": "PARCIAL", "lojasFaltantes": ["loja-guimaraes-01"], "lojasRecebidas": ["loja-braga-01", "loja-viana-01"] } </pre>

Tabela 120: Exemplos de resposta — Estado de Consolidação

Anexo 19: Lista completa de desvios — Software de Loja

Área	O que foi alterado	Porquê	Impacto
Repositório	Adicionada operação <code>atualizarSincronizacao(int id, EstadoSincronizacao estado)</code> às interfaces <code>VendaRepositorio</code> , <code>DevolucaoRepositorio</code> e <code>RemessaRepositorio</code> .	O <code>FechoDiaServico</code> precisa de marcar registos como <code>EM_TRANSITO</code> antes de enviar e como <code>CONFIRMADO</code> após resposta do servidor; a operação não estava prevista explicitamente no design.	Diagrama de classes — Repositório
Repositório	Adicionada operação <code>guardarSangria(Sangria sangria)</code> a <code>SessaoCaixaRepositorio</code> , que insere na tabela <code>Sangria</code> e persiste os <code>DetalheNumerario</code> associados.	As sangrias são parte da sessão de caixa mas têm ciclo de vida próprio; o design não tinha operação dedicada para as persistir.	Diagrama de classes — Repositório
Serviço	Adicionada operação <code>buscarSessaoAtiva(int idUtilizador)</code> a <code>ICaixaServico</code> .	O <code>VendaController</code> precisa de consultar a sessão activa do utilizador para verificar o limite de caixa; o <code>CaixaController</code> também a usa para refrescar o estado da UI.	Diagrama de classes — Serviço
Repositório / UI	Adicionada operação <code>Map<Integer, Integer> stockPorProduto()</code> a <code>LoteRepositorio</code> . A coluna “Stock” do catálogo passou a mostrar o somatório das quantidades dos lotes em vez do <code>stockMinimo</code> .	A UI do catálogo precisava de mostrar o stock actual; o domínio não expunha essa agregação e o valor anteriormente apresentado estava semanticamente errado.	Diagrama de classes — Repositório

Repositório / Arquitetura	Adicionada operação <code>limparSessoesAtivas()</code> a <code>UtilizadorRepositorio</code> , invocada pelo <code>AppContext</code> no arranque para colocar todos os utilizadores fora de sessão.	Encerramentos abruptos da aplicação deixavam o flag <code>emSessao</code> a 1, bloqueando novos logins. A recuperação é análoga à reversão de <code>EM_TRANSITO</code> → <code>PENDENTE</code> para registos sincronizáveis.	Diagrama de classes — Repositório
Apresentação	Adicionado botão “Novo Pedido” no separador “Pedidos de Remessa” e respetivo diálogo em <code>RemessaController.onNovoPedido()</code> .	O caso de uso “Gestor cria pedido de remessa a fornecedor” existia no design mas não tinha ponto de entrada na UI — o serviço <code>criarPedidoRemessa</code> estava implementado mas inacessível.	Diagrama de sequência — Criar Pedido de Remessa
Apresentação	Adicionados métodos <code>onVerDetalhesProduto()</code> e <code>mostrarDetalhesLotes(Produto, List<Lote>)</code> ao <code>CatalogoController</code> para exibir um diálogo com os lotes FEFO do produto selecionado.	A vista do catálogo precisava de um ponto de entrada para inspecionar os lotes de um produto sem sair do ecrã principal.	Diagrama de classes — Apresentação
Domínio / Serviço / Repositório / UI	Introduzida a relação <code>FornecedorProduto</code> (N:N) com atributo <code>precoFornecedor</code> , nova exceção <code>ProdutoNaoFornecidoException</code> , novo <code>FornecedorProdutoRepositorio</code> , novo <code>IFornecedorServico</code> , validação adicional em <code>RemessaServico.registarRemessa</code> e <code>criarPedidoRemessa</code> , nova vista <code>FornecedorView</code> e correspondente item de menu, filtro de produtos por fornecedor no fluxo de pedido, e label “Sugerido” no diálogo de chegada.	O modelo original permitia que qualquer fornecedor fornecesse qualquer produto, o que não reflete a realidade de uma loja e permitia gerar pedidos inválidos. A validação explícita evita erros humanos e dá base para automatizar o cálculo do <code>valorTotalGuia</code> .	Diagrama de classes — Domínio, Repositório, Serviço; Schema da BD
Apresentação / Arquitetura	Removida a vista autónoma <code>DevolucaoView.fxml/DevolucaoController</code> . A lógica de devolução foi integrada em <code>VendaController</code> como segundo separador num <code>TabPane</code> . O botão “Devolução” do menu lateral foi eliminado.	O fluxo de devolução está intimamente ligado ao contexto de venda (pesquisa por número de fatura, devolução de artigos de uma venda anterior). Ter uma página separada fragmentava desnecessariamente a navegação.	Diagrama de componentes; Diagrama de classes — Apresentação
Serviço / Apresentação	Adicionada operação “Abater lote” acessível a partir da tabela de lotes no painel de detalhes do catálogo. Restrita a GESTOR/CEO, registada em auditoria; alertas de validade são recarregados após o abate.	O sistema não tinha forma de registar a retirada física de lotes danificados ou expirados — o stock ficava inflacionado mesmo quando os produtos eram removidos da prateleira.	Diagrama de classes — Serviço; Diagrama de sequência — Abater Lote
Serviço	O <code>FechoDiaServico.executarFecho()</code> passou a registar entrada de auditoria com <code>TipoAcao.FECHO_DIA</code> após fecho bem-sucedido, completando a cobertura do registo de auditoria a todas as ações críticas previstas no enum <code>TipoAcao</code> .	O requisito exige que todas as ações críticas sejam registadas em logs de auditoria; o código já registava a maioria, faltava apenas o fecho de dia.	Diagrama de sequência — DS3
Repositório / Serviço / UI	Adicionados <code>buscarTodos()</code> e <code>buscarPorTipo(TipoAcao)</code> a <code>LogAuditoriaRepositorio</code> e métodos correspondentes em <code>IAuditoriaServico</code> . Nova vista <code>LogAuditoriaView</code> com <code>TableView</code> e filtro por tipo de ação; novo <code>LogAuditoriaController</code> e item de menu “Auditoria” visível a GESTOR/CEO.	O sistema precisava de uma interface para visualização dos logs de auditoria, permitindo aos gestores verificar o histórico de operações.	Diagrama de classes — Repositório, Serviço, Apresentação
Apresentação	Adicionado <code>ComboBox</code> de filtro por categoria nas vistas de Venda e Catálogo. Os controladores extraem categorias distintas dos produtos activos e populam dinamicamente o filtro.	Melhoria de usabilidade — permite encontrar produtos mais rapidamente em catálogos com muitas entradas.	Sem impacto em diagramas
Apresentação / Serviço	Os DTOs em <code>PacoteFechoDT0</code> foram reescritos para o contrato denormalizado esperado pelo Servidor Central: <code>VendaDT0</code> , <code>LinhaVendaDT0</code> ,	O servidor central usa um modelo de <i>analytics</i> denormalizado e não mantém tabelas de catálogo. Receber IDs locais	Contrato API; Diagrama de sequência — DS3

	RemessaDT0, DevolucaoDT0, PagamentoDT0, SessaoCaixaDT0 e LogAuditoriaDT0 foram simplificados para transportar idOriginalLoja e dados denormalizados (nomes de produto, categoria, fornecedor, utilizador). dataFecho passou a String ISO. O FechoDiaServico ganhou quatro novas dependências de repositório usadas exclusivamente para denormalização no envio.	seria inútil — precisa dos nomes resolvidos no momento do envio.	
Domínio / Repositório / Schema	Introduzido o atributo booleano temValidade em Produto (default true); Lote.dataValidade e LinhaRemessa.dataValidade passaram a NULL-aceitáveis. Diálogos de catálogo e remessa adaptados. FEFO ordenado com lotes sem validade no fim. Repositórios escrevem/lêem o campo de forma null-safe.	Nem todos os artigos da loja têm prazo de validade (pilhas, produtos não-pecíveis). Forçar uma data artificial corrompia os lotes e gerava alertas falsos.	Diagrama de classes — Domínio; Schema da BD
Apresentação	O menu lateral do gestor passou de 9 para 7 itens através de duas vistas agregadoras (AprovisionamentoView com Remessas + Fornecedores; AdministracaoView com Utilizadores + Auditoria). O registo de chegada de remessa deixou de ser modal — RemessaView ganhou um terceiro painel interno alternável.	O menu tornara-se denso com nove opções, muitas representando funcionalidades relacionadas. A reorganização reduz a carga cognitiva sem alterar funcionalidade.	Diagrama de componentes
Domínio / Repositório / Serviço / UI	Adicionado campo descricao VARCHAR(255) a LogAuditoria e à respetiva tabela. IAuditoriaServico.registar() aceita descricao como parâmetro adicional; todos os <i>call sites</i> foram atualizados para passar mensagens contextuais (ex: “Alterou preço de ‘Leite UHT’ de 1,20 € para 1,35 €”). O ecrã de logs apresenta a descrição em vez do nome do <i>enum</i> . O campo é incluído no PacoteFechoDT0.	Os logs registavam apenas o tipo de ação sem indicar o que concretamente foi feito, tornando-os pouco úteis para auditoria real. O design original não especificava o nível de detalhe.	Diagrama de classes — Domínio, Serviço; Schema da BD
Apresentação / Apresentação	ConfiguracaoTerminal passou a expor morada, localidade, nif e email lidos de config.properties. O GeradorFaturaPdf foi atualizado para apresentar logótipo da loja, dados de contacto no cabeçalho, e agrupar artigos por categoria.	O levantamento de requisitos identificou que a fatura real da TrasmUM inclui morada, localidade, NIF, e-mail e logótipo, e que os artigos devem ser apresentados por secção. O design original não contemplava estes campos nem este agrupamento.	Sem impacto em diagramas
Domínio / Serviço / Repositório / UI / Schema	Introduzida a entidade de domínio Loja (id, nome, morada, localidade, nif, email, limiteMaximoCaixa, diasAlertaValidade) com tabela ConfiguracaoLoja no schema, novo LojaRepositorio, novo ILojaServico, novo separador “Loja” na vista de Administração. ConfiguracaoTerminal tornou-se parcialmente mutável para permitir alterações em tempo de execução. O AppContext faz <i>bootstrap</i> a partir de config.properties na primeira execução.	Os dados de identidade da loja e os parâmetros operacionais (limite de caixa, dias de alerta de validade) estavam apenas em config.properties, sem interface de edição. A entidade em BD permite editá-los via UI, auditá-los, e garantir efeito imediato sem reiniciar a aplicação.	Diagrama de classes — Domínio, Repositório, Serviço; Schema da BD
Domínio / Serviço / UI / Schema	Revisão do fluxo de sessão de caixa: abertura simplificada para introduzir apenas o valor total do fundo (sem contagem por denominação); fecho mantém a contagem detalhada com novos campos saldoContado e diferencaFecho em SessaoCaixa; validação de sangria movida para o serviço com SangriaInvalidaException; logout bloqueado se houver sessão de caixa aberta; fecho	A contagem por denominação na abertura era trabalhosa e proporcionava pouco valor. A contagem detalhada no fecho tem utilidade real (reconciliação física). As restrições de <i>logout</i> e fecho de dia garantem que nenhum fecho ocorre com dinheiro em aberto.	Diagrama de classes — Domínio, Serviço; Diagrama de sequência — Sessão de Caixa; Schema da BD

de dia bloqueado se houver sessões abertas no sistema.

Tabela 121: Desvios Design - Software de Loja

Anexo 20: Lista completa de desvios — Software de Servidor Central

Área	O que foi alterado	Porquê	Impacto
Infra	A pasta ServidorCentral/ passou a conter dois subprojetos: frontend/ (Vue 3 + Vite, movido sem alterações estruturais) e backend/ (Maven + Javalin, criado de raiz). O README.md original do Vite foi movido para frontend/.	Manter os dois projetos completamente isolados em termos de <i>build</i> , dependências e arranque.	Sem impacto em diagramas
Backend	Cada interface de repositório envolvida na ingestão expõe duas variantes: a descrita no prompt (que abre a sua própria ligação) e uma adicional guardar(Connection conn, Entity) / criarSeNaoExistir(Connection conn, Loja).	O processamento de um PacoteFechoDTO é executado numa única transação JDBC. Sem aceitar a Connection externa em cada repositório, teria de se abrir uma ligação nova por tabela, quebrando a transação.	Diagrama de classes — Repositório
Backend	IngestaoServico.verificarHash limpa o campo hashIntegridade, serializa o pacote com o mesmo Gson configurado de forma idêntica ao do Software de Loja, calcula SHA-256 hexadecimal e compara com o valor recebido.	Espelhar exatamente o procedimento em HttpSincronizacaoGateway do Loja para que o hash coincida.	Sem impacto em diagramas
Apresentação	src/api/mockData.ts passou a re-exportar apenas os tipos Store e Shipment. networkService.ts foi reescrito para chamar a API real e adaptar as respostas ao formato esperado pelos componentes. Campos sem equivalente no servidor (dailySales, stockAlerts) ficam a 0.	Não alterar a estrutura visual nem a lógica dos componentes Vue. Manter os tipos legados com um adaptador garante isso, ao custo de manter campos sem significado real.	Contrato API
Frontend	Criado src/api/http.ts com request/get/post que adicionam automaticamente Authorization: Bearer <token> lido de localStorage. Em resposta HTTP 401, limpa o token e redireciona para /login.	Necessidade de <i>interceptor</i> único e persistência do <i>token</i> em localStorage para todos os pedidos autenticados.	Sem impacto em diagramas
Frontend	Nova rota /login com LoginView.vue minimalista. <i>Navigation guard</i> no Vue Router redireciona para /login quando não há <i>token</i> , exceto em rotas com meta.publica. App.vue esconde a Sidebar em rotas públicas. Botão “Terminar Sessão” ligado a authService.logout().	Conformidade com os requisitos de autenticação mantendo a estrutura visual existente. Introduce estado de autenticação no <i>frontend</i> , que não estava descrito no design visual original.	Diagrama de componentes
Backend	AutenticacaoCEOService remove a conta admin de <i>bootstrap</i> apenas após um <i>login</i> bem-sucedido com uma conta cujo nomeUtilizador seja diferente de admin. Enquanto for a única conta, continua válida.	Se a conta fosse removida no primeiro <i>login</i> da própria conta admin, não seria possível voltar a entrar antes de existir outra conta CEO definitiva.	Sem impacto em diagramas
Backend	Acrescentado endpoint GET /ping que devolve {"ok": true}.	O Software de Loja usa este endpoint no HttpSincronizacaoGateway para verificar conectividade antes de enviar o fecho.	Contrato API
Frontend	http.ts usa import.meta.env.VITE_API_URL se definido, com <i>fallback</i> para http://localhost:8080.	Permite apontar o <i>frontend</i> para uma instância remota do <i>backend</i> sem recompilar.	Sem impacto em diagramas

Backend / Frontend / Schema	Adicionada coluna descricao VARCHAR(255) a LogAuditoriaCentral e ao DTO LogAuditoriaDTO. O <i>dashboard</i> de loja no <i>frontend</i> passou a exibir a descrição textual em vez do <i>enum</i> TipoAcao.	Tornar os logs legíveis sem necessidade de interpretação técnica do <i>enum</i> ; permitir auditoria real pelo CEO/gestor. Espelha a alteração homóloga no Software de Loja.	Diagrama de classes — Domínio; Schema da BD
Backend / Frontend	Os <i>dashboards</i> global e por loja expõem totalDespesas (soma de valorTotalGuia das remessas no período) e lucroLiquido (vendas menos devoluções menos despesas). A decomposição por loja foi estendida com despesas e lucro. Novos <i>cards</i> no <i>frontend</i> com coloração dinâmica.	Os dados de remessas eram recebidos e guardados mas nunca usados analiticamente. A visibilidade do lucro líquido é necessária para suporte à tomada de decisão pelo CEO.	Contrato API; Diagrama de classes — Serviço

Tabela 122: Desvios Design - Software de Servidor Central

Anexo 21: Listagem completa dos testes unitários — Servidor Central

Nº	Classe	Método de Teste	Comportamento Verificado	Resultado
1	AutorizacaoServico	exigirPerfil_perfilPermitido_naoLancaExcecao	Utilizador com perfil autorizado não lança exceção.	Passou
2	AutorizacaoServico	exigirPerfil_perfilNaoPermitido_lancaAcessoNegadoException	Utilizador sem perfil adequado lança AcessoNegadoException.	Passou
3	AutorizacaoServico	temPerfil_perfilCorreto_devolveTrue	Retorna true quando o perfil do utilizador corresponde ao pedido.	Passou
4	AutorizacaoServico	temPerfil_perfilErrado_devolveFalse	Retorna false quando o perfil não corresponde.	Passou
5	VendaServico	adicionarLinha_produtoNaoEncontrado_lancaExcecao	Código de barras inexistente lança ProdutoNaoEncontradoException.	Passou
6	VendaServico	adicionarLinha_produtoInativo_lancaExcecao	Produto desativado lança ProdutoInativoException.	Passou
7	VendaServico	adicionarLinha_stockInsuficiente_lancaExcecao	Quantidade pedida superior ao stock disponível lança StockInsuficienteException.	Passou
8	VendaServico	adicionarLinha_fefoComDoisLotes_ambosOsLotesAtualizados	Consumo FEFO sobre dois lotes: lote com validade mais próxima esgotado primeiro; ambos os lotes atualizados no repositório.	Passou
9	VendaServico	finalizarVenda_numerarioInsuficiente_lancaIllegalArgumentException	Valor entregue em numerário inferior ao total lança IllegalArgumentException.	Passou
10	VendaServico	finalizarVenda_semSessaoCaixa_lancaSessaoCaixaNaoEncontradaException	Ausência de sessão de caixa ativa lança SessaoCaixaNaoEncontradaException.	Passou
11	VendaServico	finalizarVenda_caminhoFeliz_guardaVendaAtualizaSessaoRegistaAuditoria	Caminho feliz: venda persistida, sessão de caixa atualizada com saldo correto, auditoria registada.	Passou
12	VendaServico	finalizarVenda_saldoExcedeLimite_naoLancaExcecao	Saldo acima do limite não lança exceção no serviço; venda persistida e saldo atualizado.	Passou
13	VendaServico	calcularTroco_valorInferior_lancaIllegalArgumentException	Valor entregue inferior ao total lança IllegalArgumentException.	Passou
14	VendaServico	calcularTroco_valorSuperior_devolveDiferenca	Troco calculado corretamente como diferença entre valor entregue e total.	Passou
15	VendaServico	iniciarVenda_semSessaoAtiva_lancaSessaoCaixaNaoEncontradaException	Ausência de sessão ativa ao iniciar venda lança SessaoCaixaNaoEncontradaException.	Passou

16	VendaServico	iniciarVenda_caminhoFeliz_devolveVendaEmCurso	Nova venda criada com estado EM_CURSO quando sessão de caixa existe.	Passou
17	VendaServico	anularVenda_repoeStockEGuardaVendaAnulada	Anulação repõe stock no lote, define estado ANULADA e persiste a venda.	Passou
18	VendaServico	obterVendasPendentes_delegaParaRepositorio	Método devolve exatamente a lista retornada pelo repositório.	Passou
19	VendaServico	buscarPorNumeroFatura_delegaParaRepositorio	Método devolve exatamente o objeto retornado pelo repositório para o número de fatura fornecido.	Passou
20	Devolucao-Servico	processar_faturaNula_lancaFaturaNaoEncontradaException	Número de fatura sem correspondência no repositório lança FaturaNaoEncontradaException.	Passou
21	Devolucao-Servico	processar_vendaSemFatura_lancaFaturaNaoEncontradaException	Venda encontrada mas sem fatura associada lança FaturaNaoEncontradaException.	Passou
22	Devolucao-Servico	processar_artigoNaoConstaNafatura_lancaArtigoNaoConstaNafaturaException	Produto não presente nas linhas da fatura lança ArtigoNaoConstaNafaturaException.	Passou
23	Devolucao-Servico	processar_quantidadeTotalmenteDevolvida_lancaDevolucaoJaProcessadaException	Quantidade devolvida igual à vendida lança DevolucaoJaProcessadaException.	Passou
24	Devolucao-Servico	processar_quantidadeExcedeDisponivel_lancaQuantidadeDevolucaoExcedidaException	Quantidade a devolver superior à disponível lança QuantidadeDevolucaoExcedidaException.	Passou
25	Devolucao-Servico	processar_loteValido_repoeStockEPersisteDevolucao	Lote dentro da validade: stock reposto e devolução persistida.	Passou
26	Devolucao-Servico	processar_loteExpirado_naoRepoeStockMasPersisteDevolucao	Lote expirado: stock não reposto mas devolução persistida.	Passou
27	Devolucao-Servico	obterDevolucoesPendentes_delegaParaRepositorio	Método devolve exatamente a lista retornada pelo repositório.	Passou
28	FechoDia-Servico	executarFecho_perfilFuncionario_lancaAcessoNegadoEGatewayNuncaInvocado	Perfil Funcionário bloqueia o fecho; gateway de sincronização nunca é invocado.	Passou
29	FechoDia-Servico	executarFecho_envioFalha_lancaExcecaoEReverte	Falha no envio lança EnvioFechoFalhouException e reverte os registos para estado Pendente.	Passou
30	FechoDia-Servico	executarFecho_sucesso_guardaFechoERegistaAuditoria	Envio bem-sucedido persiste o fecho de dia e regista evento de auditoria.	Passou
31	FechoDia-Servico	executarFecho_comListasNaoVazias_cobremMapeadoresELambdas	Orquestração completa com todos os tipos de dados pendentes: marcação EM_TRANSITO e CONFIRMADO em ambas as fases do envio.	Passou
32	FechoDia-Servico	reenviar_utilizadorNulo_devolveFalse	Reenvio com utilizador nulo devolve false imediatamente.	Passou
33	FechoDia-Servico	reenviar_envioFalha_devolveFalse	Reenvio com falha no gateway devolve false.	Passou
34	FechoDia-Servico	reverterEmTransito_invocaReverterEmTodosOsRepositorios	Reversão em cascata: os sete repositórios de entidades sincronizáveis são todos invocados.	Passou

Tabela 123: Testes unitários — Software de Loja

Anexo 22: Listagem completa dos testes unitários — Servidor Central

Nº	Classe	Método de Teste	Comportamento Verificado	Result
----	--------	-----------------	--------------------------	--------

1	Autentica- caoCEOSer- vico	autenticar_utilizadorInexistente_ lancaCredenciaisInvalidasException	Nome de utilizador sem correspon- dência lança CredenciaisInvalidasEx- ception.	Passou
2	Autentica- caoCEOSer- vico	autenticar_contaBloqueada_ lancaContaBloqueadaException	Conta com bloqueio ativo lança Conta- BloqueadaException.	Passou
3	Autentica- caoCEOSer- vico	autenticar_palavraPasseIncorreta_ lancaCredenciaisInvalidasERegistaFalhado	Palavra-passe errada lança exceção e incrementa o contador de tentativas.	Passou
4	Autentica- caoCEOSer- vico	autenticar_caminhoFeliz_ retornaCeoELimpaBloqueio	Autenticação bem-sucedida devolve o CEO, reinicia as tentativas a zero e limpa o bloqueio.	Passou
5	Autentica- caoCEOSer- vico	registarFalhado_abaixoDoLimiar_ incrementaContadorSemBloquear	Tentativa falhada abaixo do limiar incre- menta o contador sem definir bloquea- doAte.	Passou
6	Autentica- caoCEOSer- vico	registarFalhado_aoAtingirLimiar_ defineBloqueioEAatualiza	Ao atingir o limiar de tentativas, blo- queadoAte é definido para o futuro.	Passou
7	Autentica- caoCEOSer- vico	autenticar_naoAdminCom BootstrapExistente_apagaBootstrap	Login de CEO não-admin com conta bootstrap existente elimina a conta de bootstrap.	Passou
8	Autentica- caoCEOSer- vico	autenticar_naoAdminSemBootstrap_ naoApagaNada	Login de CEO não-admin sem conta bo- otstrap não invoca eliminação.	Passou
9	Autentica- caoCEOSer- vico	estaBloqueado_bloqueadoAteNulo_ devolveFalse	bloqueadoAte nulo — conta não está bloqueada.	Passou
10	Autentica- caoCEOSer- vico	estaBloqueado_bloqueado AteNoPassado_devolveFalse	bloqueadoAte no passado — bloqueio expirado, conta desbloqueada.	Passou
11	Autentica- caoCEOSer- vico	estaBloqueado_bloqueado AteNoFuturo_devolveTrue	bloqueadoAte no futuro — conta efeti- vamente bloqueada.	Passou
12	Autentica- caoCEOSer- vico	criar_nomeUtilizadorEmBranco_ lancaIllegalArgumentException	Nome de utilizador em branco lança Il- legalArgumentException sem invocar o repositório.	Passou
13	Autentica- caoCEOSer- vico	criar_contaDefinitivaJaExiste_ lancaContaCEOJaExisteException	Quando já existe conta CEO definitiva, lança ContaCEOJaExisteException sem invocar repo.criar.	Passou
14	Autentica- caoCEOSer- vico	criar_caminhoFeliz_ persisteCeoComHashBCrypt	Conta criada com sucesso; CEO persis- tido tem o nome correto, hash BCrypt válido para a palavra-passe fornecida, zero tentativas e bloqueio nulo.	Passou
15	IngestaoSer- vico	processarPacote_hashAusente_ lancaIntegridadeInvalidaException	Campo hashIntegridade nulo lança In- tegridadeInvalidaException.	Passou
16	IngestaoSer- vico	processarPacote_hashEmBranco_ lancaIntegridadeInvalidaException	Campo hashIntegridade em branco lança IntegridadeInvalidaException.	Passou
17	IngestaoSer- vico	processarPacote_hashIncorreto_ lancaIntegridadeInvalidaException	Hash SHA-256 incorreto lança Integri- dadeInvalidaException.	Passou
18	IngestaoSer- vico	processarPacote_hashCorreto_ conclui_commitInvocado_rollbackNunca	Pacote com hash válido conclui sem exceção; commit invocado e rollback nunca chamado.	Passou
19	IngestaoSer- vico	processarPacote_sqlException NoCommit_lancaRuntimeException EFazRollback	SQLException no commit lança Runti- meException e invoca rollback.	Passou
20	IngestaoSer- vico	processarPacote_comTodosTiposDto_ delegaParaRepositorios	Pacote com todos os tipos de DTO (venda, devolução, remessa, paga-	Passou

			mento, sessão, log) — todos os repositórios são invocados com a mesma Connection.	
--	--	--	---	--

Tabela 124: Testes unitários — Software de Servidor Central

Prompts Utilizados

Descrição	Geração de uma contextualização narrativa, concisa e acessível, que apresente o cenário, as personagens e o projeto de desenvolvimento de software trabalhado ao longo do caso de estudo.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Assume o papel de um redator criativo especializado em narrativas de enquadramento para projetos de engenharia de software. O teu objetivo é redigir uma breve história introdutória, escrita rigorosamente em português de Portugal, que servirá de contexto para um caso de estudo académico sobre o desenvolvimento de um sistema de gestão para uma rede de lojas de conveniência. O público-alvo é constituído por estudantes e docentes universitários, pelo que a narrativa deve adotar um tom relaxado, ligeiro e acessível, evitando formalismos excessivos, mas mantendo a coerência estrutural adequada a um trabalho académico. O resultado final deve ser apresentado em texto corrido, estruturado num máximo de três parágrafos. A história deve centrar-se na figura de Hélder, o bem-sucedido proprietário de uma empresa denominada Prestige Limousines. Num gesto de apreço, Hélder decide adquirir uma rede composta por exatamente três lojas de conveniência, localizadas no distrito de Bragança, com o propósito de a oferecer à sua esposa, conhecida como Cátia das unhas pintadas. É fundamental explicar na narrativa que esta rede foi comprada às anteriores proprietárias, a Dona Zulmira e a Dona Acácia, que optaram por vender o negócio devido à sua idade avançada e à crescente dificuldade em acompanhar o rápido crescimento comercial da cadeia. Após oficializar a surpresa à sua esposa, Hélder constata que a gestão das lojas ainda assenta integralmente numa metodologia analógica, dependente de registos em papel. Consequentemente, decide contratar quatro jovens programadores para desenvolverem o novo sistema informático desta rede, motivado pela excelente experiência prévia que teve com estes mesmos jovens quando lhes encomendou o software de gestão para a sua frota de limousines. Como restrição absoluta, não deves efetuar qualquer menção direta aos problemas operacionais decorrentes da atual gestão em papel, nem delinear motivações futuras, objetivos de projeto ou antecipar soluções arquiteturais. Foca-te estritamente na apresentação da narrativa de transição, do cenário e das personagens descritas.
Fontes Fornecidas	Enunciado do trabalho prático.
Reflexão Final	O texto gerado revelou-se bem estruturado e cumpriu os objetivos pretendidos. A qualidade do resultado deveu-se em grande medida ao nível de detalhe e às restrições explícitas incluídas no prompt, que guiaram o modelo de forma eficaz. Foi necessário apenas acrescentar manualmente uma data no início do primeiro parágrafo, dado que o modelo não dispunha de contexto temporal suficiente para a inferir autonomamente.
⬆ Voltar à secção	

Prompt 2: Criação da Contextualização

Descrição	Geração do texto da secção “Caso de Estudo”, que expõe as ineficiências operacionais e os prejuízos causados pela gestão inteiramente analógica praticada pelas antigas proprietárias da rede de lojas.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Assume novamente o papel de redator criativo especializado em narrativas de enquadramento para projetos de engenharia de software. O teu objetivo agora é redigir a secção “Caso de Estudo” que dá continuidade à história de Hélder, Cátia das unhas pintadas, e da rede de três lojas de conveniência no distrito de Bragança, recentemente adquirida à Dona Zulmira e à Dona Acácia. O texto deve manter um tom acessível e ligeiro da secção anterior, mas ligeiramente mais técnico e formal, evitando expressões coloquiais. A extensão deve compreender entre três a quatro parágrafos bem estruturados. O propósito exclusivo desta secção é expor ao leitor os graves problemas, as ineficiências operacionais e os prejuízos causados pela gestão inteiramente analógica e baseada em papel praticada pelas antigas proprietárias.

	Na tua redação, deves descrever em texto corrido como a utilização de cadernos e folhas de papel para a gestão de inventário resultava frequentemente em graves discrepâncias, originando tanto ruturas de produtos essenciais como excesso de artigos perecíveis nas prateleiras, com consequentes perdas financeiras. Deves ainda introduzir e detalhar duas outras ineficiências críticas: a primeira deve focar-se no caos do fecho de caixa diário, descrevendo como as antigas proprietárias perdiam horas ao telefone com cada uma das três lojas para apurar os lucros do dia, num processo lento e sujeito a erros de cálculo humano; a segunda deve abordar a gestão desastrosa de fornecedores, relatando como faturas e guias de remessa eram frequentemente perdidas, dificultando pagamentos atempados e causando atritos nas encomendas de nova mercadoria. Como restrição absoluta, deves limitar-te ao diagnóstico e à exposição dos problemas e das suas consequências operacionais. Sob nenhuma circunstância deves introduzir, sugerir ou mencionar as soluções tecnológicas que os quatro jovens programadores irão desenvolver. Garante que o texto flui de forma natural e está redigido em português de Portugal.
Fontes Fornecidas	Enunciado do trabalho prático; relatório do trabalho prático (versão atual).
Reflexão Final	O texto inicialmente gerado apresentava um registo mais informal do que o pretendido, consequência direta da instrução para “manter o tom relaxado e ligeiro da secção anterior”. Essa indicação conduziu o modelo a produzir expressões marcadamente coloquiais, como “o cenário revelou-se digno de uma cena de um filme de terror” ou “mítica hora do fecho da caixa”, incompatíveis com o registo académico pretendido. O prompt foi reformulado com restrições explícitas ao uso deste tipo de linguagem e com a instrução de adotar um registo ligeiramente mais técnico e formal, o que se revelou eficaz e produziu o texto final utilizado na secção.
↑ Voltar à secção	

Prompt 3: Criação da Motivação

Descrição	Geração do texto da secção “Motivação e Objetivos”, justificando a necessidade do sistema e listando os objetivos a alcançar, mantendo a continuidade narrativa dos personagens.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Atua como um Consultor de Sistemas de Informação encarregue de redigir a secção de Motivação e Objetivos de um relatório académico para o projeto “Sistema de Gestão Integrada para uma Cadeia de Pequenas Lojas de Conveniência”. O teu objetivo é justificar a necessidade do software e listar o que se pretende alcançar, utilizando uma narrativa que reflita o ponto de vista dos personagens envolvidos (Hélder Cerqueira e Cátia das unhas pintadas). O texto deve ser redigido exclusivamente em português de Portugal, com um tom profissional mas narrativo e acessível, evitando tecnicismos excessivos e dando ênfase aos personagens. O conteúdo deve ser uma extensão natural da “Contextualização” e do “Caso de Estudo”, focando-se na resolução dos problemas de inventário, fecho de caixa e gestão de fornecedores. Evita usar o termo “académico” para descrever o tom, pois tende a induzir um registo demasiado formal, próximo de artigos científicos. Secção de Motivação (3 a 4 parágrafos): descreve a motivação de Hélder e Cátia em transformar o negócio analógico num modelo de eficiência digital; enfatiza o desejo de Cátia em eliminar os processos manuais e o caos administrativo (como as caixas de sapatos e os registos em papel); menciona a confiança depositada na equipa de engenharia para replicar o sucesso obtido anteriormente no negócio das limousines, agora aplicado ao retalho em Trás-os-Montes; inclui obrigatoriamente os nomes Hélder, Cátia, Dona Zulmira e Dona Acácia para manter a continuidade da narrativa. Secção de Objetivos (mínimo 5 itens em lista): define o que o sistema deverá disponibilizar para solucionar as dificuldades mencionadas, incluindo centralização de dados no fecho do dia (não em tempo real), controlo automático de stock, automação financeira, e registo e anulamento de vendas e devoluções; os objetivos devem ser claros e diretamente ligados à melhoria da tomada de decisão estratégica por parte da CEO.
Fontes Fornecidas	Enunciado do trabalho prático; relatório do trabalho prático (versão atual).
Reflexão Final	Foram necessárias várias iterações sobre o prompt inicial, uma vez que o texto gerado apresentava uma formalidade e expressividade técnica excessivas para o registo pretendido nesta secção. Verificou-se que o modelo interpreta o termo “académico” de forma estrita, associando-o a formatos altamente formalizados como artigos científicos ou dissertações, o que justifica parte do problema. O prompt foi reformulado com instruções explícitas para adotar um

	tom narrativo centrado nos personagens, evitando esse termo. Foram ainda realizadas duas correções de conteúdo no texto final: a adição de um objetivo relativo ao registo e anulamento de vendas e devoluções, e a correção do objetivo de centralização de informação, que descrevia erroneamente uma sincronização em tempo real quando, na verdade, ocorre apenas no fecho do dia.
↑ Voltar à secção	

Prompt 4: Criação dos Objetivos

Descrição	Geração do texto da secção de recursos necessários para o projeto TrasmUM, cobrindo os recursos humanos internos e externos, os recursos de hardware e os recursos de software necessários ao desenvolvimento e documentação do sistema.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Tendo em conta o relatório do projeto TrasmUM desenvolvido até ao momento, redige a secção de Recursos Necessários. A secção deve estar organizada em dois grandes grupos: recursos humanos e recursos materiais. Os recursos humanos devem distinguir o pessoal externo (equipa de desenvolvimento) do pessoal interno (representantes da organização TrasmUM). Os recursos materiais devem cobrir hardware e software, incluindo ferramentas de gestão de projeto, comunicação, modelação UML, redação do relatório e a pilha tecnológica de implementação. O texto deve ser redigido em português de Portugal, com um registo formal e objetivo, em coerência com o restante documento.
Fontes Fornecidas	Relatório do trabalho prático (versão atual).
Reflexão Final	O output gerado foi aceite com ajustes pontuais. A estrutura proposta pelo modelo revelou-se adequada e coerente com o exemplo fornecido como referência.
↑ Voltar à secção	

Prompt 5: Criação da Secção de Recursos Necessários

Descrição	Geração do texto da secção “Análise de Contexto e Restrições”, definindo o ambiente operacional do software, as fronteiras do projeto e as exclusões de âmbito decorrentes da natureza académica e temporal do desenvolvimento.
LLM Utilizado	Google Gemini 2.5 Pro (output original); Claude Sonnet 4.6 (análise complementar)
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico encarregue de redigir a secção “Análise de Contexto e Restrições” para o relatório académico do Sistema de Gestão Integrada da rede de lojas TrasmUM. O teu objetivo é definir claramente o ambiente operacional do software e estabelecer as fronteiras do projeto, tendo em conta tratar-se de um projeto académico desenvolvido num prazo de cerca de cinco meses. O texto deve ser redigido rigorosamente em português de Portugal, com um tom formal, analítico e objetivo. Adota uma estrutura mista: começa com um ou dois parágrafos de texto corrido e finaliza com uma lista em bullet points. Nos parágrafos introdutórios, contextualiza o ambiente em que o sistema irá operar. Explica que o software funcionará como uma plataforma centralizada de gestão de dados de três lojas autónomas, focando-se na otimização de fluxos de informação (vendas, stocks e fornecedores) para apoiar a tomada de decisão da administração. Salienta que o foco principal da equipa de desenvolvimento reside na arquitetura de software, na fiabilidade dos dados e na criação de uma interface funcional. Em seguida, introduz uma lista em bullet points dedicada exclusivamente às Restrições e Exclusões de Âmbito. Inclui obrigatoriamente os seguintes pontos: ausência de integração com hardware físico (leitura de códigos de barras, gavetas de dinheiro ou impressoras de talões); exclusão de processamento de pagamentos reais (o sistema apenas regista o método de pagamento, sem comunicação com gateways bancárias ou terminais Multibanco/SIBS); e não integração com sistemas fiscais externos (a emissão de faturas é uma simulação interna, sem comunicação automática com o portal da Autoridade Tributária). Acrescenta mais uma ou duas restrições técnicas coerentes com um trabalho prático universitário de Engenharia de Software.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); enunciado do trabalho prático.

Reflexão Final	O output inicial encontrava-se bem estruturado e alinhado com as restrições acordadas, mas considerou-se que nem todas as limitações desejadas ficaram explicitamente contempladas. Por precaução, a secção foi igualmente submetida ao Claude Sonnet 4.6, com o objetivo de identificar eventuais restrições adicionais relevantes para o contexto académico do projeto. Esta análise complementar permitiu identificar restrições pertinentes que não tinham sido geradas inicialmente, nomeadamente a inexistência de migração de dados históricos, que foram posteriormente incorporadas na especificação. Foram ainda eliminadas algumas restrições consideradas demasiado específicas e irrelevantes para o âmbito definido, e adicionadas outras julgadas mais pertinentes.
↑ Voltar à secção	

Prompt 6: Criação da Análise de Contexto e Restrições

Descrição	Geração das secções “Análise de Viabilidade” e “Medidas de Sucesso”, avaliando a exequibilidade do projeto sob a perspetiva dos stakeholders e definindo métricas mensuráveis que traduzam os objetivos do negócio em alvos técnicos concretos.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico especializado em engenharia de software. O teu objetivo é redigir as secções “Análise de Viabilidade” e “Medidas de Sucesso” para o relatório académico do Sistema de Gestão Integrada da rede de três lojas de conveniência pertencente a Helder Cerqueira e gerida por Cátia das unhas pintadas. O texto deve ser redigido estritamente em português de Portugal, adotando um tom formal, analítico e profissional, sem recorrer a jargão tecnológico impenetrável. Utiliza uma estrutura mista: parágrafos de texto corrido para as introduções e contextualizações, seguidos de listas em bullet points para elencar os objetivos e as métricas específicas. Evita referências aos problemas atuais já descritos em secções anteriores, como cadernos de argolas ou registos em papel. Para a secção “Análise de Viabilidade”, redige um ou dois parágrafos introdutórios que avaliem a exequibilidade do projeto sob a perspetiva dos principais stakeholders. De seguida, apresenta uma lista em bullet points com objetivos quantificáveis e realistas que justifiquem o investimento, incluindo estimativas de redução de erros no pedido de stock, diminuição do tempo de apuramento financeiro diário, melhoria na pontualidade de liquidação de faturas e melhoria no controlo de vendas. Para a secção “Medidas de Sucesso”, inicia com um parágrafo a explicar como Helder e Cátia definiram critérios de avaliação rigorosos para a equipa de desenvolvimento. Cria uma lista em bullet points com indicadores de sucesso mensuráveis, incluindo taxa de precisão do inventário automatizado, tempos máximos de resposta na consolidação diária de dados entre as três lojas, taxa de adoção do sistema pelos funcionários e integridade do registo de vendas e devoluções.
Fontes Fornecidas	Relatório do trabalho prático (versão atual).
Reflexão Final	Foram necessárias algumas iterações para alcançar metas de desempenho com as quais a equipa concordasse. Verificou-se também que o modelo tendia a repetir informação já apresentada em secções anteriores, nomeadamente sobre os problemas operacionais da cadeia, tendo sido necessário adicionar uma restrição explícita no prompt para evitar esse comportamento. Por fim, foram introduzidos manualmente dois pontos que não constavam no output inicial: um referente ao melhoramento do controlo de vendas na secção de Viabilidade, e outro relativo à integridade do registo de vendas e devoluções nas Medidas de Sucesso, funcionalidades que, apesar de implícitas, a equipa considerou importante tornar explícitas.
↑ Voltar à secção	

Prompt 7: Criação da Análise de Viabilidade e Medidas de Sucesso

Descrição	Geração das secções “Plano de Execução do Projeto” e “Revisão e Aprovação do Projeto”, descrevendo as reuniões de alinhamento e validação com a administração e formalizando a aprovação para avançar para a fase de levantamento de requisitos.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico encarregue de concluir o capítulo introdutório de um relatório académico de Engenharia de Software. O teu objetivo é

	redigir duas breves secções finais: “Plano de Execução do Projeto” e “Revisão e Aprovação do Projeto”, referentes ao desenvolvimento do Sistema de Gestão Integrada para a rede de lojas TrasmUM, liderada por Hélder Cerqueira e Cátia das unhas pintadas. O texto deve ser redigido em português de Portugal, num tom formal, objetivo e extremamente conciso, sem excesso de texto ou floreios narrativos. O resultado deve ser em texto corrido, sem bullet points. Para a secção “Plano de Execução do Projeto”, redige apenas um parágrafo. Descreve de forma direta a realização de uma reunião de alinhamento entre a equipa de quatro jovens programadores e a administração, onde foram delineadas as etapas do ciclo de desenvolvimento para um horizonte temporal de cinco meses. Termina referindo que a calendarização foi formalizada através de um Diagrama de Gantt, ilustrado na figura abaixo. Para a secção “Revisão e Aprovação do Projeto”, redige também apenas um parágrafo muito direto. Relata a realização de uma reunião de validação com os stakeholders principais, cujo propósito foi a revisão final dos objetivos, métricas e planeamento, culminando na aprovação formal do projeto pela administração e na autorização para avançar para o Levantamento e Análise de Requisitos.
Fontes Fornecidas	Relatório do trabalho prático (versão atual).
Reflexão Final	Identificou-se que a qualidade das respostas do modelo estava a degradar-se de forma significativa, com informação excessivamente repetitiva, texto prolixo e alguns indícios de incoerência. Este fenómeno foi associado à utilização continuada do mesmo chat desde o início do processo, levando à acumulação progressiva de contexto e ao aumento da probabilidade de respostas menos precisas. Para restabelecer a qualidade das respostas, optou-se pela criação de um novo chat, o que se revelou eficaz.
↑ Voltar à secção	

Prompt 8: Criação do Plano de Execução e Revisão e Aprovação do Projeto

Descrição	Geração de uma página HTML autónoma com o Diagrama de Gantt do projeto TrasmUM, ilustrando o plano de execução com as quatro fases do calendário académico de 2025/2026, com design visual polido e tipografia distintiva.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Tens à tua disposição o plano de trabalho do projeto TrasmUM, organizado em quatro capítulos sequenciais com datas de início e fim definidas: Capítulo 1 - Conceção e Engenharia de Requisitos Assistida por LLM, de 9 de fevereiro a 2 de março de 2026; Capítulo 2 - Arquitetura e Design do Software, de 2 a 23 de março; Capítulo 3 - Implementação e Desenvolvimento Assistido por LLM, de 23 de março a 13 de abril; e Capítulo 4 - Verificação, Validação e Avaliação da Qualidade, de 13 de abril a 4 de maio. Para cada capítulo foram ainda especificadas as tarefas concretas a realizar. Gera um ficheiro HTML autónomo que apresente um Diagrama de Gantt semanal com estas quatro fases e respetivas tarefas, respeitando rigorosamente as datas estipuladas. O diagrama deve ter design cuidado, com uma paleta de cores distinta por capítulo, tipografia legível, cabeçalho com identificação do projeto e badges de período, e ser adequado para inclusão num relatório académico.
Fontes Fornecidas	Enunciado do trabalho prático.
Reflexão Final	O output foi gerado corretamente na primeira tentativa. O ficheiro HTML produzido foi utilizado na íntegra, sem necessidade de ajustes ao código ou à distribuição das tarefas pelas semanas.
↑ Voltar à secção	

Prompt 9: Geração do Diagrama de Gantt em HTML

Descrição	Geração do texto da secção referente à metodologia de levantamento de requisitos, descrevendo as cinco técnicas de eliciação planeadas e fundamentando a sua aplicação no contexto do projeto TrasmUM.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico encarregue de redigir a secção referente ao Método de Levantamento de Requisitos para um relatório académico de Engenharia de Software. O teu objetivo é descrever a metodologia planeada para a eliciação dos

	requisitos do Sistema de Gestão Integrada da rede de lojas TrasmUM. O texto deve ser redigido exclusivamente em português de Portugal, adotando um registo formal e objetivo, sem recurso a informação redundante já apresentada em capítulos anteriores. Inicia o texto mencionando que os requisitos serão levantados em concordância com as orientações metodológicas de Sommerville, 2015, páginas 102 a 134. De seguida, detalha o plano de elicitação assente em cinco técnicas complementares: uma reunião formal com a administração (Cátia e Hélder) para alinhamento estratégico; uma entrevista semiestruturada conduzida com Hélder e Cátia, para captar a perspetiva macro e as expectativas dos decisores; observação direta do funcionamento das lojas durante uma semana, com recurso a princípios de etnografia; questionários dirigidos aos operadores de caixa para recolha de dados quantitativos e qualitativos; e uma análise documental das guias de remessa e faturas arquivadas nas lojas para mapear as debilidades no controlo de fornecedores.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); Sommerville, I. — <i>Software Engineering</i> (2015).
Reflexão Final	O output cumpriu os requisitos esperados com necessidade de apenas alguns refinamentos menores, comportamento previsível dado o nível de detalhe fornecido no prompt. Foi necessário corrigir um detalhe de conteúdo: o prompt inicial referia a entrevista como sendo conduzida apenas com Hélder, quando na realidade foi realizada com Hélder e Cátia em conjunto, tendo esta clarificação sido incorporada na versão final do prompt acima apresentada.
↑ Voltar à secção	

Prompt 10: Criação dos Métodos de Levantamento

Descrição	Geração do texto da subsecção relativa à reunião formal com a administração, descrevendo os seus objetivos, os recursos disponibilizados por Hélder e Cátia e os requisitos iniciais emergentes da conversa.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Pretende-se preencher o ponto relativo à Reunião Formal com a Administração. Esta secção não necessita de ser muito extensa. Importa referir que Cátia e Hélder forneceram informações relevantes, nomeadamente sobre os documentos que a equipa poderia consultar e sobre a autorização para a realização da etnografia. De forma geral, esta reunião teve como principal objetivo definir o conjunto de fontes de informação disponíveis para o projeto, embora, no decurso da conversa, tenham também emergido alguns requisitos iniciais.
Fontes Fornecidas	Relatório do trabalho prático (versão atual).
Reflexão Final	O output foi gerado de forma adequada e utilizado sem necessidade de alterações relevantes.
↑ Voltar à secção	

Prompt 11: Criação da subsecção da Reunião Formal com a Administração

Descrição	Geração do texto introdutório da subsecção de Questionários, fundamentando a utilização desta técnica de elicitação e enquadrando a apresentação das questões dirigidas aos operadores de caixa.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Atua como um Analista de Sistemas e Redator Técnico especializado em Engenharia de Software para redigir o texto introdutório da secção 3.1.4, intitulada Questionários, referente ao levantamento de requisitos da rede de lojas TrasmUM. O teu objetivo é produzir um parágrafo conciso e formal que fundamente a utilização desta técnica de elicitação, servindo de enquadramento direto para a figura que apresenta as questões dirigidas aos operadores de caixa. O texto deve ser redigido em português de Portugal, adotando um registo académico e objetivo, sem linguagem excessivamente complexa ou floreios narrativos. Deves explicar que a aplicação de questionários estruturados visou a recolha sistemática de dados junto dos utilizadores da linha da frente, com o propósito de identificar as dificuldades críticas no atual registo manual de vendas e quantificar a frequência de erros no apuramento de caixa. É fundamental referir que o objetivo principal desta abordagem foi validar a necessidade de automatização, nomeadamente no que concerne à atualização de stock e à melhoria da eficiência operacional no fecho de expediente.

Fontes Fornecidas	Relatório do trabalho prático (versão atual).
Reflexão Final	O output inicial apresentava uma linguagem demasiado pesada e complexa para o registo pretendido nesta subsecção. Foi solicitada uma nova geração com instruções explícitas para adotar uma linguagem mais acessível, o que produziu o resultado posteriormente incluído no relatório.
↑ Voltar à secção	

Prompt 12: Criação da subsecção dos Questionários

Descrição	Simulação de uma entrevista semiestruturada com Hélder Cerqueira, onde a LLM assumiu o papel do entrevistado para ajudar a estruturar e clarificar os requisitos do sistema de forma iterativa e alinhada com o enunciado.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Com base no enunciado e no trabalho que temos vindo a desenvolver, queremos realizar a entrevista com o Hélder. Partimos de alguns bullet points iniciais definidos internamente e, a partir daí, pedimos que ajudes a estruturar e clarificar os requisitos, sempre alinhados com o enunciado e com o objetivo de manter o sistema o mais simples possível, sem o complicar desnecessariamente. Vou fazer perguntas como se estivéssemos na entrevista e tu respondes como se fosses o Hélder, assumindo que muitos dos requisitos foram sendo esclarecidos por ele ao longo da conversa. Quero respostas relativamente curtas, num tom natural e descontraído, em português de Portugal, e sempre que fizer sentido podes referir os requisitos como tendo sido indicados ou confirmados por ele.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); apontamentos internos da equipa referentes às funcionalidades pretendidas para o sistema.
Reflexão Final	O output consistiu em várias respostas às questões colocadas pela equipa ao longo de uma sessão iterativa, estando toda a informação detalhada no anexo Entrevista do Hélder . Não foram necessárias alterações ao prompt, tendo o modelo assumido o papel do entrevistado de forma adequada e coerente com o contexto do projeto.
↑ Voltar à secção	

Prompt 13: Criação da Entrevista com Hélder Cerqueira

Descrição	Geração do texto da subsecção relativa às entrevistas realizadas, descrevendo sumariamente as duas entrevistas conduzidas com Hélder e Cátia no âmbito do processo de elicitação de requisitos.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Tendo em conta o presente relatório, elabora um texto com estrutura semelhante aos restantes desta secção, relativo às entrevistas realizadas. Foram realizadas duas entrevistas: uma com o Hélder, já disponível em anexo, e outra com a Cátia, a incluir posteriormente. Usa linguagem de português de Portugal e um registo formal e conciso, coerente com o restante documento.
Fontes Fornecidas	Relatório do trabalho prático (versão atual).
Reflexão Final	O output foi gerado de forma adequada e sem necessidade de alterações de fundo, cumprindo os objetivos de extensão e registo pretendidos.
↑ Voltar à secção	

Prompt 14: Criação da secção de Entrevistas

Descrição	Geração do texto da subsecção de Análise Documental, com base na análise de imagens de uma fatura e de uma guia de remessa reais fornecidas como contexto ao modelo.
LLM Utilizado	Google Gemini 2.5 Pro (mesmo chat utilizado para a subsecção dos questionários)

Prompt	Elabora agora a parte relativa à análise documental. Vou fornecer duas imagens de documentos reais - uma fatura e uma guia de remessa -, que servirão de base para a redação desta subsecção.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); imagem de uma fatura; imagem de uma guia de remessa.
Reflexão Final	Optou-se por manter o mesmo chat da subsecção dos questionários, aproveitando o contexto acumulado sobre o projeto e o registo de escrita já estabelecido, o que permitiu garantir coerência de linguagem sem necessidade de re-contextualização. O output não necessitou de alterações relevantes.
↑ Voltar à secção	

Prompt 15: Criação da subsecção da Análise Documental

Descrição	Geração de uma narrativa na primeira pessoa de um operador de caixa TrasmUM durante a hora de ponta, identificando problemas operacionais e as funcionalidades tecnológicas que os resolveriam, seguida da sua transcrição para o formato de user stories de Sommerville.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Age como uma operadora de caixa de uma das lojas TrasmUM. Escreve uma narrativa na primeira pessoa (cerca de 3 a 4 parágrafos) a descrever o stress de atender clientes durante a hora de ponta. Na tua história, deves expressar os seguintes problemas e os recursos tecnológicos que te salvariam o dia: o stress de errar nos cálculos mentais quando os clientes pagam em notas, precisando que o sistema calcule o troco automaticamente; o tempo perdido a procurar artigos no armazém ou em catálogos de papel, exigindo uma barra de pesquisa no sistema que encontre qualquer um dos 2.000 produtos em menos de 1 segundo; e as perguntas constantes dos clientes sobre validade e disponibilidade, destacando a necessidade de o sistema mostrar imediatamente no ecrã o preço atualizado, o stock atual e o prazo de validade assim que o artigo é registado na venda. De seguida, e separadamente, transcreve os pontos identificados para o método de user stories de Sommerville, no formato: Como [papel], quero [funcionalidade], para que [benefício].
Fontes Fornecidas	Relatório do trabalho prático (versão atual); apontamentos internos da equipa referentes às funcionalidades pretendidas para o sistema.
Reflexão Final	Foram necessárias algumas iterações até obter uma narrativa com o registo e o nível de detalhe pretendidos. O output final foi utilizado na íntegra, sem qualquer edição ou adaptação.
↑ Voltar à secção	

Prompt 16: Criação da User Story do Funcionário

Descrição	Geração de uma narrativa na primeira pessoa da administradora Cátia a descrever um dia típico de gestão e as dificuldades com o sistema atual, identificando necessidades empresariais concretas e transpondo-as para o formato de user stories de Sommerville.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Age como a Cátia das Unhas Pintadas, a administradora de uma rede de lojas chamada TrasmUM. Escreve uma narrativa na primeira pessoa (cerca de 3 a 4 parágrafos) a descrever um dia típico de gestão e as dificuldades que enfrentas com o sistema atual. Na tua história, deves mencionar de forma natural e contextualizada as seguintes necessidades: a dificuldade em saber quem fez o quê na caixa, expressando a necessidade de autenticação individual por turno; a frustração de não conseguires comunicar rapidamente com a equipa, desejando um sistema de envio de anúncios para os ecrãs das lojas; o caos na gestão de fornecedores, pedindo uma listagem unificada de pagamentos pendentes; o desperdício por falta de visibilidade do inventário, necessitando de relatórios de produtos em excesso ou em risco de rutura; e uma forte preocupação com a segurança dos dados financeiros, exigindo controlo de acessos por perfil. De seguida, e separadamente, transcreve os pontos identificados para o método de user stories de Sommerville, no formato: Como [papel], quero [funcionalidade], para que [benefício].
Fontes Fornecidas	Relatório do trabalho prático (versão atual); apontamentos internos da equipa referentes às funcionalidades pretendidas para o sistema.

Reflexão Final	O output foi adequado na primeira tentativa e utilizado na íntegra, sem necessidade de edições ou iterações adicionais.
↑ Voltar à secção	

Prompt 17: Criação da User Story da Cátia

Descrição	Geração da secção 3.2 — Levantamento de Requisitos, introduzindo e contextualizando o conjunto de requisitos recolhidos através dos métodos de elicitação descritos, e antecedendo a apresentação visual de uma seleção de requisitos e a lista completa em anexo.
LLM Utilizado	Google Gemini 2.5 Pro (mesmo chat utilizado para as subsecções anteriores de metodologia de levantamento)
Prompt	Elabora a Secção 3.2 - Levantamento de Requisitos. Nesta secção será incluída uma captura de ecrã com alguns dos requisitos identificados e, em anexo, a lista completa dos requisitos obtidos através dos métodos anteriormente mencionados. Para cada requisito são indicados o próprio requisito, a respetiva área e a fonte de onde foi obtido. A secção não precisa de ser muito extensa. Utiliza uma linguagem coerente com a do restante relatório, em português de Portugal.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); ficheiro Excel com os requisitos levantados.
Reflexão Final	Optou-se por manter o mesmo chat das subsecções anteriores de metodologia de levantamento, pelas mesmas razões de coerência de contexto e registo já descritas. O output necessitou apenas de uma correção de conteúdo: a remoção da referência à observação etnográfica enquanto método de elicitação, dado que esta técnica foi abandonada no decorrer do projeto.
↑ Voltar à secção	

Prompt 18: Criação da secção de Requisitos Levantados

Descrição	Geração da secção — Refinamento de Requisitos Ambíguos, identificando os requisitos com formulações críticas para a estabilidade arquitetural do sistema e apresentando a sua versão refinada em tabelas Typst. Inclui ainda o refinamento de uma ambiguidade adicional identificada pela equipa numa fase posterior de revisão.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico especializado em Engenharia de Requisitos. O teu objetivo é redigir a secção 3.3, referente ao Refinamento de Requisitos Ambíguos, para o relatório académico do Sistema de Gestão Integrada da rede de lojas TrasmUM. O texto deve ser produzido estritamente em português de Portugal, adotando um registo académico, formal e objetivo, em total coerência com a linguagem utilizada nas secções anteriores. Inicia com um parágrafo introdutório conciso que explique que, devido à natureza da linguagem natural e à complexidade do domínio do negócio, alguns requisitos iniciais apresentavam formulações suscetíveis de interpretações múltiplas, exigindo refinamento técnico antes da sua especificação formal. Analisa de forma autónoma a lista de requisitos em bruto fornecida e seleciona entre três a quatro exemplos cuja ambiguidade seja genuinamente crítica para a viabilidade e estabilidade arquitetural. Para cada um, segue a estrutura dicotômica de Ambiguidade Identificada e Correção Elaborada. Algumas observações: no requisito da devolução, o sistema deve verificar a validade do lote devolvido - se estiver dentro do prazo, repõe o produto no stock para revenda; caso contrário, descarta-o. A decisão de aceitar ou recusar a devolução é da exclusiva responsabilidade do funcionário. No requisito de entrada de remessa, as unidades já existentes em stock devem manter as suas validades originais, não sendo substituídas pela validade do novo lote recebido.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); lista completa de requisitos levantados.
Reflexão Final	O output inicial cumpriu os objetivos pretendidos, necessitando apenas de um ajuste relativo ao RF06, decorrente de uma clarificação posterior da equipa quanto ao comportamento esperado na gestão de lotes — as unidades já existentes em stock devem manter as suas validades originais. Foi igualmente solicitada a reformatação de todo o conteúdo gerado para tabelas Typst, seguindo a estrutura de formatação definida pela equipa. Numa fase de revisão posterior, a equipa identificou autonomamente uma ambiguidade crítica adicional no RF15 (Fecho de Caixa e Envio de Dados), não detetada pelo modelo, relativa ao comportamento em caso de falha no envio — o fecho só

Prompt 19: Criação da Secção de Refinamento de Requisitos Ambíguos

Descrição	Geração da secção de Requisitos Funcionais, transformando os requisitos em bruto numa especificação formal estruturada com Requisito de Utilizador, Área do Sistema, Requisitos de Sistema e Relevância para a Aplicação, em conformidade com a metodologia de Sommerville (2015).										
LLM Utilizado	Google Gemini 2.5 Pro										
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico especializado em Engenharia de Requisitos, encarregue de redigir a secção de Requisitos Funcionais para o relatório académico do Sistema de Gestão Integrada da rede de lojas TrasmUM. O teu objetivo é processar um conjunto de requisitos em bruto e transformá-los numa especificação formal, clara e exhaustiva. Todo o texto deve cumprir as normas da língua portuguesa de Portugal, adotando um registo académico, formal e objetivo. A elaboração deve fundamentar-se na metodologia de Ian Sommerville, <i>Software Engineering</i>, 2015, garantindo distinção clara entre Requisitos de Utilizador (abstratos, compreensíveis para não técnicos) e Requisitos de Sistema (técnicos e detalhados, orientados para a equipa de desenvolvimento). Para cada requisito funcional, redige uma breve descrição introdutória em texto corrido e gera de seguida uma tabela Typst com quatro linhas: Requisito de Utilizador, Área do Sistema (Loja ou Servidor Central), Requisitos de Sistema (afirmações atómicas e numeradas) e Relevância para a Aplicação. Não deves inventar dados ou integrações externas não previstas no contexto académico. Utiliza exatamente o seguinte formato Typst para cada tabela: <table><tr><td>Requisito de Utilizador</td><td>Descrição abstrata</td></tr><tr><td>Fonte</td><td>Fonte de onde se extraiu o requisito</td></tr><tr><td>Analista</td><td>Analista que extraiu o requisito</td></tr><tr><td>Requisitos de Sistema</td><td>1. Primeiro requisito atómico 2. Segundo 3. Terceiro</td></tr><tr><td>Relevância para a Aplicação</td><td>Justificação estratégica</td></tr></table> Tabela 125: Especificação do Requisito Funcional Os requisitos a processar são os seguintes (lista completa de RF01 a RF26, conforme especificado no relatório).	Requisito de Utilizador	Descrição abstrata	Fonte	Fonte de onde se extraiu o requisito	Analista	Analista que extraiu o requisito	Requisitos de Sistema	1. Primeiro requisito atómico 2. Segundo 3. Terceiro	Relevância para a Aplicação	Justificação estratégica
Requisito de Utilizador	Descrição abstrata										
Fonte	Fonte de onde se extraiu o requisito										
Analista	Analista que extraiu o requisito										
Requisitos de Sistema	1. Primeiro requisito atómico 2. Segundo 3. Terceiro										
Relevância para a Aplicação	Justificação estratégica										
Fontes Fornecidas	Relatório do trabalho prático (versão atual); Sommerville, I. — <i>Software Engineering</i> (2015).										
Reflexão Final	O output foi gerado de forma estruturada e sem necessidade de alterações de fundo, cumprindo os objetivos de especificação formal pretendidos.										

[↑ Voltar à secção](#)

Prompt 20: Criação dos Requisitos Funcionais

Descrição	Geração da secção de Requisitos Não Funcionais, transformando os vinte requisitos em bruto numa especificação formal com classificação taxonómica segundo Sommerville (2015), área do sistema e relevância estratégica para o projeto TrasmUM.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico especializado em Engenharia de Requisitos, encarregue de redigir a secção de Requisitos Não Funcionais para o relatório académico do Sistema de Gestão Integrada da rede de lojas TrasmUM. O teu objetivo é processar vinte requisitos não funcionais e transformá-los numa especificação formal, clara e exhaustiva, em português de Portugal, com registo académico, formal e objetivo. A elaboração deve fundamentar-se na metodologia de Sommerville (2015), segundo a qual os requisitos não funcionais descrevem propriedades emergentes do sistema ou restrições sobre a sua implementação.

	Para cada requisito, segue estritamente os seguintes passos: redige um parágrafo introdutório em texto corrido que classifique o requisito segundo a taxonomia de Sommerville e explique o seu propósito técnico; gera de seguida uma tabela Typst com três linhas - Requisito Não Funcional (texto exato do requisito), Área do Sistema (Loja, Servidor Central ou Global/ Ambas) e Relevância para a Aplicação. Utiliza exatamente o seguinte formato: <table><tr><td>Requisito Não Funcional</td><td>Texto exato do requisito</td></tr><tr><td>Área do Sistema</td><td>Área correspondente</td></tr><tr><td>Relevância para a Aplicação</td><td>Justificação estratégica</td></tr></table> Tabela 126: Especificação do Requisito Não Funcional Os vinte requisitos não funcionais a processar abrangem desempenho, fiabilidade, segurança, usabilidade, escalabilidade e operação autónoma da loja, conforme especificado no relatório.	Requisito Não Funcional	Texto exato do requisito	Área do Sistema	Área correspondente	Relevância para a Aplicação	Justificação estratégica
Requisito Não Funcional	Texto exato do requisito						
Área do Sistema	Área correspondente						
Relevância para a Aplicação	Justificação estratégica						
Fontes Fornecidas	Relatório do trabalho prático (versão atual); Sommerville, I. — <i>Software Engineering</i> (2015).						
Reflexão Final	O output foi gerado de forma estruturada e sem necessidade de alterações de fundo.						
↑ Voltar à secção							

Prompt 21: Criação dos Requisitos Não Funcionais

Descrição	Geração da secção de Casos de Uso, cobrindo todas as interações do sistema com os atores Operador de Loja e Administração, com especificação em tabelas Typst incluindo fluxos normais e de exceção.														
LLM Utilizado	Claude Sonnet 4.6														
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico especializado em Engenharia de Software. O teu objetivo é redigir a secção de Casos de Uso para o relatório académico do Sistema de Gestão Integrada da rede de lojas TrasmUM. O texto deve ser redigido estritamente em português de Portugal, adotando um registo académico, formal e muito conciso. A secção deve iniciar com uma breve introdução que enquadre o propósito dos casos de uso na transposição dos requisitos para interações concretas. Analisa a totalidade dos requisitos funcionais e gera os casos de uso que cubram todas as interações com os dois atores definidos: Operador de Loja e Administração. Agrega interações logicamente relacionadas num mesmo caso de uso sempre que aplicável. Para cada caso de uso, redige um curto parágrafo identificando os requisitos funcionais de origem e justificando as opções tomadas. Gera a especificação em tabelas Typst com o seguinte formato exato: <table><tr><td>Caso de uso</td><td>Registar Venda</td></tr><tr><td>Ator</td><td>Operador de Loja</td></tr><tr><td>Descrição</td><td>...</td></tr><tr><td>Pré-condição</td><td>...</td></tr><tr><td>Pós Condição</td><td>...</td></tr><tr><td>Fluxo normal</td><td>...</td></tr><tr><td>Fluxos de exceção</td><td>...</td></tr></table> Tabela 127: Caso de uso Registar Venda Garante que fluxos de exceção referenciam explicitamente o passo de divergência e indicam se há ou não retorno ao fluxo normal.	Caso de uso	Registar Venda	Ator	Operador de Loja	Descrição	...	Pré-condição	...	Pós Condição	...	Fluxo normal	...	Fluxos de exceção	...
Caso de uso	Registar Venda														
Ator	Operador de Loja														
Descrição	...														
Pré-condição	...														
Pós Condição	...														
Fluxo normal	...														
Fluxos de exceção	...														
Fontes Fornecidas	Relatório do trabalho prático (versão atual); lista de requisitos extraídos.														
Reflexão Final	O output inicial apresentou uma estrutura de casos de uso diferente da habitualmente utilizada pela equipa, tendo sido necessário reformular vários aspetos dos casos de uso gerados, tanto ao nível da estrutura como do conteúdo. Foram feitas diversas correções manuais para alinhar o resultado com as convenções do relatório.														
↑ Voltar à secção															

Prompt 22: Criação dos Casos de Uso

Descrição	Geração do código PlantUML para os diagramas de casos de uso, organizados por área operacional em quatro diagramas distintos e um diagrama global, com base nos catorze casos de uso especificados no relatório.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Tendo em conta os catorze casos de uso especificados no relatório do sistema TrasmUM, gera o código PlantUML para os respetivos diagramas de casos de uso. Organiza os diagramas por área operacional, criando quatro diagramas distintos e um diagrama global. As áreas operacionais são: Terminal de Loja - Operação Diária; Gestão de Inventário e Fornecedores; Gestão Administrativa; e Servidor Central - Supervisão Estratégica. Os atores do sistema são três: Funcionário, manager e CEO. A relação de generalização entre atores é a seguinte: manager herda de Funcionário, e CEO herda de manager. Usa obrigatoriamente stickmen (skinparam actorStyle stick). Os nomes dos casos de uso devem ser exatamente os mesmos que constam no relatório. No diagrama global, representa as duas fronteiras de sistema (Terminal de Loja e Servidor Central) e todos os catorze casos de uso distribuídos pelas respetivas fronteiras.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); casos de uso especificados na secção 3.6.
Reflexão Final	O output foi gerado de forma estruturada e coerente com os casos de uso definidos. Foram realizados três ajustes manuais: correção dos nomes dos casos de uso para correspondência exata com os do relatório, revisão da distribuição dos casos de uso pelos diagramas por área operacional, e adição do diagrama global após verificação da distribuição correta pelas duas fronteiras de sistema.
↑ Voltar à secção	

Prompt 23: Criação dos Diagramas de Casos de Uso em PlantUML

Descrição	Geração do texto introdutório e das descrições contextuais de cada diagrama de casos de uso, complementando a especificação textual existente na secção 3.6 e justificando as opções de decomposição por área operacional.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Tendo em conta os diagramas de casos de uso gerados para o sistema TrasmUM, organizados em quatro áreas operacionais distintas mais um diagrama global, redige o texto da secção relativa a esses diagramas. O texto deve ser redigido estritamente em português de Portugal, com registo académico, formal e conciso. A secção deve conter: uma introdução que justifique a utilização de diagramas de casos de uso como complemento à especificação textual, inspirada na perspetiva de Sommerville sobre o papel dos use cases na engenharia de requisitos; uma justificação para a opção de separar os diagramas por área operacional, relacionando esta decisão com a arquitetura federada do sistema TrasmUM e com a hierarquia de perfis de utilizador; uma pequena descrição contextual de cada um dos quatro diagramas por área operacional; e uma descrição do diagrama global, explicando o seu propósito complementar e utilidade como instrumento de validação com os stakeholders. Não incluas os próprios diagramas no texto, apenas os marcadores de posição para as figuras.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); Sommerville, I. — <i>Software Engineering</i> (2015), secção 4.4.3; diagramas PlantUML gerados no prompt anterior.
Reflexão Final	O output foi revisto e ajustado em três aspetos: garantia de consistência de linguagem com o restante relatório, acrescento de referências explícitas à arquitetura federada do TrasmUM e à hierarquia de atores para reforçar a justificação da decomposição, e reformulação da descrição do diagrama global para clarificar o seu papel complementar e não substitutivo face aos diagramas individuais.
↑ Voltar à secção	

Prompt 24: Redação do Texto da Secção de Diagramas de Casos de Uso

Descrição	Geração de cinco diagramas de máquinas de estado em plantUml, cobrindo os ciclos de vida da sessão de utilizador, da transação de venda, do lote de produto, do aprovisionamento e tesouraria, e do fecho de dia e sincronização.
LLM Utilizado	Google Gemini 2.5 Pro
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico especializado em Engenharia de Software. O teu objetivo é redigir a secção referente aos Diagramas de Máquinas de Estado para o relatório académico do Sistema de Gestão Integrada da rede de lojas TrasmUM. O texto deve ser produzido estritamente em português de Portugal, adotando um registo académico,

	formal e objetivo. O formato de saída deve apresentar o código de cada diagrama em sintaxe plantUml, seguido imediatamente por um parágrafo explicativo. Deves elaborar exatamente cinco diagramas: o primeiro representa o ciclo de vida da sessão do utilizador no terminal da loja (terminal inativo, sessão ativa, conta bloqueada); o segundo ilustra o processo transacional de uma venda (iniciação, em curso, concluída ou anulada); o terceiro modela o ciclo de vida unitário do lote de um produto (criação, disponível, em risco de expiração, esgotado ou expirado); o quarto unifica o processo de aprovisionamento e tesouraria (pedido de remessa, receção confirmada, pagamento liquidado); e o quinto aborda o ciclo de vida do Fecho de Dia e Sincronização (caixa aberta, fecho pendente, jornada encerrada e sincronizada). Para cada diagrama, redige uma explicação em texto corrido de aproximadamente cinco linhas, justificando as transições de estado com base nas restrições operacionais e regras de negócio da TrasmUM. Não utilizes listas na tua resposta.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); enunciado do trabalho prático.
Reflexão Final	Foram necessárias várias iterações para afinar os diagramas gerados. As descrições de cada diagrama foram revistas e melhoradas, e foram adicionados estados e transições mais específicos para aumentar o detalhe e a precisão da modelação. Foi ainda adicionado um diagrama adicional referente ao ciclo de vida do fecho de dia, de forma a contemplar a distinção entre a perspetiva da loja e do servidor central na arquitetura do sistema.
↑ Voltar à secção	

Prompt 25: Construção de Diagramas de Máquina de Estado

Descrição	Análise de padrões e estilos arquiteturais adequados ao sistema TrasmUM, com identificação de vantagens, desvantagens e adequação ao contexto específico do projeto, organizada por nível de abstração: estilos de distribuição global, padrões de organização interna e padrões de gestão de dados.
LLM Utilizado	Claude Sonnet 4.5
Prompt	Tens em anexo o relatório da primeira fase do projeto TrasmUM, desenvolvido no âmbito da unidade curricular de Laboratórios de Informática IV da Universidade do Minho. Este relatório contém a definição do domínio do sistema, a identificação de stakeholders, a análise de contexto e restrições, os requisitos funcionais e não funcionais, os casos de uso e os diagramas de máquinas de estado. Estamos agora a iniciar a segunda fase do projeto, focada na Arquitetura e Design do Software. Precisamos da tua ajuda como assistente de análise arquitetural. Com base na leitura integral do relatório, identifica os padrões e estilos arquiteturais que melhor se adequam a este sistema, tendo em conta os seguintes constrangimentos fundamentais: operação offline-first obrigatória em cada loja durante o horário de funcionamento; comunicação unidirecional e exclusivamente no momento do fecho de dia; três lojas geograficamente dispersas no distrito de Bragança; equipa de quatro estudantes com três semanas de implementação disponíveis. Para cada alternativa arquitetural que identificares, apresenta o nome e descrição do padrão, como se adequa concretamente ao contexto do TrasmUM, as vantagens para este sistema específico, e as desvantagens ou riscos para este sistema específico. Não tomes decisões por nós - o objetivo é ter uma análise fundamentada que nos permita tomar uma decisão informada.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); enunciado do trabalho prático.
Reflexão Final	A resposta do modelo foi organizada em três níveis de abstração — distribuição global, organização interna e dados — o que se revelou uma estrutura adequada para a tomada de decisão. O sumário comparativo final e as questões em aberto identificadas pelo modelo foram particularmente úteis, tendo orientado diretamente as decisões tomadas pela equipa e conduzido à formulação do Prompt 26.
↑ Voltar à secção	

Prompt 26: Exploração de Alternativas Arquiteturais

Descrição	Concretização técnica das decisões arquiteturais preliminares da equipa, cobrindo a organização das camadas de cada um dos dois softwares, a estrutura de packages, a integração do servidor HTTP Javalin no Software de Servidor Central, os comportamentos reativos no Software de Loja, e o mecanismo de Delta Sync em MySQL.
LLM Utilizado	Claude Sonnet 4.5
Prompt	Com base na análise anterior, a equipa tomou as seguintes decisões preliminares: linguagem Java para ambos os softwares; padrão de distribuição global de arquitetura federada com cliente-servidor assimétrico; padrão de organização interna de Layered Architecture com MVC; padrão de dados de Offline-First com Delta Sync; base de dados MySQL em todos os terminais de loja e no Servidor Central. Decisão crítica: o sistema é composto por dois softwares distintos e independentes. O Software de Loja é instalado em todos os terminais da rede e é responsável pela interface de caixa, gestão de stock e lotes, remessas, catálogo, utilizadores, base de dados local MySQL, e fecho de dia com envio HTTP para o Servidor Central. O Software de Servidor Central é instalado exclusivamente na máquina que serve de servidor central da rede, sendo responsável pelo servidor HTTP que recebe os pacotes de fecho de dia, base de dados central agregada MySQL, dashboards e relatórios analíticos, e consulta consolidada de remessas e pagamentos. Com isto em mente, responde às seguintes questões: como se organizam as camadas de cada um dos dois softwares em Java; que estrutura de packages sugeres para cada projeto de forma a garantir separação clara de responsabilidades; para o servidor HTTP do Software de Servidor Central, que biblioteca Java leve recomenda e como se integra no ciclo de vida da aplicação; como implementas os comportamentos reativos de alertas de stock mínimo, alertas de validade e monitorização de caixa dentro da Layered Architecture no Software de Loja sem introduzir Event-Driven completo; e como estruturas o mecanismo de Delta Sync em Java e o que precisa de existir na base de dados MySQL local para que o terminal saiba exatamente o que ainda não foi enviado para o Servidor Central. Mantém as respostas práticas e adequadas a uma equipa de quatro estudantes com três semanas de implementação.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); enunciado do trabalho prático.
Reflexão Final	A resposta do modelo forneceu um nível de detalhe técnico elevado e diretamente utilizável na fase de implementação. A estrutura de packages proposta para cada software foi adotada com ajustes menores. A nota final do modelo sobre a sequência recomendada de implementação — modelos, repositórios, serviços, controllers e vistas — foi incorporada no plano de trabalho da equipa. A revisão crítica posterior (Prompt 27) identificou uma lacuna no mecanismo de Delta Sync proposto, que conduziu à adoção de idempotência por chave composta ao nível do registo individual, conforme documentado no ADR-05. A tecnologia de persistência foi posteriormente atualizada de SQLite para MySQL, refletindo a necessidade de suporte nativo a acesso concorrente em rede por múltiplos terminais, o que implicou a revisão do prompt original e dos ADRs correspondentes.

[↑ Voltar à secção](#)

Prompt 27: Aprofundamento Técnico da Arquitetura Escolhida

Descrição	Identificação de vulnerabilidades, cenários de falha não cobertos e decisões com potencial impacto negativo na implementação ou nos testes, com base na arquitetura final definida pela equipa, organizada por severidade.
LLM Utilizado	Claude Sonnet 4.5
Prompt	Com base em toda a análise anterior, a equipa tomou as seguintes decisões arquiteturais finais para o sistema TrasmUM: dois softwares distintos e independentes – Software de Loja e Software de Servidor Central; Layered Architecture com MVC com camadas de Apresentação, Serviço, Repositório e Dados em ambos os softwares; servidor HTTP do Software de Servidor Central implementado com Javalin; Delta Sync com campo sincronizado em todas as tabelas operacionais do Software de Loja; idempotência garantida por chave composta loja_id mais id_registo nas tabelas centrais; e comportamentos reativos implementados com JavaFX Properties e Bindings nativos no Software de Loja. Agora precisamos que atues como arquiteto revisor crítico. A tua tarefa é identificar os pontos fracos, riscos e potenciais falhas desta arquitetura no contexto específico do TrasmUM. Em particular, analisa se existem cenários de falha que a arquitetura atual não cobre adequadamente; se existem decisões que podem criar problemas na fase de implementação ou de testes; se existem requisitos funcionais ou não funcionais do relatório original que esta arquitetura não satisfaz completamente; e o que mudarias ou reforçarias se tivesses de defender esta arquitetura perante um cliente real.

	Sê honesto e direto - o objetivo é melhorar a arquitetura antes de começar a implementação, não validá-la.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); enunciado do trabalho prático.
Reflexão Final	Esta foi a interação mais valiosa do processo arquitetural. O modelo identificou vulnerabilidades organizadas por severidade, das quais duas conduziram a ajustes formais na arquitetura: a adoção de idempotência por chave composta ao nível do registo individual (ADR-05) e a adoção de três estados de sincronização em vez de campo binário (ADR-04). Os restantes pontos foram documentados como limitações conhecidas ou trabalho futuro. A organização da resposta por severidade revelou-se um formato particularmente útil para priorizar decisões com impacto imediato na implementação.
↑ Voltar à secção	

Prompt 28: Revisão Crítica da Arquitetura Final

Descrição	Redação da subsecção que apresenta as alternativas arquiteturais analisadas para o sistema TrasmUM, organizadas por nível de abstração: estilos de distribuição global, padrões de organização interna e padrões de gestão de dados.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Conversa conduzida no chat de acompanhamento do projeto. Com base na análise produzida no Prompt 25 (Exploração de Alternativas Arquiteturais), foi solicitada a redação da subsecção do relatório, apresentando as alternativas identificadas em linguagem académica formal e em português de Portugal, mantendo coerência com o estilo e registo do Capítulo 1.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); output do Prompt 25.
Reflexão Final	O texto gerado foi aceite com alterações menores de adaptação ao contexto específico do projeto. A organização por níveis de abstração proposta na análise original foi mantida, por se revelar uma estrutura clara e pedagogicamente adequada para apresentar a diversidade de padrões considerados.
↑ Voltar à secção	

Prompt 29: Redação da Secção — Alternativas Arquiteturais Identificadas

Descrição	Redação da subsecção que descreve a decisão arquitetural central de desenvolver dois softwares distintos e independentes — o Software de Loja e o Software de Servidor Central — incluindo as responsabilidades de cada um, as vantagens desta separação e o custo adicional de manutenção de dois projetos distintos.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Conversa conduzida no chat de acompanhamento do projeto. Foi solicitada a redação da subsecção do relatório, descrevendo em detalhe a estrutura dos dois softwares e justificando a opção por dois projetos Java distintos em vez de uma única aplicação com módulos, em linguagem académica formal e em português de Portugal.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); output do Prompt 26; decisões arquiteturais da equipa.
Reflexão Final	O texto gerado foi aceite sem alterações significativas. A justificação da separação física entre os dois softwares — eliminação do risco de acesso indevido a funcionalidades do Servidor Central a partir de um terminal de loja, e independência dos ciclos de vida e deployment — foi considerada clara e adequada para comunicar a decisão a leitores com formação técnica. O custo adicional de manutenção de dois projetos foi mantido como desvantagem explícita, por demonstrar maturidade crítica na análise da decisão.
↑ Voltar à secção	

Prompt 30: Redação da Secção — Estrutura dos Softwares

Descrição	Redação dos oito registos de decisão arquitetural que documentam formalmente as decisões estruturais do sistema TrasmUM, cobrindo a arquitetura de distribuição global, a separação em dois softwares distintos, a base de dados
------------------	--

	MySQL partilhada por loja, o mecanismo de Delta Sync com três estados, a idempotência por chave composta, o bootstrapping de utilizadores, os logs de auditoria no pacote de fecho, e a ausência de cifra na comunicação HTTP.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Conversa conduzida no chat de acompanhamento do projeto. Com base no conjunto completo de decisões arquiteturais tomadas pela equipa ao longo da conversa – incluindo as que resultaram da revisão crítica do Prompt 27 – foi solicitada a redação de oito ADRs no formato estruturado: contexto, decisão, alternativas consideradas, justificação e consequências, adaptados ao formato de tabela Typst utilizado no relatório.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); outputs dos Prompts 25, 26 e 27; decisões arquiteturais da equipa.
Reflexão Final	Os ADRs foram gerados diretamente no formato Typst solicitado, o que eliminou trabalho de formatação posterior. O conteúdo de cada ADR foi revisto pela equipa para garantir coerência entre si — nomeadamente as referências cruzadas entre ADR-04 e ADR-05, que documentam decisões interdependentes. O ADR-02 foi posteriormente reescrito para refletir a decisão final de dois softwares distintos, em substituição da abordagem inicial de módulo único que tinha sido considerada mas abandonada. O ADR-03 foi atualizado para refletir a substituição de SQLite por MySQL como tecnologia de persistência, documentando a arquitetura cliente-servidor nativa do MySQL como justificação para o suporte robusto ao acesso concorrente de múltiplos terminais via rede local. A referência a INSERT OR IGNORE foi igualmente corrigida para INSERT IGNORE, em conformidade com a sintaxe MySQL.
↑ Voltar à secção	

Prompt 31: Redação dos Registos de Decisão Arquitetural (ADR-01 a ADR-08)

Descrição	Geração de cinco diagramas de casos de uso em PlantUML, organizados por área operacional: operação diária no terminal de loja, gestão de inventário e fornecedores, gestão administrativa, supervisão estratégica no Servidor Central, e visão global do sistema TrasmUM.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Assume o papel de um Analista de Sistemas especializado em Engenharia de Software e modelação UML. O teu objetivo é gerar cinco diagramas de casos de uso em sintaxe PlantUML para o Sistema de Gestão Integrada da rede de lojas TrasmUM, com base nos casos de uso especificados no relatório fornecido. Os diagramas devem ser organizados por área operacional distinta, evitando um diagrama monolítico único. O primeiro diagrama deve representar as funcionalidades disponíveis a todos os perfis de utilizador no terminal local durante o horário de funcionamento, incluindo início de sessão, registo de vendas e devoluções, consulta de produtos e monitorização de caixa. O segundo deve agrupar as funcionalidades de responsabilidade exclusiva do manager ao nível da gestão operacional do estabelecimento, incluindo pedidos de remessa, registo de remessas, gestão de catálogo, documentos de fornecedores, pagamentos pendentes e descontos por validade. O terceiro deve representar as funcionalidades administrativas e de encerramento diário acessíveis ao manager e ao CEO, incluindo gestão de utilizadores e fecho de dia. O quarto deve delimitar as funcionalidades exclusivas do perfil CEO no Software de Servidor Central, incluindo consulta de remessas da rede, dashboards analíticos, início de sessão no Servidor Central e consulta do estado de consolidação da rede. O quinto deve apresentar uma visão global do sistema agregando as duas fronteiras de sistema, os três perfis de utilizador e a totalidade dos casos de uso. Em todos os diagramas, representa a relação de generalização entre atores de forma a refletir a hierarquia operacional da organização: o manager herda todas as funcionalidades do Funcionário, e o CEO herda todas as funcionalidades do manager. Utiliza a sintaxe PlantUML correta para fronteiras de sistema, atores, casos de uso e relações de generalização.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); especificação dos casos de uso.
Reflexão Final	Foram necessárias várias iterações para afinar os diagramas gerados, nomeadamente na representação correta das relações de generalização entre atores e na delimitação das fronteiras de sistema. A decomposição em cinco diagramas por área operacional revelou-se a abordagem mais adequada para garantir legibilidade. O quarto diagrama foi atualizado para refletir a separação em dois softwares distintos, passando a representar o Software de Servidor Central como fronteira própria e incluindo os novos casos de uso de início de sessão e consulta do estado de consolidação da rede.

Prompt 32: Criação dos Diagramas de Casos de Uso (PlantUML)

Descrição	Redação do texto introdutório e dos parágrafos explicativos de cada diagrama de casos de uso, justificando as opções de organização por área operacional e descrevendo o conteúdo e propósito de cada diagrama no contexto do sistema TrasmUM.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Assume o papel de um Analista de Sistemas e Redator Técnico especializado em Engenharia de Software. O teu objetivo é redigir o texto da secção referente aos Diagramas de Casos de Uso para o relatório académico do Sistema de Gestão Integrada da rede de lojas TrasmUM. O texto deve ser produzido estritamente em português de Portugal, adotando um registo académico, formal e objetivo. Deves redigir um parágrafo introdutório que justifique a utilidade da representação visual dos casos de uso como complemento à especificação textual já produzida, com referência à metodologia de Sommerville. De seguida, deves justificar a opção de organizar os diagramas por área operacional distinta em vez de um único diagrama monolítico, explicando como esta decisão decorre da separação em dois softwares distintos e da diferenciação de perfis de utilizador. Por fim, deves redigir um parágrafo introdutório para cada um dos cinco diagramas produzidos, descrevendo o seu conteúdo, o seu propósito e a sua relação com os requisitos e casos de uso especificados anteriormente. O texto deve ser redigido em prosa contínua, sem listas ou bullet points, e cada parágrafo deve preceder imediatamente a figura do diagrama correspondente.
Fontes Fornecidas	Relatório do trabalho prático (versão atual); especificação dos casos de uso; diagramas PlantUML produzidos no Prompt 32.
Reflexão Final	O texto gerado foi aceite com alterações pontuais de adaptação ao contexto específico do projeto. A justificação da decomposição por área operacional foi atualizada para refletir a separação em dois softwares distintos em vez da arquitetura de módulo único anterior. Os parágrafos introdutórios do quarto diagrama foram revistos para descrever corretamente o Software de Servidor Central como software independente com os seus próprios casos de uso de autenticação e consulta de estado de consolidação.
↑ Voltar à secção	

Prompt 33: Redação do Texto da Secção de Diagramas de Casos de Uso

Descrição	Geração do texto da secção 3.9, descrevendo a reunião de validação realizada com a Diretora-Geral da rede TrasmUM e formalizando a aprovação do trabalho desenvolvido na fase de Levantamento e Análise de Requisitos.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Tendo em conta o relatório do projeto TrasmUM, reescreve corretamente a secção 3.9 – Revisão e Aprovação do Projeto, para refletir que foi realizada uma reunião formal com a Cátia das Unhas Pintadas, onde esta aprovou o trabalho desenvolvido na fase de levantamento e análise de requisitos. A secção deve seguir o mesmo padrão e registo da secção 2.9 já presente no relatório.
Fontes Fornecidas	Relatório do trabalho prático (versão atual).
Reflexão Final	O output foi aceite sem alterações de fundo. A estrutura adotada segue o padrão já estabelecido na secção 2.9, garantindo coerência entre as duas fases do projeto.
↑ Voltar à secção	

Prompt 34: Redação da Secção de Revisão e Aprovação do Projeto — Levantamento e Análise de Requisitos

Descrição	Geração de um ficheiro com guidelines de design para realização do esboço do projeto.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Tens acesso ao relatório de requisitos do sistema TrasmUM, um sistema de gestão integrada para uma cadeia de 3 lojas de conveniência no distrito de Bragança. Com base na totalidade dos requisitos funcionais, não funcionais, casos de uso e diagramas de estado descritos no relatório, quero que elabores um documento estruturado de guidelines de design para prototipagem, destinado a ser usado como briefing para uma ferramenta de prototipagem AI

	(Pencil) com o kit de componentes shadcn/ui. O documento deve estar organizado da seguinte forma: 1. Visão Geral do Sistema Arquitetura do sistema (federado/local vs central), perfis de utilizador existentes e hierarquia entre eles, e regras globais de navegação e autenticação. 2. Lista Completa de Ecrãs Para cada ecrã, indica: Nome do ecrã Perfil(is) de utilizador que acedem Objetivo do ecrã Dados exibidos Ações disponíveis Componentes shadcn/ui sugeridos Ecrãs anteriores e posteriores no fluxo de navegação Organiza os ecrãs pelos seguintes contextos: Autenticação Terminal de Loja - Funcionário Terminal de Loja - Manager Loja Central - CEO 3. Fluxos de Navegação Descreve os fluxos principais em formato de lista sequencial de ecrãs, cobrindo obrigatoriamente: Fluxo de autenticação e gestão de sessão Fluxo de registo de venda completo (incluindo desconto de validade e pagamento em numerário) Fluxo de registo de devolução Fluxo de receção de remessa e geração de pagamento pendente Fluxo de fecho de dia (loja comum) Fluxo de fecho de dia (Loja Central) 4. Componentes Globais e Padrões de UI Identifica componentes shadcn/ui reutilizáveis em múltiplos ecrãs, padrões de feedback ao utilizador (erros, alertas, confirmações), e padrões de tabelas, formulários e dashboards. 5. Regras de Negócio Críticas para o UI Lista as regras que têm impacto direto no comportamento da interface, como bloqueio de sessão, lógica FEFO, alertas de stock e validade, e restrições por perfil. Sê exaustivo e preciso. Usa os nomes exatos dos requisitos e casos de uso do relatório. O documento deve ser suficientemente detalhado para que uma ferramenta de prototipagem AI consiga gerar wireframes consistentes sem necessitar de contexto adicional.
Fontes Fornecidas	Relatório do trabalho prático (versão atual).
Reflexão Final	O output foi aceite sem alterações. Descreve bem os ecrãs necessários
↑ Voltar à secção	

Prompt 35: Elaboração do ficheiro de Guidelines

Descrição	Realização do Esboço da Interface através da ferramenta Pencil
LLM Utilizado	Gemini 3 PRO CLI
Prompt	Tens acesso ao servidor MCP do Pencil, a um ficheiro .pen com o kit de componentes shadcn/ui já configurado, e ao documento de design guidelines do sistema TrasmUM. O teu objetivo é criar um protótipo de baixa/média fidelidade do sistema TrasmUM no Pencil, seguindo rigorosamente as guidelines fornecidas. Segue esta ordem de trabalho: 1. Configuração Inicial Abre o ficheiro .pen com o kit shadcn/ui como base Garante que todos os componentes do kit estão acessíveis antes de começar 2. Criação dos Ecrãs Cria cada ecrã descrito nas guidelines, um a um, pela ordem dos contextos definidos Para cada ecrã, usa exclusivamente os componentes shadcn/ui definidos nas guidelines Respeita os dados exibidos, ações disponíveis e perfis de utilizador indicados para cada ecrã Mantém consistência visual entre ecrãs do mesmo contexto 3. Validação Após cada contexto concluído, verifica se os ecrãs criados cobrem todos os casos de uso descritos nas guidelines antes de avançar para o contexto seguinte Se detetares inconsistências entre os ecrãs criados e as guidelines, corrige antes de prosseguir

	Não inventes componentes, dados ou fluxos que não estejam descritos nas guidelines. Em caso de ambiguidade, opta pela solução mais simples e consistente com o resto do protótipo.
Fontes Fornecidas	Design Guidelines estabelecidas em cima
Reflexão Final	O primeiro output foi de boa qualidade, apenas foram realizadas pequenas alterações para alcançar o nosso objetivo.
↑ Voltar à secção	

Prompt 36: Esboço da Interface

Descrição	Geração dos diagramas de classe UML em PlantUML para as quatro camadas do Software de Loja (Domínio, Repositório, Serviços e Apresentação), organizados por módulo funcional, seguida de revisão crítica com pedido explícito de identificação de inconsistências, métodos em falta e violações de princípios de design.
LLM Utilizado	Claude Opus 4
Prompt	Numa primeira fase, foi solicitada a geração dos diagramas de classe em PlantUML para o Software de Loja, refletindo a arquitetura de quatro camadas definida no ADR-09: Domínio, Repositório, Serviços e Apresentação. Foi fornecido ao modelo o relatório completo das fases anteriores como contexto (requisitos funcionais, não funcionais, casos de uso, ADRs e decisões arquiteturais), juntamente com as seguintes especificações: a classe abstrata <code>RegistoSincronizavel</code> encapsula o mecanismo de Delta Sync (ADR-04) com os estados <code>PENDENTE</code>, <code>EM_TRANSITO</code> e <code>CONFIRMADO</code>, sendo estendida pelas classes <code>Venda</code>, <code>Devolucao</code>, <code>Remessa</code>, <code>Pagamento</code>, <code>SessaoCaixa</code>, <code>FechoDia</code> e <code>LogAuditoria</code>; o <code>Lote</code> tem associação unidirecional ao <code>Produto</code>, com <code>Desconto</code> como value object; a <code>Venda</code> é composta por <code>LinhaVenda</code> e pode originar uma <code>Fatura</code>; a <code>SessaoCaixa</code> contém fundo inicial como <code>List<DetalheNumerario></code> e suporta <code>Sangria</code>; o padrão <code>Repository</code> deve ser aplicado com interface e implementação concreta para cada entidade relevante. Os diagramas foram organizados por módulo funcional: <code>Core/Segurança/Auditoria</code>, <code>Catálogo/Inventário/Fornecedores</code>, <code>Vendas/Faturas</code> e <code>Tesouraria/Caixa</code>. Numa segunda fase, os diagramas gerados foram submetidos a revisão crítica. Foram fornecidos ao modelo todos os ficheiros PlantUML produzidos com o pedido de análise de coerência entre camadas, identificação de métodos em falta nos repositórios, validação das relações de herança com <code>RegistoSincronizavel</code>, e avaliação da conformidade com os princípios de inversão de dependência e encapsulamento modular. Foi igualmente solicitada a extração de interfaces para todos os serviços e a avaliação do acoplamento do <code>FechoDiaServico</code> com repositórios de outros módulos.
Fontes Fornecidas	Relatório das fases anteriores (requisitos, casos de uso, máquinas de estado); ADRs 01–10; diagramas PlantUML gerados na primeira fase.
Reflexão Final	A geração inicial produziu diagramas funcionais e bem organizados por módulo, mas a revisão crítica identificou vários problemas que conduziram a correções relevantes. A hierarquia de utilizadores por herança (<code>Funcionário</code> → <code>Manager</code> → <code>CEO</code>) foi substituída por um enum <code>PerfilUtilizador</code> com três valores, simplificando a modelação sem perder expressividade. O <code>FechoDia</code> como agregador de <code>List<RegistoSincronizavel></code> foi substituído por uma entidade leve que regista apenas a data e o utilizador, com a compilação dos pendentes delegada ao <code>FechoDiaServico</code> . O <code>limiteMaximo</code> foi movido de <code>SessaoCaixa</code> para <code>ConfiguracaoTerminal</code> , dado que é um parâmetro da loja e não da sessão individual. A revisão identificou ainda seis correções adicionais: remoção de herança duplicada de <code>Remessa</code> ; adição de <code>RegistoSincronizavel</code> a <code>Pagamento</code> e <code>SessaoCaixa</code> ; adição de <code>atualizar()</code> ao <code>LogAuditoriaRepositorio</code> e de <code>buscarPendentes()</code> ao <code>FechoDiaRepositorio</code> e <code>SessaoCaixaRepositorio</code> ; extração de interfaces para todos os serviços; e refatoração do <code>FechoDiaServico</code> para consumir interfaces de serviço em vez de aceder diretamente a repositórios de outros módulos. Esta última alteração melhorou significativamente a legibilidade do diagrama de componentes, eliminando ligações verticais que cruzavam camadas. Todas as correções foram validadas pela equipa antes de serem aplicadas.
↑ Voltar à secção	

Prompt 37: Geração e Revisão Crítica dos Diagramas de Classe do Software de Loja

Descrição	Geração dos diagramas de classe UML em PlantUML para o Servidor Central, seguida de revisão crítica com as mesmas regras aplicadas ao Software de Loja.
LLM Utilizado	Claude Opus 4

Prompt	Pretende-se que sejam gerados diagramas de classe em PlantUML para o Software de Servidor Central, cobrindo as camadas de Domínio (entidades core, transacionais, VOs de análise e DTOs), Repositório, Serviços e Apresentação (handlers HTTP Javalin). Especificações fornecidas: • A classe Loja é criada automaticamente ao receber o primeiro pacote de uma loja desconhecida. • As entidades transacionais (VendaCentral, DevolucaoCentral, RemessaCentral, etc.) são versões de leitura sem mecanismo de sincronização, com chave composta idLoja + idOriginalLoja para idempotência (ADR-05). • Os repositórios utilizam guardarSeNaoExistir() em vez de guardar(). • O RecepcaoPacoteServico decompõe o PacoteFechoDTO e persiste cada entidade. • Os serviços de análise (DashboardServico, RelatorioServico, etc.) produzem value objects (MetricasGlobais, MetricasLoja, Relatorio). • A autenticação do CEO inclui bloqueio temporário após tentativas falhadas. • Devem ser extraídas interfaces para todos os serviços, com handlers a depender das interfaces. Após a geração, foi solicitada revisão crítica com as mesmas regras do Software de Loja.
Fontes Fornecidas	Relatório completo, ADRs 01–08, Diagramas de classe do Software de Loja (para garantir consistência inter-sistema)
Reflexão Final	A geração inicial produziu uma estrutura sólida mas a revisão identificou várias correções: a entidade SessaoCaixaCentral e o correspondente SessaoCaixaDTO estavam em falta no pacote de transferência e no domínio transacional — foram adicionados para manter consistência com a loja, onde SessaoCaixa herda de RegistoSincronizavel; o AlertaStock foi removido por não existirem dados de stock no servidor (inconsistência de design); FECHO_DIA foi adicionado ao enum TipoAcaoCentral; a composição PedidoFecho *-- FechoDiaCentral foi corrigida para associação simples; e foram adicionadas relações entre enums e as entidades que os utilizam, que estavam em falta. Todas as correções foram validadas pela equipa.
↑ Voltar à secção	

Prompt 38: Geração e Revisão dos Diagramas de Classe do Servidor Central

Descrição	Utilização do LLM como consultor arquitetural para a construção manual dos diagramas de componentes em Visual Paradigm, abrangendo ambos os softwares.
LLM Utilizado	Claude Opus 4
Prompt	Os diagramas de componentes foram construídos manualmente pela equipa em Visual Paradigm. O LLM foi utilizado como consultor através de várias interações focadas em: 1. Definição dos componentes a representar em cada camada e das interfaces expostas (lollipop) e consumidas (socket) por cada um. 2. Estratégia de simplificação para os serviços transversais (AuditoriaServico e AutorizacaoServico), cuja ligação a todos os serviços tornava o diagrama ilegível - resultando na decisão de os omitir e documentar via nota UML. 3. Validação de que o FechoDiaServico refatorado (com dependências horizontais a interfaces de serviço em vez de verticais a repositórios) simplificava o diagrama de componentes. 4. Definição do componente externo Software de Loja no diagrama do Servidor Central e vice-versa (SincronizacaoGateway). 5. Decisão de não representar o Domínio como componentes com interfaces, dado que são classes partilhadas por todas as camadas.
Fontes Fornecidas	Diagramas de classe PlantUML de ambos os softwares, ADRs 01–08
Reflexão Final	A abordagem consultiva revelou-se adequada — o LLM orientou as decisões de layout e simplificação sem produzir o diagrama diretamente, permitindo à equipa manter controlo total sobre a ferramenta e o resultado visual. A principal decisão influenciada por esta interação foi a utilização de notas UML para os serviços transversais, que transformou um diagrama ilegível num artefacto claro e comunicável. A refatoração do FechoDiaServico feita na entrada 38, embora motivada por correção de design, teve como benefício colateral a simplificação do diagrama de componentes.
↑ Voltar à secção	

Prompt 39: Consultoria Arquitetural para Diagramas de Componentes

Descrição	Geração das secções do relatório em Typst para os diagramas de componentes e de classe de ambos os softwares.
LLM Utilizado	Claude Opus 4
Prompt	Solicita-se a geração de quatro secções em Typst para o relatório: 1. Diagrama de componentes do Software de Loja - descrição das camadas, interfaces, regras de dependência e simplificações visuais adotadas, com o diagrama em anexo rotado para landscape. 2. Diagramas de classe do Software de Loja - abordagem bottom-up (Domínio → Repositório → Serviços → Apresentação), com todos os 16 diagramas como figuras, em texto corrido sem subsecções desnecessárias. 3. Diagrama de componentes do Servidor Central - mesma estrutura, destacando as diferenças face à loja. 4. Diagramas de classe do Servidor Central - mesma abordagem bottom-up, com todos os 10 diagramas. Requisitos de escrita: português formal académico coerente com o resto do relatório, sem redundância entre as secções de componentes e classes (os componentes explicam o “porquê” e o “quem depende de quem”, as classes explicam o “o quê”), referências corretas aos ADRs, e utilização da macro \#figure() com os caminhos de imagem indicados.
Fontes Fornecidas	Diagramas de classe e componentes de ambos os softwares, ADRs 01–08, Secções anteriores do relatório (para manter tom e estilo)
Reflexão Final	A primeira geração produziu secções de boa qualidade mas com redundância entre as secções de componentes e de classes — nomeadamente a re-explicação do princípio de inversão de dependência e a descrição detalhada do FechoDiaServico em ambas. Após identificação, o modelo indicou quatro cortes concretos na secção de classes (remoção de frases sobre inversão de dependência já ditas nos componentes e encurtamento do parágrafo do FechoDiaServico com referência cruzada). As secções do Servidor Central foram geradas com instrução explícita de não repetir conceitos já explicados na loja, resultando num texto mais conciso e focado nas diferenças.

[↑ Voltar à secção](#)

Prompt 40: Geração do Texto das Secções de Componentes e Classes

Descrição	Geração iterativa do schema MySQL do Software de Loja, cobrindo todos os módulos funcionais do sistema — Core e Segurança, Catálogo e Inventário, Remessas e Fornecedores, Vendas e Faturas, e Tesouraria e Caixa — com discussão e validação das decisões de design ao longo do processo.
LLM Utilizado	Claude Sonnet 4.6
Prompt	O processo de geração do schema não seguiu um prompt único mas sim uma conversa iterativa estruturada em várias fases. Numa primeira fase, foi solicitada a geração do script SQL completo com base nos diagramas de classe do Software de Loja, nos ADRs e nas decisões arquiteturais documentadas no relatório. Numa segunda fase, foram discutidas e validadas as decisões de design não triviais - nomeadamente a representação de valores enumerados como ENUM do MySQL, a utilização de DECIMAL(10,2) para valores monetários, a dualidade de propósito da tabela DetalheNumerario com duas chaves estrangeiras opcionais, a nulabilidade de idLoteGerado em LinhaRemessa, e a ausência de ConfiguracaoTerminal no schema. Numa terceira fase, foi identificado e corrigido um problema de design relevante: o estado EM_CURSO da entidade Venda não deve ser persistido na base de dados, uma vez que a venda vive em memória durante o atendimento e só é escrita na base de dados no momento da finalização ou anulação. Esta correção conduziu à simplificação do ENUM de estado da Venda para apenas dois valores - FINALIZADA e ANULADA - e à remoção da nulabilidade dos campos metodoPagamento e totalFaturado. Foi igualmente discutida e validada a aplicabilidade desta decisão às restantes entidades do schema, concluindo-se que apenas a Venda requeria correção.
Fontes Fornecidas	Diagramas de classe PlantUML do Software de Loja; ADRs 01–08; decisões arquiteturais documentadas no relatório.
Reflexão Final	O processo iterativo revelou-se mais produtivo do que a geração de um prompt único, permitindo identificar e corrigir problemas de design que não seriam detetados numa geração direta. A decisão mais relevante que emergiu da discussão foi a separação entre estado de aplicação e estado persistido na entidade Venda — uma distinção que tem implicações diretas na simplicidade do schema e na correção dos dados armazenados. O script final foi importado no MySQL Workbench via File → Import → Reverse Engineer MySQL Create Script para geração do diagrama EER.

[↑ Voltar à secção](#)

Prompt 41: Geração do Schema SQL do Software de Loja

Descrição	Geração iterativa do schema MySQL do Software de Servidor Central, cobrindo as entidades de autenticação, as entidades transacionais com idempotência por chave composta, e as tabelas de consolidação de fecho de dia, com discussão e validação das decisões de design específicas do contexto de agregação do servidor.
LLM Utilizado	Claude Sonnet 4.6
Prompt	À semelhança do processo seguido para o Software de Loja, a geração do schema do Servidor Central decorreu de forma iterativa. Numa primeira fase, foi solicitada a geração do script SQL com base nos diagramas de classe do Servidor Central e nas especificidades arquiteturais do ADR-05, nomeadamente a idempotência por chave composta (idLoja, idOriginalLoja) em todas as entidades transacionais e a desnormalização intencional de campos descritivos como nomeProduto, categoria e nomeFornecedor. Numa segunda fase, foram discutidas decisões específicas do contexto do servidor: a nulabilidade de dataPagamento e tipoPagamento em PagamentoCentral, que reflete o facto de um pagamento poder chegar ao servidor ainda pendente de liquidação; e a nulabilidade de dataEncerramento em SessaoCaixaCentral. Esta última decisão foi objeto de discussão mais aprofundada, tendo-se concluído que apenas sessões encerradas devem ser incluídas no pacote de fecho de dia - sessões ainda abertas no momento do fecho ficam em estado PENDENTE na loja e são enviadas no pacote seguinte. Esta invariante garante que o Servidor Central nunca recebe sessões abertas, tornando dataEncerramento um campo obrigatório e permitindo adicionar uma constraint CHECK que garante consistência temporal entre dataAbertura e dataEncerramento.
Fontes Fornecidas	Diagramas de classe PlantUML do Servidor Central; ADRs 01–08; schema SQL do Software de Loja (para garantir consistência inter-sistema).
Reflexão Final	A decisão de design mais relevante que emergiu da discussão foi a invariante das sessões de caixa — a conclusão de que apenas sessões encerradas são enviadas ao servidor não estava explicitamente documentada nos requisitos nem nos ADRs, tendo sido identificada como consequência natural da arquitetura de Delta Sync. Esta decisão foi posteriormente documentada na secção de dicionário de dados como decisão de design explícita, garantindo que a diferença de nulabilidade de dataEncerramento entre os dois schemas não seja interpretada como inconsistência mas como consequência intencional da arquitetura. O script final foi importado no MySQL Workbench pelo mesmo processo utilizado para o Software de Loja.
↑ Voltar à secção	

Prompt 42: Geração do Schema SQL do Software de Servidor Central

Descrição	Geração das secções do relatório em Typst para a conceção do sistema de dados de ambos os softwares, cobrindo o modelo lógico, o dicionário de dados integrado com as decisões de design, e a normalização.
LLM Utilizado	Claude Sonnet 4.6
Prompt	À semelhança do processo seguido para as secções de componentes e classes (Prompt 41), foi solicitada a geração das secções do relatório em Typst para a conceção do sistema de dados. Para o Software de Loja, as secções cobrem o modelo lógico com apresentação do diagrama EER, um dicionário de dados seletivo integrado com as decisões de design das tabelas com estrutura não trivial - Utilizador, Lote, Venda, Pagamento, DetalheNumerario e LinhaRemessa - e uma secção de normalização que demonstra em prosa que o schema satisfaz a Terceira Forma Normal. Para o Software de Servidor Central, as secções seguem a mesma estrutura mas focam-se nas diferenças arquiteturais face ao Software de Loja - ausência de Delta Sync, idempotência por chave composta e desnormalização intencional - evitando repetir informação já apresentada na secção anterior. O dicionário seletivo do Servidor Central cobre as tabelas CEO, Loja, FechoDiaCentral, PagamentoCentral e SessaoCaixaCentral.
Fontes Fornecidas	Scripts SQL de ambos os softwares; diagramas EER gerados no MySQL Workbench; secções anteriores do relatório (para manter tom e estilo); ADRs 01–08.
Reflexão Final	A opção de integrar o dicionário de dados com as decisões de design numa secção única — em vez de as separar como no relatório de referência consultado — revelou-se mais coerente com o estilo do restante documento, onde a justificação de cada decisão precede imediatamente o artefacto que a materializa. A secção de normalização foi mantida deliberadamente mais curta do que o exemplo de referência, argumentando em prosa sem construir diagramas de dependências funcionais para cada tabela, adequando o nível de detalhe ao âmbito do projeto. A principal dificuldade foi evitar redundância entre as secções dos dois softwares — a instrução explícita de não repetir conceitos já explicados na secção da loja produziu um texto mais conciso e focado nas diferenças arquiteturais do servidor.
↑ Voltar à secção	

Prompt 43: Geração do Texto das Secções de Conceção do Sistema de Dados

Descrição	Geração das secções do relatório em Typst para a especificação da API e utilização do Swagger como ferramenta de apoio, validação e futura implementação.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Pretendo que escrevas a secção de relatório intitulada “Especificação da API e Utilização do Swagger”, integrada num projeto de Engenharia Informática. O sistema em causa é distribuído, sendo composto por um Software de Loja e um Servidor Central. O Servidor Central será desenvolvido em Java, utilizando Javalin, e irá expor uma API REST responsável pela receção de dados provenientes das lojas. A arquitetura do sistema segue um modelo em camadas (Layered Architecture com MVC), sendo a API parte integrante da camada de apresentação, através de handlers HTTP. Importa ainda referir que a fase atual do projeto corresponde à definição da arquitetura e especificação, não existindo ainda uma implementação completa. O objetivo desta secção é descrever a especificação da API do Servidor Central e explicar o papel do Swagger como ferramenta de apoio ao seu desenho, validação e futura implementação. O texto deve iniciar com uma explicação breve do que é o Swagger, limitada a um único parágrafo, apresentando-o como uma ferramenta de documentação e teste de APIs (e não como uma API), bem como o seu papel no ciclo de desenvolvimento de software. De seguida, deve ser apresentada a especificação da API a um nível conceptual, incluindo a organização dos endpoints por funcionalidades, bem como os métodos HTTP previstos. Devem ainda ser incluídos exemplos representativos de endpoints e exemplos de payloads JSON propostos, ainda que de natureza conceptual. O texto deve também abordar a integração da API no sistema, explicando o seu papel na comunicação entre o Software de Loja e o Servidor Central, bem como a sua relação com os principais casos de uso, tais como a sincronização de dados, o registo de vendas e o processamento de devoluções. Deve ainda ser feito o enquadramento da API na arquitetura global do sistema. Adicionalmente, deverá ser explicado de que forma a API foi concebida para suportar a sincronização de dados, com referência ao mecanismo de Delta Sync, incluindo considerações sobre consistência dos dados e idempotência. No que diz respeito ao Swagger, deve ser descrita a sua utilização prevista no projeto, nomeadamente como ferramenta de apoio à definição e validação da API antes da implementação, permitindo a simulação de pedidos e respostas, bem como a sua utilização futura para testes e debugging. O texto deve ser redigido em português de Portugal, com linguagem formal e adequada a um relatório académico, estruturado com subtítulos claros, focado na especificação em vez da implementação, e sempre que possível ilustrado com exemplos concretos, ainda que conceptuais. Devem ser seguidas boas práticas de engenharia de software, sem referência explícita a autores ou bibliografia.
Fontes Fornecidas	ADRs 01-10; Diagramas de Componente; Diagramas de Classe; Diagramas de Sequência; Bibliografia do Enunciado do Trabalho Prático
Reflexão Final	Foram necessárias algumas mudanças manuais no output final até ir de acordo com o que a equipa de trabalho pretendia nesta secção, tendo sido necessário simplificar o output do LLM, dado que possuía informação excessiva e que já havia sido desenvolvida em secções anteriores. A integração do Swagger como ferramenta de especificação e validação da API, revelou-se uma mais-valia no processo de design da interface de comunicação entre o Software de Loja e o Servidor Central, permitindo definir e validar o contrato da API antes da sua implementação efetiva.
↑ Voltar à secção	

Prompt 44: Geração do Texto das Secções de Especificação da API e Utilização do Swagger

Descrição	Geração de um diagrama de deployment PlantUML de alto nível que representa a topologia global do sistema TrasmUM, incluindo duas lojas com os seus terminais e bases de dados locais, o Servidor Central, e os atores do sistema com os respetivos pontos de acesso.
LLM Utilizado	Claude Sonnet 4.6
Prompt	Foi solicitada a geração de um diagrama PlantUML abstrato e generalizado que representasse a arquitetura global do sistema TrasmUM para inclusão na secção de Estrutura dos Softwares. O diagrama deveria mostrar as duas lojas individualmente com múltiplos terminais a partilhar uma base de dados local, o Servidor Central com a sua base de dados central, e os do sistema.

	Em iterações subsequentes, foram adicionados os atores Funcionário e Gestor em cada loja, e separado o acesso da CEO para um nó externo com browser web, refletindo a decisão de que o Software de Servidor Central é uma aplicação web acedida via browser e não uma aplicação desktop instalada na máquina servidora.
Fontes Fornecidas	ADRs 01–10; secção de Estrutura dos Softwares; RNF21.
Reflexão Final	A geração do diagrama decorreu em várias iterações. A primeira versão representava apenas os componentes de software e bases de dados sem atores. A segunda adicionou os atores Funcionário e Gestor em cada loja e a CEO no nó do Servidor Central — o que estava incorreto, dado que a CEO acede via browser a partir do seu próprio dispositivo. Esta incorreção motivou a identificação de uma lacuna no relatório: a natureza web do Software de Servidor Central não estava explicitamente declarada em nenhuma secção. O processo de correção do diagrama conduziu à criação do RNF21 e à adição de uma frase na secção de Estrutura dos Softwares a declarar que a interface é servida pelo Javalin e acedida via browser. A versão final do diagrama representa o dispositivo da Cátia como nó externo ao Servidor Central, com um componente Browser Web e uma seta HTTPS rotulada como acesso por browser, distinta das setas de fecho de dia provenientes das lojas.
↑ Voltar à secção	

Prompt 45: Geração e Revisão do Diagrama de Arquitetura Global do Sistema

Descrição	Pedido ao Claude Sonnet para analisar os artefactos de design produzidos nas fases anteriores e gerar prompts de implementação estruturados para fornecer ao Claude Code, cobrindo a totalidade do Software de Loja. Antes de gerar, o modelo colocou questões de clarificação sobre decisões técnicas não cobertas nos documentos de design, nomeadamente a estratégia de injeção de dependências, o mecanismo de navegação entre ecrãs JavaFX e a gestão de configuração do terminal.
LLM Utilizado	Claude Sonnet (Anthropic)
Prompt	Vou agora passar para a implementação do software da loja, o teu objetivo é criar um prompt para dar a um outro chat (claude code neste caso). Analisa toda a informação que tens sobre o software de loja, desde requisitos, a casos de uso, á sua arquitetura etc, e cria um prompt, que segue todas as boas práticas de prompt engineering, para fornecer ao claude code para ele criar este programa exatamente como nos o definimos, antes de começares, pergunta qualquer duvida que tenhas.
Fontes Fornecidas	Secção de arquitetura e design do relatório, anexos de requisitos funcionais e não funcionais, casos de uso, diagramas de classe, diagramas de sequência, schema SQL do Software de Loja, ADRs.
Reflexão Final	O modelo decidiu autonomamente dividir a implementação em dois prompts sequenciais para evitar a degradação de qualidade associada a prompts de escopo excessivo. O Prompt 1 cobre a fundação do projecto; o Prompt 2 cobre os serviços, controllers e UI. Esta decisão foi aceite pela equipa.
↑ Voltar à secção	

Prompt 46: Geração dos prompts de implementação do Software de Loja

Descrição	Prompt fornecido ao Claude Code para implementar a fundação completa do Software de Loja: estrutura Maven, schema SQL, todas as classes de domínio, todas as interfaces e implementações de repositório JDBC, infraestrutura de conexão à base de dados e exceções de domínio. O prompt especifica a stack tecnológica, a arquitetura de camadas, o schema SQL completo, os contratos de todos os repositórios e as regras de implementação JDBC.
LLM Utilizado	Claude Code (Anthropic)
Prompt	Stack tecnológica (não negociável): Java 21 LTS, JavaFX 21, MySQL 8 via JDBC puro, jBCrypt 0.4, Gson 2.10.1, Maven. Sem framework de injeção de dependências - injeção manual por raiz de composição. Arquitetura (não negociável): Layered Architecture com MVC, quatro camadas com dependências estritamente unidirecionais: Apresentação → Serviço → Repositório → Domínio. Um Controller nunca instancia um Repositório diretamente. A injeção de dependências é feita num único ApplicationContext instanciado no arranque. Estrutura de packages: pt.trasmum.loja - app/ (arranque e raiz de composição), dominio/ (core/, catalogo/, vendas/, tesouraria/, fornecedores/), repositorio/ (interfaces/, impl/), servico/ (interfaces/, impl/), apresentacao/ (controllers/), sincronizacao/. Schema da base de dados: [schema SQL completo com as tabelas Utilizador, LogAuditoria, Produto, Lote, Desconto, Fornecedor, PedidoRemessa, LinhaPedidoRemessa, Remessa, LinhaRemessa, Pagamento, Venda, LinhaVenda, Fatura, Devolucao, SessaoCaixa, Sangria, DetalheNumerario, FechoDia - com todas as constraints, ENUMs e chaves estrangeiras definidas.] Entidades de domínio: [especificação completa de todas as classes - RegistoSincronizavel (abstract), Utilizador, PerfilUtilizador, EstadoSincronizacao, Produto, Lote, Desconto, Fornecedor, Remessa, LinhaRemessa, PedidoRemessa, LinhaPedidoRemessa, Pagamento, Venda, LinhaVenda, Fatura, Devolucao, SessaoCaixa, Sangria, DetalheNumerario, Denominacao, FechoDia, TipoAcao, LogAuditoria.] Repositórios: [interfaces de UtilizadorRepositorio, LogAuditoriaRepositorio, ProdutoRepositorio, LoteRepositorio, FornecedorRepositorio, RemessaRepositorio, PedidoRemessaRepositorio, PagamentoRepositorio, VendaRepositorio, DevolucaoRepositorio, SessaoCaixaRepositorio, FechoDiaRepositorio - com todos os métodos especificados.] Regras de implementação JDBC: PreparedStatement em todas as queries. buscarLotesFEFO com ORDER BY dataValidade ASC filtrando quantidade > 0. VendaRepositorioImpl.guardar em transação única. RemessaRepositorioImpl.guardar em transação única. buscarPendentes() reverte EM_TRANSITO para PENDENTE antes de devolver. SessaoCaixaRepositorioImpl.buscarPendentes() retorna apenas sessões com dataEncerramento NOT NULL. Excepções de domínio: CredenciaisInvalidasException, SessaoDuplicadaException, AcessoNegadoException, ProdutoNaoEncontradoException, ProdutoInativoException, StockInsuficienteException, FaturaNaoEncontradaException, ArtigoNaoConstaNafaturaException, DevolucaoJaProcessadaException, QuantidadeDevolucaoExcedidaException, RemessaSemLinhasException, SessaoCaixaJaAbertaException, SessaoCaixaNaoEncontradaException, EnvioFechoFalhouException. Tarefa: implementar tudo o descrito. Não implementar ainda serviços, controllers JavaFX, ficheiros FXML ou lógica de sincronização. Após concluir, confirmar que o projeto compila sem erros e listar decisões de implementação não explicitamente especificadas.
Fontes Fornecidas	Prompt gerado na Entrada 46, artefactos de design das fases anteriores.
Reflexão Final	O código gerado compilou sem erros após a primeira iteração. Foram identificados desvios face ao design original durante a revisão — nomeadamente operações de repositório não previstas nos diagramas — documentados no ficheiro DESIGN_CHANGES.md.
↑ Voltar à secção	

Prompt 47: Implementação da fundação do Software de Loja — Prompt 1 (Claude Code)

Descrição	Prompt fornecido ao Claude Code na continuação da sessão do Prompt 1, após confirmação de compilação. Cobre a camada de serviço completa com lógica de negócio detalhada, o gateway de sincronização HTTP, a raiz de composição completo, todos os controllers JavaFX, os ficheiros FXML e a estilização CSS. A lógica crítica de negócio — FEFO, quatro guardas da devolução, Delta Sync — é especificada passo a passo para evitar que o modelo infira comportamentos incorretos.
LLM Utilizado	Claude Code (Anthropic)
Prompt	Contexto: continuação directa do Prompt 1. O projeto já tem estrutura Maven compilável, schema SQL, classes de domínio, interfaces e implementações de repositório, DatabaseConnection e ApplicationContext esqueleto. Interfaces de serviço: IAutenticacaoServico (autenticar, encerrarSessao, verificarSessaoDuplicada), IAutorizacaoServico (exigirPerfil, temPerfil), IAuditoriaServico (registrar, obterLogsPendentes), IVendaServico (iniciarVenda, adicionarLinha, finalizarVenda, anularVenda, calcularTroco, obterVendasPendentes), IDevolucaoServico (processar, obterDevolucoesPendentes), ICaixaServico (abrirSessao, registrarSangria, fecharSessao, verificarLimite, obterSesseoesPendentes), ICatalogoServico (criarProduto, editarProduto, desativarProduto, aplicarDesconto, gerarAlertasStockMinimo, gerarAlertasValidade, pesquisarProduto), IRemessaServico (registrarRemessa, associarPedidoPendente, criarPedidoRemessa, listarPedidos, obterRemessasPendentes, obterPagamentosPendentes), IPagamentoServico (listarPendentes, liquidar), IUtilizadorServico (criar, desativar, editar, validarGestorAtivo), IFechoDiaServico (executarFecho, reenviar, reverterEmTransito). Lógica crítica dos serviços: VendaServico - iniciarVenda cria Venda em memória sem persistir; adicionarLinha busca produto por código de barras, busca lotes FEFO, decrementa stock no lote, cria LinhaVenda; finalizarVenda persiste venda+linhas+fatura em transação, atualiza saldo da SessaoCaixa, verifica limite de caixa, regista log VENDA; anularVenda repõe stock nos lotes e persiste com estado ANULADA. DevolucaoServico -quatro guardas em sequência: (1) busca fatura, se null lança FaturaNaoEncontradaException; (2) verifica se artigo consta na fatura; (3) verifica unidades disponíveis para devolução; (4) busca lote por dataValidadeEmbalagem, se válido repõe stock. FechoDiaServico - recolhe pendentes de todos os serviços, marca EM_TRANSITO, compila PacoteFechoDTO, envia via SincronizacaoGateway; em falha reverte para PENDENTE; em sucesso marca CONFIRMADO exclusivamente após ACK. Estrutura de ecrãs JavaFX: LoginView.fxml, MainView.fxml com menu lateral adaptativo ao perfil, VendaView.fxml, DevolucaoView.fxml, CatalogoView.fxml, CaixaView.fxml, RemessaView.fxml, PagamentoView.fxml, UtilizadorView.fxml, FechoDiaView.fxml. Navegação por substituição do centro do BorderPane principal. Tarefa: implementar todas as interfaces e implementações de serviço, PacoteFechoDTO e DTOs de sincronização, HttpSincronizacaoGateway, ApplicationContext completo, todos os controllers JavaFX, todos os ficheiros FXML, app.css, e MainApp como ponto de entrada. Confirmar que o projeto compila, arranca e que o fluxo login → venda → fecho de dia funciona end-to-end.
Fontes Fornecidas	Prompt gerado na Entrada 46, código produzido pelo Prompt 1 (Entrada 47), mockups de interface da fase de design.
Reflexão Final	O código gerado foi revisto pela equipa. Dado que o JavaFX não suporta exportação a partir do Pencil, o frontend da loja ficou ligeiramente diferente dos esboços definidos na fase de design. Os desvios identificados foram documentados no DESIGN_CHANGES.md.
↑ Voltar à secção	

Prompt 48: Implementação dos serviços, controllers e UI do Software de Loja — Prompt 2 (Claude Code)

Descrição	Prompt fornecido ao Claude Code para implementar o backend completo do Servidor Central em Java com Javalin, incluindo a integração com o frontend Vue.js já existente. O frontend tinha sido gerado a partir dos ficheiros .pen do Pencil pelo Gemini, pelo que o backend foi desenvolvido a posteriori para servir um frontend já funcional com dados mock.
LLM Utilizado	Claude Code (Anthropic)
Prompt	Contexto: implementação do backend do Servidor Central do sistema TrasmUM. O frontend Vue.js já está completamente implementado em ServidorCentral/ e usa dados mock em src/api/mockData.ts e src/api/networkService.ts. O trabalho consiste em construir o backend Java que substitui esses dados mock por dados reais vindos da base de dados MySQL. Stack tecnológica (não negociável): Java 21, Javalin 6, MySQL 8 via JDBC puro, Gson 2.10.1, jBCrypt 0.4, Maven. Sem framework de injeção de dependências.

	Arquitetura: Layered Architecture - Handler → Serviço → Repositório → Domínio. Estrutura de packages: pt.trasmum.servidor - app/, dominio/, repositorio/ (interfaces/, impl/), servico/ (interfaces/, impl/), handler/, dto/, infra/. Schema: já existe em schema_servidor.sql. Tabelas: CEO, Loja, FechoDiaCentral, VendaCentral, LinhaVendaCentral, DevolucaoCentral, RemessaCentral, PagamentoCentral, SessaoCaixaCentral, LogAuditoriaCentral. Constraint de idempotência UNIQUE (idLoja, idOriginalLoja) em todas as tabelas transacionais. Entidades de domínio: CEO (com tentativasLogin e bloqueadoAte), Loja, FechoDiaCentral, VendaCentral, LinhaVendaCentral, DevolucaoCentral, RemessaCentral, PagamentoCentral, SessaoCaixaCentral, LogAuditoriaCentral. Enums: MetodoPagamento, EstadoPagamentoRemessa, TipoPagamento, TipoAcao. DTO de ingestão: PacoteFechoDTO com campos idLoja, nomeLoja, dataFecho, hashIntegridade, e listas de VendaDTO, DevolucaoDTO, RemessaDTO, PagamentoDTO, SessaoCaixaDTO, LogAuditoriaDTO. Serviços: IAutenticacaoCEOServico (autenticar, logout - sessões em memória com Map<"String",CEO>, expiração de 8 horas), IIngestaoServico (processarPacote - valida hash SHA-256, regista loja automaticamente se desconhecida, persiste cada entidade com INSERT IGNORE pela chave composta), IDashboardServico (calcularGlobal, calcularPorLoja - com DTOs DashboardGlobalDTO e DashboardLojaDTO), IMonitorServico (obterEstadoRede - com MonitorRedeDTO e lista de LojaMonitorDTO), IRemessaServico (listarRemessas com filtros), IRelatorioServico (gerarRelatorio com filtros). Endpoints: POST /api/auth/login, POST /api/auth/logout, POST /fecho (sem autenticação CEO - chamado pelas lojas), GET /api/dashboard/global, GET /api/dashboard/loja/:idLoja, GET /api/monitor, GET /api/remessas, GET /api/relatorios, GET /api/lojas. Middleware before em /api/verifica token Bearer excepto /api/auth/ e /fecho. Integração com frontend: reescrever src/api/networkService.ts para chamadas fetch() reais. Criar src/api/authService.ts, LoginView.vue e rota /login. Adicionar navigation guard no router. Não alterar estrutura de componentes, estilos ou lógica de apresentação. Bootstrapping: se tabela CEO vazia, criar conta admin/admin123 automaticamente. Eliminar após primeiro login com conta definitiva. Registo de decisões: qualquer decisão não especificada deve ser registada em DESIGN_CHANGES_SERVER.md com formato definido. Tarefa: implementar pom.xml, config.properties, DatabaseConnection, ApplicationContext, domínio, DTOs, repositórios, serviços, handlers, ServerMain, reescrita do networkService.ts, LoginView.vue e DESIGN_CHANGES_SERVER.md.
Fontes Fornecidas	Ficheiros .pen do Pencil exportados na fase de design, frontend Vue.js gerado pelo Gemini, schema_servidor.sql, artefactos de design das fases anteriores. Secção "Especificação da API e Utilização do Swagger" do relatório.
Reflexão Final	O backend foi desenvolvido para servir um frontend já existente, o que tornou o contrato da API implicitamente definido pelas chamadas presentes no código do frontend. As decisões de implementação não previstas nos documentos de design foram registadas no DESIGN_CHANGES_SERVER.md. A especificação OpenAPI (openapi.yaml) foi gerada no mesmo contexto, a partir da secção de Especificação da API e Utilização do Swagger, tendo-se optado por uma abordagem estática em vez do plugin javalin-openapi previsto originalmente - desvio registado em DESIGN_CHANGES_SERVER.md
↑ Voltar à secção	

Prompt 49: Implementação do backend do Servidor Central (Claude Code)

Descrição	Prompt fornecido ao Claude Code para gerar a suite de testes unitários do sistema TrasmUM. O objetivo foi cobrir a camada de serviço de ambos os softwares — Software de Loja e Servidor Central — com testes JUnit 5 e Mockito, sem alterar nenhuma classe de produção. O ponto de partida foi a ausência total de testes automatizados nos dois projetos Maven.
LLM Utilizado	Claude Code (Anthropic)
Prompt	Contexto: geração de testes unitários para o sistema TrasmUM. Os dois projetos Maven - Codigo/Loja/ e Codigo/Servidor_Central/backend/ - não têm quaisquer testes automatizados. O objetivo é cobrir a camada de serviço de cada projeto com testes JUnit 5 e Mockito, sem modificar nenhuma classe de produção existente.

Stack de testes (não negociável): JUnit 5 (junit-jupiter 5.11), Mockito 5 (mockito-core 5.12) para ambos os projetos. No Servidor Central, adicionar ainda io.javalin:javalin-testtools para testes de integração HTTP. Nenhum dos pom.xml tem o maven-surefire-plugin configurado - sem ele, o Maven usa a versão padrão 2.x que não detecta testes JUnit 5. Adicionar em ambos os projetos:

- dependências junit-jupiter e mockito-core em scope test
- maven-surefire-plugin versão 3.2.5 com <artifactId>junit-platform-launcher</artifactId> como dependência do plugin

Nota: o Servidor_Central/backend/pom.xml tem o maven-shade-plugin configurado na fase package. Não interfere com mvn test mas deve ser configurado com <skip>true</skip> no profile de testes para não conflitar com JaCoCo na fase de cobertura (etapa posterior).

Software de Loja - classes a testar:

AutorizacaoServico (sem dependências externas):

- exigirPerfil: utilizador com perfil permitido não lança exceção; utilizador com perfil não permitido lança AcessoNegadoException.
- temPerfil: devolve true para perfil correto e false para perfil incorreto.

VendaServico (mockar VendaRepositorio, ProdutoRepositorio, LoteRepositorio, SessaoCaixaRepositorio, IAuditoriaServico, ConfiguracaoTerminal):

- adicionarLinha - produto não encontrado: ProdutoNaoEncontradoException; produto inativo: ProdutoInativoException; stock insuficiente (soma dos lotes < quantidade): StockInsuficienteException; caminho feliz com dois lotes FEFO (o mais antigo esgota primeiro, o segundo é parcialmente consumido) - verificar com verify(loteRepo, times(2)).atualizar(any()) que ambos os lotes foram atualizados.
- finalizarVenda - numerário com valor insuficiente: IllegalArgumentException; sem sessão de caixa ativa: SessaoCaixaNaoEncontradaException; caminho feliz: venda guardada, saldo da sessão atualizado, auditoria registada; quando saldo + total > limiteMaximoCaixa o método deve concluir normalmente (o alerta de sangria é responsabilidade do controller, não do serviço - confirmar que nenhuma exceção é lançada).
- calcularTroco - valor inferior ao total: IllegalArgumentException; valor superior: devolve a diferença correta.

DevolucaoServico (mockar VendaRepositorio, DevolucaoRepositorio, LoteRepositorio, ProdutoRepositorio, IAuditoriaServico, ConfiguracaoTerminal):

- processar - guarda 1: fatura nula ou inexistente lança FaturaNaoEncontradaException; guarda 2: produto não consta nas linhas da fatura lança ArtigoNaoConstaNafaturaException; guarda 3: quantidade já totalmente devolvida lança DevolucaoJaProcessadaException; guarda 3b: quantidade a devolver excede disponível lança QuantidadeDevolucaoExcedidaException; guarda 4a: lote ainda válido (dataValidadeEmbalagem >= hoje) repõe stock e persiste devolução; guarda 4b: lote expirado não repõe stock mas persiste devolução.

FechoDiaServico (mockar todos os 17 parâmetros do construtor: IVendaServico, IDevolucaoServico, ICaixaServico, IRemessaServico, IAuditoriaServico, IAuthorizationServico, FechoDiaRepositorio, VendaRepositorio, DevolucaoRepositorio, RemessaRepositorio, PagamentoRepositorio, SessaoCaixaRepositorio, LogAuditoriaRepositorio, ProdutoRepositorio, LoteRepositorio, FornecedorRepositorio, UtilizadorRepositorio, SincronizacaoGateway, ConfiguracaoTerminal; configurar todos os obterPendentes() para devolver listas vazias por defeito):

- executarFecho - perfil FUNCIONARIO: autorizacaoServico.exigirPerfil lança AcessoNegadoException; verificar que o gateway nunca é invocado.
- executarFecho - envio falha (sincronizacaoGateway.enviarFechoDia devolve false): lança EnvioFechoFalhouException; verificar que vendaRepo.reverterEmTransito() é invocado (reversão total).
- executarFecho - sucesso (gateway devolve true, listas de pendentes vazias): fechoDiaRepo.guardar invocado uma vez; auditoriaServico.registar invocado com TipoAcao.FECHO_DIA.
- reverterEmTransito - verificar que os sete repositórios (vendaRepo, devolucaoRepo, sessaoCaixaRepo, remessaRepo, pagamentoRepo, logRepo, fechoDiaRepo) invocam cada um o seu reverterEmTransito().

Servidor Central - classes a testar:

AutenticacaoCE0Servico (mockar CE0Repositorio; usar Properties real com auth.tentativasMaximas=3 e auth.duracaoBloqueioMinutos=5):

- autenticar - utilizador inexistente: CredenciaisInvalidasException; conta com bloqueadoAte no futuro: ContaBloqueadaException; palavra-passe incorreta: CredenciaisInvalidasException e registarFalhado invocado; caminho feliz: retorna CE0 e invoca limparBloqueio.
- registarFalhado - abaixo do limiar: incrementa contador, não bloqueia; ao atingir o limiar: define bloqueadoAte e invoca repo.atualizar.

	• estaBloqueado - bloqueadoAte nulo: false; bloqueadoAte no passado: false; bloqueadoAte no futuro: true. IngestaoServico (método privado verificarHash testado indiretamente via processarPacote; mockar todos os repositórios, DatabaseConnection e Connection; configurar db.obter() para devolver um Connection mock com setAutoCommit e commit/rollback sem efeito): • hash ausente ou nulo: IntegridadeInvalidaException; hash incorreto: IntegridadeInvalidaException. • hash correto, pacote mínimo válido: processarPacote conclui sem exceção; verificar que conn.commit() foi invocado e conn.rollback() nunca o foi. • SQLException lançada por qualquer repositório durante a ingestão: processarPacote lança RuntimeException; verificar que conn.rollback() foi invocado (comportamento documentado em DS4). Convenções: • Uma classe de teste por classe de produção, em src/test/java/ espelhando o package de produção. • Nomes de método em português descritivo: adicionarLinha_produtoInativo_lancaExcecao. • Anotações @ExtendWith(MockitoExtension.class), @Mock e @InjectMocks onde aplicável. • Sem acesso a base de dados real - todos os repositórios são mocks. • Não alterar nenhum ficheiro de produção. • Todo o texto gerado - nomes de método, comentários, mensagens de erro e qualquer prosa - deve estar em português europeu (PT-PT). Nunca usar português do Brasil (PT-BR): usar “âmbito” em vez de “escopo”, “imposto”/“aplicado” em vez de “enforçado”, “reiniciar” em vez de “resetar”, “predefinição”/“por omissão” em vez de “por padrão”, e ortografia pós-A090 (“adotar”, “infraestrutura”, sem consoantes mudas como em “adoptado” ou “infra-estrutura”). Tarefa: atualizarCodigo/Loja/pom.xml e Codigo/Servidor_Central/backend/pom.xml com as dependências de teste; criar as classes de teste descritas acima.
Fontes Fornecidas	Código-fonte das classes VendaServico, DevolucaoServico, AutorizacaoServico, FechodiaServico (Codigo/Loja/), AutenticacaoCE0Servico e IngestaoServico (Codigo/Servidor_Central/backend/); interfaces dos repositórios correspondentes; diagramas de sequência da implementação atual — ds2_venda_fefo_v2.puml (caminhos FEFO e alerta de sangria), ds3_fecho_loja_atual.puml (fluxo de reversão e confirmação do fecho de dia), ds5_devolucao_atual.puml (quatro guardas de devolução), ds7_autenticacao_servidor_atual.puml (fluxo de bloqueio por tentativas), ds4_rececao_servidor_atual.puml (transação JDBC e rollback); artefactos de requisitos funcionais RF1–RF23 das fases anteriores.
Reflexão Final	A geração de testes sobre código existente revelou-se mais direta do que a geração de código de produção: o modelo dispunha das assinaturas, das exceções declaradas e da lógica de cada guarda, o que permitiu produzir casos de teste precisos sem ambiguidade. Os diagramas de sequência_atual foram determinantes para garantir que os casos de teste cobrem os caminhos de erro documentados na fase de design — nomeadamente o rollback transacional do IngestaoServico (DS4) e a ordem das guardas do DevolucaoServico (DS5) —, evitando que o modelo infra comportamentos apenas a partir do código. A principal dificuldade técnica foi a classe IngestaoServico, cuja dependência de DatabaseConnection e Connection JDBC exigiu configuração adicional do mock para simular o ciclo commit/rollback. O facto de a gestão do token de sessão ser responsabilidade do SessaoManager no handler (DS7) e não do AutenticacaoCE0Servico simplificou os testes deste serviço, que termina ao devolver o objeto CE0.

[↑ Voltar à secção](#)

Prompt 50: Geração de testes unitários assistida por LLM (Claude Code)

Descrição	Redação do texto da secção de Revisão e Aprovação do Projeto, correspondente ao encerramento da fase de Arquitetura e Design do Software, com base na reunião de validação realizada com o Hélder Cerqueira.
LLM Utilizado	Claude Sonnet 4.6 (Anthropic)
Prompt	Na secção de implementação do frontend, onde dizemos que o frontend de loja está diferente devido ao facto de não dar para exportar e que contactamos o Hélder, a justificação para a escolha de desktop para a loja e web para o servidor não está explicitamente desenvolvida no relatório. Propõe uma justificação e onde colocá-la. Cria também uma secção de Revisão e Aprovação do Projeto para o capítulo de Arquitetura, seguindo o padrão das secções homólogas dos capítulos anteriores, com base numa reunião de validação com o Hélder Cerqueira onde foram apresentados os esboços das interfaces e explicadas, em linguagem não técnica, as decisões de desktop para a loja e web para o servidor.

Fontes Fornecidas	Secção de Estrutura dos Softwares; RNF18; esboços das interfaces do sistema; secções de Revisão e Aprovação dos capítulos de Definição do Domínio e Levantamento de Requisitos.
Reflexão Final	O modelo identificou que a justificação para a dicotomia desktop/web nunca era articulada diretamente no relatório — estava implícita no RNF18 e na stack tecnológica, mas nunca apresentada como decisão consciente com motivação própria. A secção foi redigida tendo em conta o perfil não técnico do Hélder Cerqueira, excluindo referências a ADRs e diagramas técnicos e focando nas implicações funcionais de cada escolha.
↑ Voltar à secção	

Prompt 51: Redação da secção de Revisão e Aprovação da fase de Arquitetura

Descrição	Geração da secção de Verificação, Validação e Avaliação da Qualidade do Software Produzido do relatório, incluindo a estratégia de testes, as tabelas de testes unitários de ambos os subprojetos, a verificação dos requisitos SRS e a análise de qualidade ISO/IEC 25010.
LLM Utilizado	Claude Sonnet 4.6 (Anthropic)
Prompt	<pre> Prentende-se para a secção de Verificação, Validação e Avaliação da Qualidade do Software Produzido do relatório do projeto TrasmUM em Typst que escrevas esta secção completa em português de Portugal, com as subsecções que se seguem, respeitando o estilo e a formatação de tabelas já usados no resto do relatório. </pre>
Fontes Fornecidas	Código-fonte das classes de teste de ambos os subprojetos; Resultados de execução do mvn test; Relatório JaCoCo; Lista de requisitos funcionais e não funcionais da fase de requisitos; Enunciado do Trabalho Prático
Reflexão Final	O output cobriu integralmente o âmbito pedido e foi aceite sem alterações de substância. O nível de detalhe das tabelas e das justificações para a ausência de testes de integração estava alinhado com o contexto do projeto.
↑ Voltar à secção	

Prompt 52: Redação da secção de Verificação e Avaliação da Qualidade do Software Produzido

Descrição	Geração do texto narrativo do Capítulo 3 do relatório do projeto TrasmUM, cobrindo a metodologia de utilização de LLMs na fase de implementação, a configuração do ambiente e estrutura do projeto, o desenvolvimento por camadas do Software de Loja (domínio, repositórios, serviços, apresentação, gateway de sincronização e raiz de composição), o desenvolvimento do backend e frontend do Servidor Central, a revisão de código e deteção de smells, e os desvios ao design original. O texto foi produzido de forma iterativa, secção a secção, com revisão e ajustes pela equipa após cada proposta.
LLM Utilizado	Claude Sonnet 4.6 (Anthropic)
Prompt	<pre> Conversa iterativa conduzida ao longo de múltiplas interações, fornecendo progressivamente o conteúdo real de cada secção - código-fonte, ficheiros DESIGN_CHANGES.md e DESIGN_CHANGES_SERVER.md, prompts de implementação gerados, screenshots e excertos de código - e solicitando a redação do texto narrativo correspondente em português de Portugal, respeitando o estilo académico e a formatação Typst do restante relatório. </pre>
Fontes Fornecidas	Código-fonte do Software de Loja e do Servidor Central; ficheiros DESIGN_CHANGES.md e DESIGN_CHANGES_SERVER.md; prompts de implementação gerados na fase anterior; screenshots da aplicação em funcionamento; artefactos de design das fases anteriores.
Reflexão Final	O texto foi produzido secção a secção de forma iterativa, com revisão humana após cada proposta. Várias secções foram ajustadas pela equipa para remover repetição de conteúdo já apresentado em capítulos anteriores, corrigir imprecisões factuais e adaptar o nível de detalhe ao contexto académico do relatório.
↑ Voltar à secção	

Prompt 53: Redação do Capítulo 3 — Implementação e Desenvolvimento Assistido por LLM